

第二章 数据的机器级表示与处理

数值数据的表示

非数值数据的表示

数据的存储

数据的运算

数据的表示和运算

- 目标

- 掌握计算机内部各种数据的编码表示、存储及运算方法
- 掌握高级语言程序中的各种类型变量对应的表示形式
- 在高级语言程序中的变量、机器数和底层硬件（寄存器、加法器、ALU等）之间建立关联
- 综合运用所学知识，分析高级语言和机器级语言程序设计中遇到的各种与数据表示和运算相关的问题，解释相应的执行结果（与数学直觉的不同）

数据的表示和运算

- 分以下三个部分介绍
 - **第一讲：数值数据的表示**
 - 定点数的编码表示
 - **整数的表示**
 - 无符号整数、带符号整数
 - **浮点数的表示**
 - C语言程序的整数类型和浮点数类型
 - **第二讲：非数值数据的表示、数据的存储**
 - 逻辑值、西文字符、汉字字符
 - 数据宽度单位
 - 大端/小端、对齐存放

数据的表示和运算

- 分以下三个部分介绍（续）

- **第三讲：数据的运算**

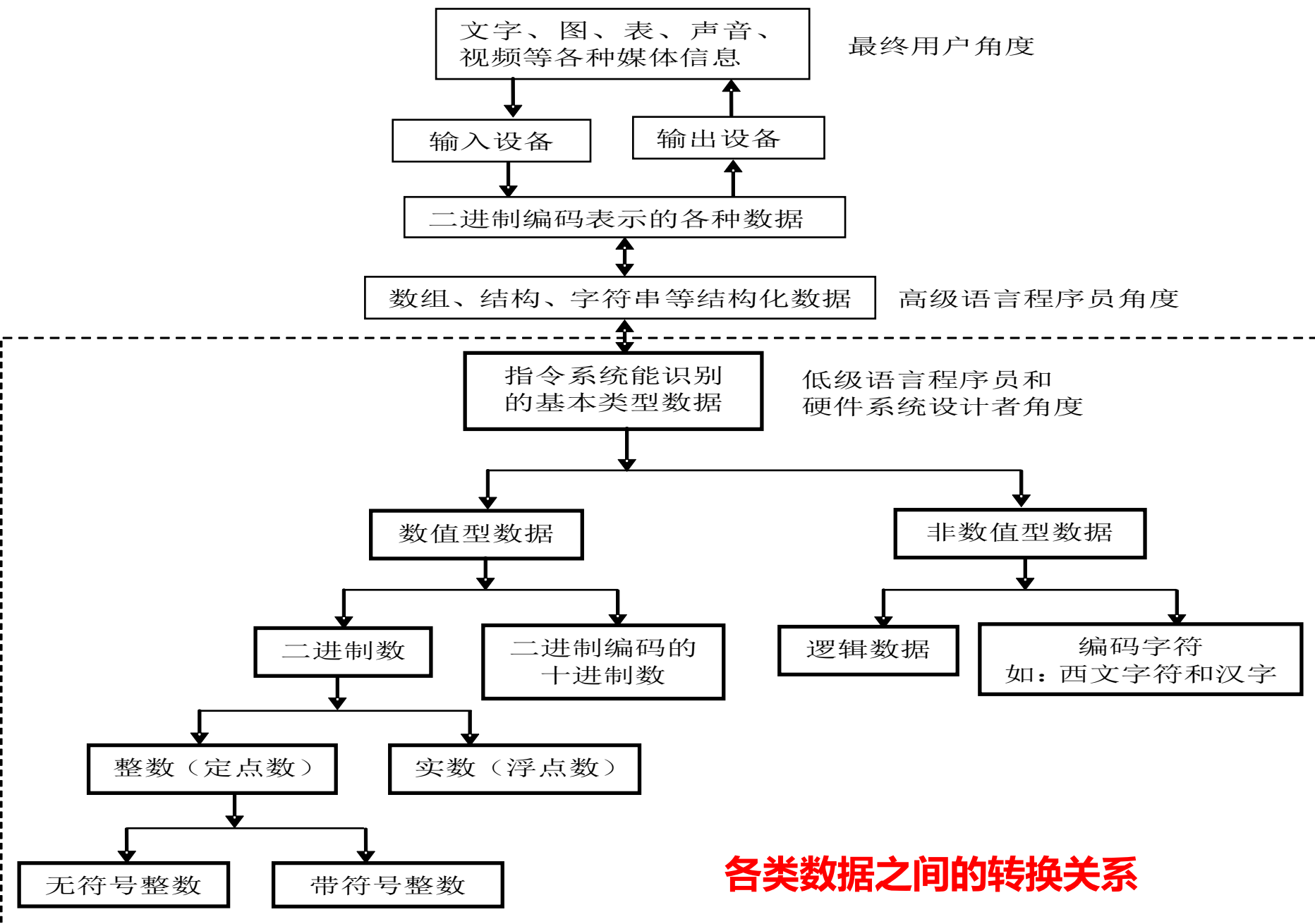
- 按位运算和逻辑运算
 - 移位运算
 - 位扩展和位截断运算
 - 无符号和带符号整数的加减运算
 - 无符号和带符号整数的乘除运算
 - 变量与常数之间的乘除运算
 - 浮点数的加减乘除运算

围绕C语言中的
运算，解释其
在底层机器级
的实现方法

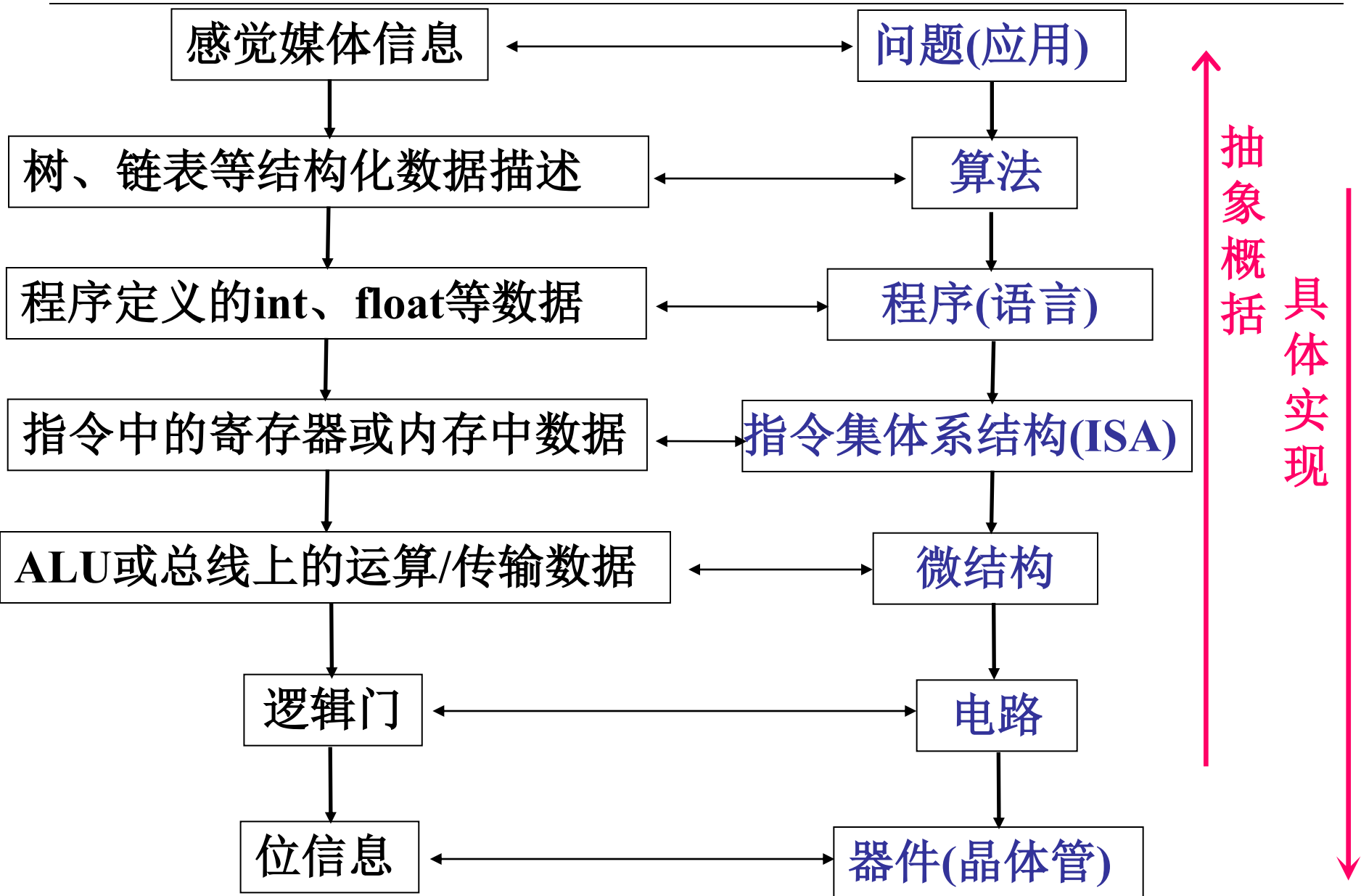
从高级语言程序中的表达式出发，用机器数在具体电路中的
执行过程，来解释表达式的执行结果

数据的表示和运算

- 分以下三个部分介绍
 - **第一讲：数值数据的表示**
 - 定点数的编码表示
 - 整数的表示
 - 无符号整数、带符号整数
 - 浮点数的表示
 - C语言程序的整数类型和浮点数类型
 - **第二讲：非数值数据的表示、数据的存储**
 - 逻辑值、西文字符、汉字字符
 - 数据宽度单位
 - 大端/小端、对齐存放



“转换”的概念在数据表示中的反映



数值数据的表示

- 数值数据表示的三要素

- 进位计数制
- 定、浮点表示
- 如何用二进制编码

即：要确定一个数值数据的值必须先确定这三个要素。

例如，机器数 **01011001** 的值是多少？ 答案是：不知道！

- 进位计数制

(编码方式、小数点位置)

- 十进制、二进制、十六进制、八进制数及其相互转换

- 定/浮点表示 (解决小数点问题)

- 定点整数、定点小数
- 浮点数 (可用一个定点小数和一个定点整数来表示)

- 定点数的编码 (解决正负号问题)

- 原码、补码、反码、移码 (反码很少用)

原码、反码、补码、移码

- 原码：取绝对值 **0不唯一**
- 反码：符号位0表示正，1表示负，正数的反码等于原码。负数的反码等于原码按位数取反，（绝对值按位求反）
- 步骤：先将这个负数化为原码，再根据原码取反。比如该负数原码是：0 0000001，那么它取反就是0 1111110
- **反码基本不用了**
- 补码：符号位0表示正，1表示负，正数的补码等于原码。负数的补码等于反码末位加1，
- 步骤：先将这个负数化为原码再化为反码，末位加1。比如该负数的反码是1 1111110，那么末位加1就是1 1111111
- **工程师们科学家们共同约定俗成的事实标准 One's Complement vs. Two's Complement**
- 移码：等于补码的符号（第一位数字）位取反。
- 步骤：先把这个数化为补码，然后再根据补码的第一位数字取反即可。（移码和补码的关系：同一数值的移码与补码符号位相反，其它各位相同。）
- 总结：
- 正数的原码，补码，反码相同；
- 负数的反码：原码的数值取反；
- 负数的补码：原码转换成反码，反码末位加1
- 负数的移码：与补码的符号位（第一位数字）相反

Sign and Magnitude （原码的表示）

Decimal	Binary
---------	--------

0	0000
---	------

1	0001
---	------

2	0010
---	------

3	0011
---	------

4	0100
---	------

5	0101
---	------

6	0110
---	------

7	0111
---	------

Decimal	Binary
---------	--------

-0	1000
----	------

-1	1001
----	------

-2	1010
----	------

-3	1011
----	------

-4	1100
----	------

-5	1101
----	------

-6	1110
----	------

-7	1111
----	------

◆ 容易理解， 但是：

- ✓ 0 的表示不唯一，故不利于程序员编程
- ✓ 加、减运算方式不统一
- ✓ 需额外对符号位进行处理，故不利于硬件设计
- ✓ 特别当 $a < b$ 时，实现 $a - b$ 比较困难

从 50年代开始，整数都采用补码来表示
但浮点数的尾数用原码定点小数表示

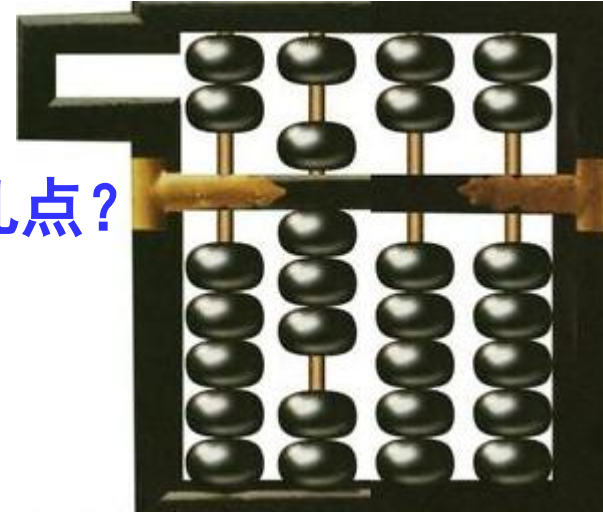
补码的表示

现实世界的模运算系统举例

例1：“钟表”模运算系统

假定时针只能顺拨，从10点倒拨4格后是几点？

$$10 - 4 = 10 + (12 - 4) = 10 + 8 = 6 \pmod{12}$$



例2：“4位十进制数”模运算系统

假定算盘只有四档，且只能做加法，则在算盘上计算
9828-1928等于多少？

$$9828 - 1928 = 9828 + (10^4 - 1928)$$

$$= 9828 + 8072$$

$$= \boxed{1}7900$$

取模即只留余数，高位“1”被丢弃！
相当于只有低4位留在算盘上。

$$= 7900 \pmod{10^4}$$

补码 - 模运算 (modular运算)

重要概念：在一个模运算系统中，一个数与它除以“模”后的余数等价。

时钟是一种模12系统 现实世界中的模运算系统

假定钟表时针指向10点，要将它拨向6点， 则有两种拨法：

① 倒拨4格： $10 - 4 = 6$

② 顺拨8格： $10 + 8 = 18 \equiv 6 \pmod{12}$

模12系统中： $10 - 4 \equiv 10 + 8 \pmod{12}$

$-4 \equiv 8 \pmod{12}$

则，称8是-4对模12的补码（即：-4的模12补码等于8）。

同样有 $-3 \equiv 9 \pmod{12}$

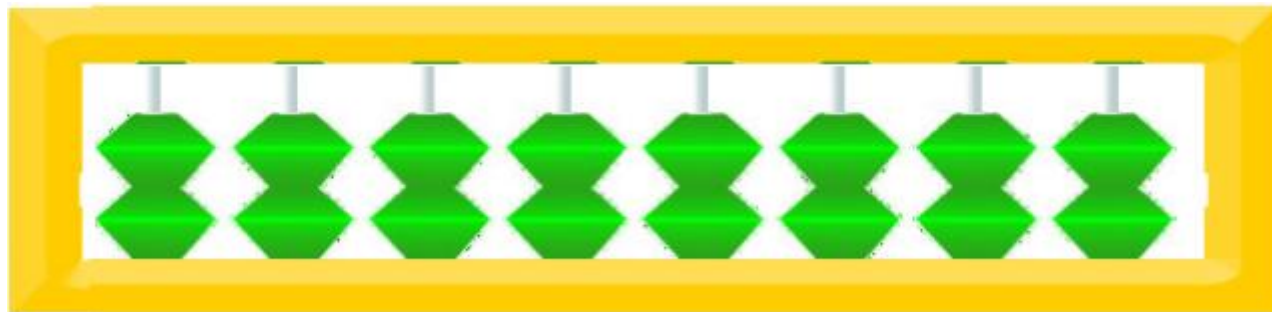
$-5 \equiv 7 \pmod{12}$ 等

结论1： 一个负数的补码等于模减该负数的绝对值。

结论2： 对于某一确定的模，某数减去小于模的另一数，总可以用该数加上另一数负数的补码来代替。

补码 (modular运算)：+ 和- 的统一

计算机中的运算器是模运算系统



8位二进制加法器模运算系统 $127-64=?$

计算 $0111\ 1111 - 0100\ 0000 = ?$

$$\begin{aligned} 0111\ 1111 - 0100\ 0000 &= 0111\ 1111 + (2^8 - 0100\ 0000) // 256 - 192 \\ &= 0111\ 1111 + 1100\ 0000 = \boxed{1}0011\ 1111 \pmod{2^8} \text{ 高位溢出丢弃} \\ &= 0011\ 1111 \end{aligned}$$

只留余数，“1”被丢弃

结论1： 一个负数的补码等于对应正数的“各位取反、末位加一” 减法变加法 无额外电路

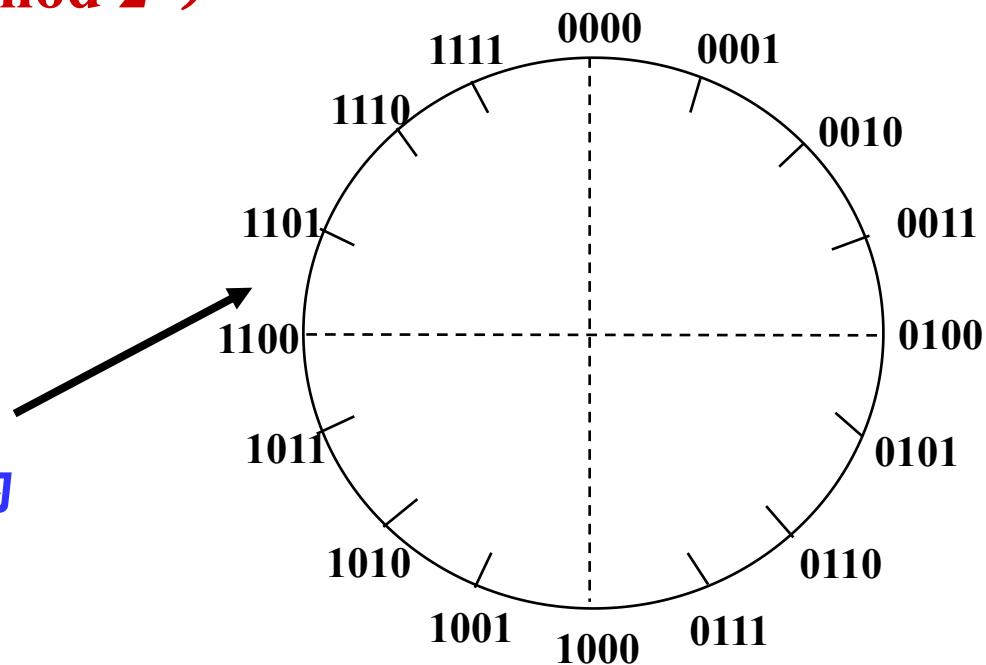
运算器适合用补码表示和运算

运算器只有有限位，假设为 n 位，则运算结果只能保留低 n 位，故可看成是个只有 n 档的二进制算盘，因此，其模为 2^n 。

补码的定义 假定补码有 n 位，则：

$$[X]_{\text{补}} = 2^n + X \quad (-2^n \leq X < 2^n, \text{ mod } 2^n)$$

当 $n=4$ 时，共有16个机器数：
0000 ~ 1111，可看成是模为
 2^4 的钟表系统。真值的范围为
-8 ~ +7



求特殊数的补码

假定机器数有n位

$$\textcircled{1} [-2^{n-1}]_{\text{补}} = 2^n - 2^{n-1} = \mathbf{10\dots0} \text{ (n-1个0)} \quad (\text{mod } 2^n)$$

$$\textcircled{2} [-1]_{\text{补}} = 2^n - 0\dots01 = \mathbf{11\dots1} \text{ (n个1)} \quad (\text{mod } 2^n)$$

$$\textcircled{3} [+0]_{\text{补}} = [-0]_{\text{补}} = \mathbf{00\dots0} \text{ (n个0)}$$

补码与真值之间的简便转换

例: 设机器数有8位, 求123和-123的补码表示。

如何快速得到123的二进制表示?

解: $123 = 127 - 4 = 01111111\text{B} - 100\text{B} = 01111011\text{B}$

$-123 = -01111011\text{B}$

$[01111011]_{\text{补}} = 2^8 + 01111011 = 100000000 + 01111011$
 $= 01111011 \pmod{2^8}$, 即 7BH。

$[-01111011]_{\text{补}} = 2^8 - 01111011 = 10000\ 0000 - 01111011$
 $= 1111\ 1111 - 0111\ 1011 + 1$
 $= 1000\ 0100 + 1 \longleftarrow \text{各位取反, 末位加1}$
 $= 1000\ 0101$, 即 85H。

Unsigned integer(无符号整数)

- 机器中字的**位排列顺序**有两种方式：（例：32位字： $0\dots01011_2$ ）
 - 高到低位从左到右： $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011$  **LSB**
 - 高到低位从右到左： $1101\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000$  **MSB**
 - Leftmost和rightmost这两个词有歧义，故用**LSB(Least Significant Bit)**来表示最低有效位，用**MSB**来表示最高有效位
 - 高位到低位多采用从左往右排列
- 一般在全部是正数运算且不出现负值结果的情况下，可使用无符号数表示。例如，**地址运算，编号表示**，等等
- 无符号整数的编码中**没有符号位**
- 能表示的最大值大于位数相同的带符号整数的最大值（Why？）
 - 例如，8位无符号整数最大是255（1111 1111）
而8位带符号整数最大为127（0111 1111）
- 总是整数，所以很多时候就**简称为“无符号数”**

Signed integer（带符号整数，定点整数）

- 计算机必须能处理正数(positive) 和负数(negative), MSB表示数符
- 有三种定点编码方式
 - Signed magnitude（原码）
现用来表示浮点（实）数的尾数
 - One's complement（反码）
现已不用于表示数值数据
 - Two's complement（补码）
50年代以来，所有计算机都用补码来表示定点整数
- 为什么用补码表示带符号整数？
 - 补码运算系统是模运算系统，加、减运算统一
 - 数0的表示唯一，方便使用
 - 比原码和反码多表示一个最小负数

C语言程序中的整数

无符号数： unsigned int (short / long)； 带符号整数： int (short / long)

常在一个数的后面加一个 “u”或 “U”表示无符号数

若同时有无符号和带符号整数，则C编译器将带符号整数强制转换为无符号数

假定以下关系表达式在32位用补码表示的机器上执行，结果是什么？

关系表达式	运算类型	结果	说明
0 == 0U			
-1 < 0			
-1 < 0U			
2147483647 > -2147483647-1			
2147483647U > -2147483647-1			
2147483647 > (int) 2147483648U			
-1 > -2			
(unsigned) -1 > -2			

C语言程序中的整数

关系表达式	类型	结果	说明
$0 == 0U$	无	1	$00...0B = 00...0B$
$-1 < 0$	带	1	$11...1B (-1) < 00...0B (0)$
$-1 < 0U$	无	0*	$11...1B (2^{32}-1) > 00...0B(0)$
$2147483647 > -2147483647 - 1$	带	1	$011...1B (2^{31}-1) > 100...0B (-2^{31})$
$2147483647U > -2147483647 - 1$	无	0*	$011...1B (2^{31}-1) < 100...0B(2^{31})$
$2147483647 > (\text{int}) 2147483648U$	带	1*	$011...1B (2^{31}-1) > 100...0B (-2^{31})$
$-1 > -2$	带	1	$11...1B (-1) > 11...10B (-2)$
$(\text{unsigned}) -1 > -2$	无	1	$11...1B (2^{32}-1) > 11...10B (2^{32}-2)$

带*的结果与常规预想的相反！

C语言程序中的整数

例如，考虑以下C代码：

```
1 int x = -1;
2 unsigned u = 2147483648;
3
4 printf ( "x = %u = %d\n" , x, x);
5 printf ( "u = %u = %d\n" , u, u);
```

在32位机器上运行上述代码时，它的输出结果是什么？为什么？

$x = 4294967295 = -1$

$u = 2147483648 = -2147483648$

- ◆ 因为-1的补码整数表示为“11...1”，作为32位无符号数解释时，其值为 $2^{32}-1 = 4\ 294\ 967\ 296-1 = 4\ 294\ 967\ 295$ 。
- ◆ 2^{31} 的无符号数表示为“100...0”，被解释为32位带符号整数时，其值为最小负数： $-2^{32-1} = -2^{31} = -2\ 147\ 483\ 648$ 。

C语言程序中的整数

- 1) 在有些32位系统上, C表达式 $-2147483648 < 2147483647$ 的执行结果为false。Why?
- 2) 若定义变量 `int i=-2147483648;` , 则 `i < 2147483647` 的执行结果为true。Why?
- 3) 如果将表达式写成 `-2147483647-1 < 2147483647` , 则结果会怎样呢? Why?

1) 在ISO C90标准下 , 2147483648为unsigned类型, 因此
 $-2147483648 < 2147483647$ 按无符号数比较,
10.....0B比01.....1大, 结果为false。

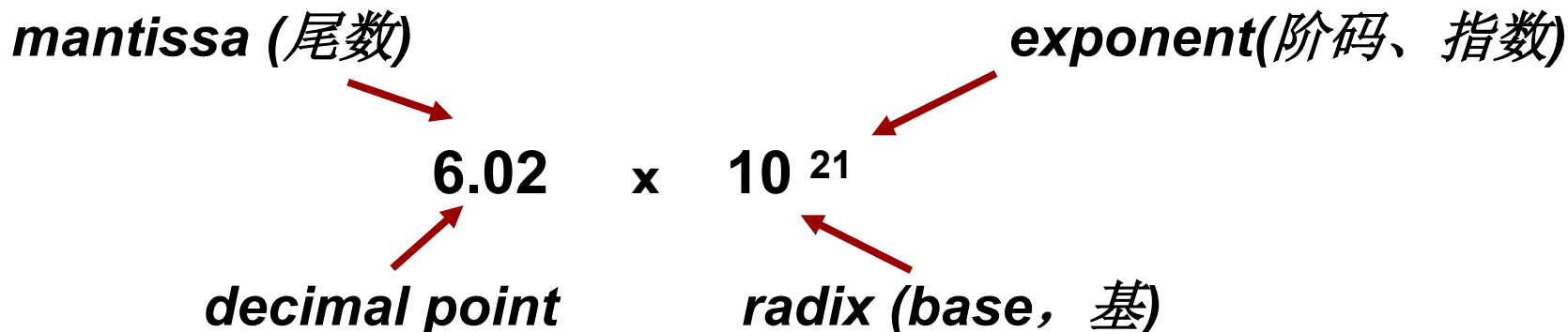
在ISO C99标准下 , 2147483648为int类型, 因此
 $-2147483648 < 2147483647$ 按带符号整数比较,
10.....0B比01.....1小, 结果为true。

2) `i < 2147483647` 按int型数比较, 结果为true。

3) `-2147483647-1 < 2147483647` 按int型比较, 结果为true。

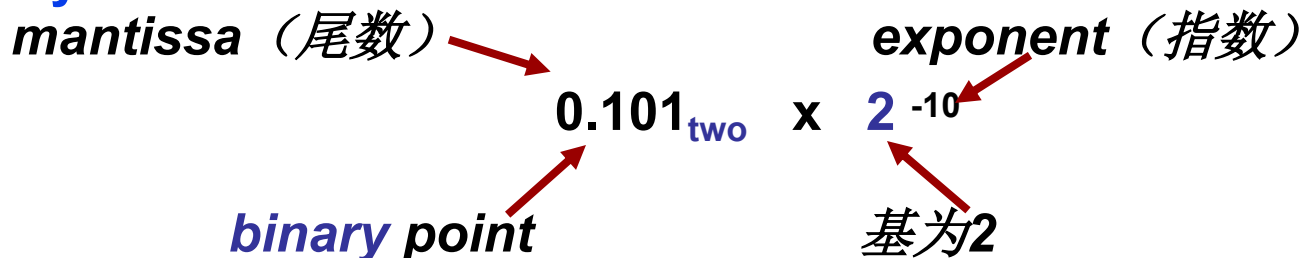
科学计数法(Scientific Notation)与浮点数

Example:



- **Normalized form (规格化形式)** : 小数点前只有一位非0数
- 同一个数有多种表示形式。例：对于数 1/1,000,000,000
 - Normalized (唯一的规格化形式): 1.0×10^{-9}
 - Unnormalized (非规格化形式不唯一) : 0.1×10^{-8} , 10.0×10^{-10}

for Binary Numbers:



只要对尾数和指数分别编码，就可表示一个浮点数（即：实数）

浮点数(Floating Point)的表示范围

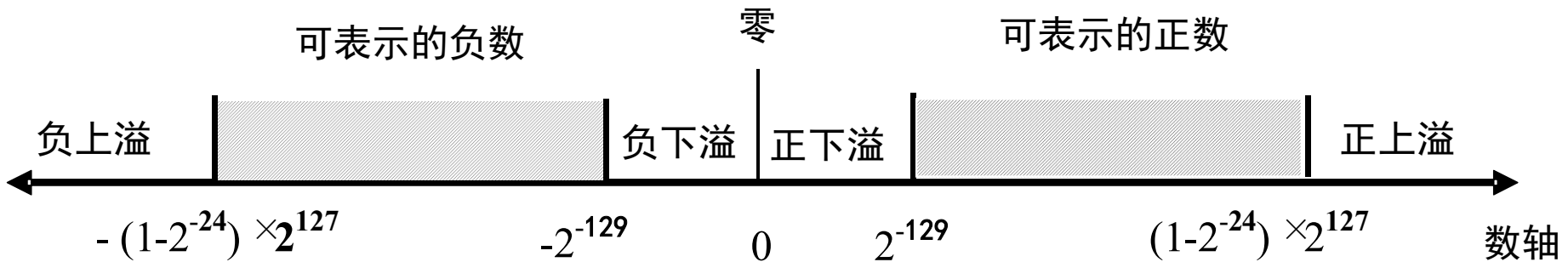
例：画出下述32位浮点数格式的规格化数的表示范围。



第0位数符S；第1~8位为8位移码表示阶码E（偏置常数为128）；第9~31位为24位二进制原码小数表示的尾数M。规格化尾数的小数点后第一位总是1，故规定第一位默认的“1”不明显表示出来。这样可用23个数位表示24位尾数。

最大正数: $0.11\dots1 \times 2^{11\dots1} = (1-2^{-24}) \times 2^{127}$ 最小正数: $0.10\dots0 \times 2^{00\dots0} = (1/2) \times 2^{-128}$

因为原码是对称的，所以其表示范围关于原点对称。



机器0：尾数为0 或 落在下溢区中的数

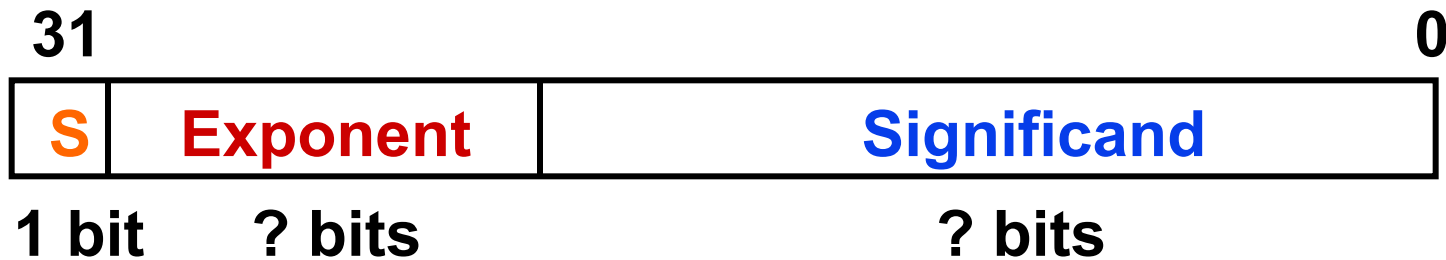
浮点数范围比定点数大，但数的个数没变多，故数之间更稀疏，且不均匀

浮点数的表示

- Normal format（规格化数形式）：

$$+/-1.\text{xxxxxxxxxxx} \times R^{\text{Exponent}}$$

- 32-bit 规格化数：



S 是符号位（Sign）

Exponent 用移码（增码）来表示

Significand 表示 xxxxxxxxxxxxxx，尾数部分

（基可以是 2 / 4 / 8 / 16，约定信息，无需显式表示）

- 早期的计算机，各自定义自己的浮点数格式

问题：浮点数表示不统一会带来什么问题？ 不通用

“Father” of the IEEE 754 standard

直到80年代初，各个机器内部的浮点数表示格式还没有统一
因而相互不兼容，机器之间传送数据时，带来麻烦

1970年代后期，IEEE成立委员会着手制定浮点数标准

1985年完成浮点数标准IEEE 754的制定

This standard was primarily the work of one person, UC Berkeley math professor William Kahan.

1989
ACM Turing
Award Winner!

[www.cs.berkeley.edu/~wkahan/
ieee754status/754story.html](http://www.cs.berkeley.edu/~wkahan/ieee754status/754story.html)



Prof. William Kahan

IEEE 754标准

规格化数: $\pm 1.\text{xxxxxxxxxx}_{\text{two}} \times 2^{\text{Exponent}}$

规定: 小数点前总是“1”, 故可隐含表示。
注意: 和前面例子规定不一样, 这里更合理!

Single Precision :

S	Exponent (非负)	Significand
1 bit	8 bits	23 bits

- Sign bit: 1 表示negative ; 0表示 positive
- Exponent (阶码 / 指数) : 全0和全1用来表示特殊值! 1-254
 - SP规格化数阶码范围为0000 0001 (-126) ~ 1111 1110 (127)
 - bias(UD)为127 (single), 1023 (double) 为什么用127? 若用128, 则阶码范围为多少?
- Significand (尾数):
 - 规格化尾数最高位总是1, 所以隐含表示, 省1位
 - 1 + 23 bits (single) , 1 + 52 bits (double)

SP: $(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-127)}$ 0000 0001 (-127) ~ 1111 1110 (126)

DP: $(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-1023)}$

Ex: Converting Binary FP to Decimal

BEE00000H is the hex. Rep. Of an IEEE 754 SP FP number

1 011 1101 110 0000 0000 0000 0000 0000

$$(-1)^S \times (1 + \text{Significand}) \times 2^{(\text{Exponent}-127)}$$

- **Sign:** 1 => negative
- **Exponent:** 移码——暴力算=真值+偏移量
 - $0111\ 1101_{\text{two}} = 125_{\text{ten}}$
 - Bias adjustment: $125 - 127 = -2$
- **Significand:** (1.110 0000 0000 0000 0000 0000)
 $1 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 0 \times 2^{-4} + 0 \times 2^{-5} + \dots$
 $= 1 + 2^{-1} + 2^{-2} = 1 + 0.5 + 0.25 = 1.75$
- **Represents:** $-1.75_{\text{ten}} \times 2^{-2} = -0.4375$

繁琐但机器处理起来高效

Ex: Converting Decimal to FP

-12.75

1. Denormalize: -12.75

2. Convert integer part:

$$12 = 8 + 4 = 1100_2$$

3. Convert fractional part:

$$.75 = .5 + .25 = .11_2$$

4. Put parts together and normalize:

$$1100.11 = 1.10011 \times 2^3$$

5. Convert exponent: $127 + 3 = 128 + 2 = 1000\ 0010_2$

11000	0010	100	1100	0000	0000	0000	0000
-------	------	-----	------	------	------	------	------

The Hex rep. is C14C0000H

Normalized numbers (规格化数)

前面的定义都是针对规格化数 (normalized form)

How about other patterns?

Exponent	Significand	Object
1-254	anything implicit leading 1	Norms
0	0	?
0	nonzero	?
255	0	?
255	nonzero	?

IEEE 754定义的特殊值-处理异常时非常有用

- 零 (0): **阶码全0, 尾数全0。**
- 无穷大 (∞): **阶码全1, 尾数全0。** 比如除以0, 结果是无穷大, 而不是直接报错。
- NaN (非数): **阶码全1, 尾数非0。** 比如对负数开根号, 结果就是NaN。
- 非规格化数: **阶码全0, 尾数非0。** 用来表示非常接近0的数, 填补了0附近的空隙。

Representation for 0

How to represent 0?

exponent: all zeros

significand: all zeros

What about sign? Both cases valid.

+0: 0 00000000 000000000000000000000000

-0: 1 00000000 000000000000000000000000

Representation for $+\infty/-\infty$

In FP, 除数为0的结果是 $\pm\infty$, 不是溢出异常. (整数除0为异常)

为什么要这样处理?

∞ : infinity

- 可以利用 $+\infty/-\infty$ 作比较。 例如: $X/0 > Y$ 可作为有效比较

How to represent $+\infty/-\infty$?

- **Exponent** : all ones (11111111B = 255)
- **Significand**: all zeros

$+\infty$: 0 11111111 000000000000000000000000

$-\infty$: 1 11111111 000000000000000000000000

Operations

$$5.0 / 0 = +\infty, \quad -5.0 / 0 = -\infty$$

$$5 + (+\infty) = +\infty, \quad (+\infty) + (+\infty) = +\infty$$

$$5 - (+\infty) = -\infty, \quad (-\infty) - (+\infty) = -\infty \quad \text{etc}$$

Representation for “Not a Number”

$\text{Sqrt}(-4.0) = ?$ $0/0 = ?$

– Called **Not a Number (NaN)** - “非数”

How to represent NaN

Exponent = 255

Significand: nonzero

NaNs can help with debugging

Operations

$\text{sqrt}(-4.0) = \text{NaN}$

$\text{op}(\text{NaN}, x) = \text{NaN}$

$+\infty - (+\infty) = \text{NaN}$

etc.

$0/0 = \text{NaN}$

$+\infty + (-\infty) = \text{NaN}$

$\infty/\infty = \text{NaN}$

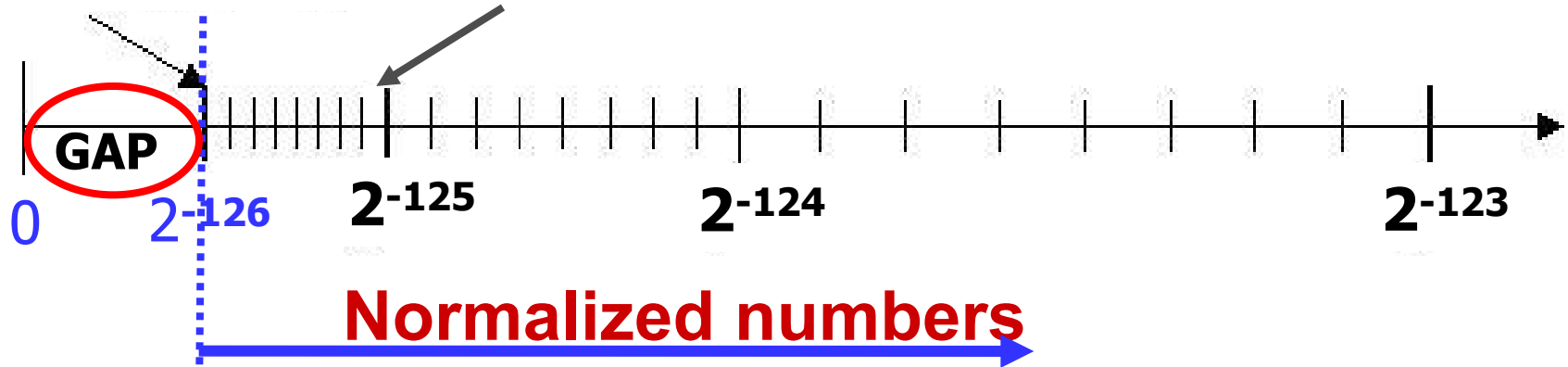
Representation for Denorms(非规格化数)

What have we defined so far? (for SP)

Exponent	Significand	Object	Used to represent Denormalized numbers
0	0	+/-0	
0	nonzero	Denorms	
1-254	anything implicit leading 1	Norms	
255	0	+/- infinity	
255	nonzero	NaN	

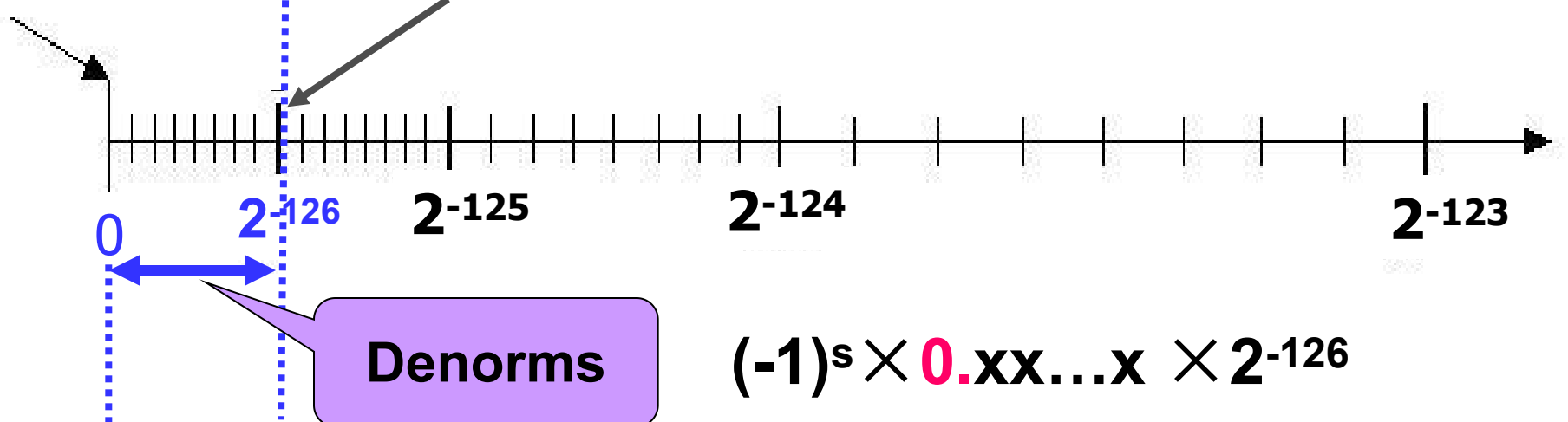
Representation for Denorms

$1.0...0 \times 2^{-126} \sim 1.1...1 \times 2^{-126}$



Normalized numbers

$0.0...0 \times 2^{-126} \sim 0.1...1 \times 2^{-126}$



Denorms

$$(-1)^s \times 0.xx...x \times 2^{-126}$$

关于浮点数精度的一个例子

```
#include <iostream>
using namespace std;
int main()
{
    float heads;
    cout.setf(ios::fixed, ios::floatfield);
    while(1)
    {
        cout << "Please enter a number: ";
        cin >> heads;
    }
```

运行结果:

```
Please enter a number: 61.419997
61.419998
Please enter a number: 61.419998
61.419998
Please enter a number: 61.419999
61.419998
Please enter a number: 61.42
61.419998
Please enter a number: 61.420001
61.420002
Please enter a number:
```

$$\log_{10} 2^{24} \approx 7.2$$

第一讲小结

- 在机器内部编码后的数称为机器数，其值称为真值
- 定义数值数据有三个要素：进制、定点/浮点、编码
- 整数的表示
 - 无符号数：正整数，用来表示地址等；带符号整数：用补码表示
- C语言中的整数
 - 无符号数：unsigned int (short / long)；带符号数：int (short / long)
- 浮点数的表示
 - 符号；尾数：定点小数；指数（阶）：定点整数（基不用表示）
- 浮点数的范围
 - 正上溢、正下溢、负上溢、负下溢；与阶码的位数和基的大小有关
- 浮点数的精度：与尾数的位数和是否规格化有关
- 浮点数的表示（IEEE 754标准）：单精度SP（float）和双精度DP（double）
 - 规格化数(SP)：阶码1~254，尾数最高位隐含为1
 - “零”（阶为全0，尾为全0）
 - ∞ （阶为全1，尾为全0）
 - NaN（阶为全1，尾为非0）
 - 非规格化数（阶为全0，尾为非0，隐藏位为0）
- 十进制数的表示：用ASCII码或BCD码表示

数据的表示和运算

- 分以下三个部分介绍
 - **第一讲：数值数据的表示**
 - 定点数的编码表示
 - 整数的表示
 - 无符号整数、带符号整数
 - 浮点数的表示
 - C语言程序的整数类型和浮点数类型
 - **第二讲：非数值数据的表示、数据的存储**
 - 逻辑值、西文字符、汉字字符
 - 数据宽度单位
 - 大端/小端、对齐存放

逻辑数据的编码表示

- 表示

- 用一位表示。例如，真：1 / 假：0
- N位二进制数可表示N个逻辑数据，或一个位串

- 运算

- 按位进行
- 如：按位与 / 按位或 / 逻辑左移 / 逻辑右移 等

- 识别

- 逻辑数据和数值数据在形式上并无差别，也是一串0/1序列，机器靠指令来识别。

- 位串（Bit String）N个二进制位可以表示N个逻辑状态

- 用来表示若干个状态位或控制位（OS中频繁使用） 逻辑vs.数值

例如，x86的标志寄存器含义如下：

				OF	DF	IF	TF	SF	ZF		AF		PF		CF
--	--	--	--	----	----	----	----	----	----	--	----	--	----	--	----

西文字符的编码表示

- 特点

- 是一种拼音文字，用有限几个字母可拼写出所有单词
- 只对有限个字母和数学符号、标点符号等辅助字符编码
- 所有字符总数不超过256个，使用7或8个二进位可表示

- 表示（常用编码为7位ASCII码）

- 十进制数字：0/1/2.../9
- 英文字母：A/B/.../Z/a/b/.../z
- 专用符号：+/-/%/*/&/.....
- 控制字符（不可打印或显示）

} 熟悉其对应的
ASCII码！ am std
code infor
interchange

- 操作

- 字符串操作，如：传送/比较 等

汉字及国际字符的编码表示

- 特点

- 汉字是表意文字，一个字就是一个方块图形。
- 汉字数量巨大，总数超过6万字，给汉字在计算机内部的表示、汉字的传输与交换、汉字的输入和输出等带来了一系列问题。

- 编码形式

- 有以下几种汉字代码：

- 输入码：对汉字用相应按键进行编码表示，用于输入
- 内码：用于在系统中进行存储、查找、传送等处理
- 字模点阵或轮廓描述：描述汉字字模点阵或轮廓，用于显示/打印

问题：西文字符有没有输入码？有没有内码？
有没有字模点阵或轮廓描述？

汉字内码

- 至少需2个字节才能表示一个汉字内码。为什么？
 - 由汉字的总数决定！
 - 可在GB2312国标码的基础上产生汉字内码
 - 为与ASCII码区别，将国标码的两个字节的第一位置“1”后得到一种汉字内码
- 例，汉字“大”在码表中位于第20行、第83列。因此区位码为0010100 1010011，国标码为00110100 01110011，即3473H。前面的34H和字符“4”的ASCII码相同，后面的73H和字符“s”的ASCII码相同，将每个字节的最高位各设为“1”后，就得到其内码：B4F3H (1011 0100 1111 0011B)，因而不会和ASCII码混淆。

汉字的字模点阵码和轮廓描述

- 为便于打印、显示汉字，汉字字形必须预先存在机内
 - 字库 (font): 所有汉字形状的描述信息集合
 - 不同字体 (如宋体、仿宋、楷体、黑体等) 对应不同字库
 - 从字库中找到字形描述信息，然后送设备输出

问题：如何知道到哪里找相应的字形信息？

编码格式、字符集 ≈ “话语权”

- 字形主要有两种描述方法：
 - 字模点阵描述（图像方式）
 - 轮廓描述（图形方式）
 - 直线向量轮廓
 - 曲线轮廓（True Type字形）

数据的基本宽度

- 比特 (bit) 是计算机中处理、存储、传输信息的最小单位
- 二进制信息的计量单位是“字节” (Byte), 也称“位组”
 - 现代计算机中, 存储器按字节编址
 - 字节是最小可寻址单位 (*addressable unit*)
 - 如果以字节为一个排列单位, 则LSB表示最低有效字节, MSB表示最高有效字节
- 除比特和字节外, 还经常使用“字” (word)作为单位
- “字” 和 “字长” 的概念不同
 - IA-32中的“字” 有多少位? 字长多少位呢?
16位 32位
 - DWORD : 32位
 - QWORD: 64位

数据的基本宽度

- “字”和“字长”的概念不同 ==> 对齐、对齐、对齐

- “字长”指数据通路的宽度。

(数据通路指CPU内部数据流经的路径以及路径上的部件，主要是CPU内部进行数据运算、存储和传送的部件，这些部件的宽度基本上要一致，才能相互匹配。因此，“字长”等于CPU内部总线的宽度、运算器的位数、通用寄存器的宽度等。)

- “字”表示被处理信息的单位，用来度量数据类型的宽度。

- 字和字长的宽度可以一样，也可不同。

例如，x86体系结构定义“字”的宽度为16位，但从386开始字长就是32位了。

数据量的度量单位

- 存储二进制信息时的度量单位要比字节或字大得多
- 容量经常使用的单位有：

- “千字节” (KB), $1\text{KB}=2^{10}\text{字节}=1024\text{B}$
- “兆字节” (MB), $1\text{MB}=2^{20}\text{字节}=1024\text{KB}$
- “千兆字节” (GB), $1\text{GB}=2^{30}\text{字节}=1024\text{MB}$

买U盘、硬盘时的“小偏差”

- “兆比特/秒” (Mb/s), $1\text{Mbps}=10^6\text{ b/s}=1000\text{ kbps}$
- “千兆比特/秒” (Gb/s), $1\text{Gbps}=10^9\text{ b/s}=1000\text{ Mbps}$
- “兆兆比特/秒” (Tb/s), $1\text{Tbps}=10^{12}\text{ b/s}=1000\text{ Gbps}$

如果把b换成B，则表示字节而不是比特（位）

例如，10MBps表示 10兆字节/秒

数据量的度量单位

文件系统的元数据（用于管理文件和目录的信息）

超级块 (Superblock)

- 存储文件系统的全局信息，如大小、块大小、空闲块数等
- 通常占用少量空间（几KB）

inode表 (Inode Table)

- 存储文件和目录的元数据（如权限、所有者、大小、时间戳等）
- 占用空间与文件数量和文件系统设计有关，通常占总容量的1%-2%

目录结构 (Directory Structure)

- 存储目录项（文件名和对应的inode号）
- 占用空间与目录数量和文件数量相关

位图 (Bitmaps)

- 用于跟踪空闲块和inode的使用情况
- 占用空间较小，通常为几KB到几十KB

日志 (Journal, 仅适用于有日志的文件系统如NTFS、ext3/ext4)

- 用于记录文件系统的操作，确保数据一致性
- 占用空间通常为几十MB

保留空间 (Reserved Space)

- 某些文件系统会保留部分空间供系统使用（如根用户或系统恢复）
- 占用空间通常为总容量的5%左右

示例：对于一个32GB的U盘

- FAT32：占用约50MB-100MB
- exFAT：占用约100MB-200MB
- NTFS：占用约200MB-500MB

程序中数据类型的宽度

- 高级语言支持多种类型、多种长度的数据

- 例如，C语言中char类型的宽度为1个字节，可表示一个字符（非数值数据），也可表示一个8位的整数（数值数据）

- 不同机器上表示的同一种类型的数据可能宽度不同

- 必须确定相应的机器级数据表示方式和相应的处理指令

C语言中数值数据类型的宽度 (单位：字节)

C声明	典型32位机器	Compaq Alpha机器
char	1	1
short int	2	2
int	4	4
long int	4	8
char*	4	8
float	4	4
double	8	8

从表中看出：同类型数据并不是所有机器都采用相同的宽度，分配的字节数随机器字长和编译器的不同而不同。

Compaq Alpha是一个针对高端应用的64位机器，即字长为64位

数据的存储和排列顺序

- 80年代开始，几乎所有机器都用**字节编址** $65535=2^{16}-1$
- ISA设计时要考虑的两个问题： $[-65535]_{\text{补}} = \text{FFFF0001H}$
 - 如何根据一个**字节地址**取到一个32位的字？ - **字的存放问题**
 - 一个字能否存放在任何字节边界？ - **字的边界对齐问题**

例如，若 $\text{int } i = -65535$ ，存放在内存100号单元（即占100# ~ 103#），则用“取数”指令访问100号单元取出 i 时，必须清楚 i 的4个字节是如何存放的。

Word:	FF	FF	00	01	little endian word 100#
	103	102	101	100	
	msb			lsb	
	100	101	102	103	big endian word 100#

大端方式（**Big Endian**）：**MSB**所在的地址是数的地址

e.g. IBM 360/370, Motorola 68k, MIPS, Sparc, HP PA

小端方式（**Little Endian**）：**LSB**所在的地址是数的地址

e.g. Intel 80x86, DEC VAX

有些机器两种方式都支持，可通过特定控制位来设定采用哪种方式。

BIG Endian versus Little Endian

Ex3: Memory layout of a instruction located in 1000

假定小端机器中指令: `mov AX, 0x12345(BX)`

其中操作码mov为40H, 寄存器AX和BX的编号分别为0001B和0010B, 立即数占32位, 则存放顺序为:



若在大端机器上, 则存放顺序如何?



00	1005	45
01	1004	23
23	1003	01
45	1002	00
12	1001	12
40	1000	40

地址

只需要考虑指令中立即数的顺序!

Byte Swap Problem (字节交换问题)

78	3
56	2
34	1
12	0

Big Endian

↑
increasing
byte
address

12	3
34	2
56	1
78	0

Little Endian

上述存放在0号单元的数据（字）是什么？ **12345678H?** **78563412H?**

存放方式不同的机器间程序移植或数据通信时，会发生什么问题？

- ◆ 每个系统内部是一致的，但在系统间通信时可能会发生问题！
- ◆ 因为顺序不同，需要进行顺序转换

音、视频和图像等文件格式或处理程序都涉及到**字节顺序问题**

ex. Little endian: GIF, PC Paintbrush, Microsoft RTF, etc

Big endian: Adobe Photoshop, JPEG, MacPaint, etc

硬件电路逻辑与人类阅读习惯之间的妥协产物！

小端算/硬件 x86文档；大端读；Leaky Abstraction；博弈；Union/char*强转int*

Alignment(对齐) 讲“排场”？

Alignment: 要求数据的地址是相应的边界地址

- 目前机器字长一般为32位或64位，而存储器地址按字节编址
- 指令系统支持对字节、半字、字及双字的运算，也有位处理指令
- 各种不同长度的数据存放时，有两种处理方式：

- 按边界对齐 （假定存储字的宽度为32位，按字节编址）

- 字地址：4的倍数（低两位为0）

- 半字地址：2的倍数（低位为0）

- 字节地址：任意

每4个字节可同时读写

- 不按边界对齐

坏处：可能会增加访存次数！存储器的组织还能想起来不？

Alignment(对齐)

如: `int i, short k, double x, char c, short j,.....`

存储器按字节

编址

每次只能读写

某个字地址开

始的4个单元中

连续的1个、2

个、3个或4个

字节

虽节省了空间，
但增加了访存次
数！

需要权衡，有时
浪费一点存储空
间没有关系！而
有时空间受限！

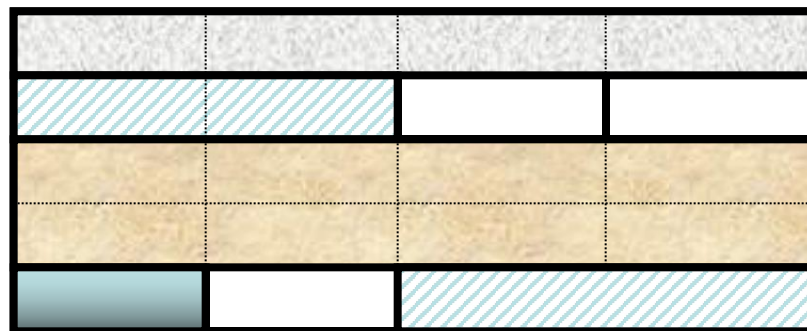
按边界对齐

x: 2个周期

j: 1个周期

00
04
08
12
16

0 字节 1 字节 2 字节 3 字节



则: `&i=0; &k=4; &x=8; &c=16; &j=18;.....`

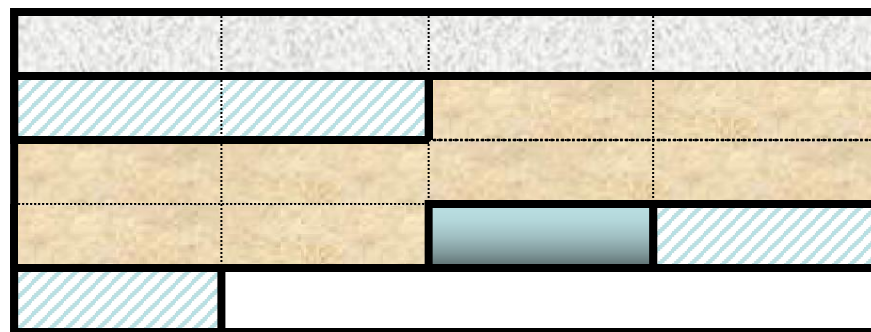
字节0 字节1 字节2 字节3

边界不对齐

x: 3个周期

j: 2个周期

00
04
08
12
16



则: `&i=0; &k=4; &x=6; &c=14; &j=15;.....`

Alignment(对齐) 举例

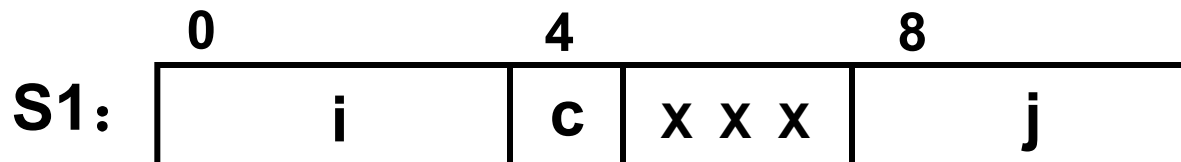
例如，考虑下列两个结构声明：

```
struct S1 {  
    int    i;  
    char   c;  
    int    j;  
};
```

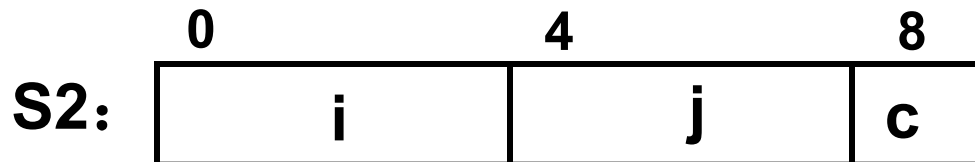
```
struct S2 {  
    int    i;  
    int    j;  
    char   c;  
};
```

在不要求对齐的情况下，哪种结构声明更好？

S2比S1好

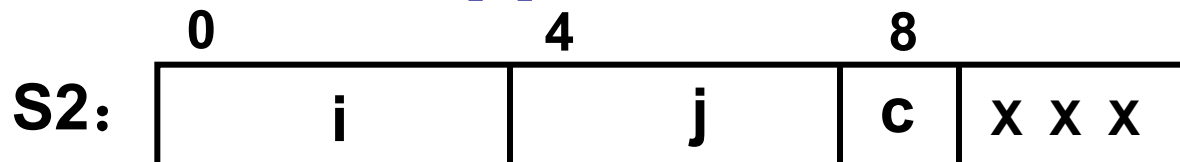


需要12个字节



只需要9个字节

对于 “struct S2 d[4]”，只分配9个字节能否满足对齐要求？ **不能！**



也需要12个字节

第二讲小结

- 非数值数据的表示

- 逻辑数据用来表示真/假或N位位串，按位运算
- 西文字符：用ASCII码表示
- 汉字：汉字输入码、汉字内码、汉字字模码

- 数据的宽度

- 位、字节、字（不一定等于字长）
- k / K / M / G / T / P / E / Z / Y 有不同的含义

- 数据的存储排列

- 数据的地址：连续若干单元中最小的地址，即：从小地址开始存放数据
 - 问题：若一个short型数据si存放在单元0x08000100和0x08000101中，那么si的地址是什么？
- 大端方式：用MSB存放的地址表示数据的地址
- 小端方式：用LSB存放的地址表示数据的地址
- 按边界对齐可减少访存次数

数据的表示和运算

- 分以下三个部分介绍（续）

- **第三讲：数据的运算**

- 按位运算和逻辑运算
 - 移位运算
 - 位扩展和位截断运算
 - 无符号和带符号整数的加减运算
 - 无符号和带符号整数的乘除运算
 - 变量与常数之间的乘除运算
 - 浮点数的加减乘除运算

围绕C语言中的
运算，解释其
在底层机器级
的实现方法

从高级语言程序中的表达式出发，用机器数在具体电路中的
执行过程，来解释表达式的执行结果

数据的运算

- 高级语言程序中涉及的运算（以C语言为例）

- 整数算术运算、浮点数算术运算
- 按位、逻辑、移位、位扩展和位截断

- 指令集中涉及到的运算

- 涉及到的定点数运算

- 算术运算

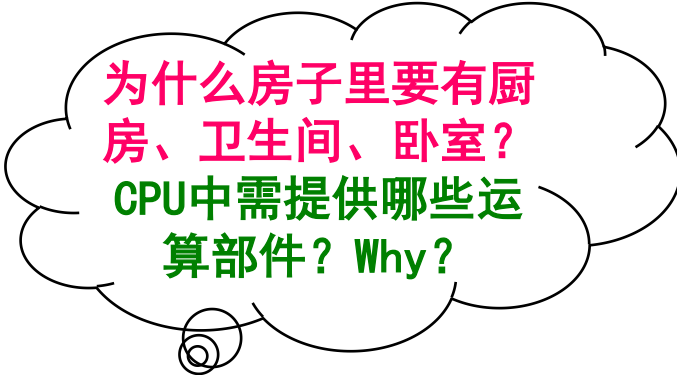
- 带符号整数运算：取负 / 符号扩展 / 加 / 减 / 乘 / 除 / 算术移位
- 无符号整数运算：0扩展 / 加 / 减 / 乘 / 除

- 逻辑运算

- 逻辑操作：与 / 或 / 非 / ...
- 移位操作：逻辑左移 / 逻辑右移

- 涉及到的浮点数运算：加、减、乘、除

- 基本运算部件ALU（Arithmetic and Logic Unit）的设计



为什么房子里要有厨房、卫生间、卧室？
CPU中需提供哪些运算部件？Why？

人的需要！ 程序的需要！

我们从程序对运算的需求开始分析。

作业：浮点数有没有移位操作和扩展操作？为什么？fscale

C语言程序中涉及的运算

- 算术运算（最基本的运算）
 - 无符号数、带符号整数、浮点数的+、-、*、/ 运算等
 - 按位运算
 - 用途
 - 对位串实现“掩码”（mask）操作或相应的其他处理
（主要用于对多媒体数据或状态/控制信息/神经网络进行处理）
 - 操作
 - 按位或：“|”
 - 按位与：“&”
 - 按位取反：“~”
 - 按位异或：“^”
- 问题：如何从16位采样数据y中提取高位字节，并使低字节为0？
可用“&”实现“掩码”操作：y & 0xFF00
例如，当y=0x2C0B时，得到结果为：0x2C00

C语言程序中涉及的运算

- 逻辑运算

- 用途

- 用于关系表达式的运算

- 例如，if (x>y and i<100) then中的 “and”运算

- 操作

- “||”表示 “OR”运算
 - “&&”表示 “AND”运算

- Ex: if ((x>y) && (i<100)) then

- “!”表示 “NOT”运算

- 与按位运算的差别

- 符号表示不同：& ~ && ; | ~ ||;
 - 运算过程不同：按位 ~ 整体
 - 结果类型不同：位串 ~ 逻辑值

C语言程序中涉及的运算

如何从16位数据y中提取高位字节？

某字长为8的机器中，x、y和z都是8位带符号整数，已知 $x=-81$ ，则 $y=x/2=?$ $z=2x=?$

- 移位运算

- 用途

- 提取部分信息
 - 扩大或缩小数值的2、4、8...倍

($y>>8$) 送8位寄存器

- 操作

- 左移:: $x<<k$; 右移: $x>>k$
 - 不区分是逻辑移位还是算术移位，由x的类型确定
 - 无符号数：逻辑左移、逻辑右移

移位! $y=-40?$ $z=-162?$

移位比除法快的经典例子

高（低）位移出，低（高）位补0，可能溢出！

问题：何时可能发生溢出？如何判断溢出？

若高位移出的是1，则左移时发生溢出

- 带符号整数：算术左移、算术右移

左移：高位移出，低位补0。可能溢出！

溢出判断：若移出的位不等于新的符号位，则溢出。

右移：低位移出，高位补符，可能发生有效数据丢失。

C语言程序中涉及的运算

• 位扩展和位截断运算

– 用途

- 类型转换时可能需要数据扩展或截断

– 操作

- 没有专门操作运算符，根据类型转换前后数据长短确定是扩展还是截断
- 扩展：短转长
 - 无符号数：0扩展，前面补0
 - 带符号整数：符号扩展，前面补符
- 截断：长转短
 - 强行将高位丢弃，故可能发生“溢出”

例1（扩展操作）：在大端机上输出si, usi, i, ui的十进制和十六进制值是什么？

```
short si = -32768;
unsigned short usi = si;
int i = si;
unsigned ui = usi;
```

```
si = -32768    80 00
usi = 32768    80 00
i = -32768     FF FF 80 00
ui = 32768     00 00 80 00
```

例2（截断操作）：i和j是否相等？

```
int i = 32768;
short si = (short) i;
int j = si;
```

不相等！

```
i = 32768    00 00 80 00
si = -32768   80 00
j = -32768   FF FF 80 00
```

原因：对i截断时发生了“溢出”，即：32768截断为16位数时，因其超出16位能表示的最大值，故无法截断为正确的16位数

科研：模型压缩量化

如何实现高级语言源程序中的运算？

- 总结：C语言程序中的基本数据类型及其基本运算类型
 - 基本数据类型
 - 无符号数、带符号整数、浮点数、位串、字符（串）
 - 基本运算类型
 - 算术、按位、逻辑、移位、扩展和截断、匹配
- 计算机如何实现高级语言程序中的运算？
 - 将各类表达式编译（转换）为指令序列
 - 计算机直接执行指令来完成运算

例：C语言赋值语句 “ $f = (g+h) - (i+j);$ ” 中变量 i 、 j 、 f 、 g 、 h 由编译器分别分配给MIPS寄存器 $\$t0 \sim \$t4$ 。寄存器 $\$t0 \sim \$t7$ 的编号对应8~15，上述程序段对应的MIPS机器代码和汇编表示（#后为注释）如下：

000000 01011 01100 01101 00000 100000 add $\$t5, \$t3, \$t4$ # $g+h$

000000 01000 01001 01110 00000 100000 add $\$t6, \$t0, \$t1$ # $i+j$

000000 01101 01110 01010 00000 100010 sub $\$t2, \$t5, \$t6$ # $f=(g+h)-(i+j)$

需要提供哪些运算类指令才能支持高级语言需求呢？

整数加减运算及其部件

- 补码加减运算公式

- $[A+B]_{\text{补}} = [A]_{\text{补}} + [B]_{\text{补}} \pmod{2^n}$

- $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^n}$

问题：Adder中执行的
是什么运算？

相当于无符号数加！

- 补码加减运算要点和运算部件

- 加、减法运算统一采用加法来处理

- 符号位(最高有效位MSB)和数值位一起参与运算

- 直接用Adder实现两个数的加运算（模运算系统）

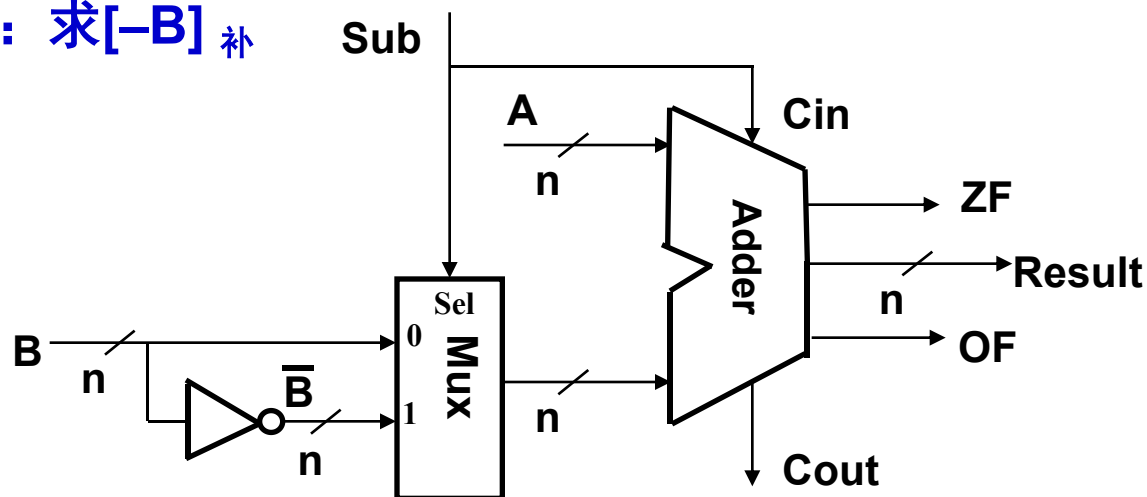
问题：模是多少？运算结果高位丢弃，保留低 n 位，相当于取模 2^n

- 实现减法的主要工作在于：求 $[-B]_{\text{补}}$

问题：如何求 $[-B]_{\text{补}}$ ？

$$[-B]_{\text{补}} = \overline{B} + 1$$

当Sub为1时，做减法
当Sub为0时，做加法



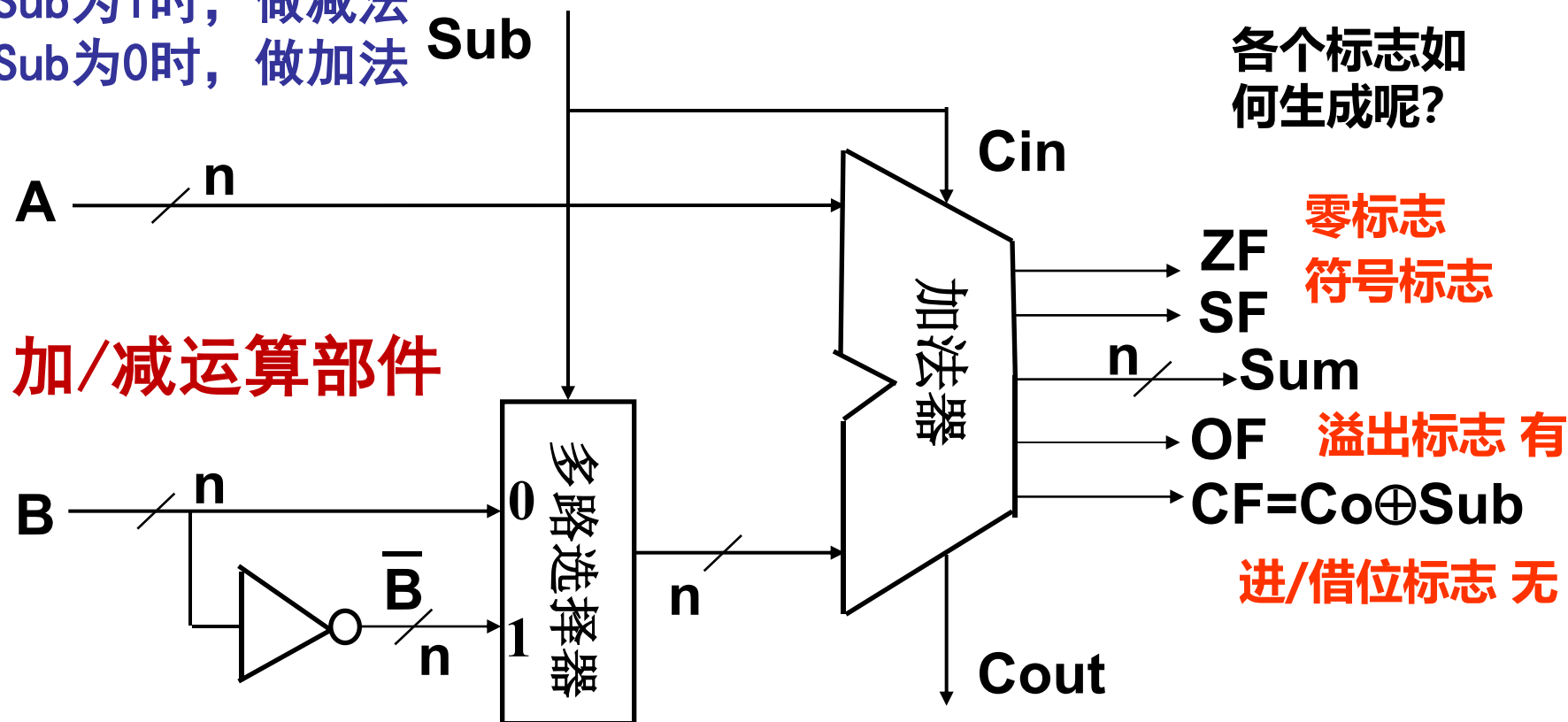
计算机中的算盘长啥样？

重要认识1：计算机中所有运算都基于加法器实现！

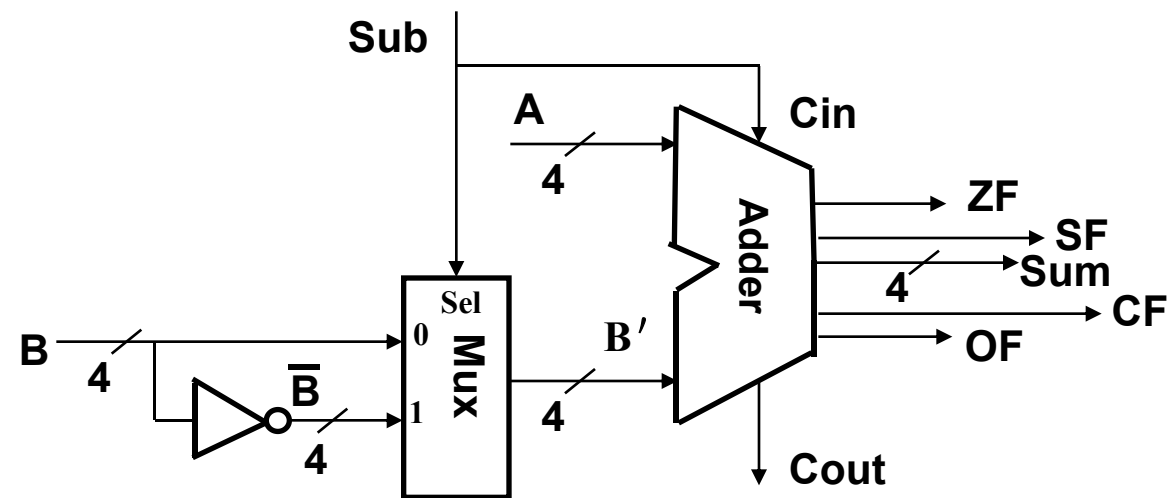
重要认识2：加法器不知道所运算的是带符号数还是无符号数。

重要认识3：加法器不判定对错，总是取低n位作为结果，并生成标志信息。

当Sub为1时，做减法
当Sub为0时，做加法



条件标志位（条件码CC）



计算机中的算盘。

所有其他运算都基于**整数加/减运算器**来实现。

整数加/减运算部件

问题：OF=? ZF=? SF=? CF=?

OF：若A与B'同号但与Sum不同号，则1；否则0。 **SF**：sum符号

ZF：如Sum为0，则1，否则0。 **CF**： $Cout \oplus sub$

• 零标志**ZF**//if (a == 0)、溢出标志**OF**、进/借位标志**CF**、符号标志**SF**//判断正负称为条件标志。

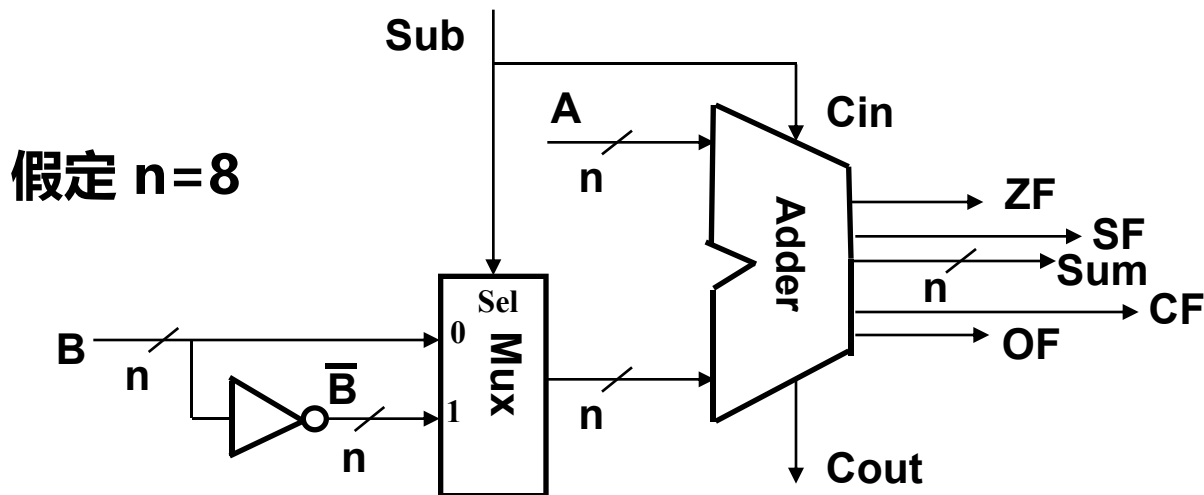
• **条件标志（Flag）**在运算电路中产生，被记录到专门的寄存器中，以便在分支指令中被用来作为条件。

• 存放标志的寄存器通常称为**程序/状态字寄存器**或**标志寄存器**。每个标志对应标志寄存器中的一个标志位。 如，IA-32中的EFLAGS寄存器

整数加减运算及其部件

```
unsigned int x=134;  
unsigned int y=246;  
int m=x;  
int n=y;  
unsigned int z1=x-y;  
unsigned int z2=x+y;  
int k1=m-n;  
int k2=m+n;
```

无符号数加减运算也用该部件执行



x和m的机器数一样: 1000 0110, y和n的机器数一样: 1111 0110

z1和k1的机器数一样: 1001 0000, **CF=1**, **OF=0**, **SF=1**

z1的值为144 (**=134-246+256**, **x-y<0**) , **k1**的值为-112。

z2和k2的机器数一样: 0111 1100, **CF=1**, **OF=1**, **SF=0**

z2的值为124 (**=134+246-256**, **x+y>256**)

k2的值为124 (**=134+246-256**, **x+y>128**, 即正溢出)

结果说明什么?

仅k1的值正确!

整数的乘运算(Shift & Add)

- 高级语言中两个n位整数相乘得到的结果通常也是一个n位整数，也即结果只取2n位乘积中的低n位。
- 例如，在C语言中，参加运算的两个操作数的类型和结果的类型必须一致，如果不一致则会先转换为一致的数据类型再进行计算。

$x^2 \geq 0$? 对于带符号整数，不一定!

例如，当n=4时， $5^2 = -7 < 0$!

结论：假定两个n位无符号整数 x_u 和 y_u 对应的机器数为 X_u 和 Y_u ，

$p_u = x_u \times y_u$ ， p_u 为n位无符号整数且对应的机器数为 P_u ；

两个n位带符号整数 x_s 和 y_s 对应的机器数为 X_s 和 Y_s ， $p_s = x_s \times y_s$ ， p_s 为n位带符号整数且对应的机器数为 P_s 。

若 $X_u = X_s$ 且 $Y_u = Y_s$ ，则 $P_u = P_s$ 。

说明可用无符号乘来实现带符号乘，但高n位无法得到，不能判断溢出。

整数的乘运算

- $X \times Y$ 的高n位可以用来判断溢出，规则如下：
 - 无符号：若高n位全0，则不溢出，否则溢出
 - 带符号：若高n位全0或全1且等于低n位的最高位，则不溢出。

运算	x	X	y	Y	$x \times y$	$X \times Y$	p	P	溢出否
无符号乘	6	0110	10	1010	60	0011 1100	12	1100	溢出
带符号乘	6	0110	-6	1010	-36	1101 1100	-4	1100	溢出
无符号乘	8	1000	2	0010	16	0001 0000	0	0000	溢出
带符号乘	-8	1000	2	0010	-16	1111 0000	0	0000	溢出
无符号乘	13	1101	14	1110	182	1011 0110	6	0110	溢出
带符号乘	-3	1101	-2	1110	6	<u>0000 0110</u>	6	0110	不溢出
无符号乘	2	0010	12	1100	24	0001 1000	8	1000	溢出
带符号乘	2	0010	-4	1100	-8	<u>1111 1000</u>	-8	1000	不溢出

整数的乘运算

- 通常硬件不判断乘法是否溢出，而是保留 $2n$ 位乘积
- 分无符号数乘指令和带符号整数乘指令
- 乘法指令无法得到溢出标志或无法自动判断是否溢出，如果程序本身不采用防止溢出的措施，而且编译器也不生成相应的用于溢出处理的代码的话，就会发生一些由于整数溢出而带来的问题。
- 乘法指令的操作数长度为 n ,而乘积长度为 $2n$ ，例如：
 - IA-32中，若指令只给出一个操作数SRC，则另一个源操作数隐含在累加器AL/AX/EAX中，将SRC和累加器内容相乘，结果存放在AX（16位时）或DX-AX（32位时）或EDX-EAX（64位时）中。
 - 在MIPS处理器中，带符号整数乘法指令mult会将两个32位带符号整数相乘得到的64位乘积置于两个32位内部寄存器Hi和Lo中，因此，可以根据Hi寄存器中的每一位是否等于Lo寄存器中的第一位来进行溢出判断。

整数溢出漏洞

- 说明以下程序存在什么漏洞，引起该漏洞的原因是什么。

/* 复制数组到堆中，count为数组元素个数 */

```
int copy_array(int *array, int count) {
```

```
    int i;
```

```
    /* 在堆区申请一块内存 */
```

```
    int *myarray = (int *) malloc(count*sizeof(int));
```

```
    if (myarray == NULL)
```

```
        return -1;
```

```
    for (i = 0; i < count; i++)
```

```
        myarray[i] = array[i];
```

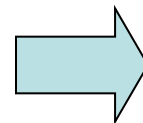
```
    return count;
```

```
}
```

2002年，Sun Microsystems公司的RPC XDR库带的xdr_array函数发生整数溢出漏洞，攻击者可利用该漏洞从远程或本地获取root权限。

攻击者可构造特殊参数来触发整数溢出，以一段**预设信息覆盖一个已分配的堆缓冲区**，造成远程服务崩溃或者改变内存数据并执行任意代码。

当参数count很大时，则count*sizeof(int)会溢出。
如count= $2^{30} + 1$ 时，
count*sizeof(int)=4。



堆 (heap) 中大量数据被破坏!

整数的除运算

- 对于n位整数除以n位整数，除了 $-2^{n-1}/-1 = 2^{n-1}$ 会发生溢出外，其余情况都不会发生溢出。Why?

因为商的绝对值不可能比被除数的绝对值更大，因而不会发生溢出，也就不会像整数乘法运算那样发生漏洞。

- 在不能整除时需要进行舍入，通常按照朝0方向舍入，即正数商取比自身小的最接近整数，负数商取比自身大的最接近整数。

例如， $7/3=?$ ， $-7/3=?$

$7/3=2$ ， $-7/3=-2$

- 除数不能为0，否则会发生“异常”，需要调出操作系统中的异常处理程序来处理。整除0的结果可以用什么机器数表示？

整除0的结果无法用一个机器数表示！

整数的除运算

代码段一：

```
int a = 0x80000000;
```

```
int b = a / -1;
```

```
printf("%d\n", b);
```

运行结果为-2147483648

用objdump看代码段一
的反汇编代码, 得知除以
-1 被优化成取负指令
neg, 故未发生除法溢出

代码段二：

```
int a = 0x80000000;
```

```
int b = -1;
```

```
int c = a / b;
```

```
printf("%d\n", c);
```

运行结果为“Floating point exception”，显然CPU检测到了异常

为什么显示是“浮点异常”呢？

为什么两者结果不同！

变量与常数之间的乘运算

- 整数乘法运算比移位和加法等运算所用时间长得多，通常一次乘法运算需要10个左右时钟周期，而一次移位、加法和减法等运算只要一个或更少的时钟周期，因此，编译器在处理变量与常数相乘时，往往以移位、加法和减法的组合运算来代替乘法运算。

例如，对于表达式 $x * 20$ ，编译器可以利用 $20 = 16 + 4 = 2^4 + 2^2$ ，将 $x * 20$ 转换为 $(x \ll 4) + (x \ll 2)$ ，这样，一次乘法转换成了两次移位和一次加法。

- 不管是无符号数还是带符号整数的乘法，即使乘积溢出时，利用移位和加减运算组合的方式得到的结果都是和采用直接相乘的结果是一样的。

变量与常数之间的除运算

- 对于整数除法运算，由于计算机中除法运算比较复杂，而且不能用流水线方式实现，所以一次除法运算大致需要30个或更多个时钟周期。为了缩短除法运算的时间，**编译器在处理一个变量与一个2的幂次形式的整数相除时，常采用右移运算来实现。**
- 无符号数除法采用逻辑右移方式，带符号整数采用算术右移方式。
- 结果一定取整数，能整除时，直接右移得到结果。

例如， $12/4=3$: $0000\ 1100 \gg 2 = 0000\ 0011$

$-12/4=-3$: $1111\ 0100 \gg 2 = 1111\ 1101$

- 不能整除时，其商采用朝零方向舍入的方式，也就是截断方式，即：移出的低位数直接丢弃。**带符号负整数则不对！（需加偏移量 (2^k-1) ，然后再右移 k 位，低位截断）**

无符号整数： $14/4=3$: $0000\ 1110 \gg 2 = 0000\ 0011$

带符号整数： $-14/4=-3$: $1111\ 0010 \gg 2 = 1111\ 1100 = -4 \neq -3$

纠偏: $k=2$, 故 $(-14+2^2-1)/4=-3$: $1111\ 0101 \gg 2 = 1111\ 1101 = -3$

浮点数运算及结果 (对阶右移 舍入 精度掉点)

设两个规格化浮点数分别为 $A=M_a \cdot 2^{E_a}$ $B=M_b \cdot 2^{E_b}$,则:

$$A \pm B = (M_a \pm M_b \cdot 2^{-(E_a-E_b)}) \cdot 2^{E_a} \quad (\text{假设 } E_a \geq E_b) \quad 1.5+1.5=?$$

$$A * B = (M_a * M_b) \cdot 2^{E_a+E_b} \quad 1.5-1.0=?$$

$$A / B = (M_a / M_b) \cdot 2^{E_a-E_b}$$

上述运算结果可能出现以下几种情况:

SP最大指数为多少? 127!

阶码上溢: 一个正指数超过了最大允许值 $\Rightarrow +\infty/-\infty/\text{溢出}$

阶码下溢: 一个负指数超过了最小允许值 $\Rightarrow +0/-0$ **SP最小指数为多少?**

尾数溢出: 最高有效位有进位 $\Rightarrow$ 右规 **尾数溢出, 结果不一定溢出** **-126!**

非规格化尾数: 数值部分高位为0 $\Rightarrow$ 左规

右规或对阶时, 右段有效位丢失 $\Rightarrow$ 尾数舍入 **运算过程中添加保护位**

IEEE建议实现时为每种异常情况提供一个**自陷允许位**。若某异常对应的位为1, 则发生相应异常时, 就调用一个特定的异常处理程序执行。

IEEE754标准规定的五种异常情况

① 无效运算（无意义）

- 运算时有一个数是非有限数，如：

加 / 减 ∞ 、 $0 \times \infty$ 、 ∞/∞ 等

- 结果无效，如：

源操作数是NaN、 $0/0$ 、 $x \text{ REM } 0$ 、 $\infty \text{ REM } y$ 等

② 除以0（即：无穷大）

③ 数太大（阶码上溢）：对于SP，指阶码 $E > 1111\ 1110$ （指数大于127）

④ 数太小（阶码下溢）：对于SP，指阶码 $E < 0000\ 0001$ （指数小于-126）

⑤ 结果不精确（舍入时引起），例如 $1/3$ ， $1/10$ 等不能精确表示成浮点数

上述情况硬件可以捕捉到，因此这些异常可设定让硬件处理，也可设定让软件处理。让硬件处理时，称为硬件陷阱。

整除0和浮点数除0的问题

```
#include <conio.h>
#include <stdio.h>
int main()
{
    int a=1, b=0;
    printf( "Division by zero:%d\n ", a/b);
    getchar();
    return 0;
}
```

作业: 为什么整数除0会发生异常而浮点数除0不会?

```
int main()
{
    double x=1.0, y=-1.0, z=0.0;
    printf( "division by zero:%f %f\n ", x/z, y/z);
    getchar();
    return 0;
}
```

问题一:为什么整除int型会产生错误? 是什么错误?

二:用double型的时候结果为1. #INF00和-1. #INF00, 作何解释???

浮点数加/减运算

- 十进制科学计数法的加法例子

$$0.123 \times 10^5 + 0.560 \times 10^2$$

其计算过程为：

$$\begin{aligned} 0.123 \times 10^5 + 0.560 \times 10^2 &= 0.123 \times 10^5 + 0.000560 \times 10^5 \\ &= (0.123 + 0.00056) \times 10^5 = 0.12356 \times 10^5 \\ &= 0.124 \times 10^5 \end{aligned}$$

进行尾数加减运算前，必须“对阶”！最后还要考虑舍入
计算机内部的二进制运算也一样！

- “对阶”操作：目的是使两数阶码相等
 - 小阶向大阶看齐，阶小的那个数的尾数右移，右移位数等于两个阶码差的绝对值
 - IEEE 754尾数右移时，要将隐含的“1”移到小数部分，高位补0，移出的低位保留到特定的“附加位”上

浮点数加减法基本要点

(假定: X_m 、 Y_m 分别是X和Y的尾数, X_e 和 Y_e 分别是X和Y的阶码)

- (1) 求阶差: $\Delta e = Y_e - X_e$ (若 $Y_e > X_e$, 则结果的阶码为 Y_e)
- (2) 对阶: 将 X_m 右移 Δe 位, 尾数变为 $X_m * 2^{X_e - Y_e}$ (保留右移部分: 附加位)
- (3) 尾数加减: $X_m * 2^{X_e - Y_e} \pm Y_m$
- (4) 规格化:

当尾数高位为0, 则需左规: 尾数左移一次, 阶码减1, 直到MSB为1

每次阶码减1后要判断阶码是否下溢 (比最小可表示的阶码还要小)

当尾数最高位有进位, 需右规: 尾数右移一次, 阶码加1, 直到MSB为1

每次阶码加1后要判断阶码是否上溢 (比最大可表示的阶码还要大)

阶码溢出异常处理: 阶码上溢, 则结果溢出; 阶码下溢, 则结果为0

- (5) 如果尾数比规定位数长, 则需考虑舍入 (有多种舍入方式)
- (6) 若运算结果尾数是0, 则需要将阶码也置0。为什么?

尾数为0说明结果应该为0 (阶码和尾数为全0)。

浮点数加法运算举例

Example: 用二进制形式计算 $0.5 + (-0.4375) = ?$

解: $0.5 = 1.000 \times 2^{-1}$, $-0.4375 = -1.110 \times 2^{-2}$

对 阶: $-1.110 \times 2^{-2} \rightarrow -0.111 \times 2^{-1}$

加 减: $1.000 \times 2^{-1} + (-0.111 \times 2^{-1}) = 0.001 \times 2^{-1}$

左 规: $0.001 \times 2^{-1} \rightarrow 1.000 \times 2^{-4}$

判溢出: 无

结果为: $1.000 \times 2^{-4} = 0.0001000 = 1/16 = 0.0625$

问题: 为何IEEE 754 加减运算右规时最多只需一次?

因为即使是两个最大的尾数相加, 得到的和的尾数也不会达到4, 故尾数的整数部分最多有两位, 保留一个隐含的“1”后, 最多只有一位被右移到小数部分。

Rounding Digits(舍入位)

举例：十进制数，最终有效位数为 3，采用两位附加位（**G**、**R**）。

问题：若没有舍入位，采用就近舍入到偶数，则结果是什么？

结果为2.36！精度没有2.37高！

$$\begin{array}{r} 2.34\mathbf{00} * 10^2 \\ 0.02\mathbf{53} * 10^2 \\ \hline 2.36\mathbf{53} * 10^2 \end{array}$$

IEEE Standard: four rounding modes (用图说明)

round to nearest (**default**)

round towards plus infinity (always round up)

round towards minus infinity (always round down)

round towards 0

round to nearest: 简称为就近舍入到偶数

round digit < 1/2 then truncate (截去)

> 1/2 then round up (add 1 to ULP)

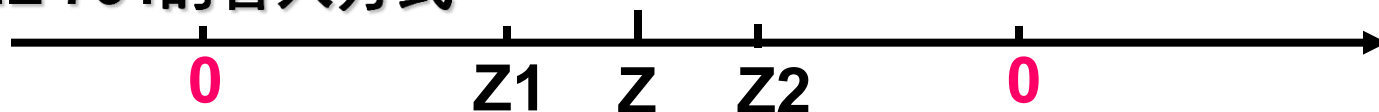
= 1/2 then round to nearest even digit

可以证明默认方式得到的平均误差最小。

注：ULP=units in
the last place.

IEEE 754的舍入方式的规则

IEEE 754的舍入方式



(Z1和Z2分别是结果Z的最近的可表示的左、右两个数)

(1) 就近舍入：舍入为最近可表示的数

非中间值：0舍1入；

中间值：强迫结果为偶数-慢

例如：附加位为

01：舍

11：入

10：(强迫结果为偶数)

例：1.1101**11** → 1.1110； 1.1101**01** → 1.1101；
1.1101**10** → 1.1110； 1.1111**10** → 10.0000；

(2) 朝 $+\infty$ 方向舍入：舍入为Z2(正向舍入)

(3) 朝 $-\infty$ 方向舍入：舍入为Z1(负向舍入)

(4) 朝0方向舍入：截去。正数：取Z1；负数：取Z2

浮点数舍入举例

例：将同一实数分别赋值给单精度和双精度类型变量，然后打印输出。

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    float a;
```

```
    double b;
```

```
    a = 123456.789e4;
```

```
    b = 123456.789e4;
```

```
    printf( "%f/n%f/n" ,a,b);
```

```
}
```

运行结果如下：

```
1234567936.000000
```

```
1234567890.000000
```

为什么float情况下输出的结果
会比原来的大？这到底有没有
根本性原因还是随机发生的？
为什么会出现这样的情况？

问题：为什么同一个实数赋值给float型变量
和double型变量，输出结果会有所不同呢？

C语言中的浮点数类型

- C语言中有**float**和**double**类型，分别对应IEEE 754单精度浮点数格式和双精度浮点数格式
- **long double**类型的长度和格式随编译器和处理器类型的不同而有所不同，IA-32中是**80位扩展精度**格式
- 从int转换为float时，不会发生溢出，但可能有数据被舍入
- 从int或 float转换为double时，因为double的有效位数更多，故能保留精确值
- 从double转换为float和int时，可能发生溢出，此外，由于有效位数变少，故可能被舍入
- 从float 或double转换为int时，因为int没有小数部分，所以数据可能会向0方向被截断

Questions about IEEE 754

- What's the range of representable values?

The largest number for single: $+1.11...1 \times 2^{127}$

约 $+3.4 \times 10^{38}$

How about double?

约 $+1.8 \times 10^{308}$

- What about following type converting: not always true!

```
if ( i == (int) ((float) i) ) {  
    printf ("true");  
}
```

How about
double?

True!

```
if ( f == (float) ((int) f) ) {  
    printf ("true");  
}
```

How about
double?

Not always
true!

- How about FP **add associative**? FALSE!

$x = -1.5 \times 10^{38}$, $y = 1.5 \times 10^{38}$, $z = 1.0$

$(x+y)+z = (-1.5 \times 10^{38} + 1.5 \times 10^{38}) + 1.0 = 1.0$

$x+(y+z) = -1.5 \times 10^{38} + (1.5 \times 10^{38} + 1.0) = 0.0$

浮点运算举例

- 1996年6月4日，Ariana 5火箭初次航行，在发射仅仅37秒钟后，偏离了飞行路线，然后解体爆炸，火箭上载有价值5亿美元的通信卫星。
- 原因是**在将一个64位浮点数转换为16位带符号整数时，产生了溢出异常**。溢出的值是火箭的水平速率，这比原来的Ariana 4火箭所能达到的速率高出了5倍。在设计Ariana 4火箭软件时，设计者确认水平速率决不会超出一个16位的整数，但在设计Ariana 5时，他们没有重新检查这部分，而是直接使用了原来的设计。
- 在不同数据类型之间转换时，往往隐藏着一些不容易被察觉的错误，这种错误有时会带来重大损失，因此，编程时要非常小心。

浮点运算举例

- 1991年2月25日，海湾战争中，美国在沙特阿拉伯达摩地区设置的爱国者导弹拦截伊拉克的飞毛腿导弹失败，致使飞毛腿导弹击中了沙特阿拉伯载赫蓝的一个美军军营，杀死了美国陆军第十四军需分队的28名士兵。其原因是由于爱国者导弹系统时钟内的一个软件错误造成的，引起这个软件错误的原因是**浮点数的精度问题**。
- 爱国者导弹系统中有一个内置时钟，用计数器实现，每隔0.1秒计数一次。程序用0.1的一个**24位定点二进制小数x**来乘以计数值作为以秒为单位的时间。
- 0.1的二进制表示是一个无限循环序列：0.00011[0011]..., $x=0.000\ 1100\ 1100\ 1100\ 1100B$ 。显然， x 是0.1的近似表示， **$0.1-x$**
 $= 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ [1100]... -$
 $0.000\ 1100\ 1100\ 1100\ 1100\ 1100B$, 即为：
 $= 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ \textcolor{blue}{1100}\ [1100]...B$
 $= 2^{-20} \times 0.1 \approx 9.54 \times 10^{-8}$

浮点运算举例

已知在爱国者导弹准备拦截飞毛腿导弹之前，已经连续工作了100小时，飞毛腿的速度大约为2000米/秒，则由于时钟计算误差而导致的距离误差是多少？

100小时相当于计数了 $100 \times 60 \times 60 \times 10 = 36 \times 10^5$ 次，因而导弹的时钟已经偏差了 $9.54 \times 10^{-8} \times 36 \times 10^5 \approx 0.343$ 秒

因此，距离误差是 $2000 \times 0.343 \text{秒} \approx 687 \text{米}$

Tips: 实际上，以色列方面已经发现了这个问题并于1991年2月11日知会了美国陆军及爱国者计划办公室（软件制造商）。以色列方面建议重新启动爱国者系统的电脑作为暂时解决方案，可是美国陆军方面却不知道每次需要间隔多少时间重新启动系统一次。1991年2月16日，制造商向美国陆军提供了更新软件，但这个软件最终却在飞毛腿导弹击中军营后的一天才运抵部队。

浮点运算举例

- 若x用float型表示，则x的机器数是什么？0.1与x的偏差是多少？系统运行100小时后的时钟偏差是多少？在飞毛腿速度为2000米/秒的情况下，预测的距离偏差为多少？
 - $0.1 = 0.0\ 0011[0011]B = +1.1\ 0011\ 0011\ 0011\ 0011\ 0011\ 00B \times 2^{-4}$ ，故x的机器数为0 011 1101 1 100 1100 1100 1100 1100 1100
 - Float型仅24位有效位数，后面的有效位全被截断，故x与0.1之间的误差为： $|x-0.1| = 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1100\ [1100]...B$ 。这个值等于 $2^{-24} \times 0.1 \approx 5.96 \times 10^{-9}$ 。100小时后时钟偏差 $5.96 \times 10^{-9} \times 36 \times 10^5 \approx 0.0215$ 秒。距离偏差 $0.0215 \times 2000 \approx 43$ 米。比爱国者导弹系统精确约16倍。
- 若用32位二进制定点小数 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ B$ 表示0.1，则误差比用float表示误差更大还是更小？
 - 当 $x = 0.000\ 1100\ 1100\ 1100\ 1100\ 1100\ 1100\ 1101\ B$ 时，与0.1之间的误差约为： $|x-0.1| = 0.000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 001100\ [1100]...B$ 。这个值等于 $2^{-30} \times 0.1 \approx 9.31 \times 10^{-11}$ 。100小时后时钟偏差 $9.31 \times 10^{-11} \times 36 \times 10^5 \approx 0.000335$ 秒。预测的距离偏差仅为 $0.000335 \times 2000 \approx 0.67$ 米。

浮点运算举例

- 从上述结果可以看出：
 - 用32位定点小数表示0.1，比采用float精度高64倍
 - 用float表示在计算速度上更慢，必须先把计数值转换为IEEE 754格式浮点数，然后再对两个IEEE 754格式的数相乘，故采用float比直接将两个二进制数相乘要慢得多
- Ariana 5火箭和爱国者导弹的例子带来的启示
 - ✓ 程序员应对底层机器级数据的表示和运算有深刻理解
 - ✓ 计算机世界，“差之毫厘，失之千里”，需要细心再细心，精确再精确
 - ✓ 不能遇到小数就用浮点数表示，有些情况下（如需要将一个整数变量乘以一个确定的小数常量），可先用一个确定的定点整数与整数变量相乘，然后再通过移位运算来确定小数点
 - ✓ 多说一句，字符集、格式定义，除了“话语权”，更多是优化设计

第三讲小结

- C语言中涉及的运算
 - 整数算术运算、浮点数算术运算
 - 按位、逻辑、移位、位扩展和位截断
- 整数的加、减运算
 - 计算机中的“算盘”：模运算系统（高位丢弃、用标志信息表示）
 - 带符号整数和无符号数的加、减都在同一个“算盘”中
 - 现实与计算机中的运算结果有差异（计算机是模运算系统）
- 整数的乘、除运算
 - 无符号整数：逻辑左移 k 位等于乘 2^k 、逻辑右移 k 位等于除 2^k
 - 带符号整数乘：算术左移 k 位等于乘 2^k
 - 带符号整数除：($x + 2^k - 1$) 算术右移 k 位等于 x 除以 2^k
- 浮点数运算
 - 加减：对阶/尾数加减/规格化/舍入(就近舍入到偶数)（大数吃小数）
 - 乘除：尾数相乘除，阶码相加减

4位乘法器计算步骤

题目：计算 6×6 （即 0110×0110 ）

目标结果：36（二进制 `0010 0100`）

硬件配置：

- A 寄存器（累加器）：4 位（初始为 0000）
- Q 寄存器（乘数）：4 位（初始为 0110）
- M 寄存器（被乘数）：0110（固定不动）
- 计数器：4（需要循环 4 次）

第 1 步

- 判断 Q 的最低位（LSB）：Q = 0110，最低位是 0。
- 操作：因为是 0，不累加（A 保持不变）。
 - A = 0000
- 操作：A 和 Q 整体逻辑右移 1 位。（因为是无符号乘法，这里用逻辑右移，高位补 0）
 - 整体看：[A][Q] = `0000 0110`
 - 右移后：`0000 0011`（A 的最低位 0 移到了 Q 的最高位，Q 的最低位 0 丢弃）
- 状态：A = 0000, Q = 0011

第 2 步

- 判断 Q 的最低位 (LSB) : $Q = 0011$, 最低位是 1。
- 操作: $A = A + M$
 - $A = 0000 + 0110 = 0110$
- 操作: A 和 Q 整体逻辑右移 1 位。
 - 整体看: $[A][Q] = 0110\ 0011$
 - 右移后: $0011\ 0001$ (A 的最低位 0 移到了 Q 的最高位)
- 状态: $A = 0011, Q = 0001$

第 3 步

- 判断 Q 的最低位 (LSB) : $Q = 0001$, 最低位是 1。
- 操作: $A = A + M$
 - $A = 0011 + 0110 = 1001$ (注意这里没有进位)
- 操作: A 和 Q 整体逻辑右移 1 位。
 - 整体看: $[A][Q] = 1001\ 0001$
 - 右移后: $0100\ 1000$ (A 的最低位 1 移到了 Q 的最高位)
- 状态: $A = 0100, Q = 1000$

第 4 步

- 判断 Q 的最低位 (LSB) : $Q = 1000$, 最低位是 0。
- 操作: 因为是 0, 不累加 (A 保持不变)。
 - $A = 0100$
- 操作: A 和 Q 整体逻辑右移 1 位。
 - 整体看: $[A][Q] = 0100\ 1000$
 - 右移后: $0010\ 0100$
- 状态: $A = 0010, Q = 0100$

最终结果

经过 4 次循环:

- A 寄存器: 0010 (高 4 位)
- Q 寄存器: 0100 (低 4 位)

把 A 和 Q 拼起来, 结果是 0010 0100。

转换成十进制: $32 + 4 = 36$ 。

Booth算法：核心思想与决策规则

核心思想：将乘数中连续的1序列（如...0111...）转化为一次减法和一次加法操作，从而显著减少加法运算的次数，提升乘法效率。

Yn (当前位)	Yn+1 (下一位)	操作指令
0	0	部分积右移一位（无操作）
0	1	加 [X]补，右移一位
1	0	减 [X]补 (加[-X]补)，右移一位
1	1	部分积右移一位（无操作）

核心价值：将连续的1序列运算转化为“减加”操作，大幅降低硬件运算复杂度与耗时

Booth算法实战：计算 -3×2 的补码乘法过程



01 题目与补码准备

$X = -3(1101)$ $Y = +2(0010)$ $[-X]_{\text{补}} = 0011$



02 寄存器初始化

$A = 0000$ | $B = 1101$ | $Q = 0010$ | $Q_{n+1} = 0$ | $C = 4$



03 循环迭代 ($C=4 \rightarrow 1$)

- $C4: 00 \rightarrow$ 右移 ($A=0000, Q=0001$)
- $C3: 10 \rightarrow$ 减B(加0011) $\rightarrow$ 右移 ($A=0001, Q=1000$)
- $C2: 01 \rightarrow$ 加B(加1101) $\rightarrow$ 右移 ($A=1111, Q=0100$)
- $C1: 00 \rightarrow$ 右移 ($A=1111, Q=1010$)



04 结果验证：11111010 = -6 (正确)

核心价值：高效的补码乘法

Booth 算法寄存器动态流程示意

初始状态:

- A (累加器): 0000
- S (被乘数补码): 1101
- Q (乘数): 0010
- Q-1 (Q 的最低位辅助位): 0
- C (循环计数器): 4

第 1 次迭代 (C = 4)

- 判断条件: $Q_0 Q_{-1} = 00$ (相等)
- 操作: 右移 (Arithmetic Right Shift)
- 状态变化:
 - A: 0000 $\rightarrow$ 0000
 - Q: 0010 $\rightarrow$ 0001
 - Q-1: 0 $\rightarrow$ 0 (原Q0移入)
- 新状态:
 - A: 0000
 - Q: 0001
 - Q-1: 0
 - C: 3

第 2 次迭代 (C = 3)

- 判断条件: $Q_0 Q_{-1} = 10$ ($Q_0=1, Q_{-1}=0$)
- 操作: $A = A + S$ (即 $A = A + [-X]_{\text{补}}$), 然后 右移
- 加法步骤: $0000 + 1101 = 1101$
- 移位步骤:
 - A: 1101 $\rightarrow$ 1110
 - Q: 0001 $\rightarrow$ 1000
 - Q-1: 0 $\rightarrow$ 1 (原Q0移入)
- 新状态:
 - A: 1110
 - Q: 1000
 - Q-1: 1
 - C: 2

第 3 次迭代 (C = 2)

- 判断条件: $Q_0 Q_{-1} = 01$ ($Q_0=0, Q_{-1}=1$)
- 操作: $A = A + B$ (即 $A = A + [X]_{\text{补}}$), 然后 右移
- 加法步骤: $1110 + 0011 = 10001$ (丢弃溢出位, 取低4位为 0001)
- 移位步骤:
 - A: 0001 $\rightarrow$ 0000
 - Q: 1000 $\rightarrow$ 1100
 - Q-1: 1 $\rightarrow$ 0 (原Q0移入)
- 新状态:
 - A: 0000
 - Q: 1100
 - Q-1: 0
 - C: 1

第 4 次迭代 (C = 1)

- 判断条件: $Q_0 Q_{-1} = 00$ (相等)
- 操作: 右移 (Arithmetic Right Shift)
- 状态变化:
 - A: 0000 $\rightarrow$ 0000
 - Q: 1100 $\rightarrow$ 0110
 - Q-1: 0 $\rightarrow$ 0 (原Q0移入)
- 新状态:
 - A: 0000
 - Q: 0110
 - Q-1: 0
 - C: 0

当 C=0 时, 循环结束。

• 最终乘积 (A || Q): 1111 1010 (注: 验证结果 1111 1010, 在实际运算中, 最后一次移位后的A其高位会因为符号扩展保持为1, 最终拼接结果为 1111 1010, 即十进制的 -6)。

示例对比——“龙虾”的统计

4.1 性能对比

指标	4 位阵列乘法器	8 位 Booth 乘法器
速度	1 时钟周期	8 时钟周期
吞吐量	高 (适合流水线)	中
延迟	低 (门延迟)	中 (时钟周期)
适用场景	高速 DSP	通用 CPU

4.2 成本对比

成本项	4 位阵列乘法器	8 位 Booth 乘法器
门数量	$O(N^2) = 16$ 个与门	$O(N) = 1$ 个加法器
寄存器	少	多 (3 个 8 位)
布线复杂度	高	低
功耗	高	低

4.3 功能对比

功能	4 位阵列乘法器	8 位 Booth 乘法器
无符号乘法	✔ 支持	✔ 支持
有符号乘法	✘ 不支持	✔ 支持 (补码)
连续 1 优化	✘ 无	✔ 有
可扩展性	差 (面积爆炸)	好

第三章 程序的转换与机器级表示

程序转换概述

IA-32 /x86-64指令系统

C语言程序的机器级表示

复杂数据类型的分配和访问

越界访问和缓冲区溢出、x86-64架构

程序的转换与机器级表示

- **主要教学目标**

- 了解高级语言与汇编语言、汇编语言与机器语言之间的关系
- 掌握有关指令格式、操作数类型、寻址方式、操作类型等内容
- 理解高级语言源程序中的语句与机器级代码之间的对应关系
- 理解复杂数据类型（数组、结构等）的机器级实现

- **主要教学内容**

- 介绍C语言程序与IA-32机器级指令之间的对应关系。
- 主要包括：程序转换概述、IA-32指令系统、C语言中控制语句和过程调用等机器级实现、复杂数据类型（数组、结构等）的机器级实现等。
- 本章所用的机器级表示主要以汇编语言形式表示为主。

采用逆向工程方法！

程序的机器级表示

- 分以下六个部分介绍

- **第一讲：程序转换概述**

- 机器指令和汇编指令
 - 机器级程序员感觉到的属性和功能特性
 - 高级语言程序转换为机器代码的过程

- **第二讲：IA-32 /x86-64指令系统**

- **第三讲：C语言程序的机器级表示**

- 过程调用的机器级表示
 - 选择语句的机器级表示
 - 循环结构的机器级表示

- **第四讲：复杂数据类型的分配和访问**

- 数组的分配和访问
 - 结构体数据的分配和访问
 - 联合体数据的分配和访问
 - 数据的对齐

- **第五讲：越界访问和缓冲区溢出**

- **第六讲：x86-64架构（自学）**

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

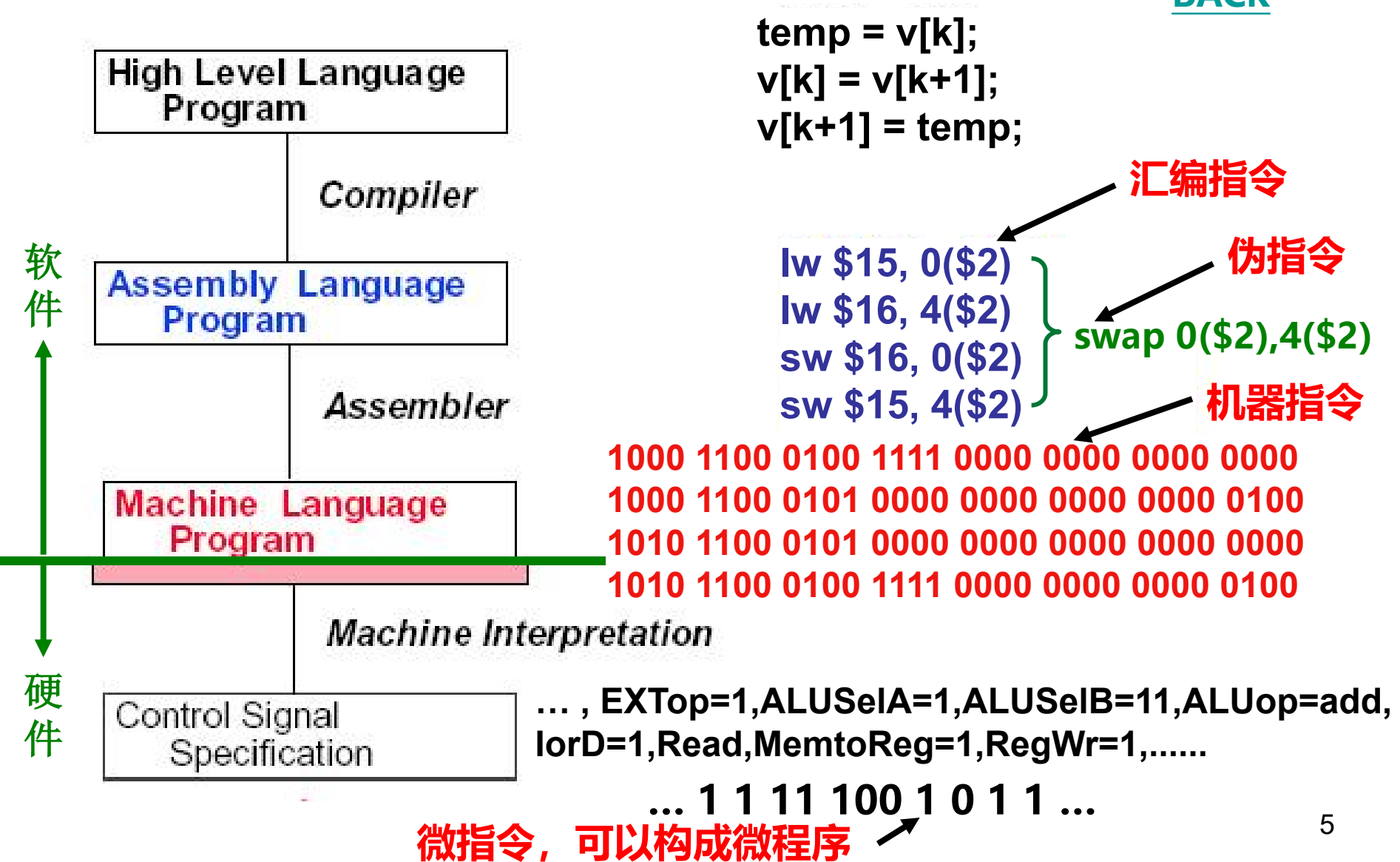
围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

“指令”的概念

- 计算机中的指令有**微指令**、**机器指令**和**伪（宏）指令**之分
- **微指令**是微程序级命令，属于硬件范畴
- **伪指令**是由若干机器指令组成的指令序列，属于软件范畴
- **机器指令**介于二者之间，处于硬件和软件的交界面
 - 本章中提及的指令都指机器指令
- **汇编指令**是机器指令的汇编表示形式，即符号表示
- 机器指令和汇编指令一一对应，它们都与具体机器结构有关，都属于**机器级指令**

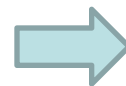
回顾：Hardware/Software Interface

[BACK](#)

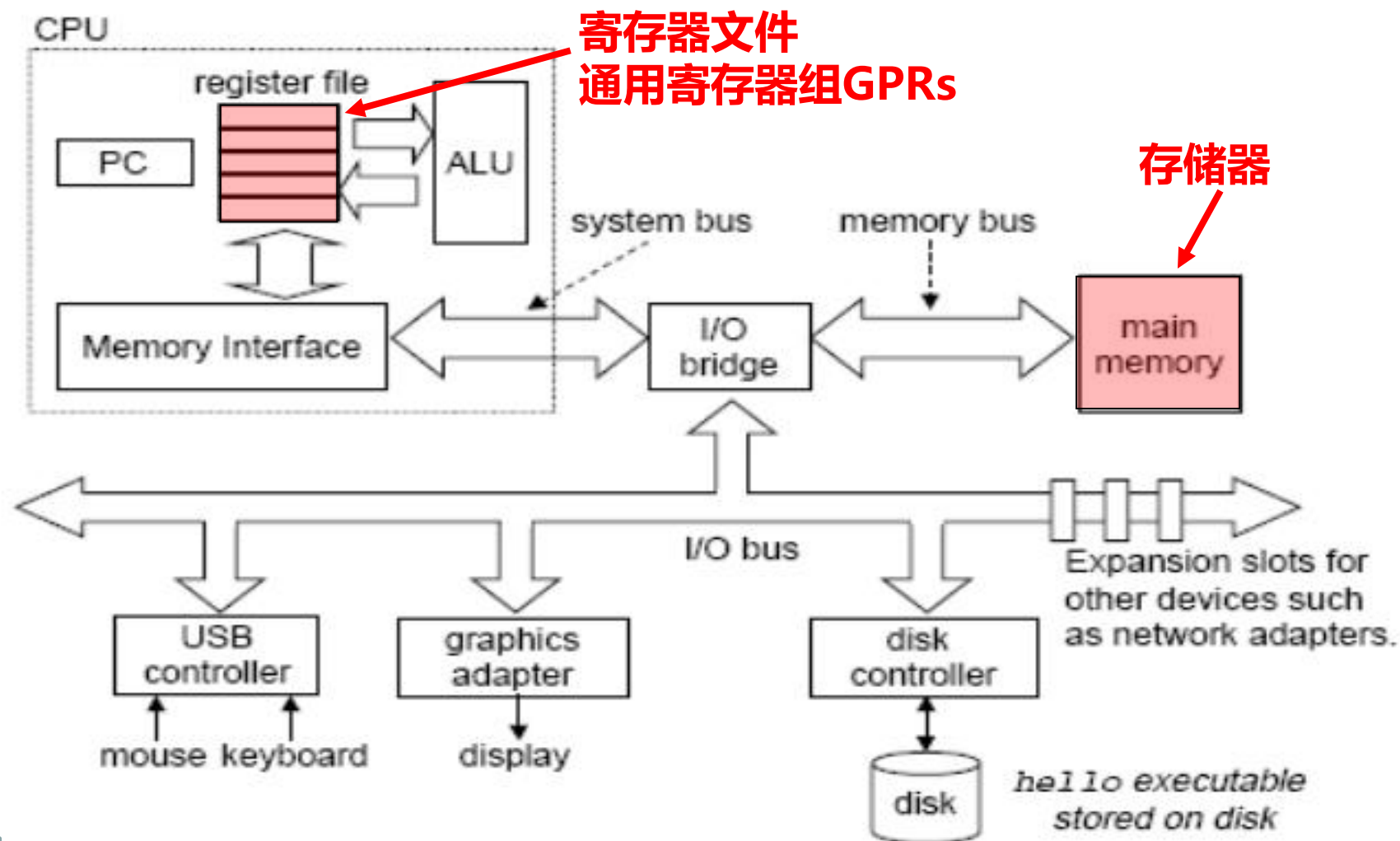


指令需给出的信息

- 操作性质（操作码）
- 源操作数1 或/和 源操作数2 （立即数、寄存器编号、存储地址）
- 目的操作数地址 （寄存器编号、存储地址）
- 存储地址的描述与**操作数的数据结构**有关！



计算机中的数据存放在哪里



机器级指令

- 机器指令和汇编指令一一对应，都是机器级指令
- 机器指令是一个0/1序列，由若干**字段**组成

补码**11111010**
的真值为多少？

100010 DW	mod	reg	r/m	disp8
100010 0 0	01	001	001	11111010

操作码

寻址方式

寄存器编号

立即数(位移量)

- 汇编指令是机器指令的符号表示（可能有不同的格式）

mov [bx+di-6], cl 或 **movb %cl, -6(%bx,%di)**

Intel格式

AT&T 格式

本课程采用
AT&T格式

mov、movb、bx、%bx等都是**助记符**

指令的功能为： **$M[R[bx] + R[di] - 6] \leftarrow R[cl]$**

寄存器传送语言 RTL (Register Transfer Language)

R: 寄存器内容
M: 存储单元内容

IA-32机器指令格式

前缀类型:

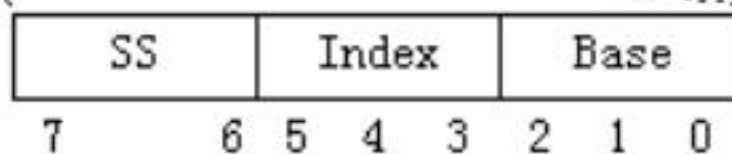
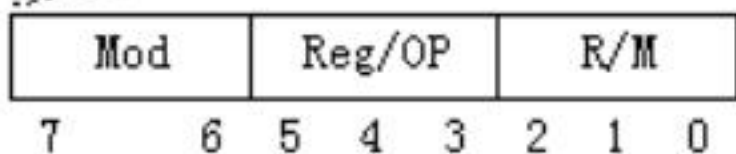
字节数:

指令前缀	段前缀	操作数长度	地址长度
0或1	0或1	0或1	0或1

指令段:

字节数:

操作码	寻址方式	SIB	位移	直接数据
1或2	0或1	0或1	1、2、4	立即数



操作码: opcode; w: 与机器模式 (16 / 32位) 一起确定寄存器位数 (AL / AX / EAX); d: 操作方向

寻址方式: mod、r/m、reg/op三个字段与w字段和机器模式一起确定操作数所在的寄存器编号或有效地址计算方式

SIB中基址B和变址I都可是8个GRS中任一个; SS给出比例因子

位移量和**立即数**都可以是: 1B/2B/4B

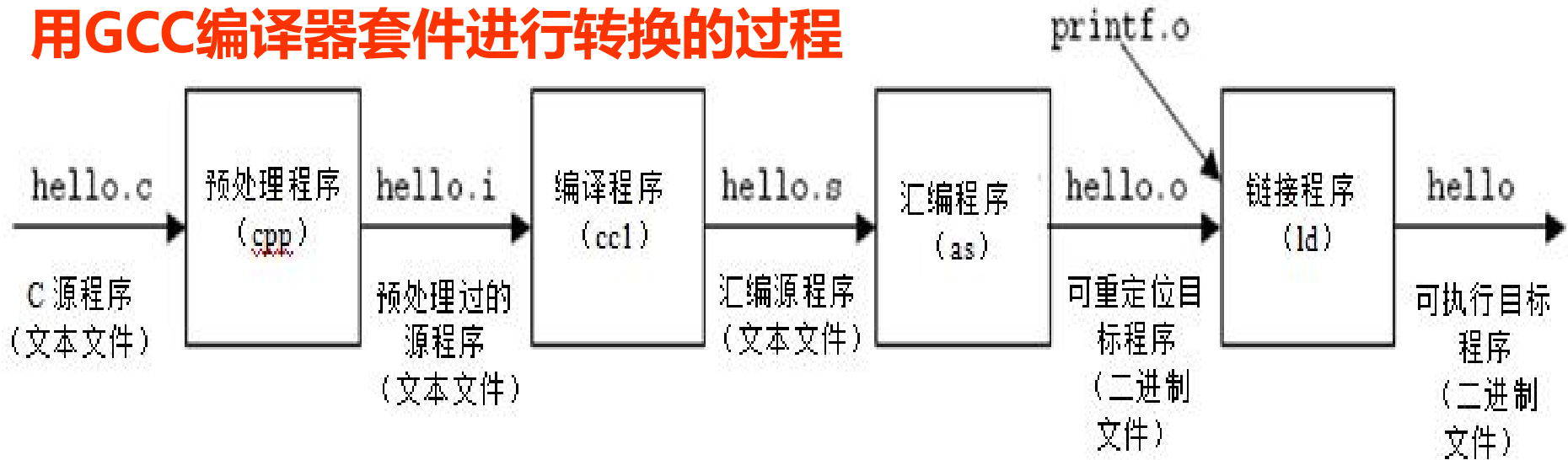
Effective Address	Mod	R/M	Mod				Reg/OP			R/M	
			7	6	5	4	3	2	1	0	
[EAX]	00	000	00	08	10	18	20	28	30	38	
[ECX]		001	01	09	11	19	21	29	31	39	
[EDX]		010	02	0A	12	1A	22	2A	32	3A	
[EBX]		011	03	0B	13	1B	23	2B	33	3B	
[--][--] ¹		100	04	0C	14	1C	24	2C	34	3C	
disp32		101	05	0D	15	1D	25	2D	35	3D	
[ESI]		110	06	0E	16	1E	26	2E	36	3E	
[EDI]		111	07	0F	17	1F	27	2F	37	3F	
disp8[EAX]	01	000	40	48	50	58	60	68	70	78	
disp8[ECX]		001	41	49	51	59	61	69	71	79	
disp8[EDX]		010	42	4A	52	5A	62	6A	72	7A	
disp8[EBX];		011	43	4B	53	5B	63	6B	73	7B	
disp8[--][--]		100	44	4C	54	5C	64	6C	74	7C	
disp8[EBP]		101	45	4D	55	5D	65	6D	75	7D	
disp8[ESI]		110	46	4E	56	5E	66	6E	76	7E	
disp8[EDI]		111	47	4F	57	5F	67	6F	77	7F	
disp32[EAX]	10	000	80	88	90	98	A0	A8	B0	B8	
disp32[ECX]		001	81	89	91	99	A1	A9	B1	B9	
disp32[EDX]		010	82	8A	92	9A	A2	AA	B2	BA	
disp32[EBX]		011	83	8B	93	9B	A3	AB	B3	BB	
disp32[--][--]		100	84	8C	94	9C	A4	AC	B4	BC	
disp32[EBP]		101	85	8D	95	9D	A5	AD	B5	BD	
disp32[ESI]		110	86	8E	96	9E	A6	AE	B6	BE	
disp32[EDI]		111	87	8F	97	9F	A7	AF	B7	BF	
EAX/AX/AL	11	000	C0	C8	D0	D8	E0	E8	F0	F8	
ECX/CX/CL		001	C1	C9	D1	D9	E1	E9	F1	F9	
EDX/DX/DL		010	C2	CA	D2	DA	E2	EA	F2	FA	
EBX/BX/BL		011	C3	CB	D3	DB	E3	EB	F3	FB	
ESP/SP/AH		100	C4	CC	D4	DC	E4	EC	F4	FC	
EBP/BP/CH		101	C5	CD	D5	DD	E5	ED	F5	FD	
ESI/SI/DH		110	C6	CE	D6	DE	E6	EE	F6	FE	
EDI/DI/BH		111	C7	CF	D7	DF	E7	EF	F7	FF	

指令集体系结构ISA

- ISA (Instruction Set Architecture) 位于软件和硬件之间
- 硬件的功能通过ISA提供出来
- 软件通过ISA规定的“指令”使用硬件
- ISA规定了：
 - 可执行的指令的集合，包括指令格式、操作种类以及每种操作对应的操作数的相应规定；
 - 指令可以接受的操作数的类型；
 - 操作数所能存放的寄存器组的结构，包括每个寄存器的名称、编号、长度和用途；
 - 操作数所能存放的存储空间的大小和编址方式；
 - 操作数在存储空间存放时按照大端还是小端方式存放；
 - 指令获取操作数的方式，即寻址方式；
 - 指令执行过程的控制方式，包括程序计数器、条件码定义等。

高级语言程序转换为机器代码的过程

用GCC编译器套件进行转换的过程



预处理：在高级语言源程序中插入所有用`#include`命令指定的文件和用`#define`声明指定的宏。

编译：将预处理后的源程序文件编译生成相应的汇编语言程序。

汇编：由汇编程序将汇编语言源程序文件转换为可重定位的机器语言目标代码文件。

链接：由链接器将多个可重定位的机器语言目标文件以及库例程（如`printf()`库函数）链接起来，生成最终的可执行目标文件。

GCC使用举例

```
1 // test.c
2 #include <stdio.h>
3 int add(int i, int j )
4 {
5     int x = i +j;
6     return x;
7 }
```

编译test1.c和test2.c, 最终生成可执行文件为test
gcc -c test2.c -o test

-O1为一级优化, -O2为二级优化, 选项-o指出输出文件名

gcc -c test.s -o test.o

可用objdump -d test.o 反汇编为汇编语言程序

00000000 <add>:

gcc -E test.c -o test.i
gcc -S test.i -o test.s
gcc -S test.c -o test.s

add:
pushl %ebp
movl %esp, %ebp
subl \$16, %esp
movl 12(%ebp), %eax
movl 8(%ebp), %edx
leal (%edx, %eax), %eax
movl %eax, -4(%ebp)
movl -4(%ebp), %eax
leave
ret

0:	55	push %ebp
1:	89 e5	mov %esp, %ebp
3:	83 ec 10	sub \$0x10, %esp
6:	8b 45 0c	mov 0xc(%ebp), %eax
9:	8b 55 08	mov 0x8(%ebp), %edx
c:	8d 04 02	leal (%edx,%eax,1), %eax
f:	89 45 fc	mov %eax, -0x4(%ebp)
12:	8b 45 fc	mov -0x4(%ebp), %eax
15:	c9	leave
16:	c3	ret

位移量 机器指令

汇编指令

编译得到的与反汇编得到的汇编指令形式稍有差异

两种目标文件

```
1 // test.c
2 #include <stdio.h>
3 int add(int i, int j )
4 {
5     int x = i +j;
6     return x;
7 }
```

test.o: 可重定位目标文件

test: 可执行目标文件

“objdump -d test.o” 结果

00000000 <add>:

0:	55	push	%ebp
1:	89 e5	mov	%esp, %ebp
3:	83 ec 10	sub	\$0x10, %esp
6:	8b 45 0c	mov	0xc(%ebp), %eax
9:	8b 55 08	mov	0x8(%ebp), %edx
c:	8d 04 02	lea	(%edx,%eax,1), %eax
f:	89 45 fc	mov	%eax, -0x4(%ebp)
12:	8b 45 fc	mov	-0x4(%ebp), %eax
15:	c9	leave	
16:	c3	ret	

“objdump -d test” 结果

080483d4 <add>:

80483d4:	55	push ...
80483d5:	89 e5	...
80483d7:	83 ec 10	...
80483da:	8b 45 0c	...
80483dd:	8b 55 08	...
80483e0:	8d 04 02	...
80483e3:	89 45 fc	...
80483e6:	8b 45 fc	...
80483e9:	c9	...
80483ea:	c3	ret

test.o中的代码从地址0开始， test中的代码从80483d4开始!

程序的机器级表示

- 分以下六个部分介绍

- 第一讲：程序转换概述

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- 第二讲：IA-32 /x86-64指令系统

- 第三讲：C语言程序的机器级表示

- 过程调用的机器级表示
- 选择语句的机器级表示
- 循环结构的机器级表示

- 第四讲：复杂数据类型的分配和访问

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- 第五讲：越界访问和缓冲区溢出

- 第六讲：x86-64架构（自学）

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

IA-32/x64指令系统概述

- x86是Intel开发的一类处理器体系结构的泛称
 - 包括 Intel 8086、80286、i386和i486等，因此其架构被称为“x86”
 - 由于数字并不能作为注册商标，因此，后来使用了可注册的名称，如Pentium、PentiumPro、Core 2、Core i7等
 - 现在Intel把32位x86架构的名称x86-32改称为IA-32
- 由AMD首先提出了一个兼容IA-32指令集的64位版本
 - 扩充了指令及寄存器长度和个数等，更新了参数传送方式
 - AMD称其为AMD64，Intel称其为Intl64（不同于IA-64）
 - 命名为“x86-64”，有时也简称为x64

本课程主要介绍IA-32，最后简要介绍x-64

IA-32支持的数据类型及格式

C 语言声明	Intel 操作数类型	汇编指令长度后缀	存储长度 (位)
(unsigned) char	整数 / 字节	b	8
(unsigned) short	整数 / 字	w	16
(unsigned) int	整数 / 双字	l	32
(unsigned) long int	整数 / 双字	l	32
(unsigned) long long int	-	-	2×32
char *	整数 / 双字	l	32
float	单精度浮点数	s	32
double	双精度浮点数	l	64
long double	扩展精度浮点数	t	80 / 96

IA-32的寄存器组织

	31	16	15	8	7	0	
EAX				AH	(AX)	AL	累加器
EBX				BH	(BX)	BL	基址寄存器
ECX				CH	(CX)	CL	计数寄存器
EDX				DH	(DX)	DL	数据寄存器
ESP				SP			堆栈指针
EBP				BP			基址指针
ESI				SI			源变址寄存器
EDI				DI			目标变址寄存器
EIP				IP			指令指针
EFLAGS				FLAGS			标志寄存器

8个通用寄存器

两个专用寄存器

6个段寄存器

CS	代码段
SS	堆栈段
DS	数据段
ES	附加段
FS	附加段
GS	附加段

IA-32的标志寄存器

31-22	21	20	19	18	17	16	15	14	13 12	11	10	9	8	7	6	5	4	3	2	1	0
保留	ID	VIP	VIF	AC	VM	RF	0	NT	IOPL	0	D	I	T	S	Z	0	A	0	P	1	C

- 6个条件标志

- OF、SF、ZF、CF各是什么标志（条件码）？
- AF：辅助进位标志（BCD码运算时才有意义）
- PF：奇偶标志

- 3个控制标志

- DF（Direction Flag）：方向标志（自动变址方向是增还是减）
- IF（Interrupt Flag）：中断允许标志（仅对外部可屏蔽中断有用）
- TF（Trap Flag）：陷阱标志（是否是单步跟踪状态）

-

IA-32的寻址方式

- 寻址方式
 - 根据指令给定信息得到操作数或操作数地址
- 操作数所在的位置
 - 指令中：立即寻址
 - 寄存器中：寄存器寻址
 - 存储单元中（属于存储器操作数，按字节编址）：其他寻址方式
- 存储器操作数的寻址方式与微处理器的工作模式有关
- 存储地址的描述与操作数的数据结构有关！

微处理器的工作模式

- **实地址模式（基本用不到）**
 - 为与8086/8088兼容而设，加电或复位时
 - 20位地址： $(CS) \ll 4 + (IP)$
 - 寻址空间为1MB
- **保护模式（需要掌握）**
 - 加电后进入，采用虚拟存储管理，多任务情况下隔离、保护
 - 80286以上高档微处理器最常用的工作模式
 - 32位线性地址分段（段基址+段内偏移量）
 - 寻址空间为 $2^{32}B$

为什么存储地址的描述与操作数的数据结构有关

```
1 // 一个整型数组
2 int numbers = {10, 20, 30, 40, 50};<websource>source_group_web_1</websource>
3
4 // 一个表示学生信息的结构体
5 struct Student {
6     int id;      // 假设占4字节
7     char grade;  // 假设占1字节
8     float score; // 假设占4字节
9 };
10 struct Student student1;
```

- 访问 numbers (值为30)

数据结构：一维数组，元素在内存中是连续存放。

地址描述：CPU会采用变址寻址或基址+变址寻址。

 基地址：numbers 数组的起始地址。

 索引：2。

有效地址计算：有效地址 = 基地址 + (索引 * 元素大小) = 基地址 + (2 * 4)。

地址描述明确地利用了“**数组是等宽元素线性排列**”结构特点

- 访问 student1.grade (值为'A')

数据结构：内部成员有固定的相对位置（偏移量）。

地址描述：CPU会采用基址寻址。

 基地址：student1 结构体变量的起始地址。

 偏移量：grade 成员相对于结构体起始位置的字节数。例子中，id 占4字节，所以 grade 的偏移量是4。

有效地址计算：有效地址 = 基地址 + 偏移量 = 基地址 + 4。

地址描述利用了“**结构体成员有固定偏移**”结构特点。

示例二：二维数组的行优先 vs. 列优先

二维数组在内存中本质上是一维的，但逻辑上是二维的。不同的编程语言或编译器可能采用不同的存储方式。

假设有一个 M 行 N 列的二维数组 `arr`，每个元素占 K 字节，首地址为 `base`。

- 按行优先存储 (C/C++语言)

数据先存储第一行的所有元素，再存储第二行，以此类推。

- 地址描述：要访问 `arr[i][j]`，其地址计算公式为：

$$\text{地址} = \text{base} + ((i * N) + j) * K$$

- 这个公式反映了“先走完一整行 ($i * N$)，再走列 ($+ j$)”的行优先结构。

- 按列优先存储 (Fortran语言)

数据先存储第一列的所有元素，再存储第二列，以此类推。

- 地址描述：要访问 `arr[i][j]`，其地址计算公式为：

$$\text{地址} = \text{base} + ((j * M) + i) * K$$

- 这个公式反映了“先走完一整列 ($j * M$)，再走行 ($+ i$)”的列优先结构。

保护模式下的寻址方式

寻址方式	说明		
立即寻址	指令直接给出操作数		
寄存器寻址	指定的寄存器R的内容为操作数		
位移	$LA = (SR) + A$		} 存储器操作数
基址寻址	$LA = (SR) + (B)$		
基址加位移	$LA = (SR) + (B) + A$		
比例变址加位移	$LA = (SR) + (I) \times S + A$		
基址加变址加位移	$LA = (SR) + (B) + (I) + A$		
基址加比例变址加位移	$LA = (SR) + (B) + (I) \times S + A$		
相对寻址	$LA = (PC) + A$	跳转目标指令地址	

注：LA:线性地址 (X):X的内容 SR:段寄存器 PC:程序计数器 R:寄存器
A:指令中给定地址段的位移量 B:基址寄存器 I:变址寄存器 S:比例系数

- **SR段寄存器（间接）确定操作数所在段的段基址**
- **有效地址给出操作数在所在段的偏移地址**



存储器操作数的寻址方式

```
int x;  
float a[100];  
short b[4][4];  
char c;  
double d[10];
```

a[i]的地址如何计算?

104+i×**4**

i=99时, $104 + 99 \times 4 = 500$

b[i][j]的地址如何计算?

504+i×**8**+j×**2**

i=3、j=2时, $504 + 24 + 4 = 532$

d[i]的地址如何计算?

544+i×**8**

i=9时, $544 + 9 \times 8 = 616$

b31		b0	
d[9]			616
⋮			
d[0]			544
		c	536
b[3][3]	b[3][2]		532
⋮			
b[0][1]	b[0][0]		504
a[99]			500
⋮			
a[0]			104
x			100
⋮			25

存储器操作数的寻址方式

```
int x;  
float a[100];  
short b[4][4];  
char c;  
double d[10];
```

各变量应采用什么寻址方式?

x、c: 位移 / 基址

a[i]: $104 + i \times 4$, 比例变址 + 位移

d[i]: $544 + i \times 8$, 比例变址 + 位移

b[i][j]: $504 + i \times 8 + j \times 2$,

基址 + 比例变址 + 位移

将b[i][j]取到AX中的指令可以是:

“movw $504(\%ebp, \%esi, 2), \%ax$ ”

其中, $i \times 8$ 在EBP中, j在ESI中,

2为比例因子

b31		b0	
d[9]			616
⋮			
d[0]			544
		c	536
b[3][3]	b[3][2]		532
⋮			
b[0][1]	b[0][0]		504
a[99]			500
⋮			
a[0]			104
x			100
⋮			26 ←

IA-32常用指令类型

(1) 传送指令

— 通用数据传送指令

MOV: 一般传送, 包括movb、movw和movl等

MOVS: 符号扩展传送, 如movsbw、movswl等

MOVZ: 零扩展传送, 如movzwl、movzbl等

XCHG: 数据交换

PUSH/POP: 入栈/出栈, 如pushl, pushw, popl, popw等

— 地址传送指令

LEA: 加载有效地址, 如leal (%edx,%eax), %eax” 的功能为
 $R[edx] \leftarrow R[edx] + R[edx]$, 执行前, 若 $R[edx] = i$,
 $R[edx] = j$, 则指令执行后, $R[edx] = i + j$

— 输入输出指令

IN和OUT: I/O端口与寄存器之间的交换

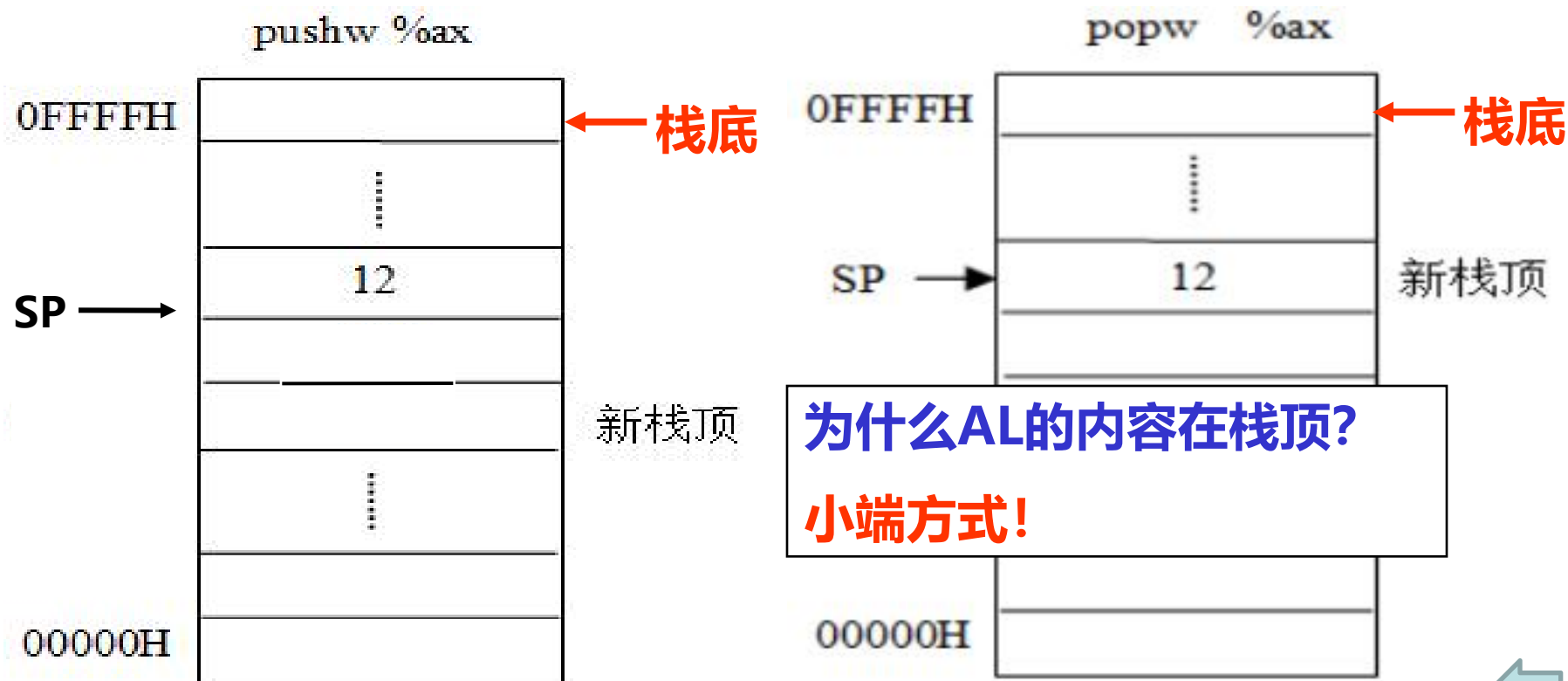
— 标志传送指令

PUSHF、POPF: 将EFLAG压栈, 或将栈顶内容送EFLAG



“入栈”和“出栈”操作

- 栈 (Stack) 是一种采用 **“先进后出”** 方式进行访问的一块存储区，用于**嵌套过程调用**。从**高地址向低地址增长**
- “栈”不等于 **“堆栈”** (由“堆”和“栈”组成)



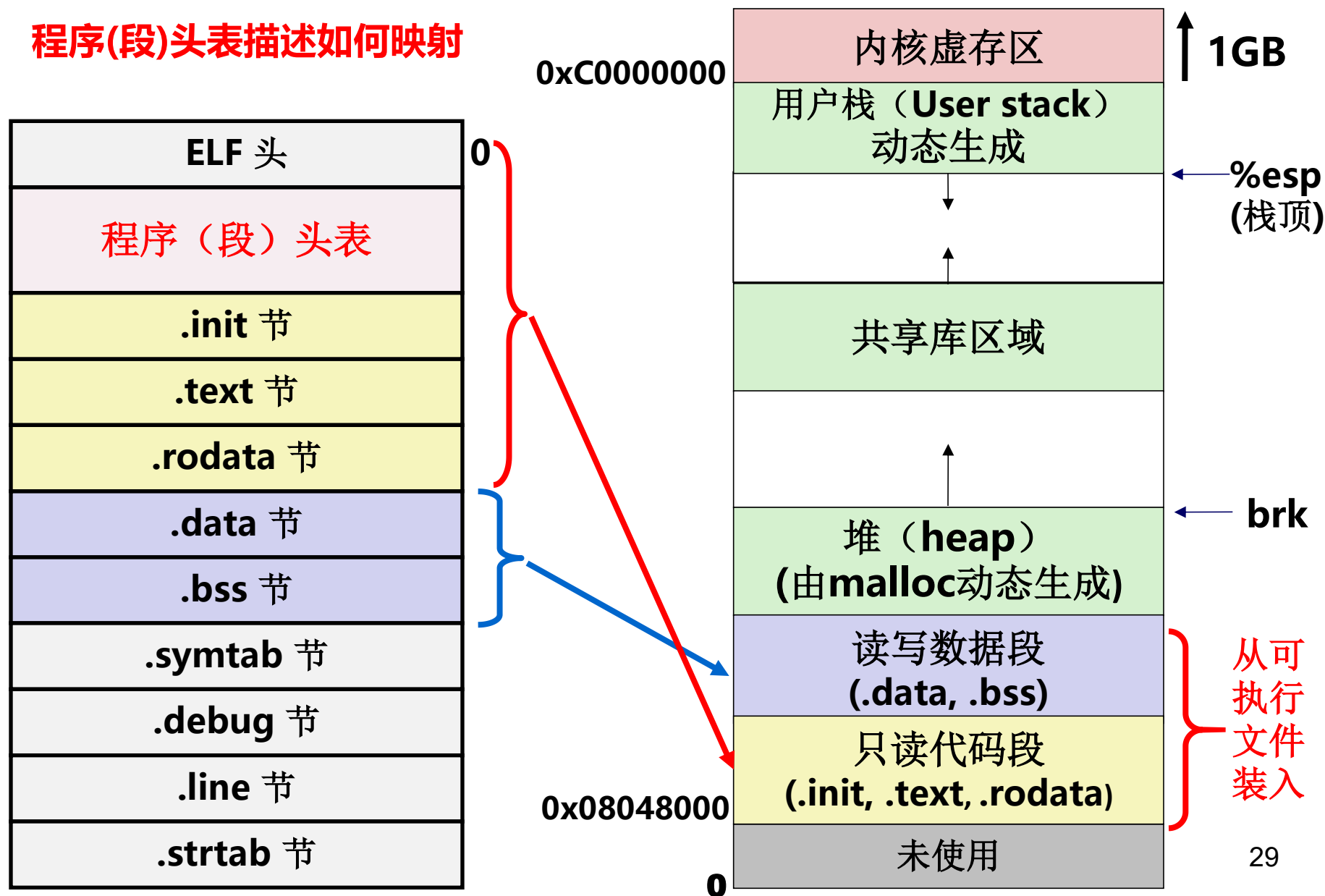
$R[sp] \leftarrow R[sp] - 2$ 、 $M[R[sp]] \leftarrow R[ax]$

$R[ax] \leftarrow M[R[sp]]$ 、 $R[sp] \leftarrow R[sp] + 2$ ²⁸



可执行文件的存储器映像

程序(段)头表描述如何映射



传送指令举例

将以下Intel格式指令转换为AT&T格式指令，并说明功能。

```
push    ebp
mov     ebp, esp
mov     edx, DWORD PTR [ebp+8]
mov     bl, 255
mov     ax, WORD PTR [ebp+edx*4+8]
mov     WORD PTR [ebp+20], dx
leax    eax, [ecx+edx*4+8]
```

```
pushl   %ebp           //R[esp]←R[esp]-4, M[R[esp]] ←R[ebp], 双字
movl    %esp, %ebp      //R[ebp] ←R[esp], 双字
movl    8(%ebp), %edx    //R[edx] ←M[R[ebp]+8], 双字
movb    $255, %bl       //R[bl]←255, 字节
movw    8(%ebp,%edx,4), %ax //R[ax]←M[R[ebp]+R[edx]×4+8], 字
movw    %dx, 20(%ebp)    //M[R[ebp]+20]←R[dx], 字
leal    8(%ecx,%edx,4), %eax //R[eax]←R[ecx]+R[edx]×4+8, 双字
```

IA-32常用指令类型

(2) 定点算术运算指令

- 加 / 减运算 (影响标志、不区分无/带符号)

ADD: 加, 包括addb、addw、addl等

SUB: 减, 包括subb、subw、subl等

- 增1 / 减1运算 (影响除CF以外的标志、不区分无/带符号)

INC: 加, 包括incb、incw、incl等

DEC: 减, 包括decb、decw、decl等

- 取负运算 (影响标志、若对0取负, 则结果为0/CF=0, 否则CF=1)

NEG: 取负, 包括negb、negw、negl等

- 比较运算 (做减法得到标志、不区分无/带符号)

CMP: 比较, 包括cmpb、cmpw、cmpl等

- 乘 / 除运算 (区分无/带符号)

MUL / IMUL: 无符号乘 / 带符号乘

DIV / IDIV: 带无符号除 / 带符号除

整数乘除指令

- 乘法指令：可给出一个、两个或三个操作数 [BACK1](#)
 - 若给出一个操作数SRC，则另一个源操作数隐含在AL/AX/EAX中，将SRC和累加器内容相乘，结果存放在AX（16位）或DX-AX（32位）或EDX-EAX（64位）中。DX-AX表示32位乘积的高、低16位分别在DX和AX中。 $n\text{位} \times n\text{位} = 2n\text{位}$
 - 若指令中给出两个操作数DST和SRC，则将DST和SRC相乘，结果在DST中。 $n\text{位} \times n\text{位} = n\text{位}$
 - 若指令中给出三个操作数REG、SRC和IMM，则将SRC和立即数IMM相乘，结果在REG中。 $n\text{位} \times n\text{位} = n\text{位}$
- 除法指令：只明显指出除数，用EDX-EAX中内容除以指定的除数
 - 若为8位，则16位被除数在AX寄存器中，商送回AL，余数在AH
 - 若为16位，则32位被除数在DX-AX寄存器中，商送回AX，余数在DX
 - 若为32位，则被除数在EDX-EAX寄存器中，商送EAX，余数在EDX

以上内容无需死记硬背，遇到具体指令时能查阅到并理解即可。 [BACK2](#)

定点算术运算指令汇总

指令	显式操作数	影响的常用标志	操作数类型	AT&T 指令助记符	对应 C 运算符
ADD	2 个	OF、ZF、SF、CF	无/带符号整数	addb、addw、addl	+
SUB	2 个	OF、ZF、SF、CF	无/带符号整数	subb、subw、subl	-
INC	1 个	OF、ZF、SF	无/带符号整数	incb、incw、incl	++
DEC	1 个	OF、ZF、SF	无/带符号整数	decb、decw、decl	--
NEG	1 个	OF、ZF、SF、CF	无/带符号整数	negb、negw、negl	-
CMP	2 个	OF、ZF、SF、CF	无/带符号整数	cmpb、cmpw、cmpl	<, <=, >, >=
MUL	1 个	无	无符号整数	mulb、mulw、mull	*
MUL	2 个	无	无符号整数	mulb、mulw、mull	*
MUL	3 个	无	无符号整数	mulb、mulw、mull	*
IMUL	1 个	无	带符号整数	imulb、imulw、imull	*
IMUL	2 个	无	带符号整数	imulb、imulw、imull	*
IMUL	3 个	无	带符号整数	imulb、imulw、imull	*
DIV	1 个	无	无符号整数	divb、divw、divl	/, %
IDIV	1 个	无	带符号整数	idivb、idivw、idivl	/, % 33

定点加法指令举例

- 假设 $R[ax]=FFFAH$, $R[bx]=FFF0H$, 则执行以下指令后

“addw %bx, %ax”

AX、BX中的内容各是什么？标志CF、OF、ZF、SF各是什么？要求分别将操作数作为**无符号数**和**带符号整数**解释并验证指令执行结果。

解：功能： $R[ax] \leftarrow R[ax] + R[bx]$, 指令执行后的结果如下

$R[ax]=FFFAH+FFF0H=FFEAH$, BX中内容不变

$CF=1$, $OF=0$, $ZF=0$, $SF=1$

若是无符号整数运算, 则 $CF=1$ 说明结果溢出

验证：FFFA的真值为 $65535-5=65530$, FFF0的真值为 65520

FFEA的真值为 $65535-21=65514 \neq 65530+65520$, 即溢出

若是带符号整数运算, 则 $OF=0$ 说明结果没有溢出

验证：FFFA的真值为 -6 , FFF0的真值为 -16

FFEA的真值为 $-22=-6+(-16)$, 结果正确, 无溢出

定点乘法指令举例

- 假设 $R[eax] = 000000B4H$, $R[ebx] = 00000011H$, $M[000000F8H] = 000000A0H$, 请问:

(1) 执行指令 “**mulb %bl**” 后, 哪些寄存器的内容会发生变化? 是否与执行 “**imulb %bl**” 指令所发生的变化一样? 为什么? 请用该例给出的数据验证你的结论。

解: “**mulb %bl**” 功能为 $R[ax] \leftarrow R[al] \times R[bl]$, 执行结果如下

无符号乘:

$R[ax] = B4H \times 11H$ (无符号整数180和17相乘)

$R[ax] = 0BF4H$, 真值为 $3060 = 180 \times 17$

$$\begin{array}{r} 1011\ 0100 \\ \times\ 0001\ 0001 \\ \hline 1011\ 0100 \\ 1011\ 0100 \\ \hline \end{array}$$

0000 1011 1111 0100
AH=? **AL=?**

“**imulb %bl**” 功能为 $R[ax] \leftarrow R[al] \times R[bl]$

$R[ax] = B4H \times 11H$ (带符号整数-76和17相乘)

$R[al] = F4H$, $R[ah] = ?$ **AH中的变化不一样!**

$R[ax] = FAF4H$, 真值为 $-1292 = -76 \times 17$

对于带符号乘, 若积只取低n位, 则和无符号相同; 若取2n位, 则采用 “**布斯**” 乘法

定点乘法指令举例

- **布斯乘法:** **"imulb %bl"**

R[ax] = B4H × 11H

```

      1 0 1 1 0 1 0 0
    x 0 0 1 -1 0 0 1 -1    0001 0001
  -----
0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0
1 1 1 1 1 1 1 1 1 0 1 1 0 1 0 0
0 0 0 0 0 1 0 0 1 1 0 0
1 1 1 1 0 1 1 0 1 0 0
-----
1 1 1 1 1 0 1 0 1 1 1 1 0 1 0 0
  AH=?      AL=?

```

R[ax]=FAF4H, 真值为-1292=-76 × 17

定点乘法指令举例

- 假设 $R[eax]=000000B4H$, $R[ebx]=00000011H$, $M[000000F8H]=000000A0H$, 请问:

(2) 执行指令 `"imull $-16, (%eax,%ebx,4), %eax"` 后哪些寄存器和存储单元发生了变化? 乘积的机器数和真值各是多少?

解: `"imull -16, (%eax,%ebx,4), %eax"`

功能为 $R[eax] \leftarrow (-16) \times M[R[eax] + R[ebx] \times 4]$, 执行结果如下

$$R[eax] + R[ebx] \times 4 = 000000B4H + 00000011H \ll 2 = 000000F8H$$

$$R[eax] = (-16) \times M[000000F8H]$$

$$= (-16) \times 000000A0H \text{ (带符号整数乘)}$$

$$= FFFFFFF60H \ll 4$$

$$= FFFFFFF600H$$

EAX中的真值为-2560

IA-32常用指令类型

(3) 按位运算指令

- 逻辑运算（仅NOT不影响标志，其他指令OF=CF=0，而ZF和SF根据结果设置：若全0，则ZF=1；若最高位为1，则SF=1）

NOT: 非, 包括 notb、notw、notl等

AND: 与, 包括 andb、andw、andl等

OR: 或, 包括 orb、orw、orl等

XOR: 异或, 包括 xorb、xorw、xorl等

TEST: 做“与”操作测试, 仅影响标志

- 移位运算（左/右移时, 最高/最低位送CF）

SHL/SHR: 逻辑左/右移, 包括 shlb、shrw、shrl等

SAL/SAR: 算术左/右移, 左移判溢出, 右移高位补符

（移位前、后符号位发生变化, 则OF=1）

ROL/ROR: 循环左/右移, 包括 rolb、rorw、roll等

RCL/RCR: 带CF循环左/右移, 将CF作为操作数一部分循环移位

以上内容不要死记硬背, 遇到具体指令时能查阅到并理解即可。

按位运算指令举例

假设short型变量x被编译器分配在寄存器AX中， $R[ax]=FF80H$ ，则以下汇编代码段执行后变量x的机器数和真值分别是多少？

`movw %ax, %dx`

`salw $2, %ax` 1111 1111 1000 0000<<2

`addw %dx, %ax` 1111 1111 1000 0000+1111 1110 0000 0000

`sarw $1, %ax` 1111 1101 1000 0000>>1=1111 1110 1100 0000

解：\$2和\$1分别表示立即数2和1。

x是short型变量，故都是算术移位指令，并进行带符号整数加。

假设上述代码段执行前 $R[ax]=x$ ，则执行 $((x \ll 2) + x) \gg 1$ 后， $R[ax]=5x/2$ 。

算术左移时，AX中的内容在移位前、后符号未发生变化，故OF=0，没有溢出。

最终AX的内容为FEC0H，解释为short型整数时，其值为-320。

验证： $x=-128$ ， $5x/2=-320$ 。经验证，结果正确。

逆向工程：从汇编指令推出高级语言程序代码

移位指令举例

```
#include <stdio.h>
void main()
{
    int a = 0x80000000;
    unsigned int b = 0x80000000;
    printf("a= 0x%X\n", a >> 1);
    printf("b= 0x%X\n", b >> 1);
}
```

```
push    %ebp
mov     %esp, %ebp
and     $0xffffffff0, %esp
sub     $0x20, %esp
movl    $0x80000000, 0x1c(%esp)
movl    $0x80000000, 0x18(%esp)
```

```
19: 8b 44 24 1c
1d: d1 f8
1f: 89 44 24 04
23: c7 04 24 00 00 00 00
2a: e8 fc ff ff ff
2f: 8b 44 24 18
33: d1 e8
35: 89 44 24 04
39: c7 04 24 0b 00 00 00
40: e8 fc ff ff ff
45: c9
46: c3
```

```
mov     0x1c(%esp), %eax
sar     %eax
mov     %eax, 0x4(%esp)
movl    $0x0, (%esp)
call    2b <main+0x2b>
mov     0x18(%esp), %eax
shr     %eax
mov     %eax, 0x4(%esp)
movl    $0xb, (%esp)
call    41 <main+0x41>
leave
ret
```

算术

逻辑

IA-32常用指令类型

(4) 控制转移指令

指令执行可**按顺序** 或 **跳转到转移目标指令处**执行

- **无条件转移指令**

JMP DST: 无条件转移到目标指令DST处执行

- **条件转移**

Jcc DST: cc为条件码, 根据标志 (条件码) 判断是否满足条件, 若满足, 则转移到目标指令DST处执行, 否则按顺序执行

- **条件设置**

SETcc DST: 将条件码cc保存到DST (通常是一个8位寄存器)

- **调用和返回指令 (用于过程调用)**

CALL DST: **返回地址RA**入栈, 转DST处执行

RET: 从栈中取出返回地址RA, 转到RA处执行

- **中断指令 (详见第7、8章)**

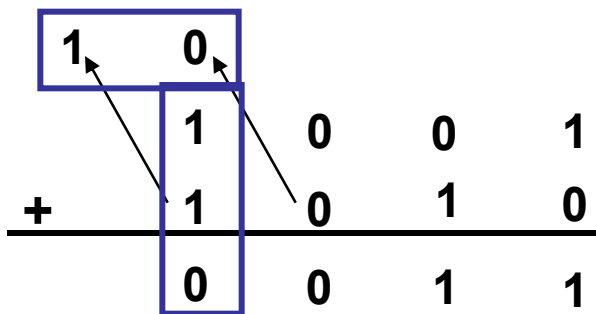
以上内容不要死记硬背, 遇到具体指令时能查阅到并理解即可。



标志信息是干什么的？

Ex1: $-7 - 6 = -7 + (-6) = +3$

$$9 - 6 = 3$$



OF=1、ZF=0

SF=0、借位CF=0

做减法以比较大小，规则：

Unsigned: CF=0时，大于

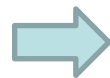
Signed: OF=SF时，大于

分三类:

(1)根据单个标志的值转移

(2)按无符号整数比较转移

(3)按带符号整数比较转移



序号	指令	转移条件	说明
1	jc label	CF=1	有进位/借位
2	jnc label	CF=0	无进位/借位
3	je/jz label	ZF=1	相等/等于零
4	jne/jnz label	ZF=0	不相等/不等于零
5	js label	SF=1	是负数
6	jns label	SF=0	是非负数
7	jo label	OF=1	有溢出
8	jno label	OF=0	无溢出
9	ja/jnbe label	CF=0 AND ZF=0	无符号整数 $A > B$
10	jae/jnb label	CF=0 OR ZF=1	无符号整数 $A \geq B$
11	jb/jnae label	CF=1 AND ZF=0	无符号整数 $A < B$
12	jbe/jna label	CF=1 OR ZF=1	无符号整数 $A \leq B$
13	jg/jnle label	SF=OF AND ZF=0	带符号整数 $A > B$
14	jge/jnl label	SF=OF OR ZF=1	带符号整数 $A \geq B$
15	jl/jnge label	SF $\neq$ OF AND ZF=0	带符号整数 $A < B$
16	jle/jng label	SF $\neq$ OF OR ZF=1	带符号整数 $A \leq B$

条件转移指令举例

```
int sum(int a[ ], unsigned len)
{
    int i, sum = 0;
    for (i = 0; i <= len-1; i++)
        sum += a[i];
    return sum;
}
```

当参数len为0时，返回值应该是0，但是在机器上执行时，却发生了存储器访问异常。 **Why?**

i 和 len 分别存放在哪个寄存器中？ %eax？ %edx？

sum:

...

.L3:

...

movl -4(%ebp), %eax

movl 12(%ebp), %edx

subl \$1, %edx

cmpl %edx, %eax

jbe .L3

...

i 在%eax中， len在%edx中

%eax: 0000 0000

%edx: 0000 0000

subl 指令的执行结果是什么？

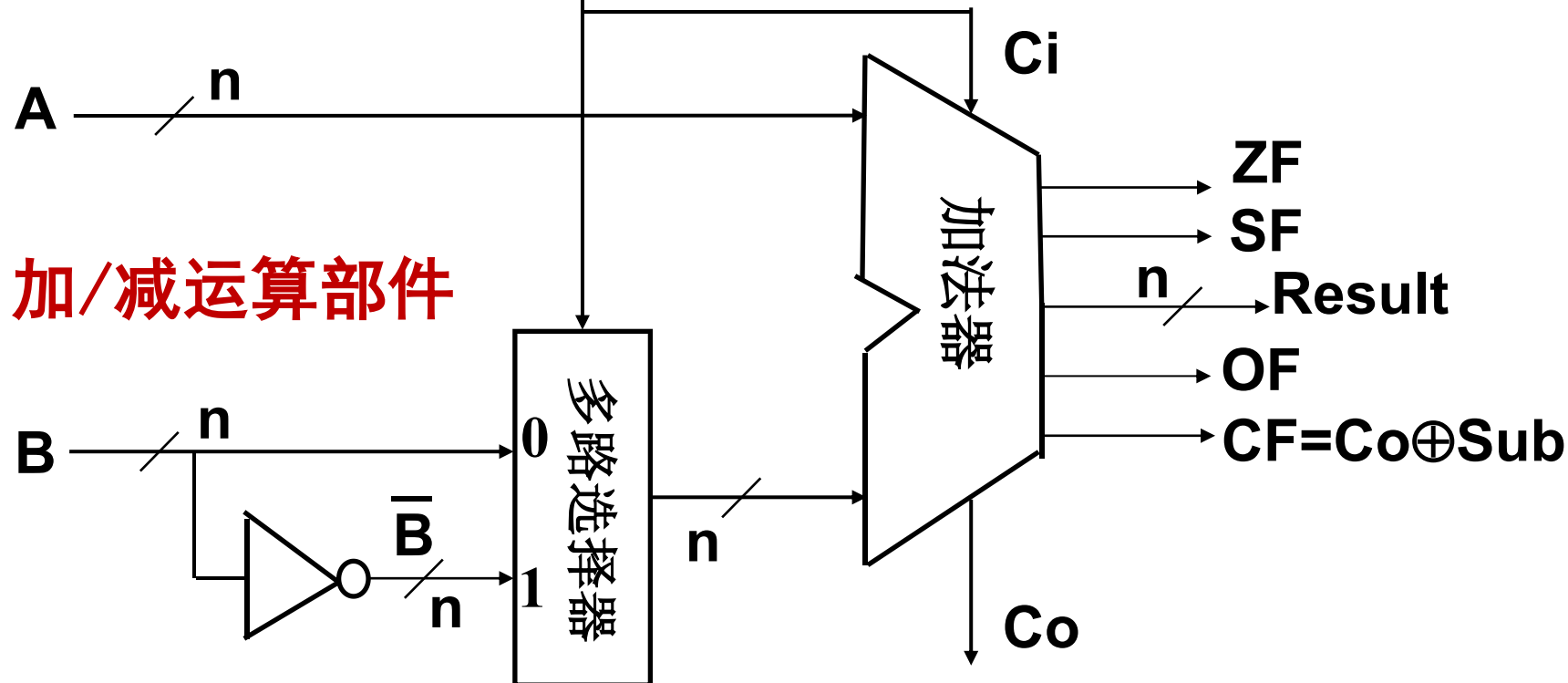
cmpl 指令的执行结果是什么？

subl \$1, %edx指令的执行结果

当Sub为1时，做减法
当Sub为0时，做加法

Sub

已知EDX中为 len=0000 0000H



“`subl $1, %edx`” 执行时：A=0000 0000H，B为00000001H，Sub=1，因此Result是32个1。

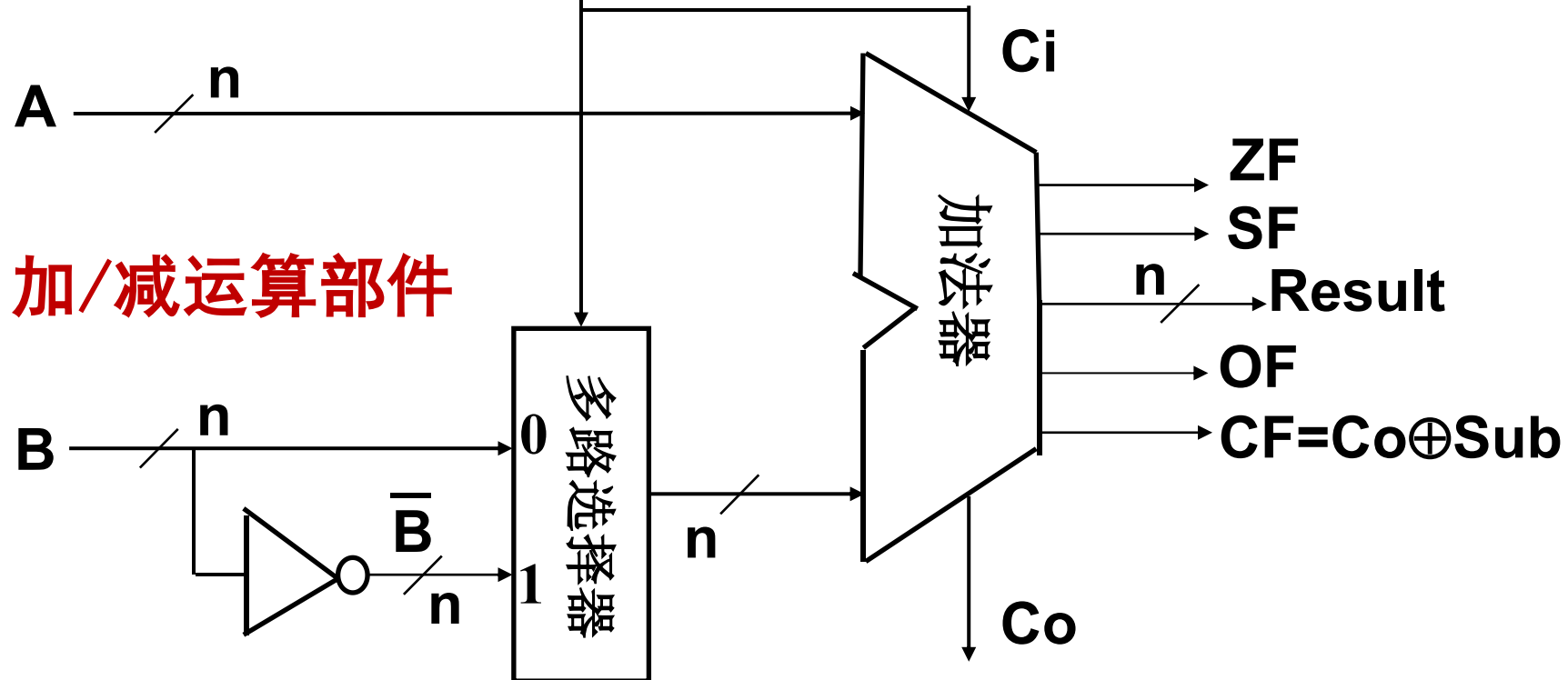
cmpl %edx,%eax指令的执行结果

当Sub为1时，做减法
当Sub为0时，做加法

Sub

已知EDX中为 len-1=FFFF FFFFH

EAX中为 i=0000 0000H



“cmpl %edx,%eax” 执行时: A=0000 0000H, B为FFFF FFFFH
Sub=1, 因此Result是0...01, CF=1, ZF=0, OF=0, SF=0

jbe .L3指令的执行结果

指令	转移条件	说明
JA/JNBE label	CF=0 AND ZF=0	无符号数A > B
JAE/JNB label	CF=0 OR ZF=1	无符号数A ≥ B
JB/JNAE label	CF=1 AND ZF=0	无符号数A < B
JBE/JNA label	CF=1 OR ZF=1	无符号数A ≤ B
JG/JNLE label	SF=OF AND ZF=0	有符号数A > B
JGE/JNL label	SF=OF OR ZF=1	有符号数A ≥ B
JL/JNGE label	SF≠OF AND ZF=0	有符号数A < B
JLE/JNG label	SF≠OF OR ZF=1	有符号数A ≤ B

“**cmpl %edx,%eax**” 执行结果是 **CF=1, ZF=0, OF=0, SF=0**
说明满足条件，应转移到.L3执行！ 显然，对于每个 i 都满足条件，因为任何无符号数都比32个1小，因此循环体被不断执行，最终导致数组访问越界而发生存储器访问异常。

条件转移指令举例

例:

```
int sum(int a[ ], int len)
{
    int i, sum = 0;
    for (i = 0; i <= len-1; i++)
        sum += a[i];
    return sum;
}
```

正确的做法是将参数len声明为int型。 **Why?**

i 和 len 分别存放在哪个寄存器中? %eax? %edx?

sum:

...

.L3:

...

movl -4(%ebp), %eax

movl 12(%ebp), %edx

subl \$1, %edx

cmpl %edx, %eax

jle .L3

...

i 在%eax中, len在%edx中

%eax: 0000 0000

%edx: 0000 0000

subl 指令的执行结果是什么?

cmpl 指令的执行结果是什么?

jle .L3指令的执行结果

指令	转移条件	说明
JA/JNBE label	CF=0 AND ZF=0	无符号数 $A > B$
JAE/JNB label	CF=0 OR ZF=1	无符号数 $A \geq B$
JB/JNAE label	CF=1 AND ZF=0	无符号数 $A < B$
JBE/JNA label	CF=1 OR ZF=1	无符号数 $A \leq B$
JG/JNLE label	SF=OF AND ZF=0	有符号数 $A > B$
JGE/JNL label	SF=OF OR ZF=1	有符号数 $A \geq B$
JL/JNGE label	SF $\neq$ OF AND ZF=0	有符号数 $A < B$
JLE/JNG label	SF $\neq$ OF OR ZF=1	有符号数 $A \leq B$

“cml %edx,%eax” 执行结果是 CF=1, ZF=0, OF=0, SF=0
说明不满足条件，应跳出循环执行，执行结果正常。

条件设置指令举例

unsigned long long

↑

long long

↑

unsigned

↑

int

↑

(unsigned)char,short

```
#include <stdio.h>
void main()
{
    unsigned int a = 1;
    unsigned short b = 1;
    char c = -1;
    int d;

    d = (a > c) ? 1:0;
    printf("%d\n",d);
    d = (b > c) ? 1:0;
    printf("%d\n",d);
}
```

条件设置指令:

猜测: 各用哪种条件设置指令?

SETcc DST: 将条件码cc保存到DST (通常是一个8位寄存器)

```

0804841c <main>:
804841c: 55          push    %ebp
804841d: 89 e5       mov     %esp,%ebp
804841f: 83 e4 f0    and     $0xfffffffff0,%esp
8048422: 83 c0 20    sub     $0x20,%esp
8048425: c7          unsigned int a=1; → movl    $0x1,0x1c(%esp)
804842c: 0f
804842d: 6b          unsigned short b=1; → movw    $0x1,0x1a(%esp)
8048434: c6 44 24 1  char c=-1; → movb    $0xff,0x19(%esp)
8048439: 0f be 44 2  d=(a>c)?1:0 → movsbl  0x19(%esp),%eax
804843e: 3b 44 24    cmp     0x1c(%esp),%eax 无符号
8048442: 0f 92 c0    setb    %al
8048445: 0f b6 c0    movzbl  %al,%eax
8048448: 89 44 24 14 mov     %eax,0x14(%esp)
804844c: 8b 44 24 14 mov     0x14(%esp),%eax
8048450: 89 44 24 04 mov     %eax,0x4(%esp)
8048454: c7 04 24 20 85 04 08 movl    $0x8048520,(%esp)
804845b: e8 a0 fe ff ff call    8048300 <printf@plt>
8048460: 0f b7 54 24 1a movzwl  0x1a(%esp),%edx
8048465: 0f be 44 2  d=(b>c)?1:0 → movsbl  0x19(%esp),%eax 带符号
804846a: 39 c2       cmp     %eax,%edx
804846c: 0f 9f c0    setg    %al
804846f: 0f b6 c0    movzbl  %al,%eax
8048472: 89 44 24 14 mov     %eax,0x14(%esp)
8048476: 8b 44 24 14 mov     0x14(%esp),%eax
804847a: 89 44 24 04 mov     %eax,0x4(%esp)
804847e: c7 04 24 20 85 04 08 movl    $0x8048520,(%esp)
8048485: e8 76 fe ff ff call    8048300 <printf@plt>
804848a: c9         leave
804848b: c3         ret

```

X87浮点指令、MMX和SSE指令

- IA-32的浮点处理架构有两种：
 - 浮点协处理器x87架构 (x87 FPU)
 - ✓ 8个80位寄存器ST(0) ~ ST(7) (采用栈结构) , 栈顶为ST(0)
 - 由MMX发展而来的SSE架构
 - ✓ MMX指令使用8个64位寄存器MM0~MM7, 借用8个80位寄存器ST(0)~ST(7)中64位尾数所占的位, 可同时处理8个字节, 或4个字, 或2个双字, 或一个64位的数据, 即SIMD (单指令多数据, 也称数据级并行) 技术
 - ✓ MMX指令并没带来3D游戏性能的显著提升, 故相继推出SSE指令集, 它们都采用SIMD
 - ✓ SSE指令集将80位浮点寄存器扩充到128位多媒体扩展通用寄存器XMM0~XMM7, 可同时处理16个字节, 或8个字, 或4个双字 (32位整数或单精度浮点数), 或两个四字的数据, 而且从SSE2开始, 还支持128位整数运算或同时并行处理两个64位双精度浮点数

IA-32中通用寄存器中的编号

编号	8 位寄存器	16 位寄存器	32 位寄存器	64/80位寄存器	128 位寄存器
000	AL	AX	EAX	MM0 / ST(0)	XMM0
001	CL	CX	ECX	MM1 / ST(1)	XMM1
010	DL	DX	EDX	MM2 / ST(2)	XMM2
011	BL	BX	EBX	MM3 / ST(3)	XMM3
100	AH	SP	ESP	MM4 / ST(4)	XMM4
101	CH	BP	EBP	MM5 / ST(5)	XMM5
110	DH	SI	ESI	MM6 / ST(6)	XMM6
111	BH	DI	EDI	MM7 / ST(7)	XMM7

反映了体系结构发展的轨迹，字长不断扩充，指令保持兼容

ST (0) ~ ST (7) 是80位，MM0 ~MM7使用其低64位

SSE指令（SIMD操作）

●用简单的例子来比较普通指令与数据级并行指令的执行速度

- ✓为使比较结果不受访存操作影响，下例中的运算操作数在寄存器中
- ✓为使比较结果尽量准确，例中设置的循环次数较大: $0x4000000 = 2^{26}$
- ✓例子只是为了说明指令执行速度的快慢，并没有考虑结果是否溢出

以下是普通指令写的程序

080484f0 <dummy_add>:

所用时间约为22.643816s

```
80484f0: 55          push %ebp
80484f1: 89 e5       mov %esp, %ebp
80484f3: b9 00 00 00 04 mov $0x4000000, %ecx
80484f8: b0 01       mov $0x1, %al
80484fa: b3 00       mov $0x0, %bl
80484fc: 00 c3       add %al, %bl
80484fe: e2 fc       loop 80484fc <dummy_add+0xc>
8048500: 5d          pop %ebp
8048501: c3          ret
```

循环400 0000H= 2^{26} 次，每次只有一个数（字节）相加

SSE指令 (SIMD操作)

以下是SIMD指令写的程序

所用时间约为1.411588s

08048510 <dummy_add_sse>:

```
8048510: 55                push %ebp
8048511: b8 00 9d 04 10    mov $0x10049d00, %eax
8048516: 89 e5             mov %esp, %ebp
8048518: 53                push %ebx
8048519: bb 20 9d 04 14    mov $0x14049d20, %ebx
804851e: b9 00 00 40 00    mov $0x400000, %ecx
8048523: 66 0f 6f 00       movdqa (%eax), %xmm0
8048527: 66 0f 6f 0b       movdqa (%ebx), %xmm1
804852b: 66 0f fc c8       paddb %xmm0, %xmm1
804852f: e2 fa             loop 804852b <dummy_add_sse+0x1b>
8048531: 5b                pop %ebx
8048532: 5d                pop %ebp
8048533: c3                ret
```

22.643816s/
1.411588s
≈16.041378,与
预期结果一致!
SIMD指令并行
执行效率高!

} SIMD指令

循环400000H=2²²次, 每次同时有128/8=16个数 (字节) 相加

第一、二讲总结

- 高级语言程序总是转换为机器代码才能在机器上执行
- 转换过程：预处理、编译、汇编、链接
- 机器代码是二进制代码，可DUMP为汇编代码表示
- ISA规定了一台机器的指令系统涉及到的所有方面，例如：
 - 所有指令的指令格式、功能
 - 通用寄存器的个数、位数、编号和功能
 - 存储地址空间大小、编址方式、大/小端
 - 指令寻址方式
- IA-32是典型的CISC（复杂指令集计算机）风格ISA
 - Intel格式汇编、AT&T格式汇编（本课程使用）
 - 指令类型（传送、算术、位操作、控制、浮点、...）
 - 寻址方式
 - 立即、寄存器、存储器 ($SR:[B] + [I]*s + A$)

程序的机器级表示

- 分以下六个部分介绍

- 第一讲：程序转换概述

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- 第二讲：IA-32 /x86-64指令系统

- 第三讲：C语言程序的机器级表示

- 过程调用的机器级表示
- 选择语句的机器级表示
- 循环结构的机器级表示

- 第四讲：复杂数据类型的分配和访问

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- 第五讲：越界访问和缓冲区溢出

- 第六讲：x86-64架构（自学）

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

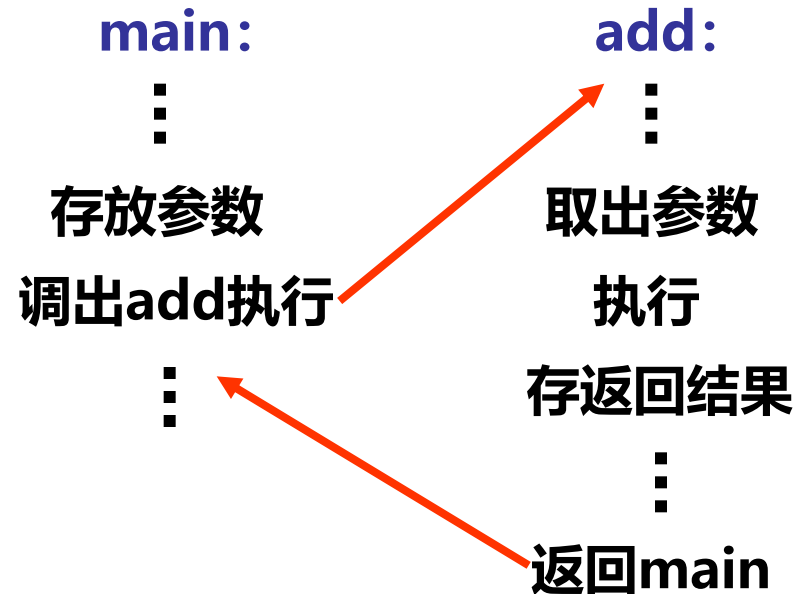
围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

过程调用的机器级表示

- 以下过程（函数）调用对应的机器级代码是什么？
- 如何将t1(125)、t2(80)分别传递给add中的形式参数x、y
- add函数执行的结果如何返回给main？

```
int add ( int x, int y ) {  
    return x+y;  
}  
  
int main ( ) {  
    int t1 = 125;  
    int t2 = 80;  
    int sum = add (t1, t2);  
    return sum;  
}
```

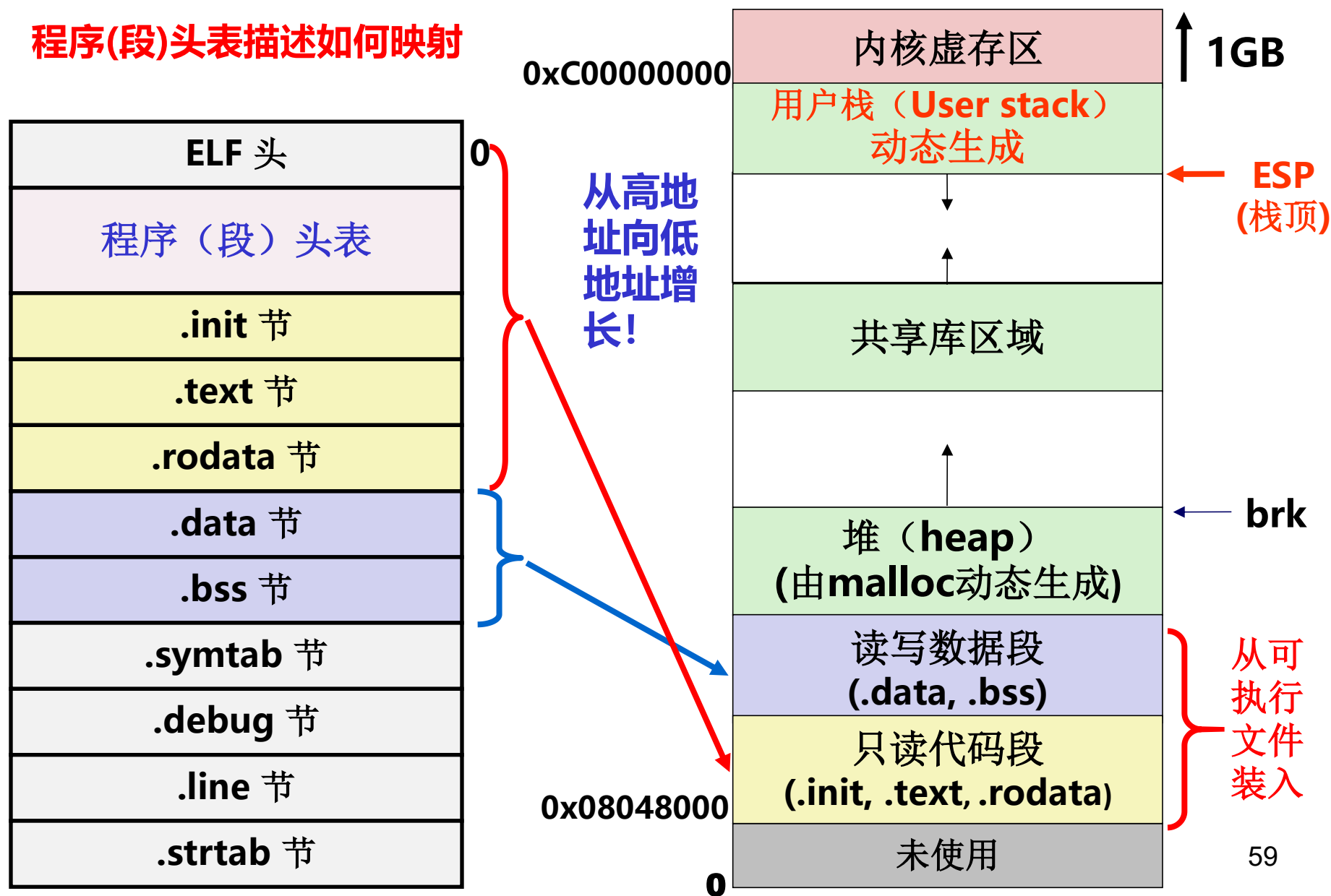
add
↑
main



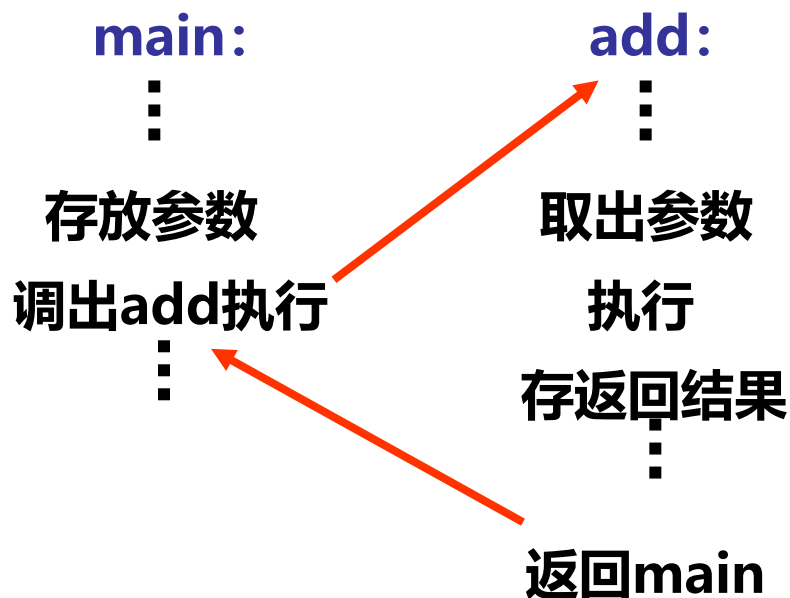
- 参数通过栈（stack）来传递！
为什么一定要用栈？ 栈在哪里？

可执行文件的存储器映像

程序(段)头表描述如何映射



过程调用的机器级表示



何为现场?

通用寄存器的内容!

为何要保存现场?

因为所有过程共享一套通用寄存器

过程调用的执行步骤(P为调用者, Q为被调用者)

- (1) P保存现场 (必要时);
 - (2) 将入口参数 (实参) 放到Q能访问到的地方;
 - (3) P保存返回地址, 然后将控制转移到Q; CALL指令
 - (4) Q保存P的现场, 并为自己的非静态局部变量分配空间;
 - (5) 执行Q的过程体 (函数体);
 - (6) Q恢复P的现场, 释放局部变量空间;
 - (7) Q取出返回地址, 将控制转移到P。RET指令
- 准备阶段: P过程, P的栈帧
- 处理阶段: Q过程, Q栈帧
- 结束阶段: Q过程, Q栈帧

过程调用的机器级表示

- **IA-32的寄存器使用约定**

- **EBP和ESP分别是帧指针寄存器和栈指针寄存器，分别用来指向当前栈帧的底部和顶部。**

- **调用者保存寄存器：EAX、EDX、ECX**

当过程P调用过程Q时，Q可以直接使用这三个寄存器，不用将它们的值保存到栈中。如果P在从Q返回后还要用这三个寄存器的话，P应在转到Q之前先保存，并在从Q返回后先恢复它们的值再使用。

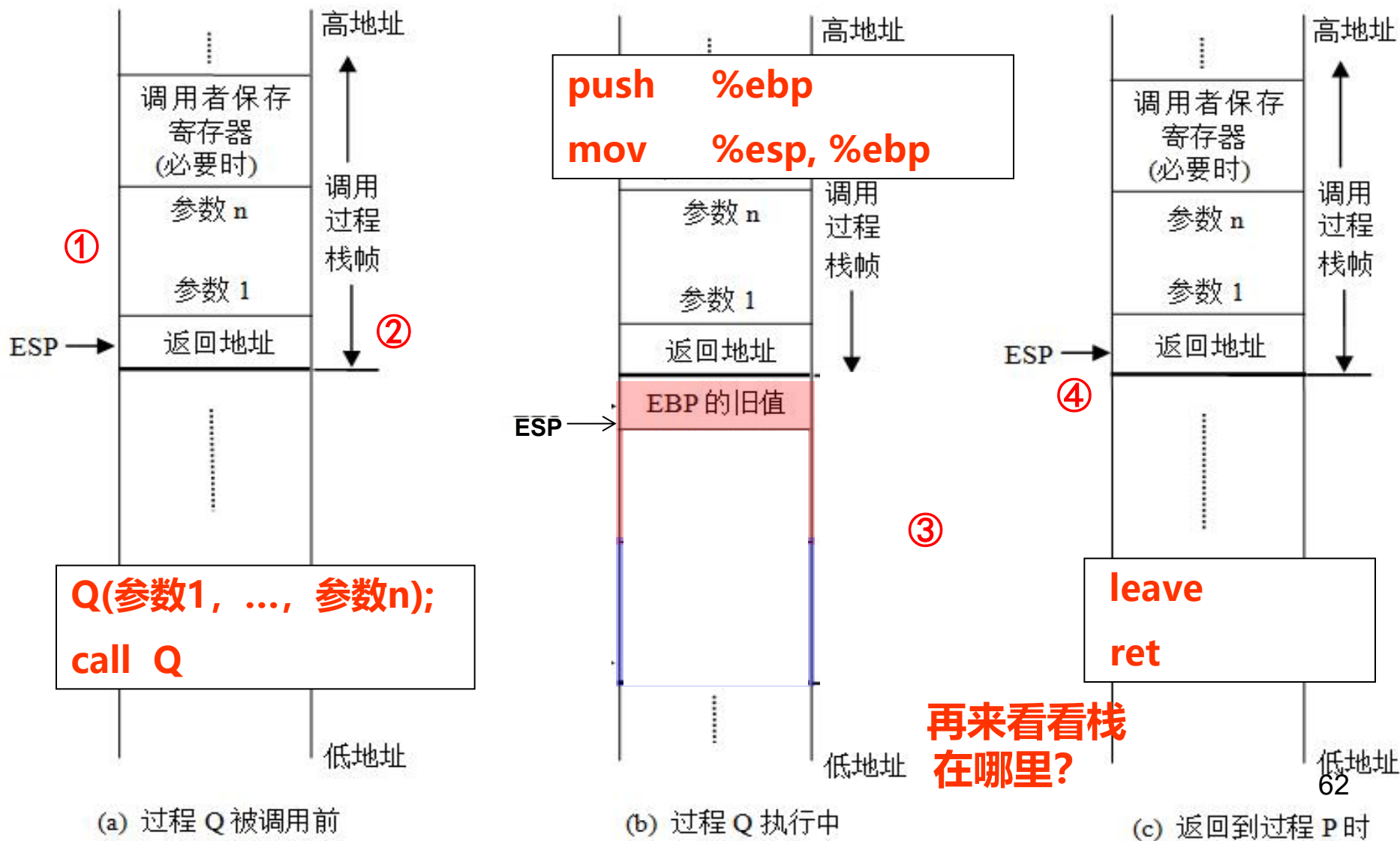
- **被调用者保存寄存器：EBX、ESI、EDI**

Q必须先将它们的值保存到栈中再使用它们，并在返回P之前恢复它们的值。

Q：为减少准备和结束阶段的开销，每个过程应先使用哪些寄存器？
EAX、ECX、EDX！

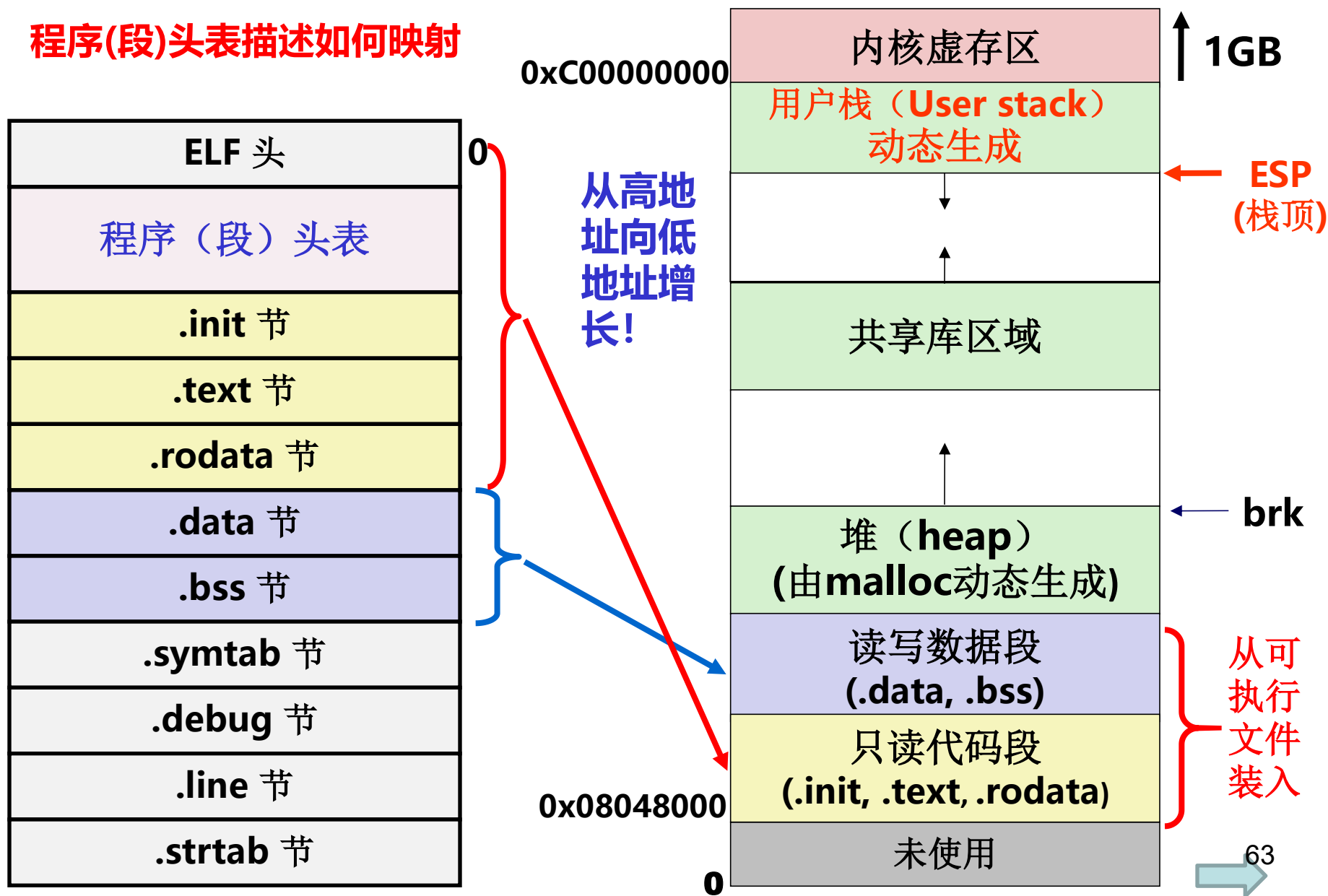
过程调用的机器级表示

- 过程调用过程中**栈和栈帧**的变化 (P和Q为调用与被调用过程)

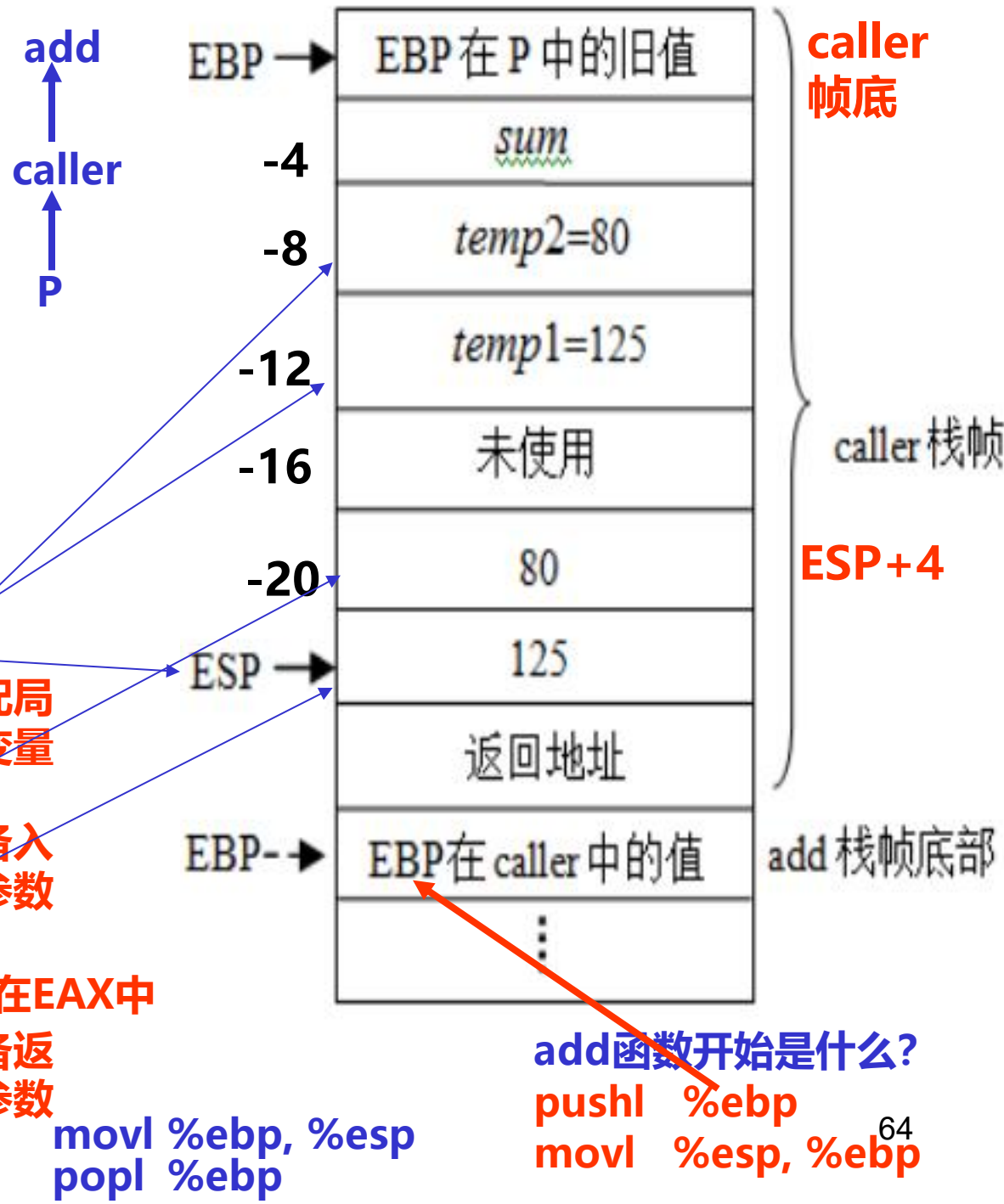


可执行文件的存储器映像

程序(段)头表描述如何映射



```
int add ( int x, int y ) {
    return x+y;
}
int caller ( ) {
    int  t1 = 125;
    int  t2 = 80;
    int  sum = add (t1, t2);
    return sum;
}
```



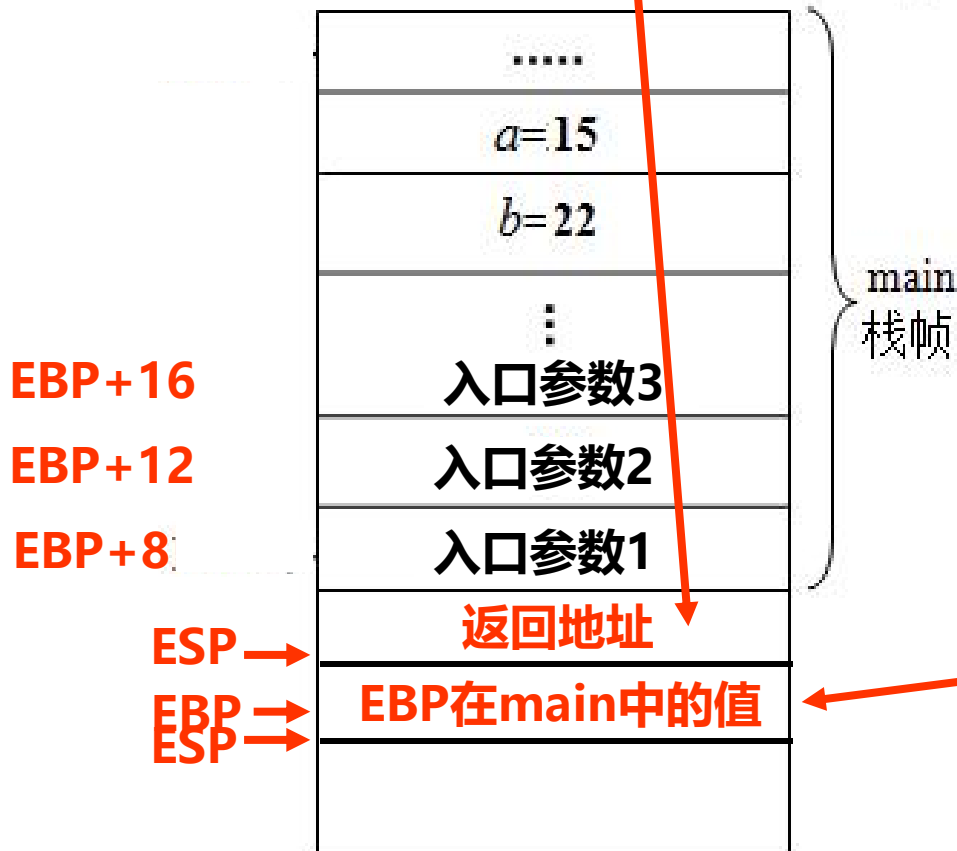
add函数开始是什么?
pushl %ebp
movl %esp, %ebp

过程（函数）的结构

- 一个C过程的大致结构如下：
 - 准备阶段
 - 形成帧底：push指令 和 mov指令
 - 生成栈帧（如果需要的话）：sub指令 或 and指令
 - 保存现场（如果有被调用者保存寄存器）：mov指令
 - 过程（函数）体
 - 分配局部变量空间，并赋值
 - 具体处理逻辑，如果遇到函数调用时
 - 准备参数：将实参送栈帧入口参数处
 - CALL指令：保存返回地址并转被调用函数
 - 在EAX中准备返回参数
 - 结束阶段
 - 退栈：leave指令 或 pop指令
 - 取返回地址返回：ret指令

入口参数的位置

```
movl 参数3, 8(%esp) } 准备  
.....             } 入口  
movl 参数1, (%esp)   } 参数  
call add R[esp]←R[esp]-4  
          M[R[esp]]←返回地址  
          R[eip]←add函数首地址
```



返回地址是什么?

call指令的下一条指令的地址!

- IA-32中, 若参数类型是 unsigned char、char或 unsigned short、short, 也都分配4个字节
- 故在被调用函数中, 使用 R[ebp]+8、R[ebp]+12、R[ebp]+16作为有效地址来访问函数的入口参数
- 每个过程开始两条指令

```
pushl %ebp  
movl %esp, %ebp
```

过程调用参数传递举例

程序一

```
#include <stdio.h>
main ( )
{
    int a=15, b=22;
    printf ("a=%d\tb=%d\n", a, b);
    swap (&a, &b);
    printf ("a=%d\tb=%d\n", a, b);
}
swap (int *x, int *y )
{
    int t=*x;
    *x=*y;
    *y=t;
}
```

按地址传递参数

执行结果？为什么？

程序一的输出：

a=15 b=22
a=22 b=15

程序二

```
#include <stdio.h>
main ( )
{
    int a=15, b=22;
    printf ("a=%d\tb=%d\n", a, b);
    swap (a, b);
    printf ("a=%d\tb=%d\n", a, b);
}
swap (int x, int y )
{
    int t=x;
    x=y;
    y=t;
}
```

按值传递参数

程序二的输出：

a=15 b=22
a=15 b=22

过程调用参数传递举例

按地址传递参数 swap (&a, &b)

main:

leal -8(%ebp), %eax

movl %eax, 4(%esp)

leal -4(%ebp), %eax

movl %eax, (%esp)

call swap

.....

ret

swap:

pushl %ebp

movl %esp, %ebp

pushl %ebx **EBX是被调用者保存**

movl 8(%ebp), %edx

movl (%edx), %ecx

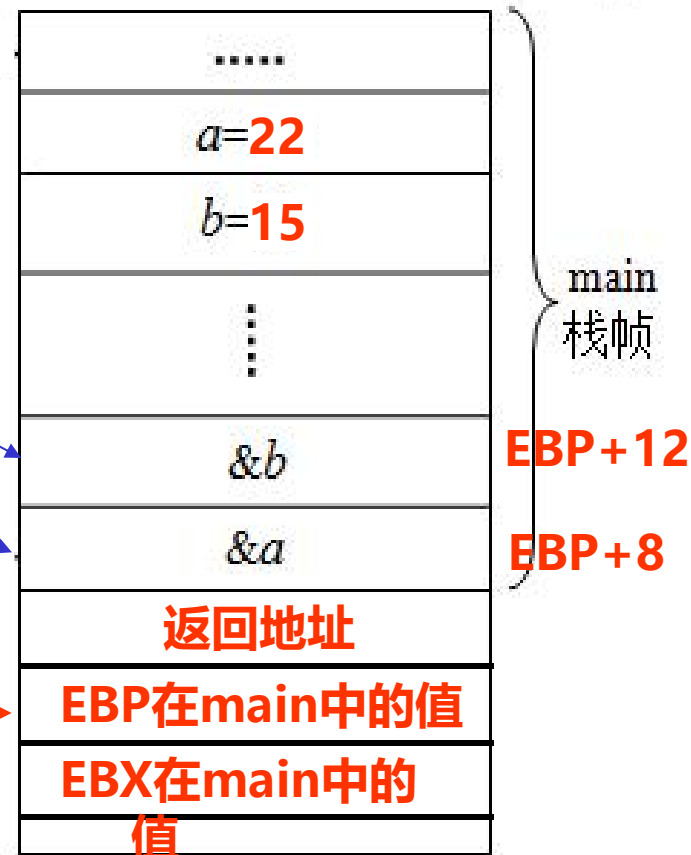
movl 12(%ebp), %eax

movl (%eax), %ebx

movl %ebx, (%edx)

movl %ecx, (%eax)

```
int t=*x;
*x=*y;
*y=t;
```



$R[ecx] \leftarrow M[&a] = 15$

$R[ebx] \leftarrow M[&b] = 22$

$M[&a] \leftarrow R[ebx] = 22$

$M[&b] \leftarrow R[ecx] = 15$

局部变量a和b确实交换

过程调用参数传递举例

按值传递参数 swap (a, b)

main:

movl -8(%ebp), %eax

movl %eax, 4(%esp)

movl -4(%ebp), %eax

movl %eax, (%esp)

call swap

.....

ret

swap:

pushl %ebp

movl %esp, %ebp

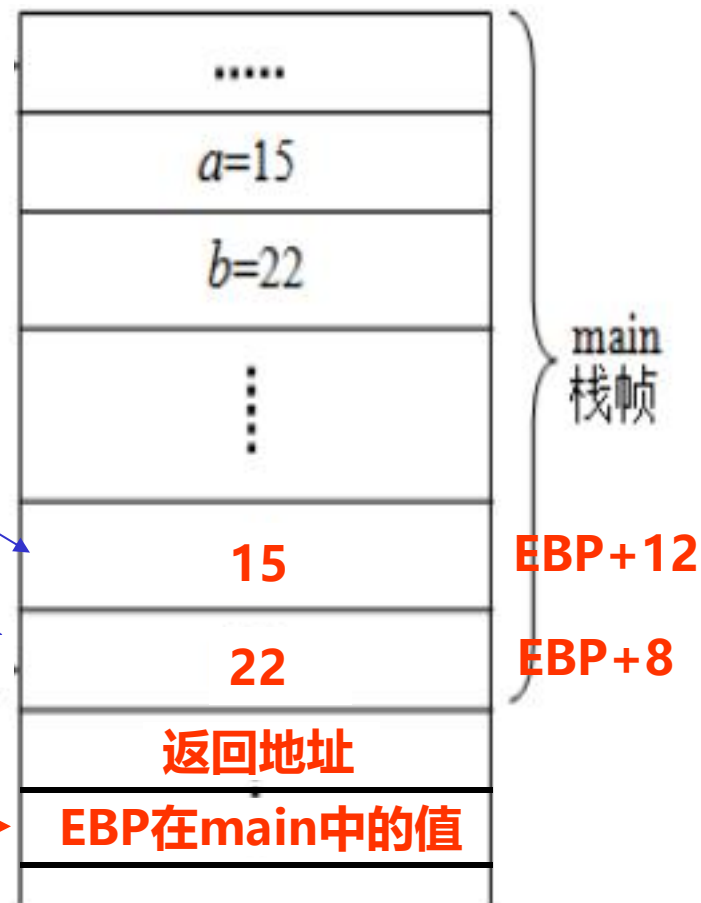
movl 8(%ebp), %edx

movl 12(%ebp), %eax

movl %eax, 8(%ebp)

movl %edx, 12(%ebp)

```
int t=x;  
x=y;  
y=t;
```



EBP →

$R[edx] \leftarrow 15$

$R[eax] \leftarrow 22$

$M[R[ebp]+8] \leftarrow R[eax] = 22$

$M[R[ebp]+12] \leftarrow R[edx] = 15$

局部变量a和b没有交换，
交换的仅是入口参数

过程调用

```
1 void test ( int x, int *ptr )
2 {
3     if ( x>0 && *ptr>0 )
4         *ptr+=x;
5 }
6
7 void caller (int a, int y )
8 {
9     int x = a>0 ? a : a+100;
10    test (x, &y);
11 }
```

100 200

test

caller

P

则函数返回400

若return x+y;

&y: 300

&a: 100

返址

EBP 旧值

.....

&y

x=100

返址

EBP 旧值

.....

EBP →

ESP →

P

caller

调用caller的过程为P，P中给出形参a和y的
实参分别是100和200，画出相应栈帧中的状态，

(1) test的形参是按值传递还是按地址传递？test的形参ptr对应的实参是一个什么类型的值？
前者按值、后者按地址。一定是一个地址

(2) test中被改变的*ptr的结果如何返回给它的调用过程caller？

第10行执行后，P帧中200变成300，test退帧后，caller中通过y引用该值300

(3) caller中被改变的y的结果能否返回给过程P？为什么？

第11行执行后caller退帧并返回P，因P中无变量与之对应，故无法引用该值300

练习

```
void test ( int x, int *ptr )  
{  
    int y = 10;  
    *ptr += x + y;  
}
```

```
void caller ( )  
{  
    int x = 10, y = 20;  
    test (x, &y);  
}
```

请画出：

- 1) caller调用test前后相应栈帧中的状态；
- 2) test调用结束后相应栈帧中的状态。

```
int nn_sum ( int n)
{
```

```
    int result;
```

```
    if (n<=0 )
```

```
        result=0;
```

```
    else
```

```
        result=n+nn_sum(n-1);
```

```
    return result;
```

```
}
```

```
pushl %ebp
```

```
movl %esp, %ebp
```

```
pushl %ebx
```

```
subl $4, %esp
```

```
movl 8(%ebp), %ebx
```

```
movl $0, %eax
```

```
cmpl $0, %ebx
```

```
jle .L2
```

```
leal -1(%ebx), %eax
```

```
movl %eax, (%esp)
```

```
call nn_sum
```

```
addl %ebx, %eax
```

```
.L2
```

```
addl $4, %esp
```

```
popl %ebx
```

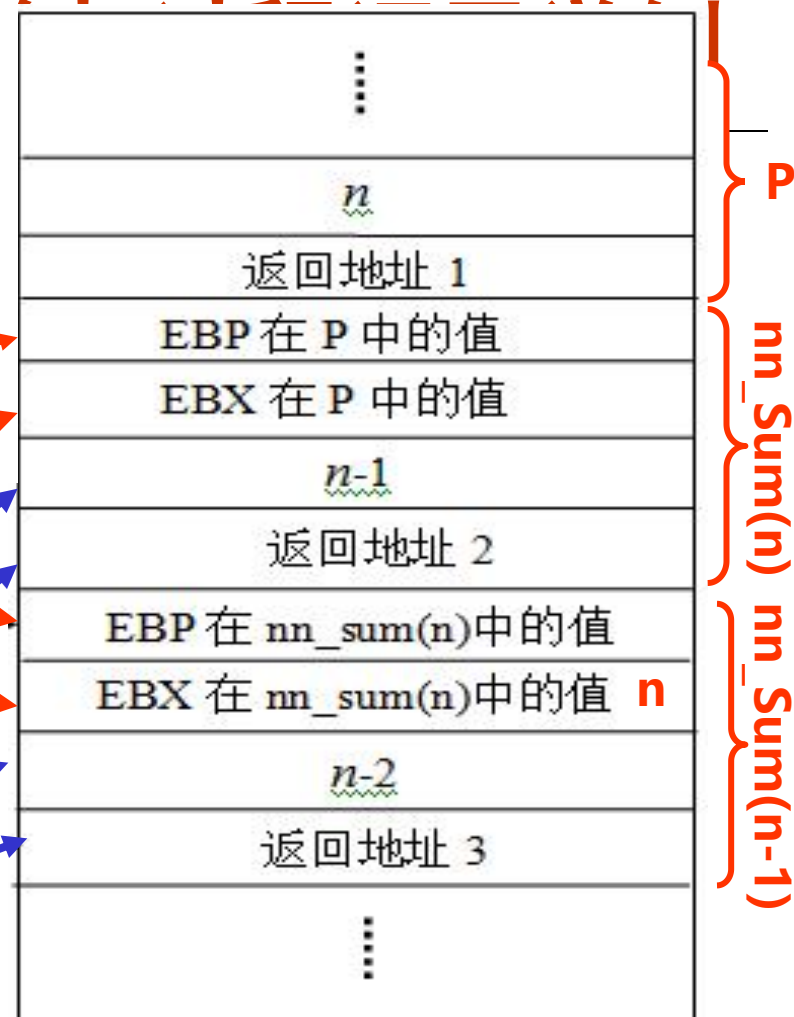
```
popl %ebp
```

```
ret
```

nn_sum(n-1)

nn_sum(n)

P



$R[ebx] \leftarrow n$

$R[eax] \leftarrow 0$

if ($n \leq 0$) 转L2

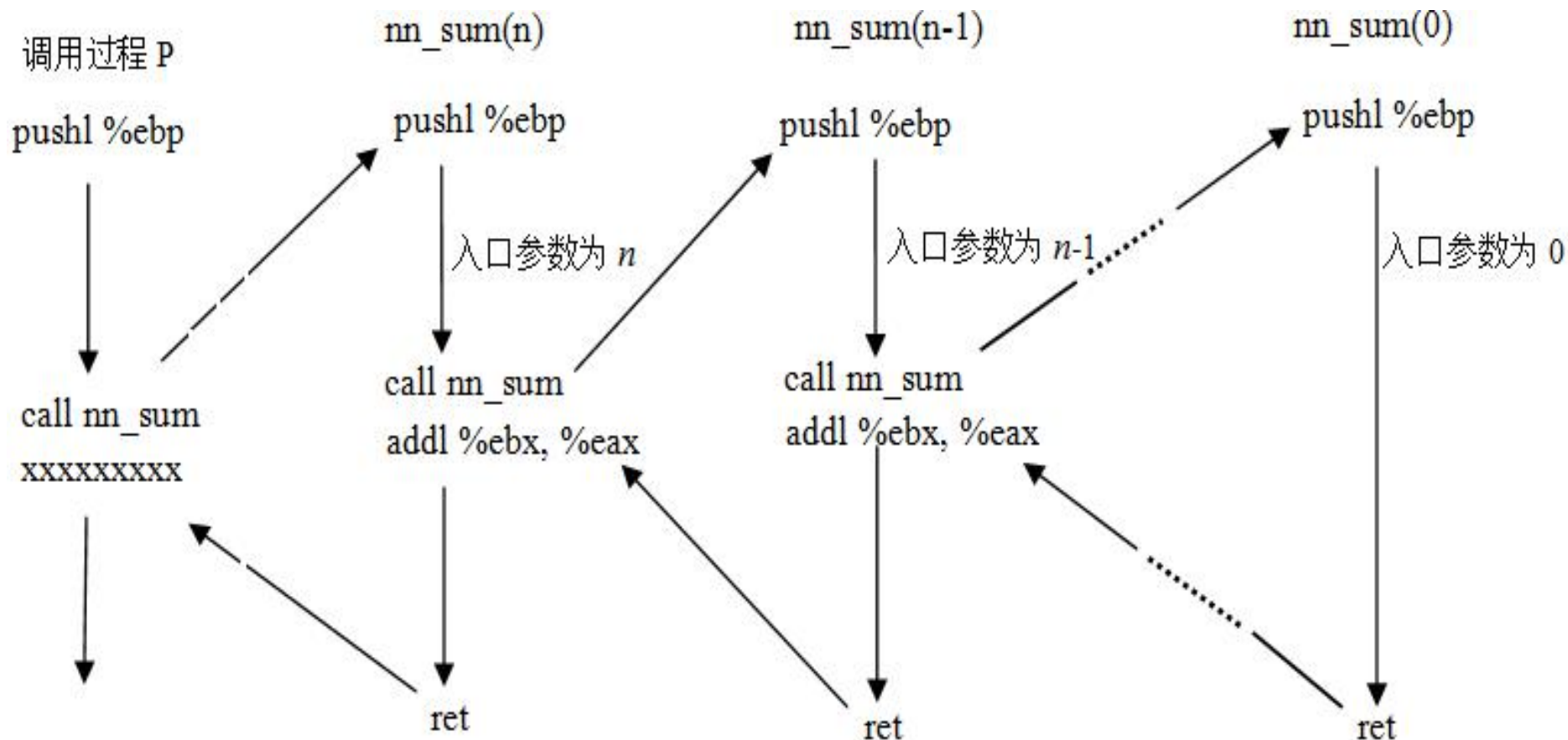
$R[eax] \leftarrow n-1$

$R[eax] \leftarrow 0+1+2+\dots+(n-1)+n$

每次递归调用都会增加一个栈帧，所以空间开销很大。

过程调用的机器级表示

- 递归函数 `nn_sum` 的执行流程



过程功能由过程体实现，为支持过程调用，每个过程包含准备阶段和结束阶段。因而每增加一次过程调用，就要增加许多条包含在准备阶段和结束阶段的额外指令，它们对程序性能影响很大，应尽量避免不必要的过程调用，特别是递归调用。

过程调用举例

例：应始终返回d[0]中的3.14，但并非如此。 **Why?**

```
double fun(int i)
{
    volatile double d[1] = {3.14};
    volatile long int a[2];
    a[i] = 1073741824; /* Possibly out of bounds */
    return d[0];
}
```

fun(0) → 3.14

fun(1) → 3.14

fun(2) → 3.1399998664856

fun(3) → 2.00000061035156

fun(4) → 3.14, 然后存储保护错

为何每次返回不一样?

为什么会引起保护错?

栈帧中的状态如何?

不同系统上执行结果可能不同

例如，编译器对局部变量分配方式可能不同

```
double fun(int i)
{
    volatile double d[1] = {3.14};
    volatile long int a[2];
    a[i] = 1073741824;
    return d[0];
}
```

当i=0或1, OK

当i=2, d3~d0=0x40000000

低位部分 (尾数) 被改变

当i=3, d7~d3=0x40000000

高位部分被改变

当i=4, EBP被改变

<fun>:

```
push    %ebp
mov     %esp,%ebp
sub     $0x10,%esp
fldl    0x8048518
fstpl   -0x8(%ebp)
```

```
mov     0x8(%ebp),%eax
movl    $0x40000000,-0x10(%ebp,%eax,4)
```

```
fldl    -0x8(%ebp)
leave
ret
```

EBP

ESP

EBP的旧值		
d7 ... d4		3
d3 ... d0		2
a[1]		1
a[0]		0

a[i]=1073741824;

0x40000000

=2³⁰=1073741824

fun(2) = 3.1399998664856

fun(3) = 2.000000061035156

fun(4) = 3.14, 然后存储保护错

选择结构的机器级表示

- if ~ else语句的机器级表示

```
c=cond_expr;  
if (!c)
```

条件转移
指令

```
    goto false_label;
```

```
    then_statement
```

```
    goto done;
```

```
false_label:
```

```
    else_statement
```

```
done:
```

无条件转
移指令

```
if (cond_expr)  
    then_statement  
else  
    else_statement
```

```
c=cond_expr;
```

```
if (c)
```

条件转移
指令

```
    goto true_label;
```

```
    else_statement
```

```
    goto done;
```

```
true_label:
```

```
    then_statement
```

```
done:
```

无条件转
移指令



If-else语句举例

```
int get_cont( int *p1, int *p2 )
{
    if ( p1 > p2 )
        return *p2;
    else
        return *p1;
}
```

p1和p2对应实参的存储地址分别为R[ebp]+8、R[ebp]+12，EBP指向当前栈帧底部，结果存放在EAX。

作业：为何这里是“jbe”指令？

```
movl 8(%ebp), %eax    //R[eax] ← M[R[ebp]+8], 即 R[eax]=p1
movl 12(%ebp), %edx    //R[edx] ← M[R[ebp]+12], 即 R[edx]=p2
cmpl %edx, %eax        //比较 p1 和 p2, 即根据 p1-p2 结果置标志
jbe .L1                //若 p1 ≤ p2, 则转 L1 处执行
movl (%edx), %eax
jmp .L2                //无条件跳转到 L2 执行
.L1:
    movl (%eax), %eax
.L2
```

switch-case语句举例

```
int sw_test(int a, int b, int c)
{
    int result;
    switch(a) {
    case 15:
        c=b&0x0f;
    case 10:
        result=c+50;
        break;
    case 12:
    case 17:
        result=b+50;
        break;
    case 14:
        result=b;
        break;
    default:
        result=a;
    }
    return result;
}
```

```
    movl 8(%ebp), %eax
    subl $10, %eax
    cmpl $7, %eax
    ja .L5
    jmp *.L8(, %eax, 4)
.L1:
    movl 12(%ebp), %eax
    andl $15, %eax
    movl %eax, 16(%ebp)
.L2:
    movl 16(%ebp), %eax
    addl $50, %eax
    jmp .L7
.L3:
    movl 12(%ebp), %eax
    addl $50, %eax
    jmp .L7
.L4:
    movl 12(%ebp), %eax
    jmp .L7
.L5:
    addl $10, %eax
.L7:
```

$R[eax] = a - 10$

if $(a - 10) > 7$ 转 L5

间接跳转指令

转 $.L8 + 4 * i$ 处的地址

跳转表在目标文件的只读节中，按4字节边界对齐。

.section	.rodata
.align 4	
.L8	a =
.long .L2	10
.long .L5	11
.long .L3	12
.long .L5	13
.long .L4	14
.long .L1	15
.long .L5	16
.long .L3	17

循环结构的机器级表示

- do~while循环的机器级表示

```
do loop_body_statement  
   while (cond_expr);
```

```
loop:  
    loop_body_statement  
    c=cond_expr;  
    if (c) goto loop;
```

红色处为条件转移指令!

- for循环的机器级表示

```
for (begin_expr; cond_expr; update_expr)  
    loop_body_statement
```

- while循环的机器级表示

```
while (cond_expr)  
    loop_body_statement
```

```
    c=cond_expr;  
    if (!c) goto done;  
loop:  
    loop_body_statement  
    c=cond_expr;  
    if (c) goto loop;  
done:
```

```
begin_expr;  
c=cond_expr;  
if (!c) goto done;  
loop:  
    loop_body_statement  
    update_expr;  
    c=cond_expr;  
    if (c) goto loop;  
done:
```

循环结构与递归的比较

递归函数nn_sum仅为说明原理，实际上可直接用公式，为说明循环的机器级表示，这里用循环实现。

```
int nn_sum ( int n)
{
    int i;
    int result=0;
    for (i=1; i <=n; i++)
        result+=i;
    return result;
}
```

```
movl 8(%ebp), %ecx
movl $0, %eax
movl $1, %edx
cmpl %ecx, %edx
jg .L2
.L1:
addl %edx, %eax
addl $1, %edx
cmpl %ecx, %edx
jle .L1
.L2
```

局部变量 i 和 result 被分别分配在EDX和EAX中。

通常复杂局部变量被分配在栈中，而这里都是简单变量

SKIP

过程体中没用到被调用过程保存寄存器。因而，该过程栈帧中仅需保留EBP，即其栈帧仅占用4字节空间，而递归方式则占用了 $(16n+12)$ 字节栈空间，多用了 $(16n+8)$ 字节，每次递归调用都要执行16条指令，一共多了n次过程调用，因而，递归方式比循环方式至少多执行了 $16n$ 条指令。由此可以看出，为了提高程序的性能，若能用非递归方式执行则最好用非递归方式。

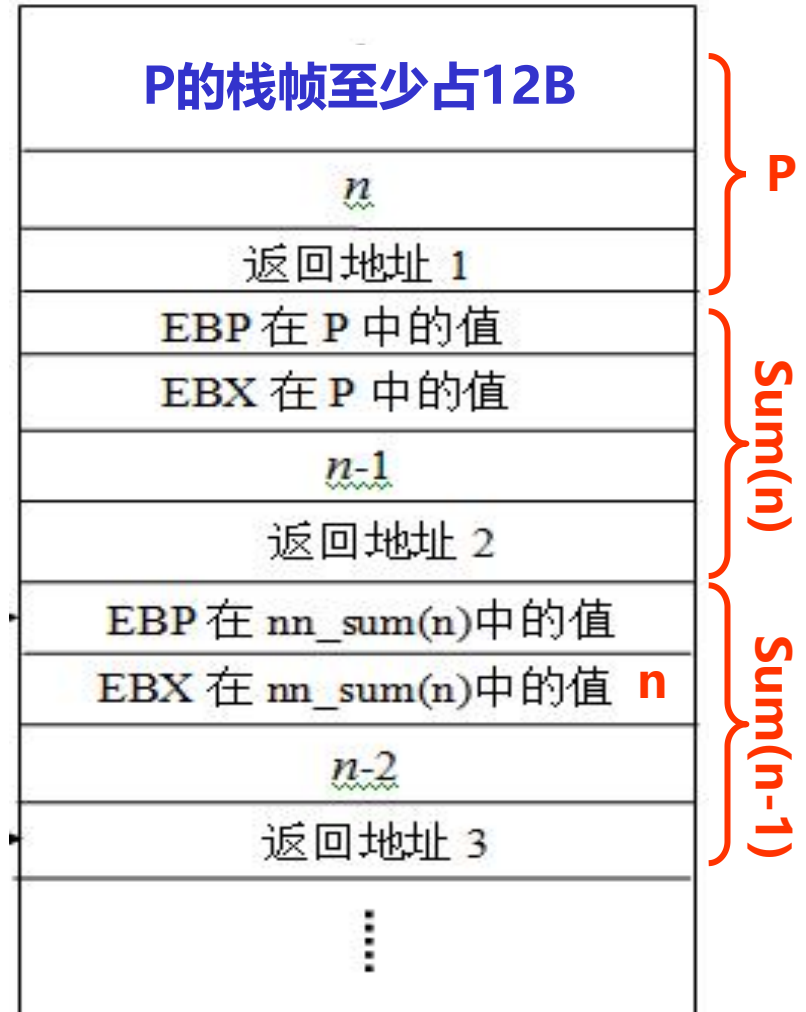
递归过程调用举例

```
int nn_sum ( int n)
{
    int result;
    if (n<=0 )
        result=0;
    else
        result=n+nn_sum(n-1);
    return result;
}
```

```
pushl    %ebp
movl     %esp, %ebp
pushl    %ebx
subl     $4, %esp
movl     8(%ebp), %ebx
movl     $0, %eax
cmpl     $0, %ebx
jle      .L2
leal     -1(%ebx), %eax
movl     %eax, (%esp)
call     nn_sum
addl     %ebx, %eax
```

BACK

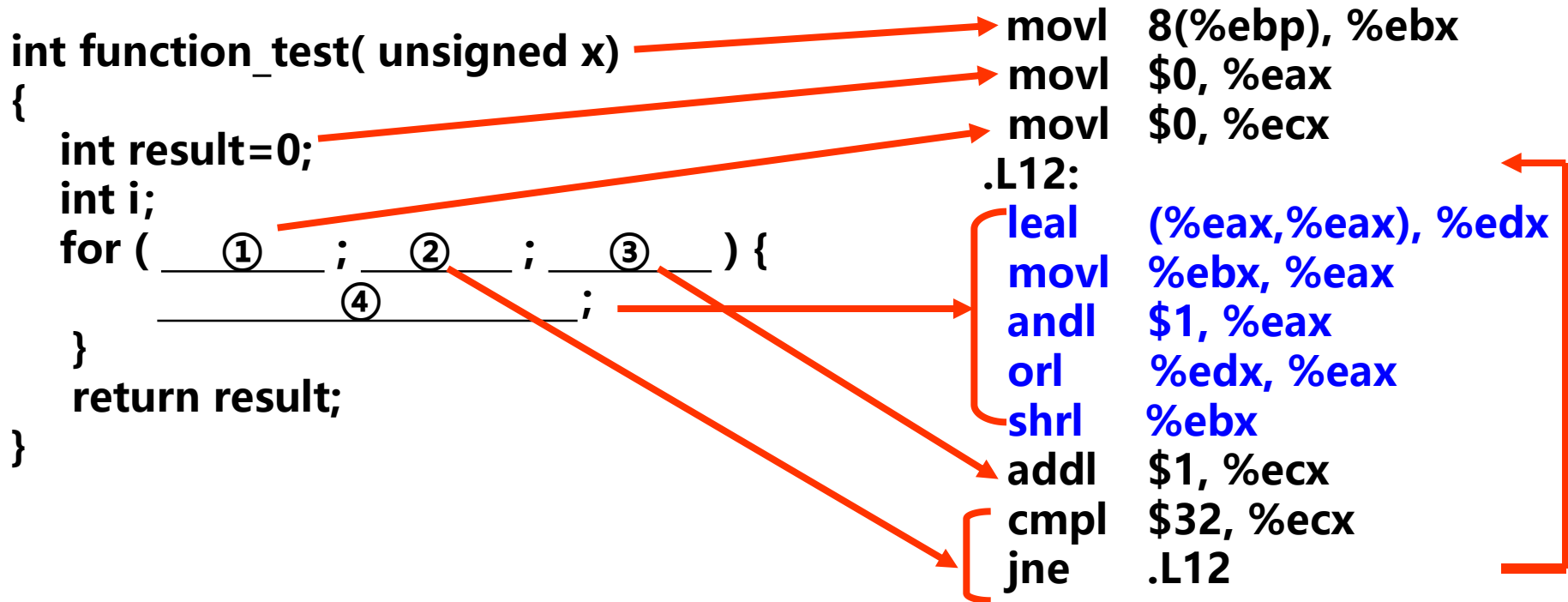
```
.L2
addl     $4, %esp
popl     %ebx
popl     %ebp
ret
```



时间开销：每次递归执行16条指令，共16n条指令

空间开销：一次调用增加16B栈帧，共16n+12

逆向工程举例



① 处为 `i=0`，② 处为 `i != 32`，③ 处为 `i++`。

入口参数 `x` 在 `EBX` 中，返回参数 `result` 在 `EAX` 中。LEA 实现 “`2*result`”，即：将 `result` 左移一位；第6和第7条指令则实现 “`x&0x01`”；第8条指令实现 “`result=(result<<1) | (x & 0x01)`”，第9条指令实现 “`x>>=1`”。综上所述，④ 处的C语言语句是 “`result=(result<<1) | (x & 0x01); x>>=1;`”。

程序的机器级表示

- 分以下六个部分介绍

- 第一讲：程序转换概述

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- 第二讲：IA-32 /x86-64指令系统

- 第三讲：C语言程序的机器级表示

- 过程调用的机器级表示
- 选择语句的机器级表示
- 循环结构的机器级表示

- 第四讲：复杂数据类型的分配和访问(自学)

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- 第五讲：越界访问和缓冲区溢出

- 第六讲：x86-64架构（自学）

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

数组的分配和访问

- 数组元素在内存的存放和访问

- 例如，定义一个具有4个元素的静态存储型 short 数据类型数组A，可以写成 “static short A[4];”
- 第 i ($0 \leq i \leq 3$) 个元素的地址计算公式为
- 假定数组A的首地址存放在EDX中， i 存放在ECX中，现要将A[i]取到AX中，则所用的汇编指令是什么？

movw (%edx, %ecx, 2), %ax 比例因子是2！

其中，ECX为**变址（索引）寄存器**，在循环体中增量

数组定义	数组名	数组元素类型	数组元素大小 (B)	数组大小 (B)	起始地址	元素 i 的地址
char S[10]	S	char	1	10	&S[0]	&S[0]+i
char * SA[10]	SA	char *	4	40	&SA[0]	&SA[0]+4*i
double D[10]	D	double	8	80	&D[0]	&D[0]+8*i
double * DA[10]	DA	double *	4	40	&DA[0]	&DA[0]+4*i

数组元素在内存的存放和访问

- 分配在**静态区**的数组的初始化和访问

```
int buf[2] = {10, 20};  
int main ( )  
{  
    int i, sum=0;  
    for (i=0; i<2; i++)  
        sum+=buf[i];  
    return sum;  
}
```

buf是在静态区分配的数组，链接后，buf
在可执行目标文件的数据段中分配了空间

08048908 <buf> :
08048908:

此时，buf=&buf[0]=0x08048908

编译器通常将其先存放到寄存器(如EDX)中

假定 i 被分配在ECX中，sum被分配在EAX中，则

“sum+=buf[i];” 和 i++ 可用什么指令实现？

addl buf(, %ecx, 4), %eax 或 addl 0(%edx , %ecx, 4), %eax

addl &1, %ecx

数组元素在内存的存放和访问

- auto型数组的初始化和访问

```
int adder ( )
```

```
{
```

```
    int buf[2] = {10, 20};
```

```
    int i, sum=0;
```

```
    for (i=0; i<2; i++)
```

```
        sum += buf[i];
```

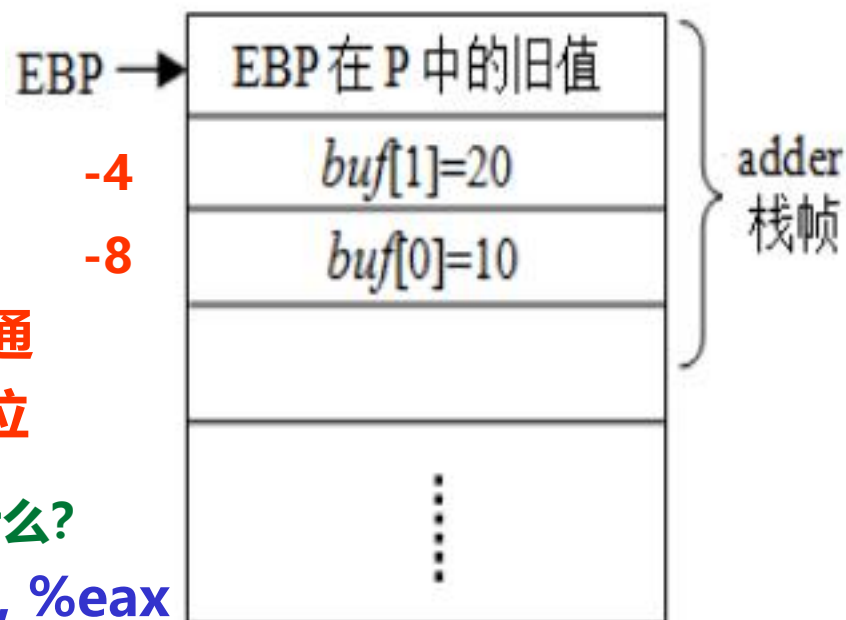
```
    return sum;
```

```
}
```

分配在栈中，
故数组首址通
过EBP来定位

EDX、ECX各是什么？

`addl (%edx, %ecx, 4), %eax`



对buf进行初始化的指令是什么？

`movl $10, -8(%ebp)` //buf[0]的地址为R[ebp]-8，将10赋给buf[0]

`movl $20, -4(%ebp)` //buf[1]的地址为R[ebp]-4，将20赋给buf[1]

若要将buf首址放入EDX中，则对应指令是什么？

`leal -8(%ebp), %edx` //buf[0]的地址为R[ebp]-8，将buf首址送EDX

数组元素在内存的存放和访问

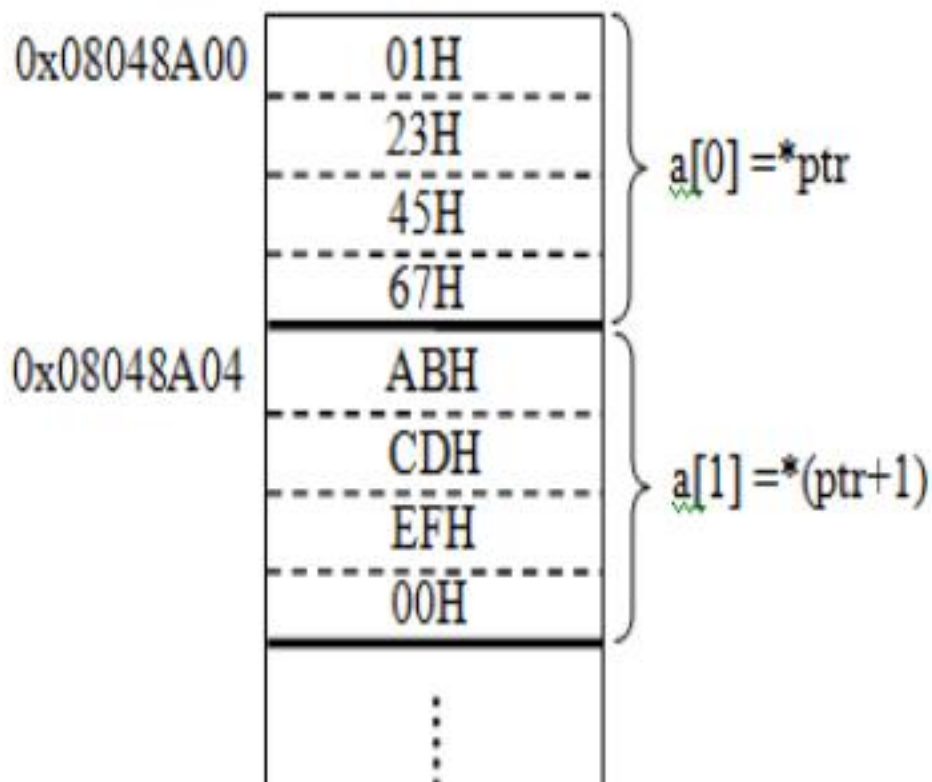
- 数组与指针

- ✓ 在指针变量目标数据类型与数组类型相同的前提下，指针变量可以指向数组中任意元素

- ✓ 以下两个程序段功能完全相同，都是使ptr指向数组a的第0个元素a[0]。a的值就是其首地址，即a=&a[0]，因而a=ptr，从而有&a[i]=ptr+i=a+i以及a[i]=ptr[i]=*(ptr+i)=*(a+i)

-

```
(1) int a[10];  
    int *ptr=&a[0];  
  
(2) int a[10], *ptr;  
    ptr=&a[0];
```



小端方式下a[0]=?,a[1]=?

a[0]=0x67452301, a[1]=0x0efcdab

数组首址0x8048A00在ptr中，ptr+i并不是用0x8048A00加i得到，而是等于0x8048A00+4*i

数组元素在内存的存放和访问

• 数组与指针

序号	表达式	类型	值的计算方式	汇编代码
1	A	int *	问题： 假定数组A的首址SA在ECX中，i在EDX中，表达式结果在EAX中，各表达式的计算方式以及汇编代码各是什么？	
2	<u>A[0]</u>	int		
3	<u>A[i]</u>	int		
4	&A[3]	int *		
5	&A[i]-A	int		
6	<u>*(A+i)</u>	int		
7	<u>*(&A[0]+i-1)</u>	int		
8	A+i	int *		

2、3、6和7对应汇编指令都需访存，指令中源操作数的寻址方式分别是“基址”、“基址加比例变址”、“基址加比例变址”和“基址加比例变址加位移”的方式，因为数组元素的类型为int型，故比例因子为4。

数组元素在内存的存放和访问

- 数组与指针 假设A首址SA在ECX, i 在EDX, 结果在EAX

序号	表达式	类型	值的计算方式	汇编代码
1	A	int *	SA	leal (%ecx), %eax
2	A[0]	int	M[SA]	movl (%ecx), %eax
3	A[i]	int	M[SA+4*i]	movl (%ecx, %edx, 4), %eax
4	&A[3]	int *	SA+12	leal 12(%ecx), %eax
5	&A[i]-A	int	(SA+4*i-SA)/4=i	movl %edx, %eax
6	*(A+i)	int	M[SA+4*i]	movl (%ecx, %edx, 4), %eax
7	*(&A[0]+i-1)	int	M[SA+4*i-4]	movl -4(%ecx, edx, 4), %eax
8	A+i	int *	SA+4*i	leal (%ecx, %edx, 4), %eax

2、3、6和7对应汇编指令都需访存，指令中源操作数的寻址方式分别是“基址”、“基址加比例变址”、“基址加比例变址”和“基址加比例变址加位移”的方式，因为数组元素的类型为int型，故比例因子为4。

数组元素在内存的存放和访问

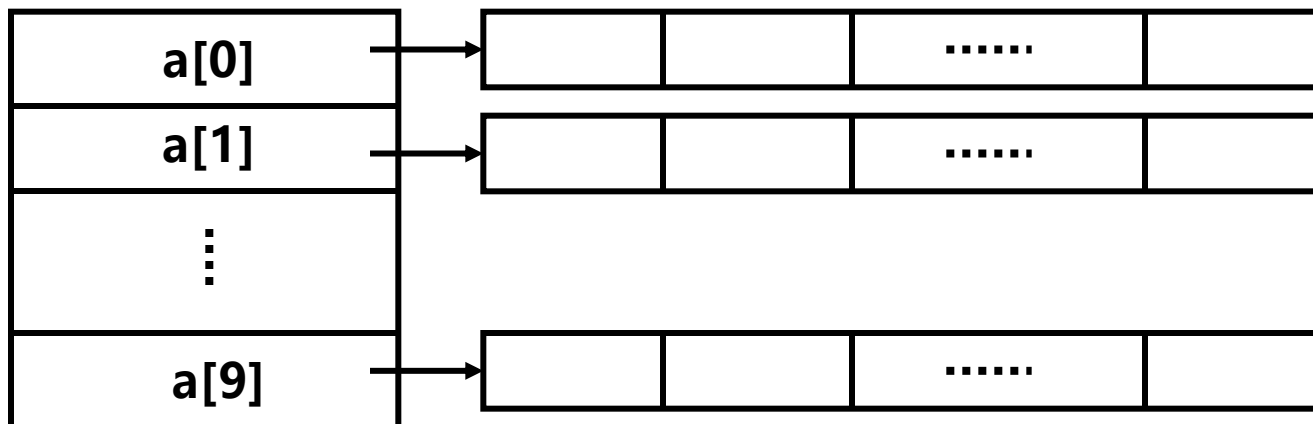
- 指针数组和多维数组

- 由若干指向同类目标的指针变量组成的数组称为指针数组。
- 其定义的一般形式如下：

存储类型 数据类型 *指针数组名[元素个数];

- 例如， “int *a[10];” 定义了一个指针数组a，它有10个元素，每个元素都是一个指向int型数据的指针。

- 一个指针数组可以实现一个二维数组。



数组元素在内存的存放和访问

- 指针数组和二维数组

- 计算一个两行四列整数矩阵中每一行数据的和。

```
main ( )
```

```
{
```

```
    static short num[ ][4]={ {2, 9, -1, 5},  
                               {3, 8, 2, -6}};
```

```
    static short *pn[ ]={num[0], num[1]};
```

```
    static short s[2]={0, 0};
```

```
    int i, j;
```

```
    for (i=0; i<2; i++) {
```

```
        for (j=0; j<4; j++)
```

```
            s[i] += *pn[i] ++;
```

```
        printf (sum of line %d: %d\n" , i+1, s[i]);
```

```
    }
```

```
}
```

当i=1时, $pn[i] = *(pn+i) = M[pn+4*i] = 0x8049308$

若处理 “ $s[i] += *pn[i] ++;$ ” 时 i 在 ECX, s[i]在AX, 则对应指令序列可以是什么? 假设要先将pn[i]放在EDX

movl pn(,%ecx,4), %edx

addw (%edx), %ax

addl \$2, pn(, %ecx, 4)

pn[i] + " 1" → pn[i]

若num=0x8049300,则数组num、pn和s在存储区中如何存放?

08049300 <num>: num=&num[0]=0x8049300

08049300:

08049310 <pn>:

08049310:

08049318 <s>:

08049318:

pn=&pn[0]=0x8049310

pn[0]=num[0]=0x8049300

pn[1]=num[1]=0x8049308

结构体数据的分配和访问

- 结构体成员在内存的存放和访问
 - 分配在栈中的auto结构型变量的首地址由EBP或ESP来定位
 - 分配在静态区的结构型变量首地址是一个确定的静态区地址
 - 结构型变量 x 各成员首址可用 “**基址加偏移量**” 的寻址方式

若变量x被分配在地址0x8049200开始的区域，那么

```
struct cont_info {  
    char id[8];  
    char name [12];  
    unsigned post;  
    char address[100];  
    char phone[20];  
};
```

```
struct cont_info x={ "0000000" , "ZhangS" , 210022, "273 long  
street, High Building #3015" , "12345678" };
```

x初始化后，在地址0x8049208到0x804920D处是字符串 “ZhangS” ，
0x804920E处是字符 ‘\0’ ，从0x804920F到0x8049213处都是空字符。

“unsigned xpost=x.post;” 对应汇编指令为

结构体数据的分配和访问

- 结构体数据作为入口参数

按地址调用

```
void stu_phone1 ( struct cont_info *s_info_ptr)
{
    printf ( "%s phone number: %s" , (*s_info_ptr).name, (*s_info_ptr).phone);
}
```

stu_phone1 (&x)

```
void stu_phone2 ( struct cont_info s_info)
{
    printf ( "%s phone number: %s" , s_info.name, s_info.phone);
}
```

按值调用 stu_phone2 (x)

- 当结构体变量需要作为一个函数的形参时，形参和调用函数中的实参应具有相同结构

```
struct cont_info x={ "0000000" , "ZhangS" , 210022, "273 long  
street, High Building #3015" , "12345678" };
```

- 若采用按值传递，则结构成员都要复制到栈中参数区，这既增加时间开销又增加空间开销，且更新后的数据无法在调用过程使用
- 通常应按地址传递，即：在执行CALL指令前，仅需传递指向结构体的指针而不需复制每个成员到栈中

结构体数据的分配和访问

- 结构体数据作为入口参数 (若对应实参是x)

stu_phone2 (x)

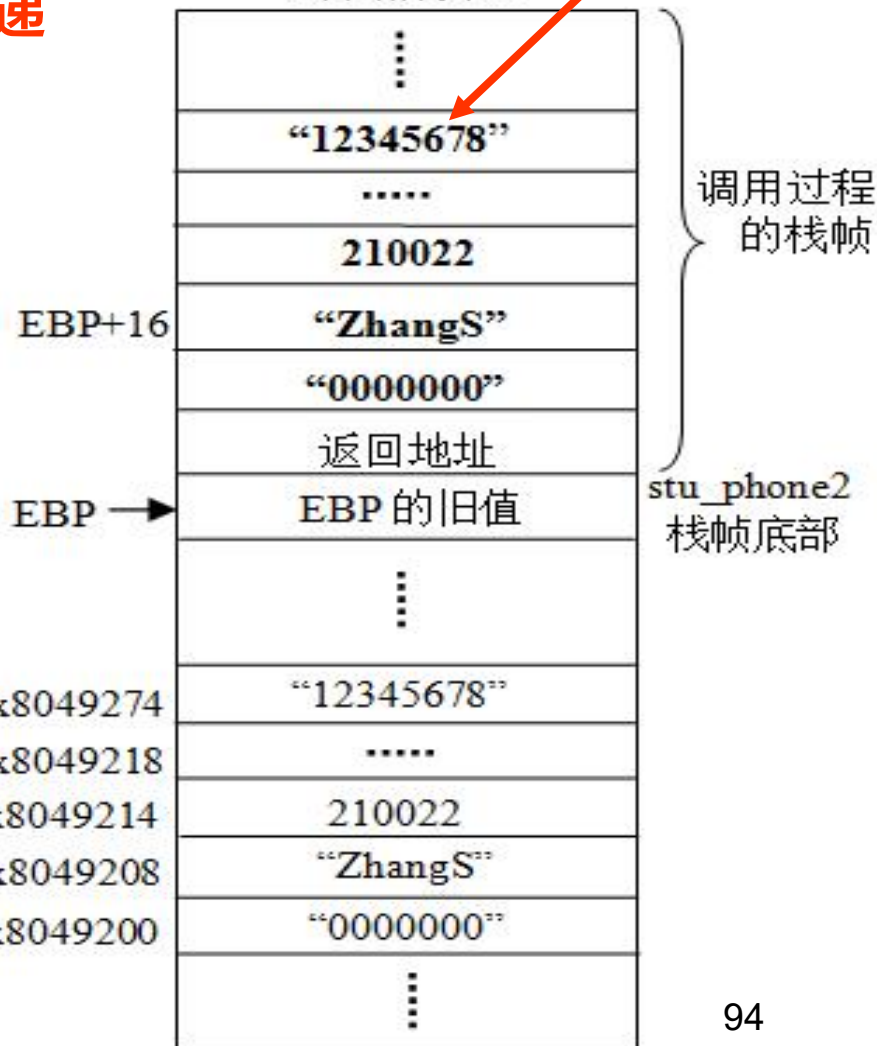
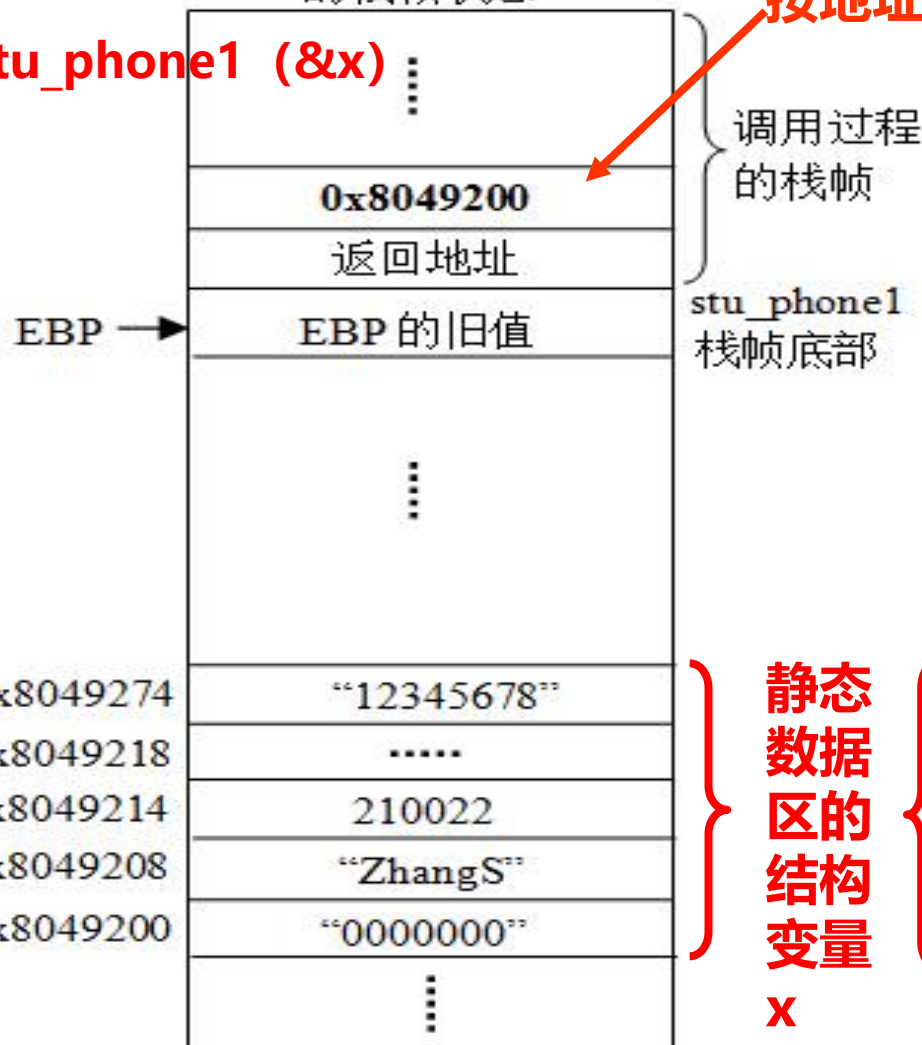
调用 stu_phone1 时的
栈帧状态

调用 stu_phone2 时的
栈帧状态

按值传递

按地址传递

stu_phone1 (&x)



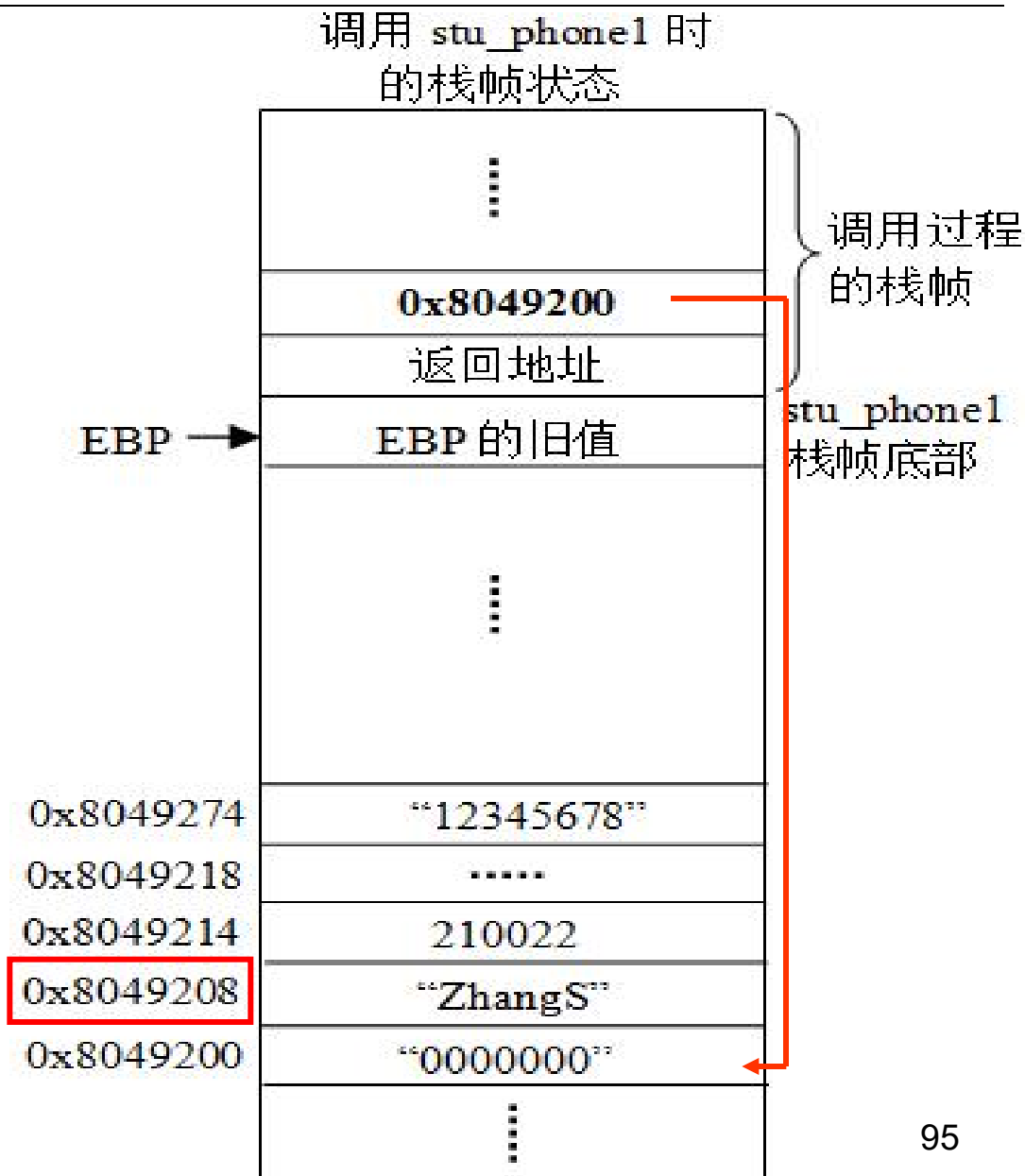
结构体数据的分配和访问

- 按地址传递参数

`(*s_info_ptr).name`可写成
`s_info_ptr->name`,
执行以下两条指令后:

```
movl 8(%ebp), edx
```

```
leal 8(%edx), eax
```



结构体数据的分配和访问

- 按值传递参数

x所有成员值作为实参
存到参

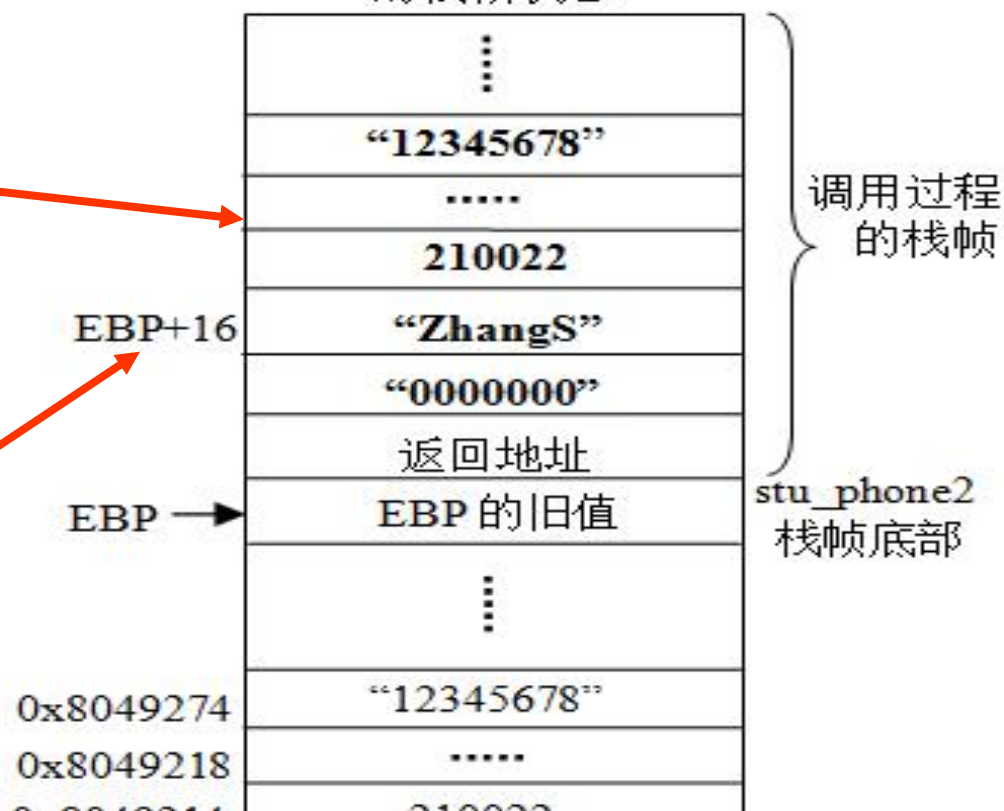
s_info.name送EAX
的指令序列为：

```
leal 8(%ebp), edx
```

```
leal 8(%edx), eax
```

EAX中存放的是
“ZhangS” 的栈内
参数区首址。

调用 stu_phone2 时的
栈帧状态



- stu_phone1和stu_phone2功能相同，但两者的时、空开销都不一样。后者开销大，因为它需对结构体成员整体从静态区复制到栈中，需要很多条mov或其他指令，从而执行时间更长，并占更多栈空间和代码空间，并且无法获得更新后的结果。

联合体数据的分配和访问

联合体各成员共享存储空间，按最大长度成员所需空间大小为目标

```
union uarea {  
    char c_data;  
    short s_data;  
    int i_data;  
    long l_data;  
};
```

IA-32中编译时，long和int长度一样，故uarea所占空间为4个字节。而对于与uarea有相同成员的结构型变量来说，其占用空间大小至少有11个字节，对齐的话则占用更多。

- 通常用于特殊场合，如，当事先知道某种数据结构中的不同字段的使用时间是互斥的，就可将这些字段声明为联合，以减少空间。
- 但有时会得不偿失，可能只会减少少量空间却大大增加处理复杂性。

联合体数据的分配和访问

- 还可实现对相同位序列进行不同数据类型的解释

```
unsigned float2unsign( float f)
{
    union {
        float f;
        unsigned u;
    } tmp_union;
    tmp_union.f=f;
    return tmp_union.u;
}
```

过程体为:

movl 8(%ebp), %eax

movl %eax, -4(%ebp)

movl -4(%ebp) , %eax

} 可优化掉!

将存放在地址R[ebp]+8处的入口参数 f
送到EAX (返回值)

问题: float2unsign(10.0)=? $2^{30} + 2^{24} + 2^{21} = 1092616192$

从该例可看出: 机器级代码并不区分所处理对象的数据类型, 不管高级语言中将其说明成float型还是int型或unsigned型, 都把它当成一个0/1序列来处理。

IA-32的寄存器组织

	31	16	15	8	7	0	
EAX				AH	(AX)	AL	累加器
EBX				BH	(BX)	BL	基址寄存器
ECX				CH	(CX)	CL	计数寄存器
EDX				DH	(DX)	DL	数据寄存器
ESP				SP			堆栈指针
EBP				BP			基址指针
ESI				SI			源变址寄存器
EDI				DI			目标变址寄存器
EIP				IP			指令指针
EFLAGS				FLAGS			标志寄存器
				CS			代码段
				SS			堆栈段
				DS			数据段
				ES			附加段
				FS			附加段
				GS			附加段

00

IA-32的寄存器组织

编号	8 位寄存器	16 位寄存器	32 位寄存器	64 位寄存器	128 位寄存器
000	AL	AX	EAX	MM0 / ST(0)	XMM0
001	CL	CX	ECX	MM1 / ST(1)	XMM1
010	DL	DX	EDX	MM2 / ST(2)	XMM2
011	BL	BX	EBX	MM3 / ST(3)	XMM3
100	AH	SP	ESP	MM4 / ST(4)	XMM4
101	CH	BP	EBP	MM5 / ST(5)	XMM5
110	DH	SI	ESI	MM6 / ST(6)	XMM6
111	BH	DI	EDI	MM7 / ST(7)	XMM7

如果要用C语言来模拟IA-32的寄存器组织，该如何做？

```
typedef struct{  
union{
```

```
    struct {  
        uint32_t  eax;  
        uint32_t  ecx;  
        uint32_t  edx;  
        uint32_t  ebx;  
        uint32_t  esp;  
        uint32_t  ebp;  
        uint32_t  esi;  
        uint32_t  edi;};
```

```
    union{  
        uint32_t  _32;  
        uint16_t  _16;  
        uint8_t   _8[2];
```

```
    } gpr[8];
```

```
};
```

```
swaddr_t eip;
```

```
} CPU_state;
```

```
extern CPU_state cpu;
```

```
enum { R_EAX, R_ECX, R_EDX, R_EBX, R_ESP, R_EBP, R_ESI, R_EDI };
```

```
enum { R_AX, R_CX, R_DX, R_BX, R_SP, R_BP, R_SI, R_DI };
```

```
enum { R_AL, R_CL, R_DL, R_BL, R_AH, R_CH, R_DH, R_BH };
```

```
#define reg_l(index) (cpu.gpr[index]._32)
```

```
#define reg_w(index) (cpu.gpr[index]._16)
```

```
#define reg_b(index) (cpu.gpr[index & 0x3]._8[index >> 2])
```

PA中模拟的 IA-32的寄存器组织

联合体数据的

• 利用嵌套可定义链表结构

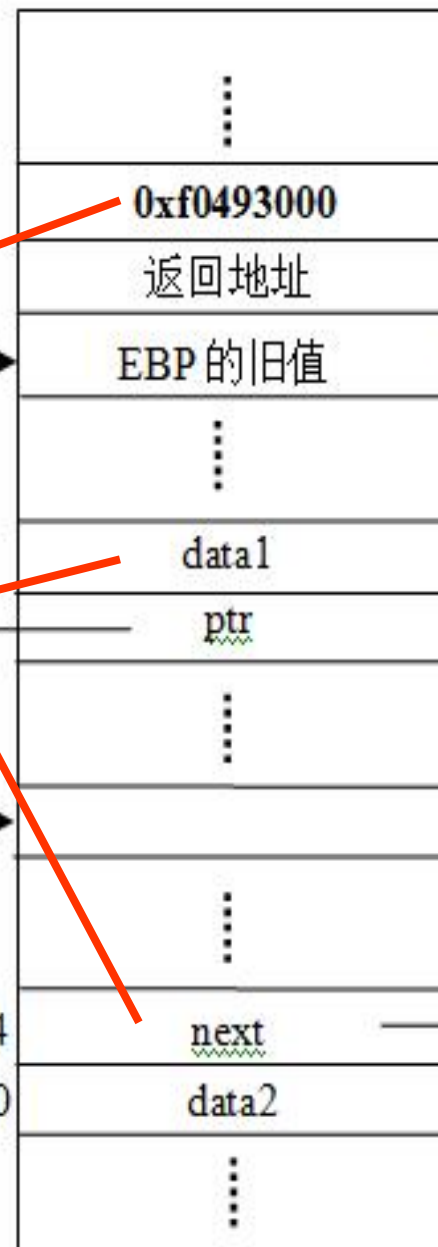
```
union node {  
    struct {  
        int *ptr;  
        int data1;  
    } node1;  
    struct {  
        int data2;  
        union node *next;  
    } node2;  
};
```

```
void node_proc ( union node *np )  
{  
    np->next->data1=*(np->next->ptr)+np->data2;  
}
```

```
movl 8(%ebp), %ecx  
movl 4(%ecx), %edx  
movl (%edx), %eax  
movl (%eax), %eax  
addl (%ecx), %eax  
movl %eax, 4(%edx)
```

问题:
(ECX)=?
(EDX)=?

EBP →

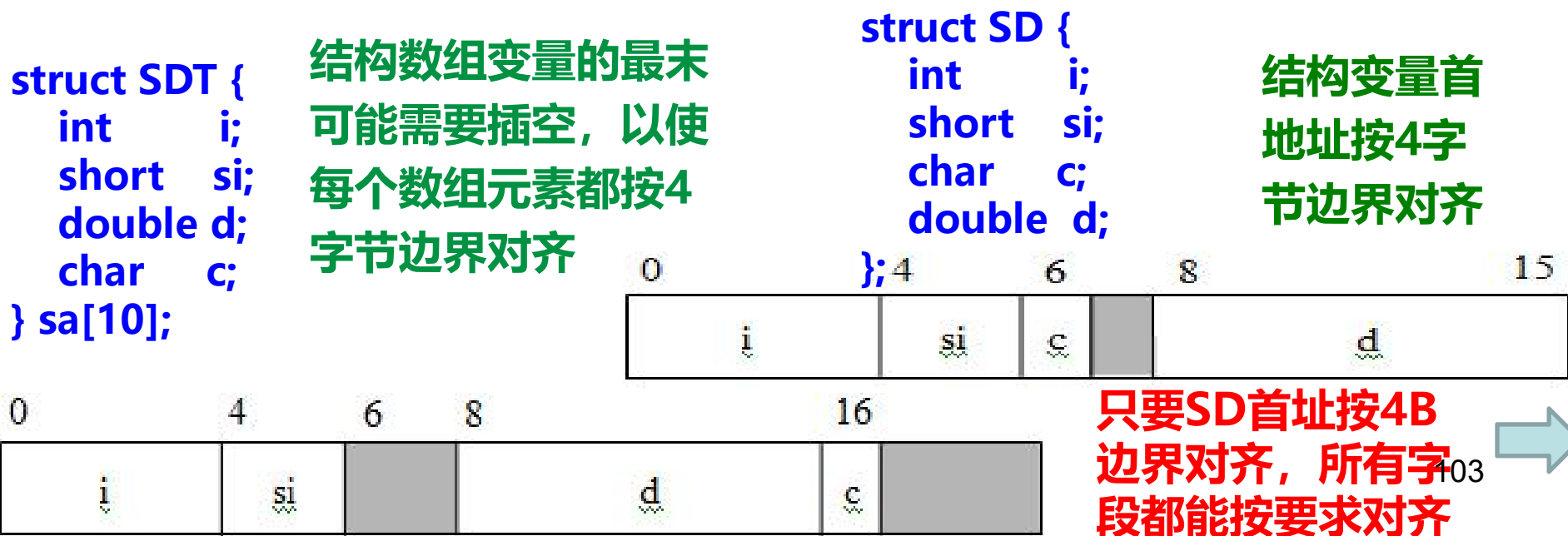


动态链表通常在堆区

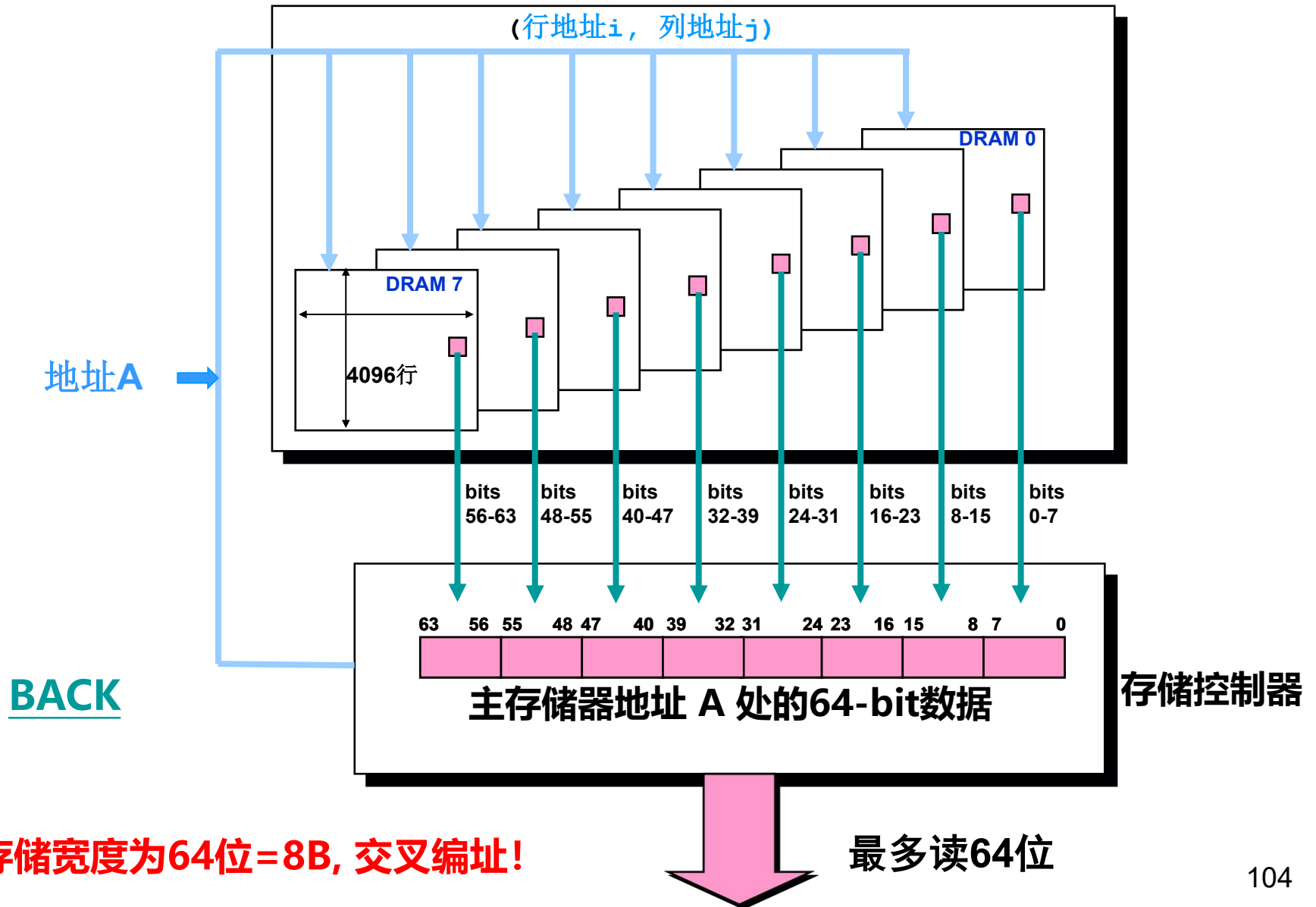


数据的对齐

- CPU访问主存时只能一次读取或写入若干特定位。例如，若每次最多读写64位，则第0字节到第7字节可同时读写，第8字节到第15字节可同时读写，.....，以此类推。按边界对齐，可使读写数据位于 $8i \sim 8i+7$ ($i=0, 1, 2, \dots$) 单元。
- 最简单的对齐策略是，按其数据长度进行对齐，例如，int型地址是4的倍数，short型地址是2的倍数，double和long long型的是8的倍数，float型的是4的倍数，char不对齐。Windows采用该策略。Linux策略更宽松：short是2的倍数，其他如int、double和指针等都是4的倍数。



主存储器的结构



程序的机器级表示

- 分以下六个部分介绍

- 第一讲：程序转换概述

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- 第二讲：IA-32 /x86-64指令系统

- 第三讲：C语言程序的机器级表示

- 过程调用的机器级表示
- 选择语句的机器级表示
- 循环结构的机器级表示

- 第四讲：复杂数据类型的分配和访问

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- 第五讲：越界访问和缓冲区溢出

- 第六讲：x86-64架构（自学）

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

越界访问和缓冲区溢出

大家还记得以下的例子吗？

```
double fun(int i)
{
    volatile double d[1] = {3.14};
    volatile long int a[2];
    a[i] = 1073741824; /* Possibly out of bounds */
    return d[0];
}
```

fun(0) → 3.14
fun(1) → 3.14
fun(2) → 3.13999998664856
fun(3) → 2.000000061035156
fun(4) → 3.14, 然后存储保护错

为什么当 $i > 1$ 就有问题？

因为数组访问越界！

Saved State	4
d7 ... d4	3
d3 ... d0	2
a[1]	1
a[0]	0

越界访问和缓冲区溢出

- C语言中的**数组元素可使用指针来访问**，因而对数组的引用没有边界约束，也即程序中对数组的访问可能会有意或无意地超越数组存储区范围而无法发现。
- 数组存储区可看成是一个缓冲区，**超越数组存储区范围的写入操作称为缓冲区溢出**。
- 例如，对于一个有10个元素的char型数组，其定义的缓冲区有10个字节。若写一个字符串到这个缓冲区，那么只要写入的字符串多于9个字符（结束符 ‘\0’ 占一个字节），就会发生 **“写溢出”**。
- 缓冲区溢出是一种**非常普遍、非常危险的漏洞**，在各种操作系统、应用软件中广泛存在。
- **缓冲区溢出攻击**是利用缓冲区溢出漏洞所进行的攻击行动。利用缓冲区溢出攻击，可导致程序运行失败、系统关机、重新启动等后果。

main()函数的原型

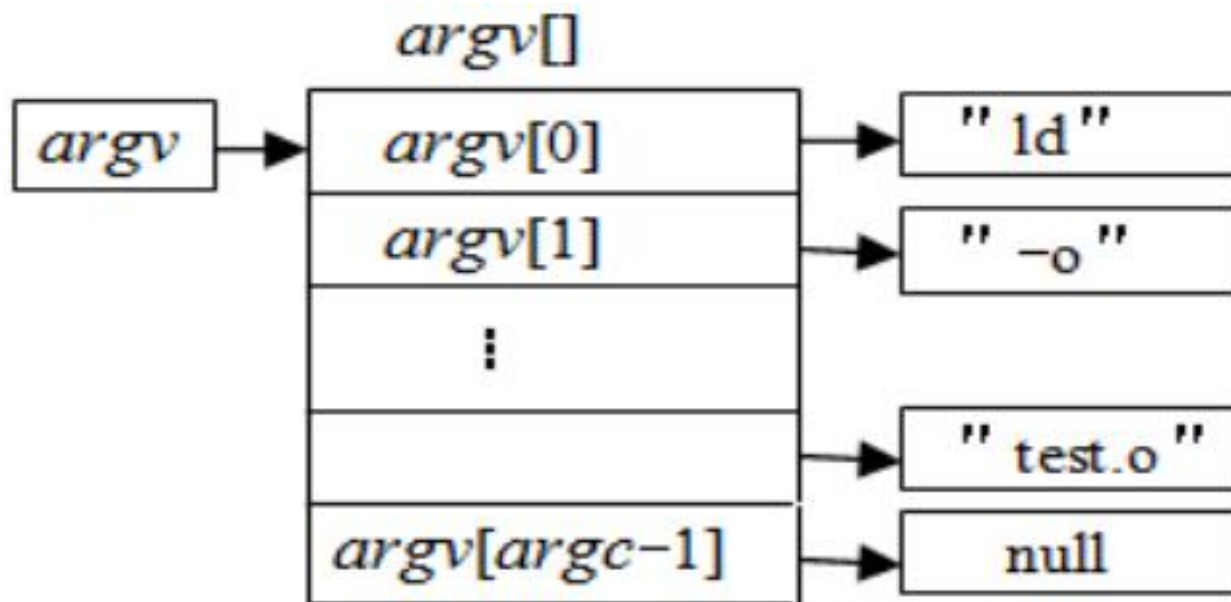
- 主函数main()的原型形式如下：

`int main(int argc, char **argv, char **envp);` 或

`int main(int argc, char *argv[], char *envp[]);`

argc: 参数列表长度，参数列表中开始是命令名（可执行文件名），最后以NULL结尾。例：当 “./hello” 时，argc=2

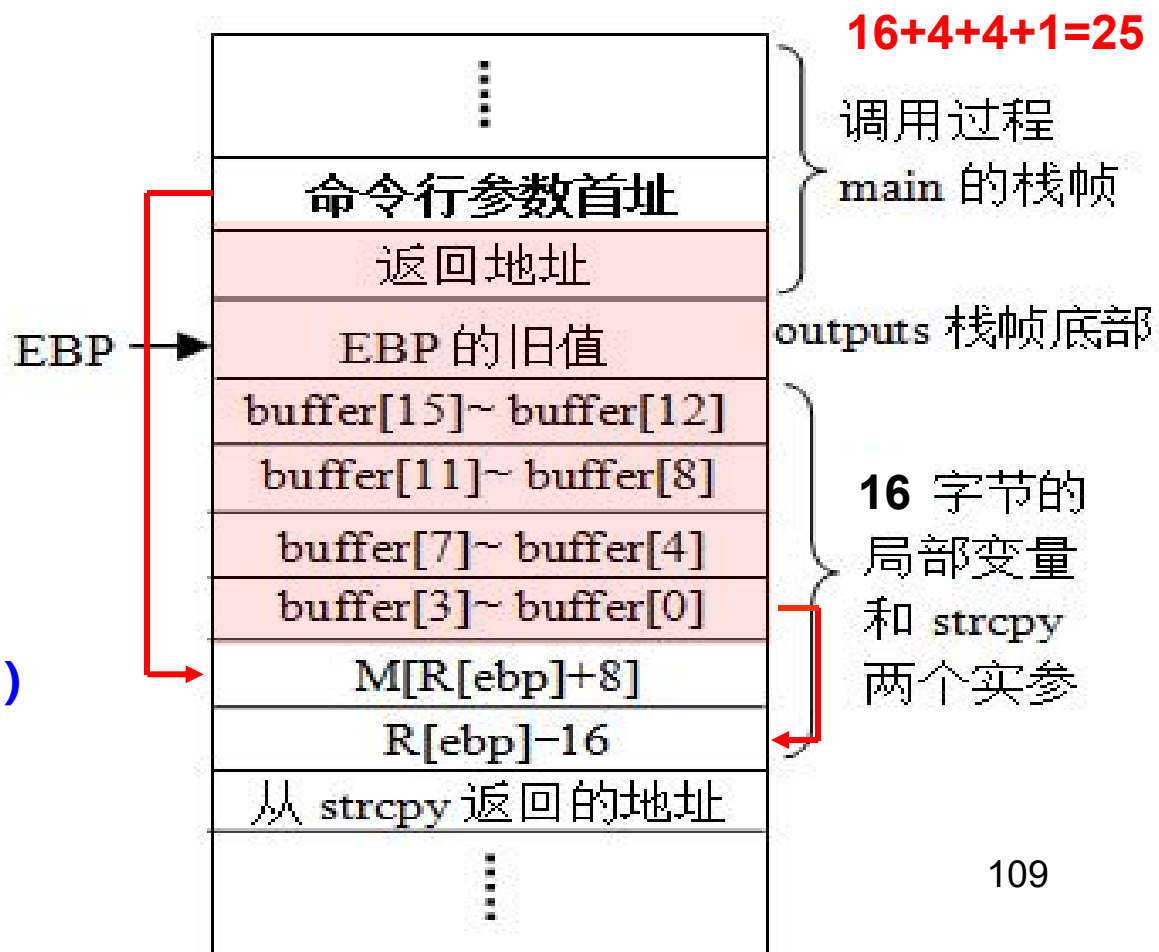
例：当 “ld -o test main.o test.o” 时，argc=6



越界访问和缓冲区溢出

- 造成缓冲区溢出的原因是程序没有对栈中作为缓冲区的数组进行越界检查。举例：利用缓冲区溢出转到自设的程序hacker去执行
outputs漏洞：当命令行中字符串超**25个字符**时，使用**strcpy**函数就会使缓冲buffer造成写溢出并破坏返址

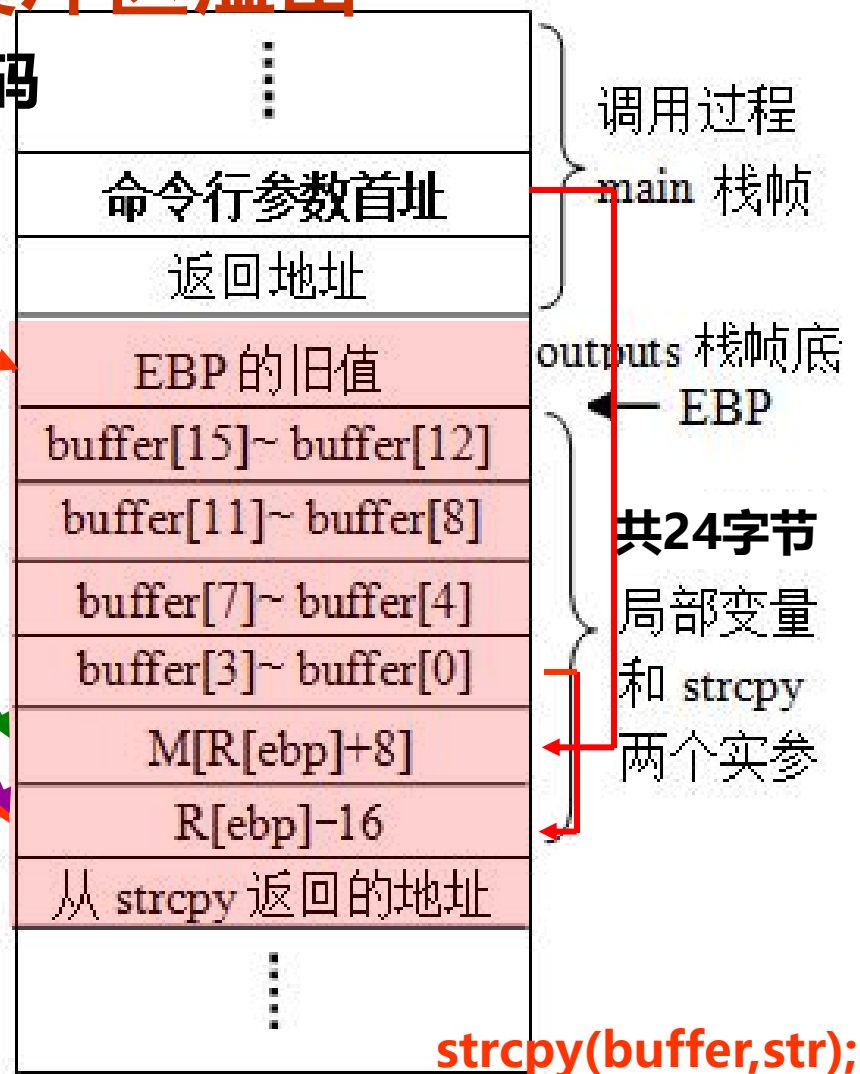
```
#include "stdio.h"
#include "string.h"
void outputs(char *str)
{
    char buffer[16];
    strcpy(buffer, str);
    printf("%s \n", buffer);
}
void hacker(void)
{
    printf("being hacked\n");
}
int main(int argc, char *argv[])
{
    outputs(argv[1]);
    return 0;
}
```



越界访问和缓冲区溢出

test被反汇编得到的**outputs**汇编代码

```
080483e4 push  %ebp
080483e5 mov  %esp,%ebp
080483e7 sub   $0x18,%esp
080483ea mov  0x8(%ebp),%eax
080483ed mov  %eax,0x4(%esp)
080483f1 lea  0xffffffff0(%ebp),%eax
080483f4 mov  %eax,(%esp)
080483f7 call 0x8048330 <strcpy>
080483fc lea  0xffffffff0(%ebp),%eax
080483ff mov  %eax,0x4(%esp)
08048403 movl $0x8048500,(%esp)
0804840a call 0x8048310
0804840f leave
08048410 ret
```



若strcpy复制了**25个字符**到buffer中，并将hacker首址置于结束符 '\0' 前4个字节，则在执行strcpy后，hacker代码首址被置于main栈帧返回地址处，当执行outputs代码的ret指令时，便会转到hacker函数实施攻击。

程序的加载和运行

- UNIX/Linux系统中，可通过调用execve()函数来加载并执行程序。
- execve()函数的用法如下：

```
int execve(char *filename, char *argv[], *envp[]);
```

filename是加载并运行的可执行文件名(如./hello)，可带参数列表argv和环境变量列表envp。若错误（如找不到指定文件filename），则返回-1，并将控制权交给调用程序；若函数执行成功，则不返回，最终将控制权传递到可执行目标中的主函数main。

- 主函数main()的原型形式如下：

```
int main(int argc, char **argv, char **envp); 或者：
```

```
int main(int argc, char *argv[], char *envp[]);
```

前述例子：“./test 0123456789ABCDEFXXXX” ,argc=3
 argv[0] argv[1]

缓冲区溢

```
#include "stdio.h"
char code[]=
    "0123456789ABCDEFXXXX"
    "\x11\x84\x04\x08"
    "\x00";
int main(void)
{
    char *argv[3];
    argv[0]="./test";
    argv[1]=code;
    argv[2]=NULL;
    execve(argv[0],argv,NULL);
    return 0;
}
```

```
#include "stdio.h"
#include "string.h"
void outputs(char *str)
{
    char buffer[16];
    strcpy(buffer,str);
    printf("%s \n", buffer);
}
void hacker(void)
{
    printf("being hacked\n");
}
int main(int argc, char *argv[])
{
    outputs(argv[1]);
    return 0;
}
```

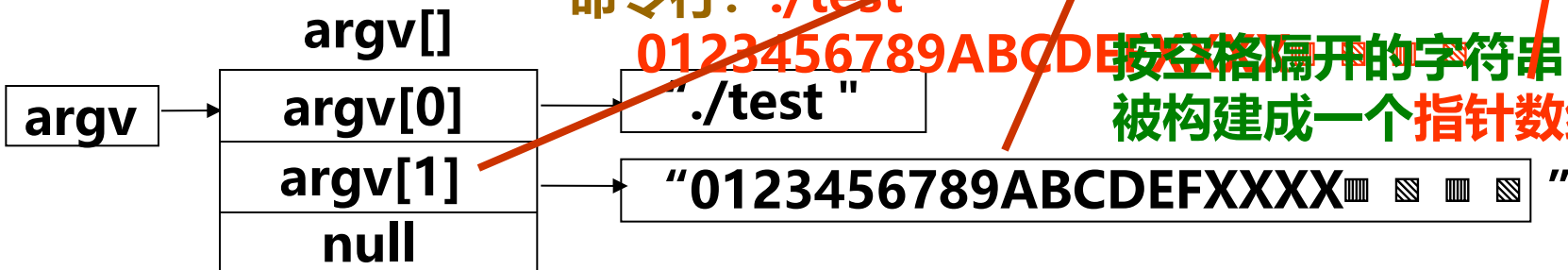
可执行文件名为test

命令行: ./test

0123456789ABCDEFXXXX

按空格隔开的字符串

被构建成一个指针数组



越界访问和缓

假定hacker首址为0x08048411

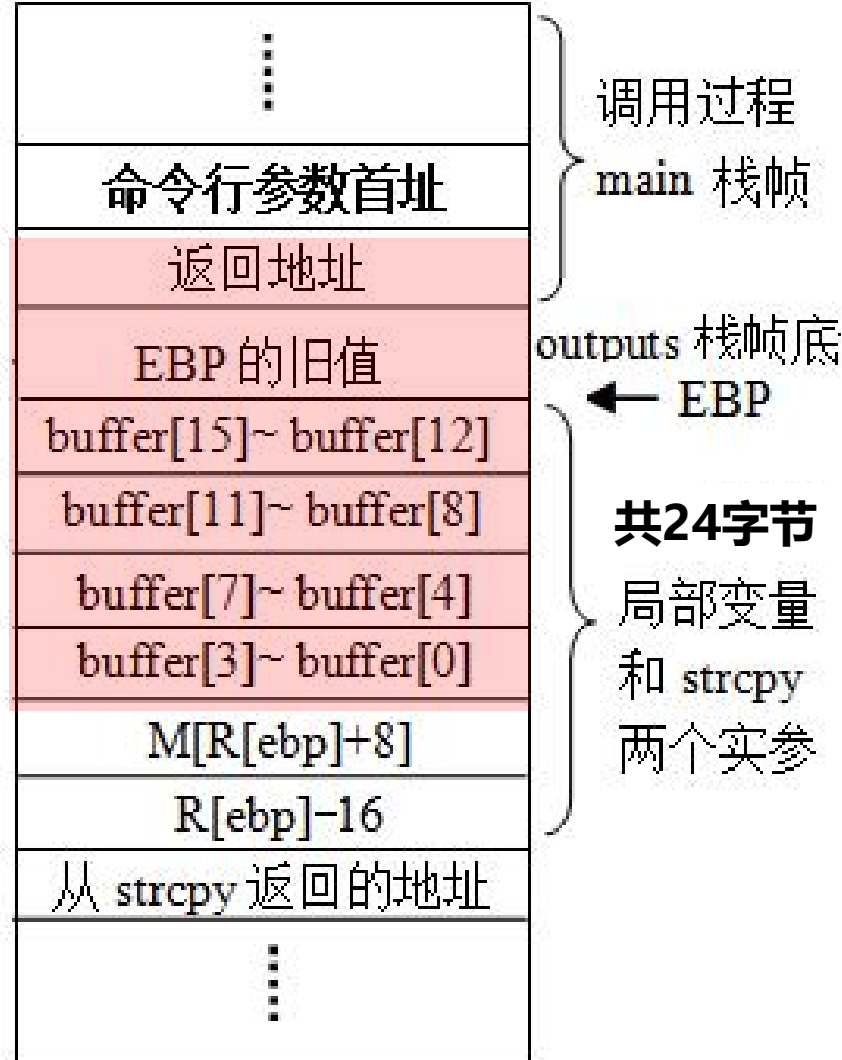
```
void hacker(void) {  
    printf("being hacked\n");  
}  
  
#include "stdio.h"  
char code[] =  
    "0123456789ABCDEFXXXX"  
    "\x11\x84\x04\x08"  
    "\x00";  
  
int main(void) {  
    char *argv[3];  
    argv[0] = "./test";  
    argv[1] = code;  
    argv[2] = NULL;  
    execve(argv[0], argv, NULL);  
    return 0;  
}
```

执行上述攻击程序后的输出结果为:

"0123456789ABCDEFXXXX" ■ ■ ■ ■

being hacked

Segmentation fault



最后显示“Segmentation fault”，原因是执行到hacker过程的ret指令时取到的“返回地址”是一个不确定的值，因而可能跳转到数据区或系统区或其他非法访问的存储区执行，因而造成段错误。

缓冲区溢出攻击的防范（自学）

- 两个方面的防范

- 从程序员角度去防范

- 用辅助工具帮助程序员查漏，例如，用grep来搜索源代码中容易产生漏洞的库函数（如strcpy和sprintf等）的调用；用fault injection查错

- 从编译器和操作系统方面去防范

- 地址空间随机化ASLR

- 是一种比较有效的防御缓冲区溢出攻击的技术

- 目前在Linux、FreeBSD和Windows Vista等OS使用

- 栈破坏检测
 - 可执行代码区域限制
 - 等等

缓冲溢出攻击防范（自学）

• 地址空间随机化

- 只要操作系统相同，则栈位置就一样，若攻击者知道漏洞程序使用的栈地址空间，就可设计一个针对性攻击，在使用该程序机器上实施攻击
- 地址空间随机化（栈随机化）的基本思路是，将加载程序时生成的代码段、静态数据段、堆区、动态库和栈区各部分的首地址进行随机化处理，使每次启动时，程序各段被加载到不同地址起始处
- 对于随机生成的栈起始地址，攻击者不太容易确定栈的起始位置

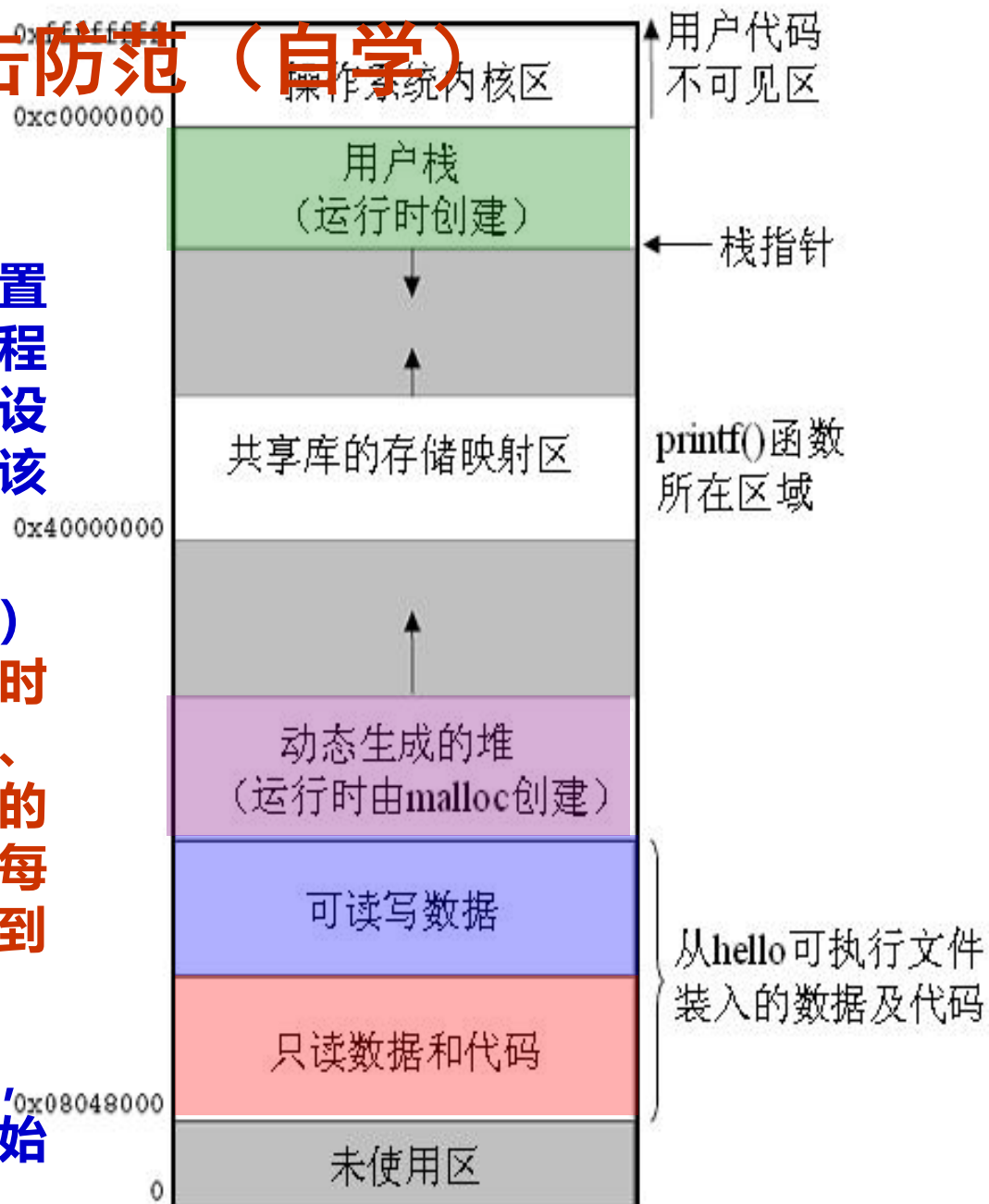
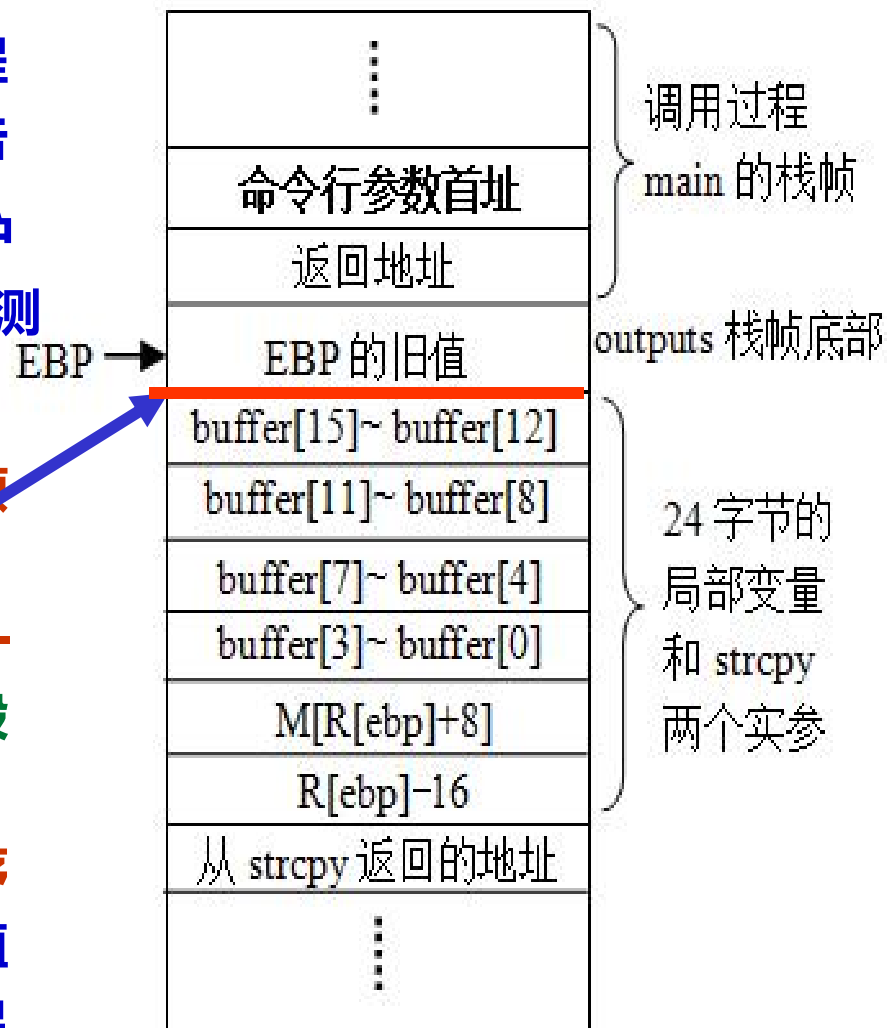


图6.30 Linux虚拟地址空间映像

缓冲区溢出攻击的防范（自学）

• 栈破坏检测

- 若在程序跳转到攻击代码前能检测出程序栈已被破坏，就可避免受到严重攻击
- 新GCC版本在代码中加入了一种栈保护者（stack protector）机制，用于检测缓冲区是否越界
- 主要思想：在函数准备阶段，在其栈帧中缓冲区底部与保存寄存器之间（如 `buffer[15]` 与保留的 `EBP` 之间）加入一个随机生成的特定值；在函数恢复阶段，在恢复寄存器并返回到调用函数前，先检查该值是否被改变。若改变则程序异常中止。因为插入在栈帧中的特定值是随机生成的，所以攻击者很难猜测出它是什么



缓冲区溢出攻击的防范（自学）

- 可执行代码区域限制

- 通过将程序栈区和堆区设置为不可执行，从而使得攻击者不可能执行被植入在输入缓冲区的代码，这种技术也被称为非执行的缓冲区技术。
- 早期Unix系统只有代码段的访问属性是可执行，其他区域的访问属性是可读或可读可写。但是，近来Unix和Windows系统由于要实现更好的性能和功能，允许在栈段中动态地加入可执行代码，这是缓冲区溢出的根源。
- 为保持程序兼容性，不可能使所有数据段都设置成不可执行。不过，可以将动态的栈段设置为不可执行，这样，既保证程序的兼容性，又可以有效防止把代码植入栈（自动变量缓冲区）的溢出攻击。

程序的机器级表示

- 分以下六个部分介绍

- 第一讲：程序转换概述

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- 第二讲：IA-32 /x86-64指令系统

- 第三讲：C语言程序的机器级表示

- 过程调用的机器级表示
- 选择语句的机器级表示
- 循环结构的机器级表示

- 第四讲：复杂数据类型的分配和访问

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- 第五讲：越界访问和缓冲区溢出

- 第六讲：x86-64架构（自学）

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

X86-64架构

- 背景

- Intel最早推出的64位架构是基于超长指令字VLIW技术的IA-64体系结构，Intel 称其为显式并行指令计算机EPIC（Explicitly Parallel Instruction Computer）。安腾和安腾2分别在2000年和2002年问世，它们是IA-64体系结构的最早的具体实现。
- AMD公司利用Intel在IA-64架构上的失败，抢先在2003年推出兼容IA-32的64位版本指令集x86-64，AMD获得了以前属于Intel的一些高端市场。AMD后来将x86-64更名为AMD64。
- Intel在2004年推出IA32-EM64T，它支持x86-64指令集。Intel为了表示EM64T的64位模式特点，又使其与IA-64有所区别，2006年开始把EM64T改名为Intel 64。

X86-64架构

- 与IA-32相比，x86-64架构的主要特点

- 新增8个64位通用寄存器：R8、R9、R10、R11、R12、R13、R14和R15。可作为8位（R8B~R15B）、16位（R8W~R15W）或32位寄存器（R8D~R15D）使用
- 所有GPRs都从32位扩充到64位。8个32位通用寄存器EAX、EBX、ECX、EDX、EBP、ESP、ESI和EDI对应扩展寄存器分别为RAX、RBX、RCX、RDX、RBP、RSP、RSI和RDI；EBP、ESP、ESI和EDI的低8位寄存器分别是BPL、SPL、SIL和DIL
- 字长从32位变为64位，故逻辑地址从32位变为64位
- long double型数据虽还采用80位扩展精度格式，但所分配存储空间从12B扩展为16B，即改为16B对齐，但不管是分配12B还是16B，都只用到低10B
- 过程调用时，通常用通用寄存器而不是栈来传递参数，因此，很多过程不用访问栈，这使得大多数情况下执行时间比IA-32代码更短
- 128位的MMX寄存器从原来的8个增加到16个，浮点操作采用基于SSE的面向XMM寄存器的指令集，而不采用基于浮点寄存器栈的指令集

IA-32中通用寄存器中的编号

编号	8 位寄存器	16 位寄存器	32 位寄存器	64/80位寄存器	128 位寄存器
000	AL	AX	EAX	MM0 / ST(0)	XMM0
001	CL	CX	ECX	MM1 / ST(1)	XMM1
010	DL	DX	EDX	MM2 / ST(2)	XMM2
011	BL	BX	EBX	MM3 / ST(3)	XMM3
100	AH	SP	ESP	MM4 / ST(4)	XMM4
101	CH	BP	EBP	MM5 / ST(5)	XMM5
110	DH	SI	ESI	MM6 / ST(6)	XMM6
111	BH	DI	EDI	MM7 / ST(7)	XMM7

反映了体系结构发展的轨迹，字长不断扩充，指令保持兼容

ST (0) ~ ST (7) 是80位，MM0 ~MM7使用其低64位

X86-64架构

- x86-64的基本指令和对齐
 - 数据传送指令（汇编指令中助记符“q”表示操作数长度为四字（即64位））
 - movabsq指令用于将一个64位立即数送到一个64位通用寄存器中；
 - movq指令用于传送一个64位的四字；
 - movsbq、movswq、movslq用于将源操作数进行符号扩展并传送到一个64位寄存器或存储单元中；
 - movzbq、movzwbq用于将源操作数进行零扩展后传送到一个64位寄存器或存储单元中；
 - pushq和popq分别是四字压栈和四字出栈指令；
 - movl指令的功能相当于movzlbq指令。

X86-64架构

- 数据传送指令举例

以下函数功能是将类型为source_type
的参数转换为dest_type型数据并返回

```
dest_type convert(source_type x) {  
    dest_type y = (dest_type) x;  
    return y;  
}
```

根据参数传递约定知，x在RDI对应的
的适合宽度的寄存器（RDI、EDI、
DI和DIL）中，y存放在RAX对应的
寄存器（RAX、EAX、AX或AL）中
，填写下表中的汇编指令以实现
convert函数中的赋值语句

source_type	dest_type	汇 编 指 令
char	long	问题：每种情况对应的 汇编指令各是什么？
int	long	
long	long	
long	int	
unsigned int	unsigned long	
unsigned long	unsigned int	
unsigned char	unsigned long	

X86-64架构

- 数据传送指令举例

以下函数功能是将类型为source_type
的参数转换为dest_type型数据并返回

```
dest_type convert(source_type x) {  
    dest_type y = (dest_type) x;  
    return y;  
}
```

根据参数传递约定知，x在RDI对应的
的适合宽度的寄存器（RDI、EDI、
DI和DIL）中，y存放在RAX对应的
寄存器（RAX、EAX、AX或AL）中
，填写下表中的汇编指令以实现
convert函数中的赋值语句

source_type	dest_type	汇 编 指 令
char	long	movsbq %dil, %rax
int	long	movslq %edi, %rax
long	long	movq %rdi, %rax
long	int	movslq %edi, %rax //符号扩展到 64 位 movl %edi, %eax // 只需x的低32位
unsigned int	unsigned long	movl %edi, %eax //零扩展到 64 位
unsigned long	unsigned int	movl %edi, %eax //零扩展到 64 位
unsigned char	unsigned long	movzbq %dil, %rax //零扩展到 64 位 ¹²⁴

X86-64架构

- 算术逻辑运算指令

- addq (四字相加)
- subq (四字相减)
- imulq (带符号整数四字相乘)
- orq (64位相或)
- leaq (有效地址加载到64位寄存器)

以下是C赋值语句

“ $x=a*b+c*d$;” 对应的x86-64汇编代码，已知x、a、b、c和d分别在寄存器RAX(x)、RDI(a)、RSI(b)、RDX(c)和RCX(d)对应宽度的寄存器中。根据以下汇编代码，推测x、a、b、c和d的数据类型

movslq %ecx, %rcx

imulq %rdx, %rcx

movsbl %sil, %esi

imull %edi, %esi

movslq %esi, %rsi

leaq (%rcx, %rsi), %rax

d从32位符号扩展为64位，故d为int型

在RDX中的c为64位long型

在RSI中的b为char型

在EDI中的a是int型

在RAX中的x是long型

X86-64架构

- 过程调用的参数传递

- 通过寄存器传送参数
- 最多可有6个整型或指针型参数通过寄存器传递
- 超过6个入口参数时，后面的通过栈来传递
- 在栈中传递的参数若是基本类型，则都被分配8个字节
- call (或callq) 将64位返址保存在栈中之前，执行 $R[rsi] \leftarrow R[rsi] - 8$
- ret从栈中取出64位返回地址后，执行 $R[rsi] \leftarrow R[rsi] + 8$

操作数宽度 (字节)	入口参数						返回 参数
	1	2	3	4	5	6	
8	RDI	RSI	RDX	RCX	R8	R9	RAX
4	EDI	ESI	EDX	ECX	R8D	R9D	EAX
2	DI	SI	DX	CX	R8W	R9W	AX
1	DIL	SIL	DL	CL	R8B	R9B	AL

X86-64架构过程调用举例

```
long caller ( )
```

```
{
```

```
char a=1;
```

```
short b=2;
```

```
int c=3;
```

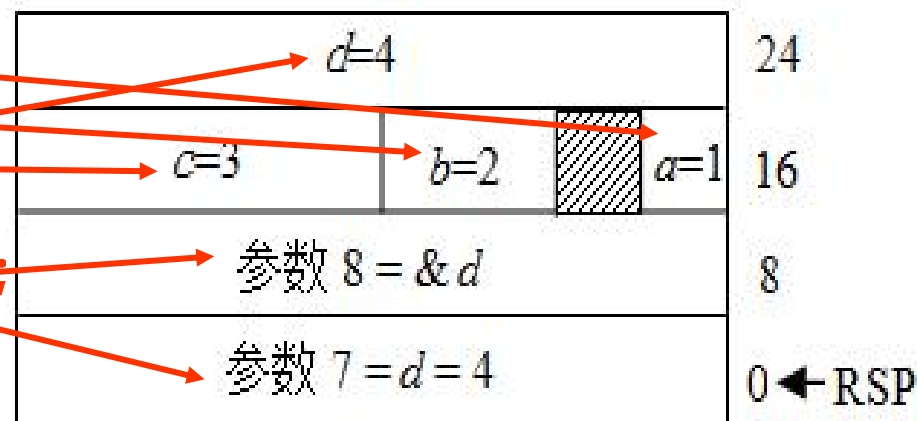
```
long d=4;
```

```
test(a, &a, b, &b, c, &c, d, &d);
```

```
return a*b+c*d;
```

```
}
```

执行到 caller 的 call 指令时栈中情况



其他6个参数在哪里?

```
void test(char a, char *ap,
short b, short *bp,
int c, int *cp,
long d, long *dp)
```

```
{
```

```
*ap+=a;
```

```
*bp+=b;
```

```
*cp+=c;
```

```
*dp+=d;
```

```
}
```

执行到caller的call指令前, 栈中的状态如何?

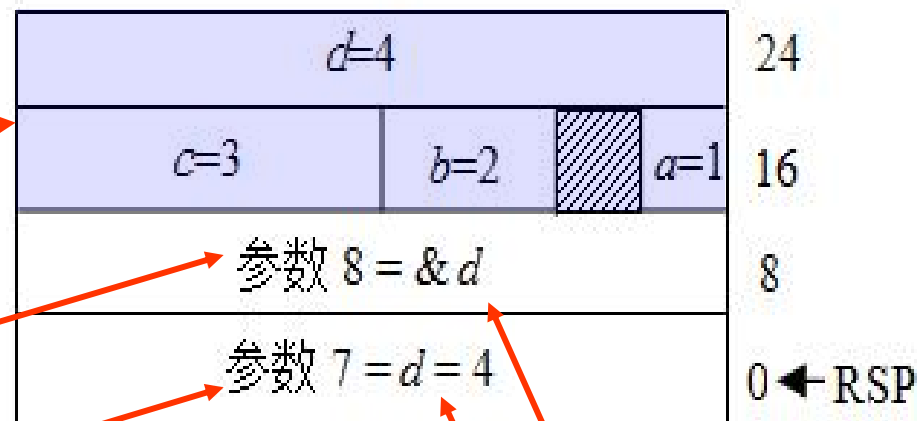
操作数宽度 (字节)	入口参数						返回 参数
	1	2	3	4	5	6	
8	RDI	RSI	RDX	RCX	R8	R9	RAX
4	EDI	ESI	EDX	ECX	R8D	R9D	EAX
2	DI	SI	DX	CX	R8W	R9W	AX
1	DIL	SIL	DL	CL	R8B	R9B	AL

X86-64架构过程调用举例

```
subq $32, %rsp    //R[rsp]←R[rsp]-32
movb $1, 16(%rsp) //M[R[rsp]+16]←1
movw $2, 18(%rsp) //M[R[rsp]+18]←2
movl $3, 20(%rsp) //M[R[rsp]+20]←3
movq $4, 24(%rsp) //M[R[rsp]+24]←4
leaq 24(%rsp), %rax //R[rax]←R[rsp]+24
movq %rax, 8(%rsp)  //M[R[rsp]+8]←R[rax]
movq $4, (%rsp)     //M[R[rsp]]←4
leaq 20(%rsp), %r9 //R[r9]←R[rsp]+20
movl $3, %r8d      //R[r8d]←3
leaq 18(%rsp), %rcx //R[rcx]←R[rsp]+18
movw $2, %dx       //R[dx]←2
leaq 16(%rsp), %rsi //R[rsi]←R[rsp]+16
movb $1, %dil      //R[dil]←1
call test
```

第15条指令

执行到 caller 的 call 指令时栈中情况



long caller ()

```
{
    char a=1;
    short b=2;
    int c=3;
    long d=4;
    test(a, &a, b, &b, c, &c, d, &d);
    return a*b+c*d;
}
```

X86-64架构过程调用举例

```

movq 16(%rsp), %r10 //R[r10]←M[R[rsp]+16]    R[r10]←&d
addb %dil, (%rsi)   //M[R[rsi]]←M[R[rsi]]+R[dil]  *ap+=a;
addw %dx, (%rcx)    //M[R[rcx]]←M[R[rcx]]+R[dx]   *bp+=b;
addl %r8d, (%r9)    //M[R[r9]]←M[R[r9]]+R[r8d]   *cp+=c;
movq 8(%rsp), %rax  //R[rax]←M[R[rsp]+8]
addq %rax, (%r10)   //M[R[r10]]←M[R[r10]]+R[rax] } *dp+=d;
ret
    
```

执行到test的ret指令前，栈中的状态如何？ret执行后怎样？

DIL、RSI、DX、RCX、R8D、R9

void test(char a, char *ap,
short b, short *bp,
int c, int *cp,
long d, long *dp)

```

{
    *ap+=a;
    *bp+=b;
    *cp+=c;
    *dp+=d;
}
    
```

d=8			32
c=6	b=4	a=2	24
参数 8 = &d			16
参数 7 = d = 4			8
返回地址=第 16 行指令所在地址			0 ← RSP

X86-64架构过程调用举例

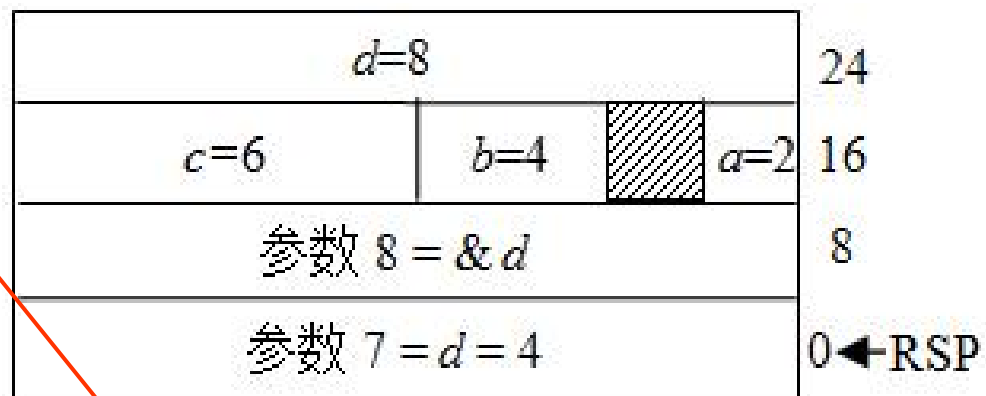
从第16条指令开始

```
movslq 20(%rsp), %rcx
movq 24(%rsp), %rdx
imulq %rdx, %rcx
movsbw 16(%rsp), %ax
movw 18(%rsp), %dx
imulw %dx, %ax
movswq %ax, %rax
leaq (%rax, %rcx), %rax
addq $32, %rsp
ret
```

释放caller的栈帧

执行到ret指令时,
RSP指向调用caller
函数时保存的返回值

执行test的ret指令后, 栈中的状态如何?



long caller ()

```
{
    char a=1;
    short b=2;
    int c=3;
    long d=4;
    test(a, &a, b, &b, c, &c, d, &d);
    return a*b+c*d;
}
```

X86-64架构

- 浮点操作与SIMD指令

- IA-32的浮点处理架构有两种

- (1) x86配套的浮点协处理器x87FPU架构，80位浮点寄存器栈

- (2) 由MMX发展而来的SSE指令集架构，采用的是单指令多数据（Single Instruction Multi Data, SIMD）技术

- 对于IA-32架构，gcc默认生成x87 FPU 指令集代码，如果想要生成SSE指令集代码，则需要设置适当的编译选项

- 在x86-64中，浮点运算采用SIMD指令

- 浮点数存放在128位的XMM寄存器中

- 采用80位双精度扩展格式

- 1位符号位s、15位阶码e（偏置常数为16 383）、1位显式首位有效位（explicit leading significant bit）j和 63位尾数f。它与IEEE 754单精度和双精度浮点格式的一个重要的区别是，它没有隐藏位，有效位数共64位。

浮点寄存器栈和多媒体扩展寄存器组

- IA-32的浮点处理架构有两种：
 - 浮点协处理器x87架构 (x87 FPU)
 - ✓ 8个80位寄存器ST(0) ~ ST(7) (采用栈结构)，栈顶为ST(0)
 - 由MMX发展而来的SSE架构
 - ✓ MMX指令使用8个64位寄存器MM0~MM7，借用8个80位寄存器ST(0)~ST(7)中64位尾数所占的位，可同时处理8个字节，或4个字，或2个双字，或一个64位的数据
 - ✓ MMX指令并没带来3D游戏性能的显著提升，故推出SSE指令，并陆续推出SSE2、SSE3、SSSE3和SSE4等采用SIMD技术的指令集，这些统称为SSE指令集
 - ✓ SSE指令集将80位浮点寄存器扩充到128位多媒体扩展通用寄存器XMM0~XMM7，可同时处理16个字节，或8个字，或4个双字（32位整数或单精度浮点数），或两个四字的数据，而且从SSE2开始，还支持128位整数运算或同时并行处理两个64位双精度浮点数

X87 FPU指令

- 数据传送类

从内存装入栈顶ST(0) 带P结尾指令表示操作数会出栈，也即ST(1)将变成ST(0)

- (1) 装入

FLD: 将数据装入浮点寄存器栈顶

FILD: 将数据从int型转换为浮点格式后，装入浮点寄存器栈顶

- (2) 存储

FSTx: x为s/l时，将栈顶ST(0)转换为单/双精度格式，然后存入存储单元

FSTPx: 弹出栈顶元素，并完成与FSTx相同的功能

FISTx: 将栈顶数据从int型转换为浮点格式后，存入存储单元

FISTP: 弹出栈顶元素，并完成与FISTx相同的功能

不作要求，大概了解一下

- 数据传送类

- (3) 交换

- FXCH: 交换栈顶和次栈顶两元素

- (4) 常数装载到栈顶

- FLD1 : 装入常数1.0

- FLDZ : 装入常数0.0

- FLDPI : 装入常数 π (=3.1415926...)

- FLDL2E : 装入常数 $\log(2)e$

- FLDL2T : 装入常数 $\log(2)10$

- FLDLG2 : 装入常数 $\log(10)2$

- FLDLN2 : 装入常数 $\text{Log}(e)2$

X87 FPU指令

- 算术运算类

(1) 加法

FADD/FADDP: 相加 / 相加后弹出

FIADD: 按int型转换后相加

(2) 减法

FSUB/FSUBP : 相减 / 相减后弹出

FSUBR/FSUBRP: 调换次序相减 / 相减后弹出

FISUB: 按int型相减

FISUBR: 按int型相减, 调换相减次序

若指令未带操作数, 则默认操作数为ST(0) 、 ST(1)

带R后缀指令是指操作数顺序变反, 例如:

fsub执行的是 $x-y$, fsubr执行的就是 $y-x$

X87 FPU指令

- 算术运算类

(3) 乘法

FMUL/FMULP: 相乘/相乘后出栈

FIMUL: 按int型相乘

(4) 除法

FDIV/FDIVP : 相除/相除后出栈

FIDIV: 按int型相除

FDIVR/FDIVRP

FIDIVR

IA-32浮点操作举例

问题：使用老版本gcc -O2编译时，程序一输出0，程序二输出却是1，是什么原因造成的？**f(10)的值是多少？机器数是多少？**

程序一：

```
#include <stdio.h>
double f(int x) {
    return 1.0 / x ;
}
void main() {
    double a, b;
    int i;
    a = f(10) ;
    b = f(10) ;
    i = a == b ;
    printf( "%d\n" , i ) ;
}
```

程序二：

```
#include <stdio.h>
double f(int x) {
    return 1.0 / x ;
}
void main() {
    double a, b, c;
    int i;
    a = f(10) ;
    b = f(10) ;
    c = f(10) ;
    i = a == b ;
    printf( "%d\n" , i ) ;
}
```

习题课讨论题

IA-32浮点操作举例

double f(int x)	8048328: 55	push %ebp
{	8048329: 89 e5	mov %esp,%ebp
 return 1.0 / x ;	804832b: d9 e8	fld1
}	804832d: da 75 08	fidivl 0x8(%ebp)
	8048330: c9	leave
	8048331: c3	ret

两条重要指令的功能如下。

fld1: 将常数1压入栈顶ST(0)

fidivl: 将指定存储单元操作数M[R[ebp]+8]中的int型数转换为double型, 再将ST(0)除以该数, 并将结果存入ST(0)中

f(10)=0.1

0.1=0.00011[0011]B= 0.00011 0011 0011 0011 0011 0011 0011...B

IA-32浮点操作举例

08048334 <main>:

```

8048334:    55                push    %ebp
8048335:    89 e5             mov     %esp,%ebp
8048337:    83 ec 08          sub     $0x8,%esp
804833a:    83 e4 f0          and     $0xffffffff0,%esp
804833d:    83 ec 0c          sub     $0xc,%esp
8048340:    6a 0a             push    $0xa
8048342:    e8 e1 ff ff ff    call   8048328 <f> //计算a=f(10)
8048347:    dd 5d f8          fstpl   0xffffffff8(%ebp) //a存入内存
804834a:    c7 04 24 0a 00 00 00 00 movl    $0xa,(%esp,1)
8048351:    e8 d2 ff ff ff    call   8048328 <f> //计算b=f(10)
8048356:    dd 45 f8          fldl    0xffffffff8(%ebp) //a入栈顶
8048359:    58                pop     %eax
804835a:    da e9            fucompp //比较ST(0)a和ST(1)b
804835c:    df e0            fnstsw %ax //把FPU状态字送到AX
804835e:    80 e4 45          and     $0x45,%ah
8048361:    80 fc 40          cmp     $0x40,%ah
8048364:    0f 94 c0          sete    %al
8048367:    5a                pop     %edx
8048368:    0f b6 c0          movzbl %al,%eax
804836b:    50                push    %eax
804836c:    68 d8 83 04 08    push    $0x80483d8
8048371:    e8 f2 fe ff ff    call   8048268 <_init+0x38>
8048376:    c9                leave
8048377:    c3                ret
    
```

```

...
a = f(10);
b = f(10);
i = a == b;
...
    
```

80位→64位

64位→80位

0.1是无限循环小数，无法精确表示，因而，比较时，a舍入过而b没有舍入过，故 a≠b

IA-32浮点操作举例

```
...  
a = f(10);  
b = f(10);  
c = f(10);  
i = a == b;  
...
```

```
8048342:  e8 e1 ff ff ff    call  8048328 <f> //计算a  
8048347:  dd 5d f8          fstpl  0xffffffff8(%ebp) //把a存回内存  
                        //a产生精度损失  
804834a:  c7 04 24 0a 00 00 00  movl  $0xa,(%esp,1)  
8048351:  e8 d2 ff ff ff    call  8048328 <f> //计算b  
8048356:  dd 5d f0          fstpl  0xffffffff0(%ebp) //把b存回内存  
                        //b产生精度损失  
8048359:  c7 04 24 0a 00 00 00  movl  $0xa,(%esp,1)  
8048360:  e8 c3 ff ff ff    call  8048328 <f> //计算c  
8048365:  dd d8            fstp  %st(0)  
8048367:  dd 45 f8          fldl  0xffffffff8(%ebp) //从内存中载入a  
804836a:  dd 45 f0          fldl  0xffffffff0(%ebp) //从内存中载入b  
804836d:  d9 c9            fxch  %st(1)  
804836f:  58              pop  %eax  
8048370:  da e9            fucompp //比较a , b  
8048372:  df e0            fnstsw %ax
```

0.1是无限循环小数，
无法精确表示，因而
，比较时，a和b都是
舍入过的，故 a=b!

IA-32浮点操作举例

- 从这个例子可以看出
 - 编译器的设计和硬件结构紧密相关。
 - 对于**编译器设计者**来说，只有真正了解底层硬件结构和真正理解指令集体系结构，才能够翻译出没有错误的目标代码，并为程序员完全屏蔽掉硬件实现的细节，方便应用程序员开发出可靠的程序。
 - 对于**应用程序开发者**来说，也只有真正了解底层硬件的结构，才有能力编制出高效的程序，能够快速定位出错的地方，并对程序的行为作出正确的判断。

IA-32浮点操作举例

C/C++ code



```
1  #include "stdafx.h"
2  int main(int argc, char* argv[])
3  {
4      int a=10;
5      double *p=(double*)&a;
6      printf("%f\n", *p);           //结果为0.000000
7      printf("%f\n", (double)a);   //结果为10.000000
8
9      return 0;
10 }
11 为什么printf("%f", *p)和printf("%f", (double)a)结果不一样呢?
```

不都是强制类型转换吗？怎么会不一样

关键差别在于一条指令：

fldl 和 **fildl**

IA-32浮点操作举例

int a = 10;

8048425: c7 44 24 28 0a 00 00 00 movl \$0xa,0x28(%esp)

double *p = (double *)&a;

804842d: 8d 44 24 28 lea 0x28(%esp),%eax

8048431: 89 44 24 2c mov %eax,0x2c(%esp)

可以看到关于指针的类型转换在汇编层次并没有体现出来，都是直接 mov 过去

printf("%lf\n", *p);

8048435: 8d 44 24 2c

mov 0x2c(%esp),%eax

fildl (%eax)

度加载到浮点栈顶 **ST(0)**)

fstpl 0x4(%esp)

的类型是 **double**，故按 **64 位** 压栈)

movl \$0x8048500,(%esp)

call 8048300 <printf@plt>

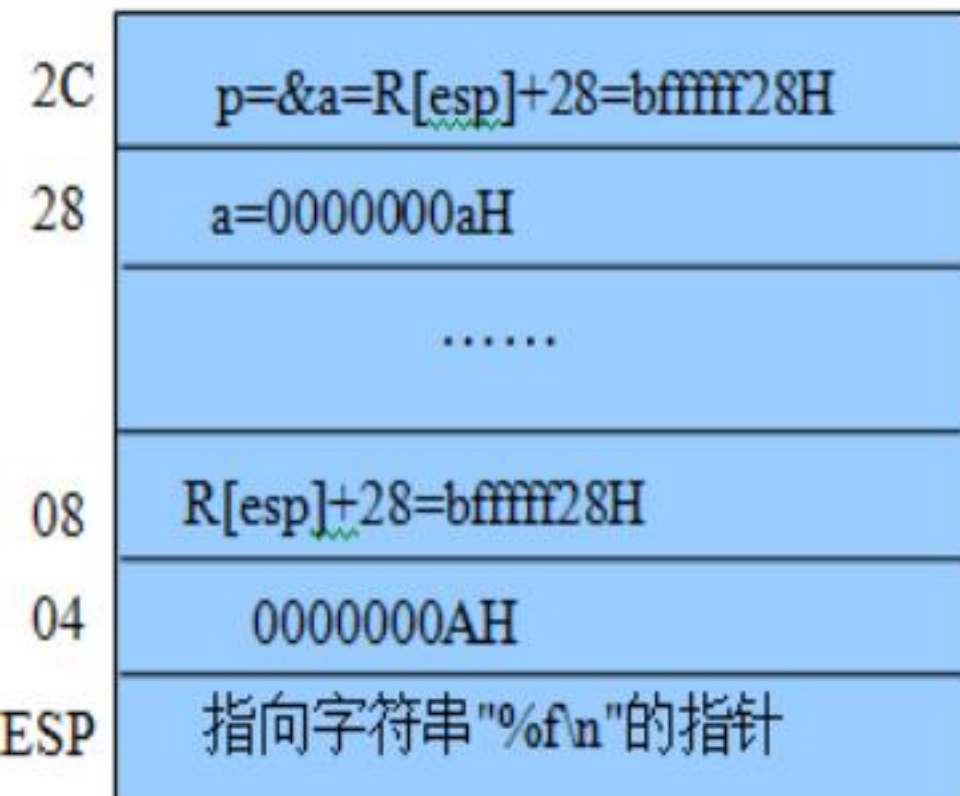
mov 0x28(%esp),%eax

mov %eax,0x1c(%esp)

操作，把变量 **a** 的值移来移去

fildl 0x1c(%esp)

是 fildl 指令，和上面用的 fildl 指令不一样!



IA-32浮点操作举例

- 有一个回帖如是说，其实也是一知半解！

(1)

首先建议你看下 IEEE754中关于浮点数的存储。然后看我给你讲的。

a是int型，内存（小端）中的表示是

0000 1010 0000 0000 0000 0000 0000 0000 后面的位不确定

double型占用8 个字节，如果将上述字节看做double，第一位是符号位，第2~12位是阶码，第12~64是位数，

0000 1010 0000 0000 0000 0000 0000 0000 后面的不确定，

那么你可以转换成 实数算下，应该很小（小数点的后6位肯定都是零），输出的时候默认为6位小数，发生截断，所以是

0.000000

(2) printf("%f\n", (double(a))); 发生类型转化，这个可以，一般 sizeof 比较小的类型可以转换成 size 比较大的类型，或者是类型提升或者是转换

请问：
这个帖子的回答中，
哪些是正确的？ 哪些
是错误的？

IA-32浮点操作举例

- 有一个回帖如是说

回复次数：14

[对我有用\[0\]](#) [丢个板砖\[0\]](#) [引用](#) | [举报](#) | [管理](#)

回复于：2012-03-05 21:30:48

#1 得分：5

一只鸡，上帝说它是狗，它就能变成只狗。

把一只鸡放一个盒子里，上帝告诉你盒子里是只狗，可其实里面还是只鸡，你当它是狗就怎么看怎么奇怪。

计算机专业人员应该是系统的“上帝”，怎么能不明白自己呢！

IA-32和x86-64的比较

例：以下是一段C语言代码：

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    double a = 10;
```

```
    printf("a = %d\n", a);
```

```
}
```

在IA-32上运行时，打印结果为a=0

在x86-64上运行时，打印一个不确定值

为什么？

$10 = 1010B = 1.01 \times 2^3$

阶码 $e = 1023 + 3 = 10000000010B$

10的double型表示为：

0 10000000010 0100...0B

即4024 0000 0000 0000H

← 先执行fldl，再执行fstpl

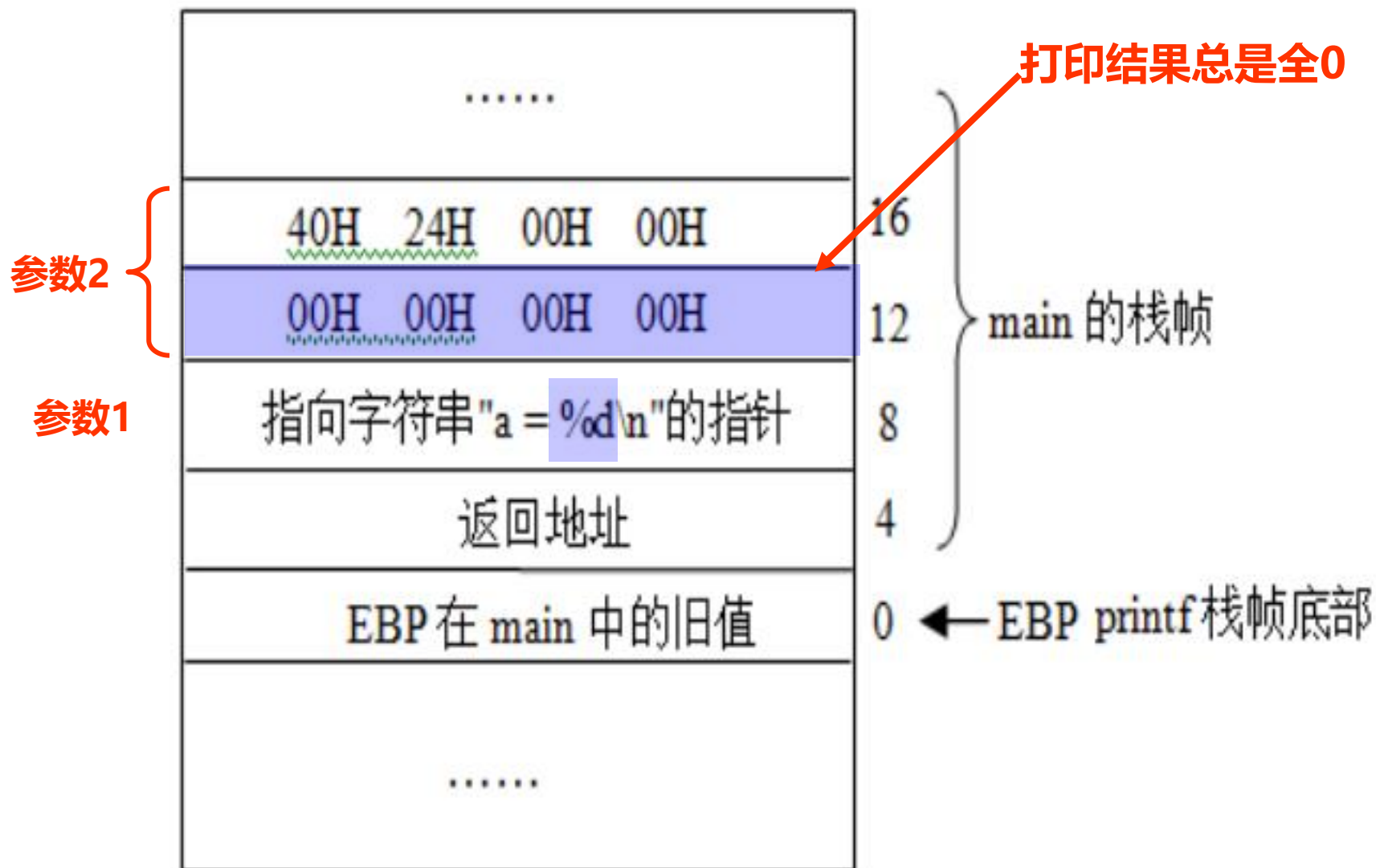
fldl：局部变量区→ST(0)

fstpl：ST(0) →参数区

在IA-32中a为float型又怎样呢？先执行flds，再执行fstpl

即：flds将32位单精度转换为80位格式入浮点寄存器栈，fstpl再将80位转换为64位送存储器栈中，故实际上与a是double效果一样！

IA-32过程调用参数传递



a的机器数对应十六进制为：40 24 00 00 00 00 00 00H

X86-64过程调用参数传递

```
main()
```

```
{
    double a = 10;
    printf("a = %d\n", a);
}
```

操作数宽度 (字节)	入口参数						返回 参数
	1	2	3	4	5	6	
8	RDI	RSI	RDX	RCX	R8	R9	RAX
4	EDI	ESI	EDX	ECX	R8D	R9D	EAX
2	DI	SI	DX	CX	R8W	R9W	AX
1	DIL	SIL	DL	CL	R8B	R9B	AL

.LC1:

.string "a = %d\n "

```
.....
movsd .LC0(%rip), %xmm0 //a送xmm0
```

```
movl $.LC1, %edi //RDI 高32位为0
```

```
movl $1, %eax //向量寄存器个数
```

```
call printf
```

```
addq $8, %rsp
```

```
ret
```

.....

.LC0:

```
.long 0 ← 00000000H
```

```
.long 1076101120 ← 40240000H
```

小端方式! 0存在低地址上

printf中为%d, 故将从ESI中取打印参数进行处理; 但a是double型数据, 在x86-64中, a的值被送到XMM寄存器中而不会送到ESI中。故在printf执行时, 从ESI中读取的并不是a的低32位, 而是一个不确定的值。

X86-64架构

- 数据的对齐

- x86-64中各类型数据遵循一定的对齐规则，而且更严格
- x86-64中存储器访问接口被设计成按8字节或16字节为单位进行存取，其对齐规则是，任何K字节宽的基本数据类型和指针类型数据的起始地址一定是K的倍数。
 - short型数据必须按2字节边界对齐
 - int、float等类型数据必须按4字节边界对齐
 - long型、double型、指针型变量必须按8字节边界对齐
 - long double型数据必须按16字节边界对齐

程序的机器级表示

- 分以下六个部分介绍

- 第一讲：程序转换概述

- 机器指令和汇编指令
- 机器级程序员感觉到的属性和功能特性
- 高级语言程序转换为机器代码的过程

- 第二讲：IA-32 /x86-64指令系统

- 第三讲：C语言程序的机器级表示

- 过程调用的机器级表示
- 选择语句的机器级表示
- 循环结构的机器级表示

- 第四讲：复杂数据类型的分配和访问

- 数组的分配和访问
- 结构体数据的分配和访问
- 联合体数据的分配和访问
- 数据的对齐

- 第五讲：越界访问和缓冲区溢出

- 第六讲：x86-64架构（自学）

从高级语言程序出发，用其对应的机器级代码以及内存（栈）中信息的变化来说明底层实现

围绕C语言中的语句和复杂数据类型，解释其在底层机器级的实现方法

第四章 程序的链接

目标文件格式

符号解析与重定位

共享库与动态链接

可执行文件的链接生成

- **主要教学目标**
 - 掌握链接器是如何工作的，从而养成良好的程序设计习惯，并增加程序调试能力。
 - 掌握可执行文件的存储器映像来进一步深入理解进程的虚拟地址空间的概念。
- **包括以下内容**
 - 链接和静态链接概念
 - 三种目标文件格式
 - 符号及符号表、符号解析
 - 使用静态库链接
 - 重定位信息及重定位过程
 - 可执行文件的存储器映像
 - 可执行文件的加载
 - 共享（动态）库链接

程序的链接

- 分以下三个部分介绍

- **第一讲：目标文件格式**

- 程序的链接概述、链接的意义与过程
 - ELF目标文件、重定位目标文件格式、可执行目标文件格式

- **第二讲：符号解析与重定位**

- 符号和符号表、符号解析
 - 与静态库的链接
 - 重定位信息、重定位过程
 - 可执行文件的加载

- **第三讲：动态链接**

- 动态链接的特性、程序加载时的动态链接、程序运行时的动态链接、动态链接举例

一个典型程序的转换处理过程

经典的 “hello.c” C-源程序

```
#include <stdio.h>

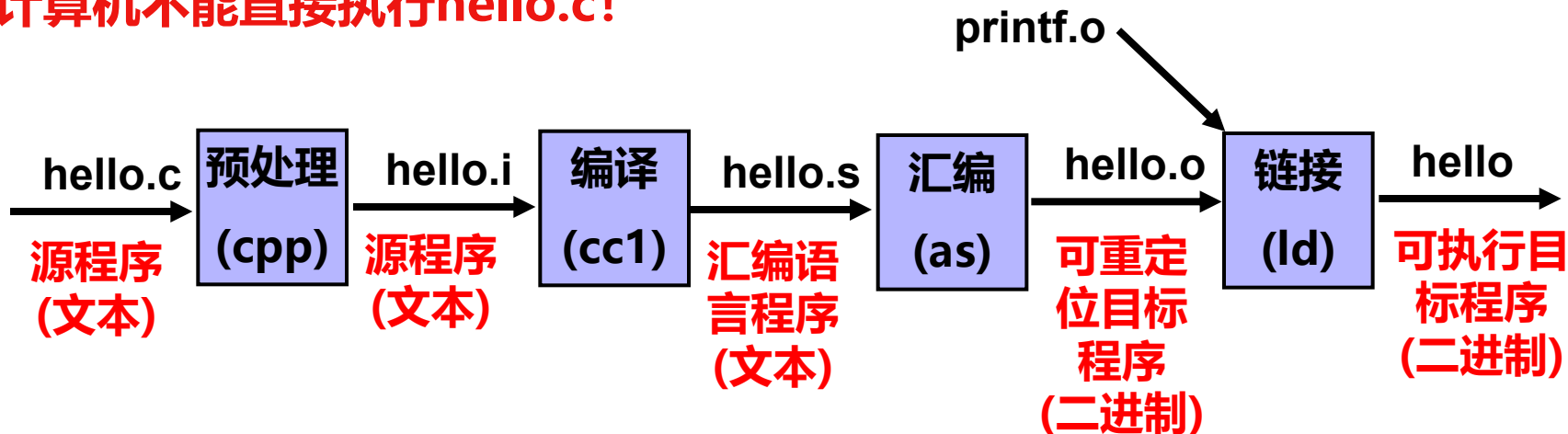
int main()
{
    printf("hello, world\n");
}
```

hello.c的ASCII文本表示 文本文件

```
# i n c l u d e < s p > < s t d i o .
35 105 110 99 108 117 100 101 32 60 115 116 100 105 111 46
h > \n \n i n t < s p > m a i n ( ) \n {
104 62 10 10 105 110 116 32 109 97 105 110 40 41 10 123
\n < s p > < s p > < s p > < s p > p r i n t f ( " h e l
10 32 32 32 32 112 114 105 110 116 102 40 34 104 101 108
l o , < s p > w o r l d \n " ) ; \n }
108 111 44 32 119 111 114 108 100 92 110 34 41 59 10 125
```

功能：输出 “hello,world”

计算机不能直接执行hello.c!



链接器的由来

- 原始的连接概念早在高级编程语言出现之前就已存在
- 最早程序员用机器语言编写程序，并记录在纸带或卡片上



穿孔表示0，未穿孔为1
假设：0010-jmp

0: 0101 0110

1: 0010 0101

2:

3:

4:

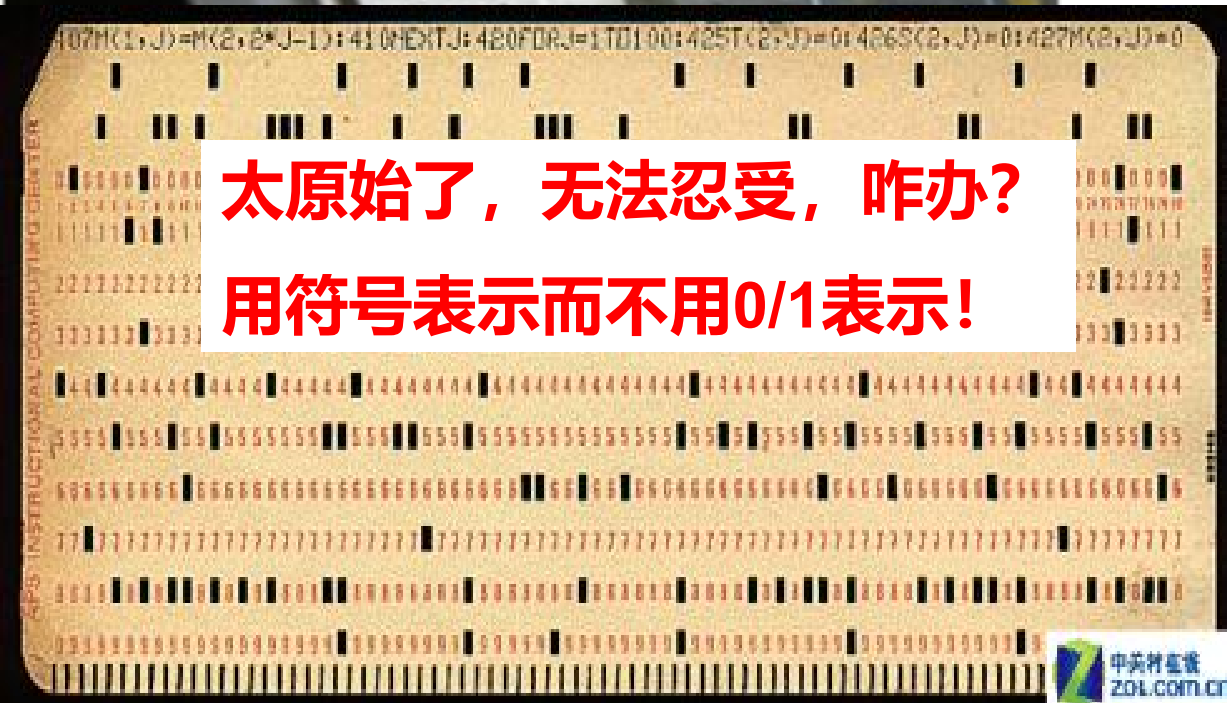
5: 0110 0111

6:

太原始了，无法忍受，咋办？

用符号表示而不用0/1表示！

若在第5条指令前加入指令，则程序员需重新计算jmp指令的目标地址（重定位），然后重新打孔。



链接器的由来

- 用**符号**表示跳转位置和变量位置，是否简化了问题？

- 汇编语言出现

- 用助记符表示操作码
- 用**符号**表示位置
- 用助记符表示寄存器
-

0:	0101 0110	add B
1:	0010 0101	jmp L0
2:		
3:		
4:		
5:	0110 0111	L0: sub C
6:		

- 更高级编程语言出现

- 程序越来越复杂，需多人开发不同的程序模块
- 子程序（函数）起始地址和变量起始地址是**符号定义**（definition）
- 调用子程序（函数或过程）和使用变量即是**符号的引用**（reference）
- 一个模块定义的符号可以被另一个模块引用
- 最终须链接（即合并），合并时须在符号引用处填入定义处的地址

如上例，先确定L0的地址，再在jmp指令中填入L0的地址

使用链接的好处

链接带来的好处1：模块化

- (1) 一个程序可以分成很多源程序文件
- (2) 可构建公共函数库，如数学库，标准C库等

链接带来的好处2：效率高

- (1) 时间上，可分开编译

只需重新编译被修改的源程序文件，然后重新链接

- (2) 空间上，无需包含共享库所有代码

源文件中无需包含共享库函数的源码，只要直接调用即可
可执行文件和运行时的内存中只需包含所调用函数的代码
而不需要包含整个共享库

一个C语言程序举例

main.c

```
int buf[2] = {1, 2};  
void swap();  
  
int main()  
{  
    swap();  
    return 0;  
}
```

swap.c

```
extern int buf[];  
int *bufp0 = &buf[0];  
static int *bufp1;  
void swap()  
{  
    int temp;  
    bufp1 = &buf[1];  
    temp = *bufp0;  
    *bufp0 = *bufp1;  
    *bufp1 = temp;  
}
```

你能说出哪些是**符号定义**？哪些是**符号的引用**？

局部变量temp分配在栈中，不会在过程外被引用，因此不是符号定义

可执行文件的生成

- 使用GCC编译器编译并链接生成可执行程序P:

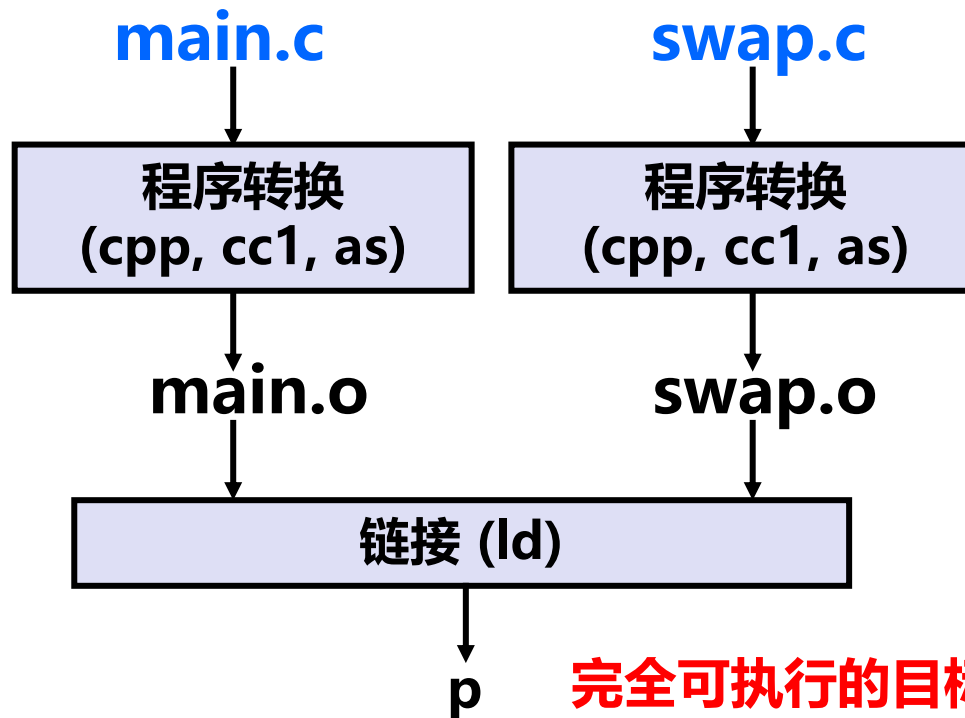
- `$ gcc -O2 -g -o p main.c swap.c`
- `$ ./p`

-O2: 2级优化

-g: 生成调试信息

-o: 目标文件名

GCC
编译
器的
静态
链接
过程



源程序文件

分别转换（预处理、编译、汇编）为可重定位目标文件

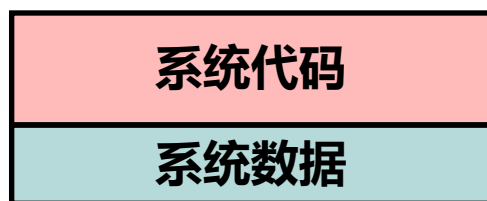
完全可执行的目标文件

链接过程的本质

链接本质：合并相同的“节”

可执行目标文件

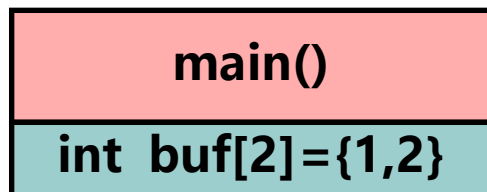
可重定位目标文件



.text

.data

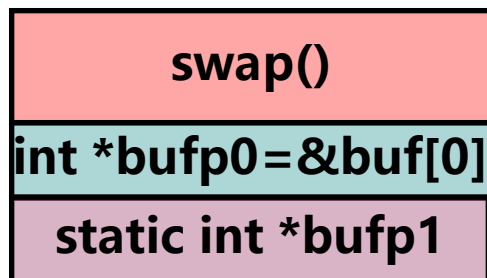
main.o



.text

.data

swap.o

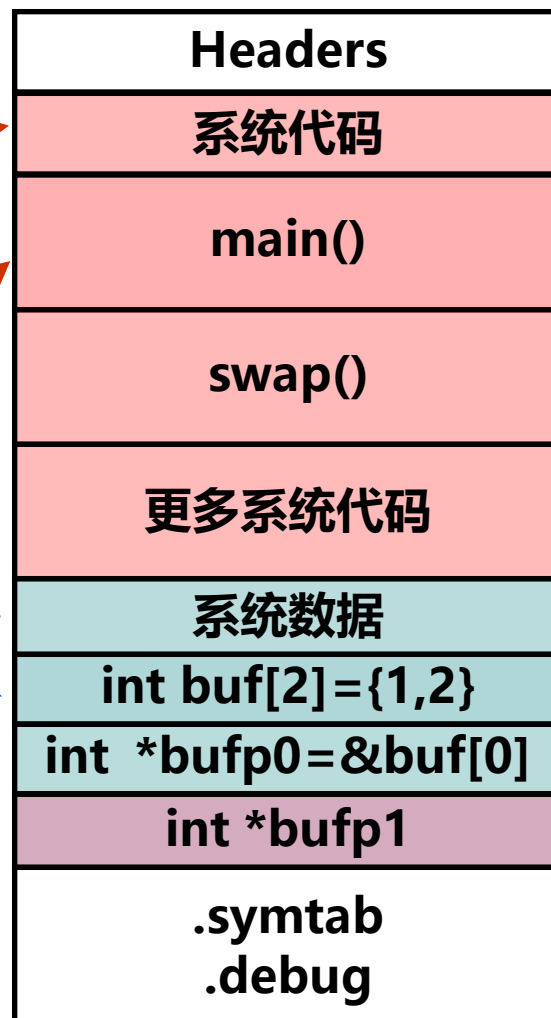


.text

.data

.bss

0



.text

.data

.bss

目标文件

```
/* main.c */
int add(int, int);
int main( )
{
    return add(20, 13);
}
```

```
/* test.c */
int add(int i, int j)
{
    int x = i + j;
    return x;
}
```

00000000

0:

55

1:

89 e5

3:

83 ec 10

6:

8b 45 0c

9:

8b 55 08

c:

8d 04 02

f:

89 45 fc

12:

8b 45 fc

15:

c9

16:

c3

<add>:

objdump -d test.o

push %ebp

mov %esp, %ebp

sub \$0x10, %esp

mov 0xc(%ebp), %eax

mov 0x8(%ebp), %edx

lea (%edx,%eax,1), %eax

mov %eax, -0x4(%ebp)

mov -0x4(%ebp), %eax

leave

ret

080483d4

80483d4:

55

80483d5:

89 e5

80483d7:

83 ec 10

80483da:

8b 45 0c

80483dd:

8b 55 08

80483e0:

8d 04 02

80483e3:

89 45 fc

80483e6:

8b 45 fc

80483e9:

c9

80483ea:

c3

<add>:

objdump -d test

push %ebp

mov %esp, %ebp

sub \$0x10, %esp

mov 0xc(%ebp), %eax

mov 0x8(%ebp), %edx

lea (%edx,%eax,1), %eax

mov %eax, -0x4(%ebp)

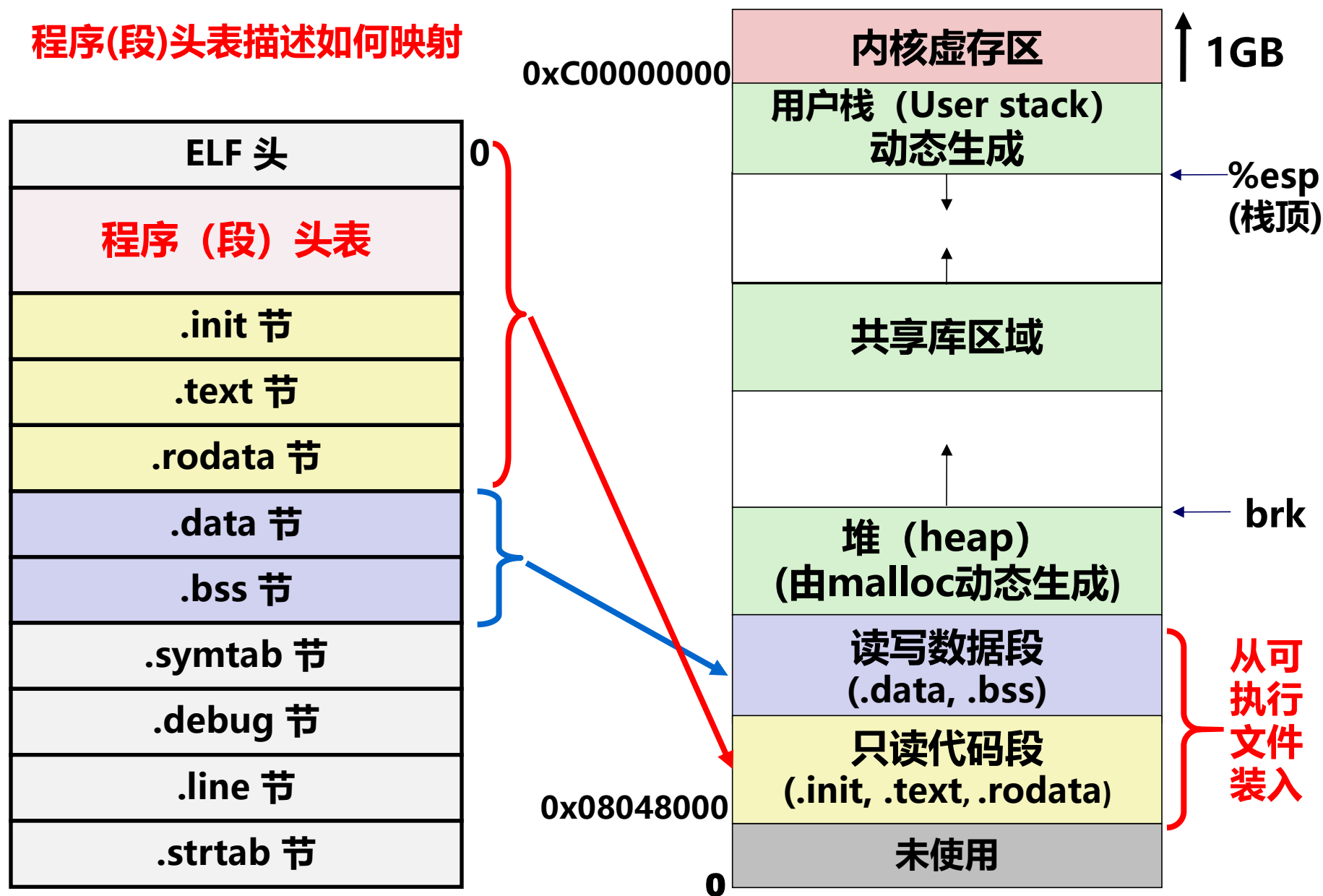
mov -0x4(%ebp), %eax

leave

ret

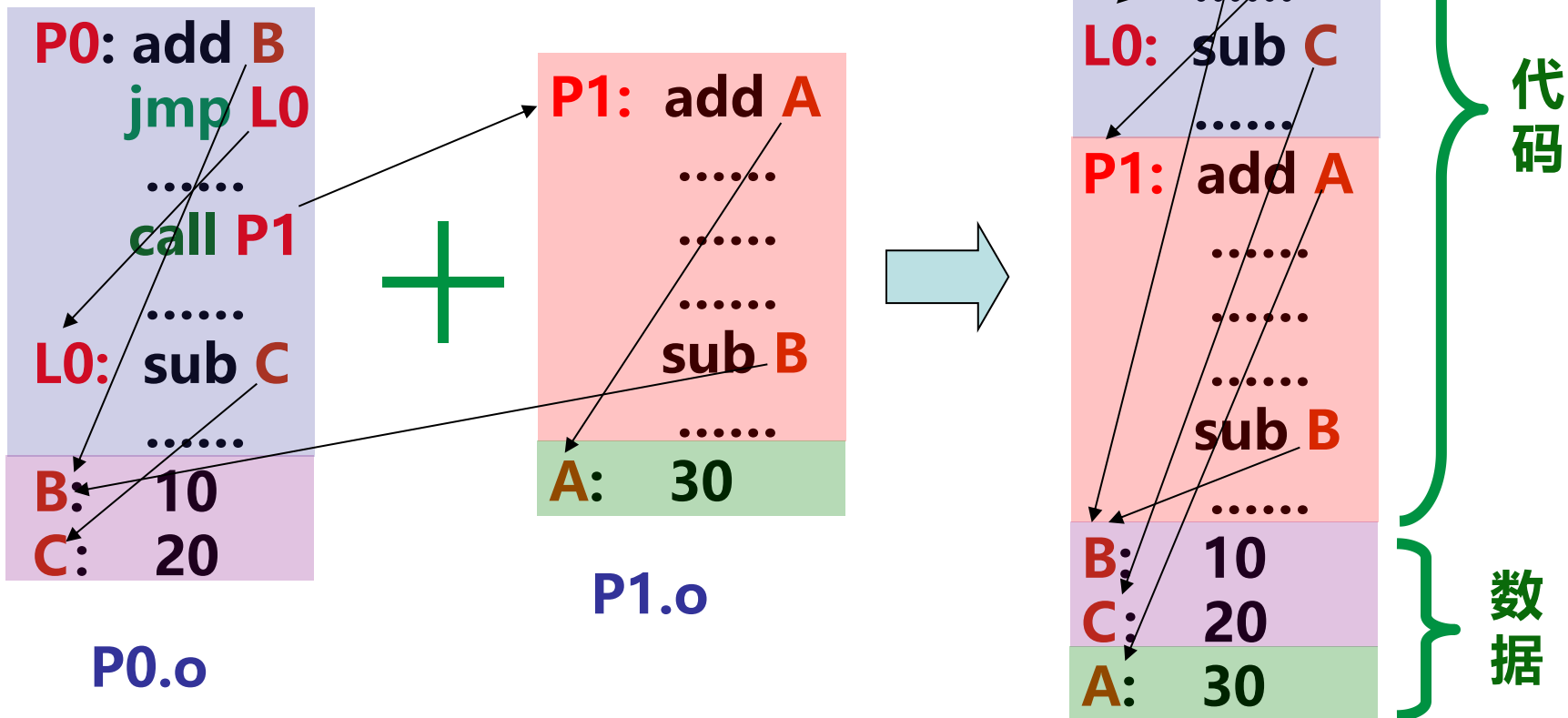
可执行文件的存储器映像

程序(段)头表描述如何映射



链接操作的步骤

- 1) 确定标号引用关系 (符号解析)
 - 2) 合并相关.o文件
 - 3) 确定每个标号的地址
 - 4) 在指令中填入新地址
- } 重定位



链接操作的步骤

add B
jmp L0
.....
.....
.....
L0: sub C
.....

- Step 1. 符号解析 (Symbol resolution)
 - 程序中有定义和引用的符号 (包括变量和函数等)
 - void swap() {...} /* 定义符号swap */
 - swap(); /* 引用符号swap */
 - int *xp = &x; /* 定义符号 xp, 引用符号 x */
 - 编译器将定义的符号存放在一个符号表 (symbol table) 中.
 - 符号表是一个结构数组
 - 每个表项包含符号名、长度和位置等信息
 - 链接器将每个符号的引用都与一个确定的符号定义建立关联
- Step 2. 重定位
 - 将多个代码段与数据段分别合并为一个单独的代码段和数据段
 - 计算每个定义的符号在虚拟地址空间中的绝对地址
 - 将可执行文件中符号引用处的地址修改为重定位后的地址信息

三类目标文件

- 可重定位目标文件 (.o)
 - 其代码和数据可和其他可重定位文件合并为可执行文件
 - 每个.o 文件由对应的.c文件生成
 - 每个.o文件代码和数据地址都从0开始
- 可执行目标文件 (默认为a.out)
 - 包含的代码和数据可以被直接复制到内存并被执行
 - 代码和数据地址为虚拟地址空间中的地址
- 共享的目标文件 (.so)
 - 特殊的可重定位目标文件，能在装入或运行时被装入到内存并自动被链接，称为共享库文件
 - Windows 中称其为 *Dynamic Link Libraries* (DLLs)

目标文件的格式

- **目标代码 (Object Code)** 指编译器和汇编器处理源代码后所生成的机器语言目标代码
- **目标文件 (Object File)** 指包含目标代码的文件
- 最早的目标文件格式是自有格式，非标准的
- 标准的几种目标文件格式
 - DOS操作系统（最简单）：**COM格式**，文件中仅包含代码和数据，且被加载到固定位置
 - System V UNIX早期版本：**COFF (common object file format) 格式**，文件中不仅包含代码和数据，还包含重定位信息、调试信息、符号表等其他信息，由一组严格定义的数据结构序列组成
 - Windows：**PE格式**（COFF的变种），称为可移植可执行（Portable Executable，简称PE）
 - Linux等类UNIX：**ELF格式**（COFF的变种），称为可执行可链接（Executable and Linkable Format，简称ELF）

Executable and Linkable Format (ELF)

- 两种视图
 - 链接视图（被链接）：Relocatable object files
 - 执行视图（被执行）：Executable object files



链接视图

节 (section) 是 ELF 文件中具有相同特征的最小可处理单位

.text节: 代码

.data节: 数据

.rodata: 只读数据

.bss: 未初始化数据

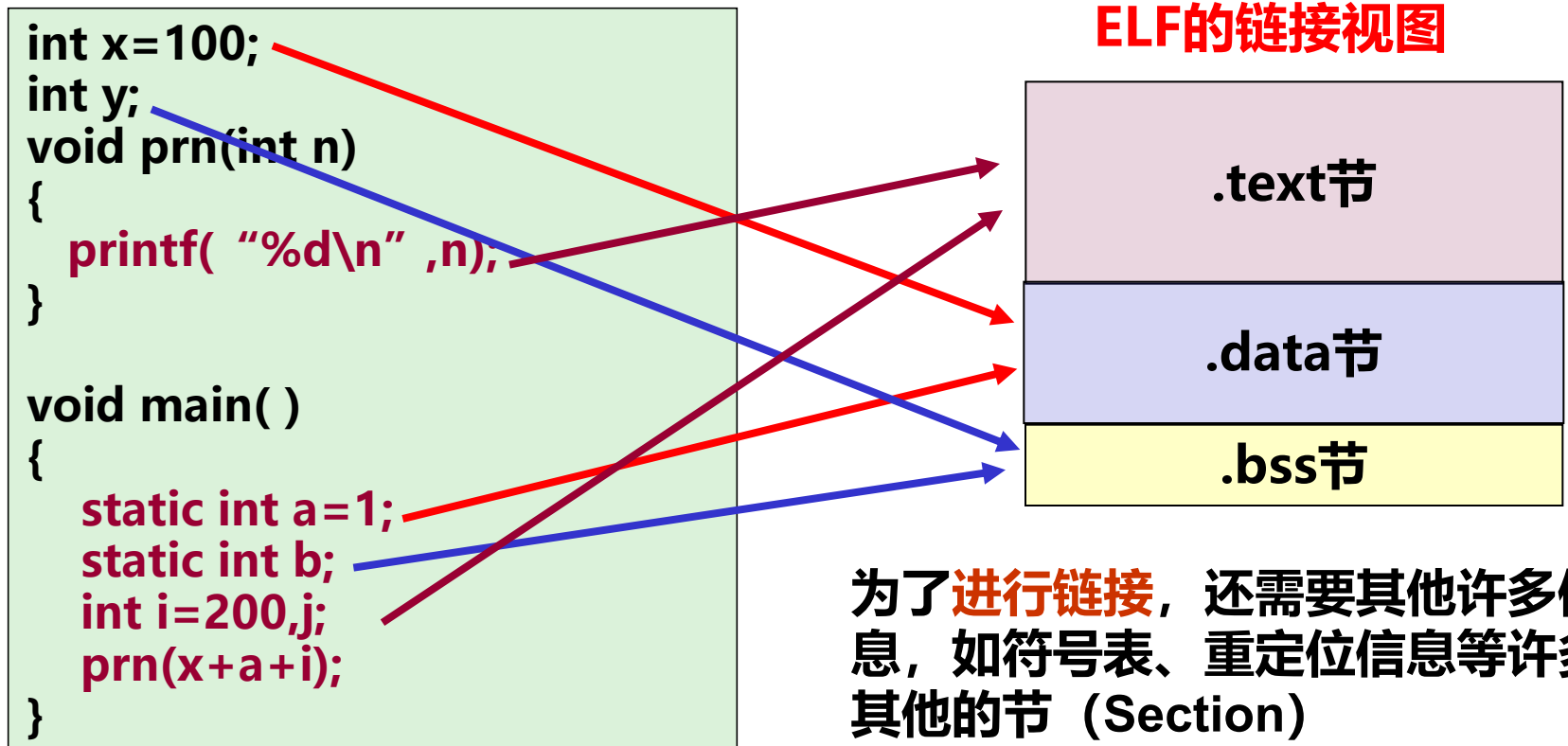


执行视图

由不同的段 (segment) 组成, 描述节如何映射到存储段中, 可多个节映射到同一段, 如: 可合并.data节和.bss节, 并映射到一个可读可写数据段中

链接视图—可重定位目标文件

- 可被链接（合并）生成可执行文件或**共享目标文件**
- **静态链接库文件**由若干个可重定位目标文件组成
- 包含代码、数据（已初始化.data和未初始化.bss）
- 包含**重定位信息**（指出哪些符号引用处需要重定位）
- 文件扩展名为.o（相当于Windows中的 .obj文件）



可重定位目标文件格式

0

ELF 头

- ✓ 占16字节，包括字长、字节序（大端/小端）、文件类型（.o, exec, .so）、机器类型（如 IA-32）、节头表的偏移、节头表的表项大小及表项个数

.text 节

- ✓ 编译后的代码部分

.rodata 节

- ✓ 只读数据，如 printf 格式串、switch 跳转表等

.data 节

- ✓ 已初始化的全局变量

.bss 节

- ✓ 未初始化全局变量，仅是占位符，不占据任何实际磁盘空间。区分初始化和非初始化是为了空间效率

ELF 头
.text 节
.rodata 节
.data 节
.bss 节
.symtab 节
.rel.txt 节
.rel.data 节
.debug 节
.strtab 节
.line 节
Section header table (节头表)


```
void main ( )
```

```
{  int x;  
    printf( "x=%d\n" , x);  
}
```

字符串 "x=%d\n" 属于只读数据

0804841c <main>:

804841c:	55	push	%ebp
804841d:	89 e5	mov	%esp,%ebp
804841f:	83 e4 f0	and	\$0xffffffff0,%esp
8048422:	83 ec 20	sub	\$0x20,%esp
8048425:	8b 44 24 1c	mov	0x1c(%esp),%eax
8048429:	89 44 24 04	mov	%eax,0x4(%esp)
804842d:	c7 04 24 d0 84 04 08	movl	\$0x80484d0,(%esp)
8048434:	e8 c7 fe ff ff	call	8048300 <printf@plt>
8048439:	c9	leave	
804843a:	c3	ret	

switch-case语句举例

```
int sw_test(int a, int b, int c)
{
    int result;
    switch(a) {
    case 15:
        c=b&0x0f;
    case 10:
        result=c+50;
        break;
    case 12:
    case 17:
        result=b+50;
        break;
    case 14:
        result=b;
        break;
    default:
        result=a;
    }
    return result;
}
```

```
    movl 8(%ebp), %eax
    subl $10, %eax
    cmpl $7, %eax
    ja   .L5
    jmp  *.L8(, %eax, 4)
.L1:
    movl 12(%ebp), %eax
    andl $15, %eax
    movl %eax, 16(%ebp)
.L2:
    movl 16(%ebp), %eax
    addl $50, %eax
    jmp  .L7
.L3:
    movl 12(%ebp), %eax
    addl $50, %eax
    jmp  .L7
.L4:
    movl 12(%ebp), %eax
    jmp  .L7
.L5:
    addl $10, %eax
.L7:
```

$R[eax] = a - 10 = i$

if $(a - 10) > 7$ 转 L5

转 $.L8 + 4 * i$ 处的地址

跳转表在目标文件的只读节中，按4字节边界对齐。

.section .rodata		
.align 4		
.L8	a =	
.long .L2	10	
.long .L5	11	
.long .L3	12	
.long .L5	13	
.long .L4	14	
.long .L1	15	
.long .L5	16	
.long .L3	17	

ELF头 (ELF Header)

- ELF头位于ELF文件开始，包含文件结构说明信息。分32位系统对应结构和64位系统对应结构（32位版本、64位版本）
- 以下是32位系统对应的数据结构

```
#define EI_NIDENT      16
typedef struct {
    unsigned char    e_ident[EI_NIDENT];
    Elf32_Half       e_type;
    Elf32_Half       e_machine;
    Elf32_Word       e_version;
    Elf32_Addr       e_entry;
    Elf32_Off        e_phoff;
    Elf32_Off        e_shoff;
    Elf32_Word       e_flags;
    Elf32_Half       e_ehsize;
    Elf32_Half       e_phentsize;
    Elf32_Half       e_phnum;
    Elf32_Half       e_shentsize;
    Elf32_Half       e_shnum;
    Elf32_Half       e_shstrndx;
} Elf32_Ehdr;
```

定义了ELF魔数、版本、小端/大端、操作系统平台、目标文件的类型、机器结构类型、程序执行的入口地址、程序头表（段头表）的起始位置和长度、节头表的起始位置和长度等

魔数：文件开头几个字节通常用来确定文件的类型或格式

a.out的魔数：01H 07H

PE格式魔数：4DH 5AH

加载或读取文件时，可用魔数确认文件类型是否正确

ELF头信息举例

\$ readelf -h main.o 可重定位目标文件的ELF头

ELF Header: **ELF文件的魔数**

Magic: 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00 00

Class: ELF32

Data: 2's complement, little endian

Version: 1 (current)

OS/ABI: UNIX - System V

ABI Version: 0

Type: REL (Relocatable file)

Machine: Intel 80386

Version: 0x1

Entry point address: 0x0

Start of program headers: 0 (bytes into file)

Start of section headers: 516 (bytes into file)

Flags: 0x0

Size of this header: 52 (bytes)

Size of program headers: 0 (bytes)

Number of program headers: 0

Size of section headers: 40 (bytes)

Number of section headers: 15

Section header string table index: 12

没有程序头表

15x40B

ELF 头

.text 节

.rodata 节

.data 节

.bss 节

.symtab 节

.rel.txt 节

.rel.data 节

.debug 节

.strtab 节

.line 节

Section header
(节头表)

节头表 (Section Header Table)

- 除ELF头之外，节头表是ELF可重定位目标文件中最重要的部分内容
- 描述每个节的节名、在文件中的偏移、大小、访问属性、对齐方式等
- 以下是32位系统对应的数据结构（每个表项占40B）

```
typedef struct {  
    Elf32_Word    sh_name; 节名字符串在.strtab中的偏移  
    Elf32_Word    sh_type; 节类型：无效/代码或数据/符号/字符串/...  
    Elf32_Word    sh_flags; 节标志：该节在虚拟空间中的访问属性  
    Elf32_Addr    sh_addr; 虚拟地址：若可被加载，则对应虚拟地址  
    Elf32_Off     sh_offset; 在文件中的偏移地址，对.bss节而言则无意义  
    Elf32_Word    sh_size; 节在文件中所占的长度  
    Elf32_Word    sh_link; sh_link和sh_info用于与链接相关的节（如  
    Elf32_Word    sh_info; .rel.text节、.rel.data节、.symtab节等）  
    Elf32_Word    sh_addralign; 节的对齐要求  
    Elf32_Word    sh_entsize; 节中每个表项的长度，0表示无固定长度表项  
} Elf32_Shdr;
```

节头表信息举例

\$ readelf -S test.o

There are 11 section headers, starting at offset 0x120:

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	Al
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	.text	PROGBITS	00000000	000034	00005b	00	AX	0	0	4
[2]	.rel.text	REL	00000000	000498	000028	08		9	1	4
[3]	.data	PROGBITS	00000000	000090	00000c	00	WA	0	0	4
[4]	.bss	NOBITS	00000000	00009c	00000c	00	WA	0	0	4
[5]	.rodata	PROGBITS	00000000	00009c	000004	00	A	0	0	1
[6]	.comment	PROGBITS	00000000	0000a0	00002e	00		0	0	1
[7]	.note.GNU-stack	PROGBITS	00000000	0000ce	000000	00		0	0	1
[8]	.shstrtab	STRTAB	00000000	0000ce	000051	00		0	0	1
[9]	.symtab	SYMTAB	00000000	0002d8	000120	10		10	13	4
[10]	.strtab	STRTAB	00000000	0003f8	00009e	00		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings)

I (info), L (link order), G (group), x (unknown)

O (extra OS processing required) o (OS specific), p (processor specific)

可重定位目标文件中，每个可装入节的起始地址总是0

ELF 头
.text 节
.rodata 节
.data 节
.bss 节
.symtab 节
.rel.txt 节
.rel.data 节
.debug 节
.strtab 节
.line 节
Section header (节头表)

节头表信息举例

\$ readelf -S test.o

There are 11 section headers, starting at offset 0x120:

Section Headers:

[Nr]	Name	Off	Size	ES	Flg	Lk	Inf	AI
[0]		000000	000000	00		0	0	0
[1]	.text	000034	00005b	00	AX	0	0	4
[2]	.rel.text	000498	000028	08		9	1	4
[3]	.data	000090	00000c	00	WA	0	0	4
[4]	.bss	00009c	00000c	00	WA	0	0	4
[5]	.rodata	00009c	000004	00	A	0	0	1
[6]	.comment	0000a0	00002e	00		0	0	1
[7]	.note.GNU-stack	0000ce	000000	00		0	0	1
[8]	.shstrtab	0000ce	000051	00		0	0	1
[9]	.symtab	0002d8	000120	10		10	13	4
[10]	.strtab	0003f8	00009e	00		0	0	1

Key to Flags:

W (write), A (alloc), X (execute), M (merge), S (strings)

I (info), L (link order), G (group), x (unknown)

有4个节将会分配 (A) 存储空间

.text: 可执行

.data和.bss: 可读可写

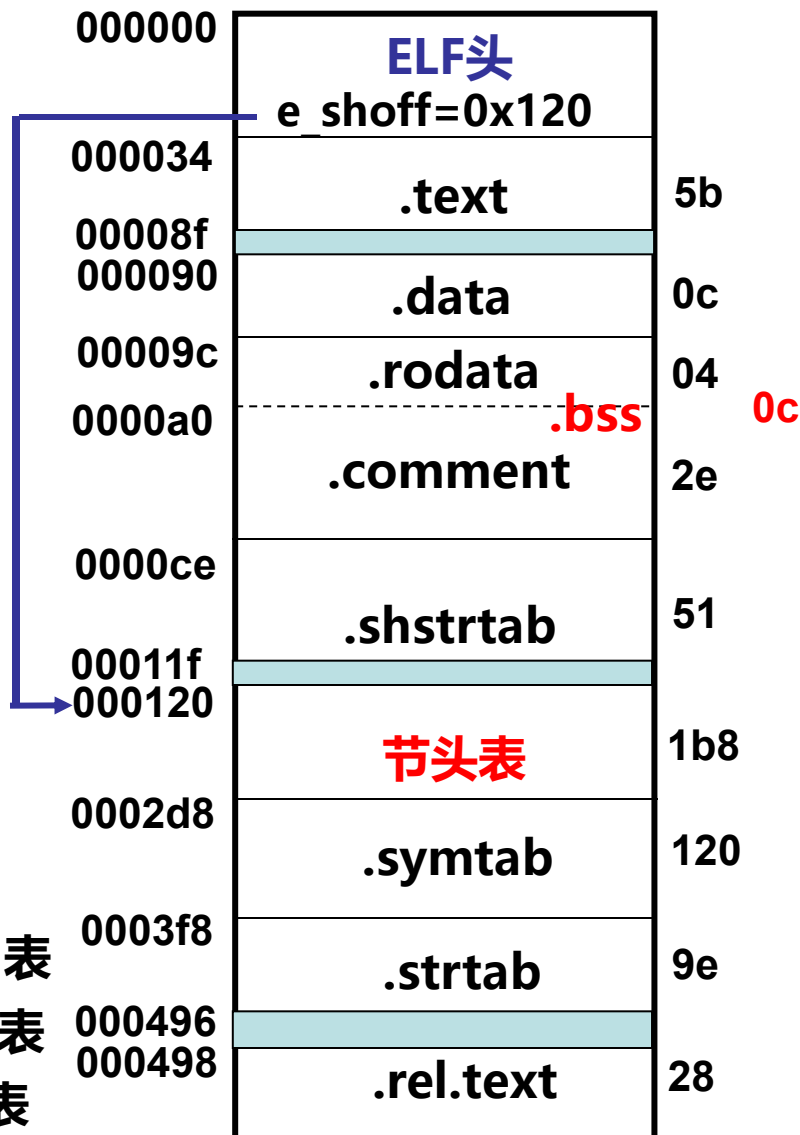
.rodata: 可读

shstrtab: 节名表

strtab: 符号名表

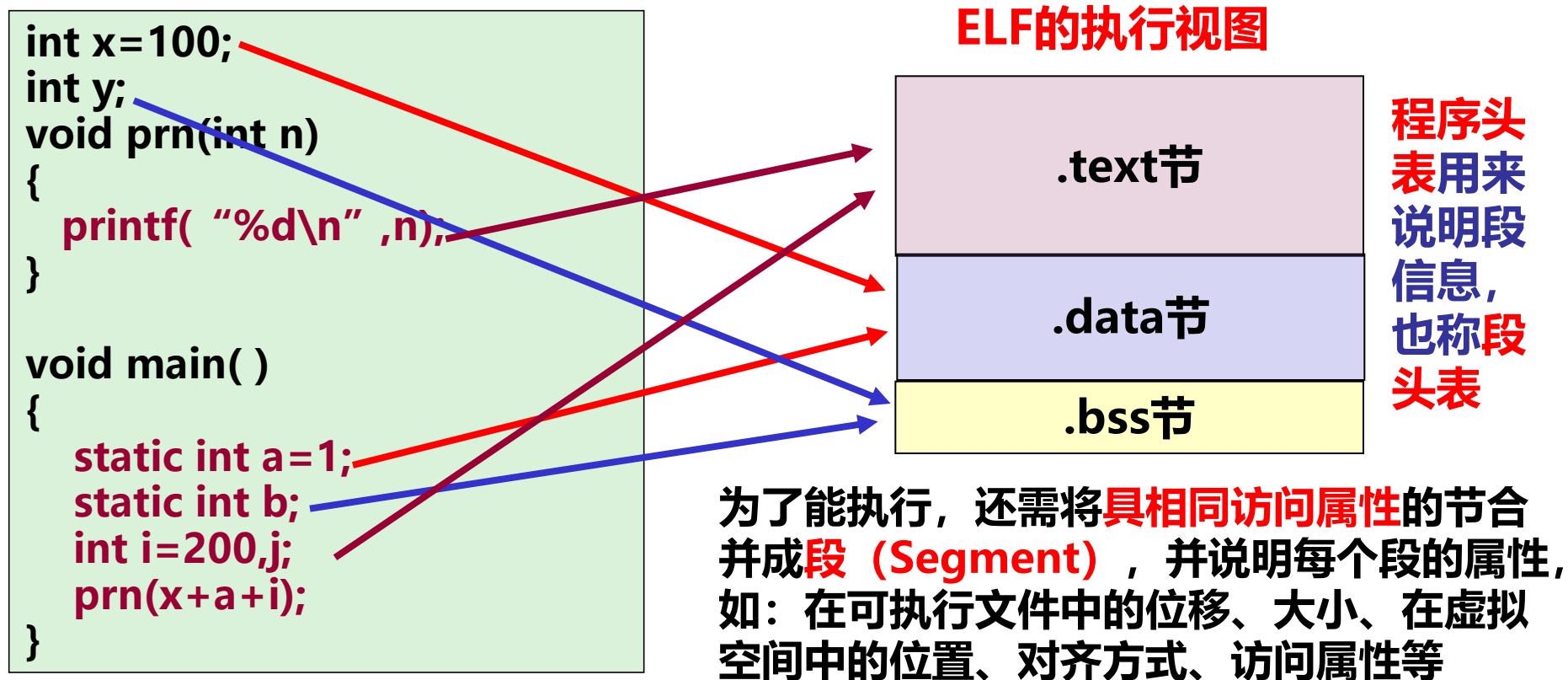
symtab: 符号表

可重定位目标文件test.o的结构



执行视图—可执行目标文件

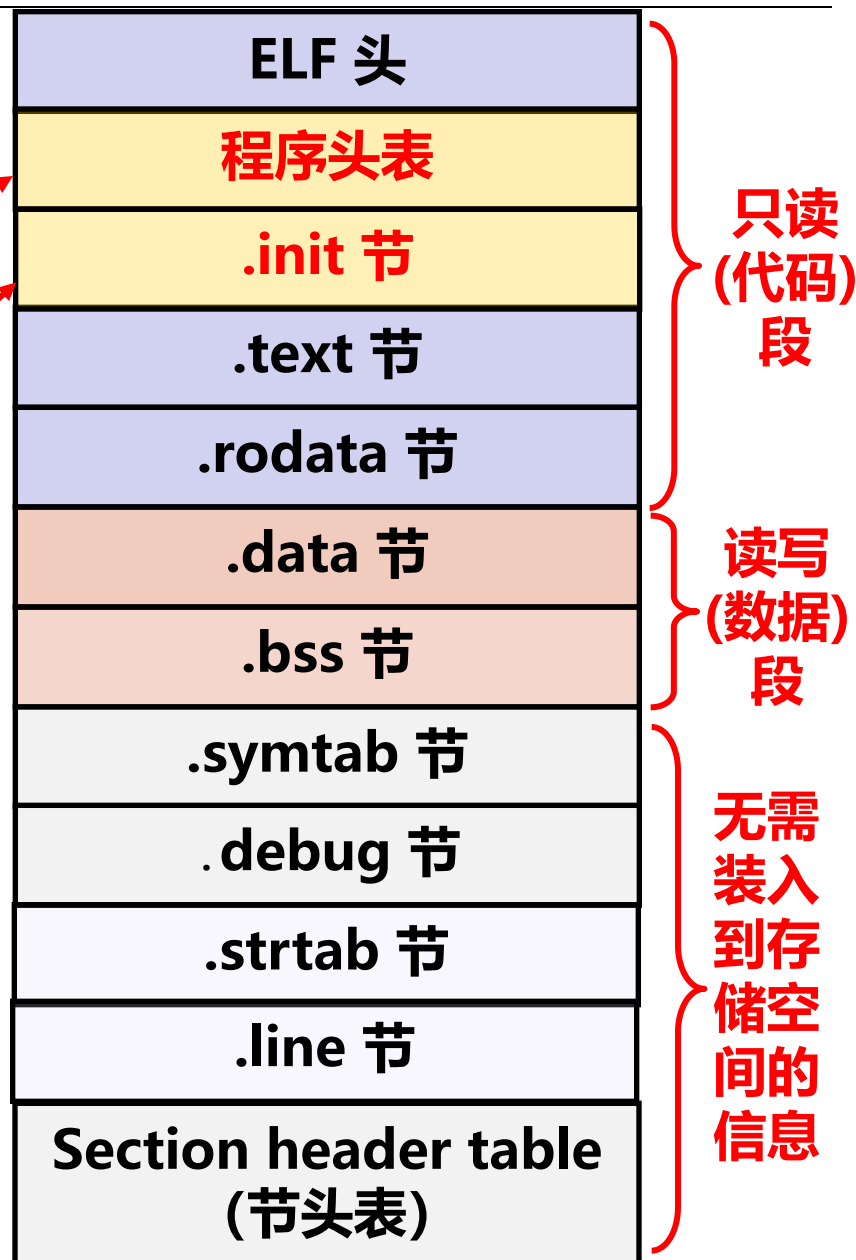
- 包含代码、数据（已初始化.data和未初始化.bss）
- 定义的所有变量和函数**已有确定地址**（虚拟地址空间中的地址）
- 符号引用处**已被重定位**，以指向所引用的定义符号
- 没有文件扩展名或默认为a.out（相当于Windows中的.exe文件）
- 可被CPU**直接执行**，指令地址和指令给出的操作数地址都是**虚拟地址**



可执行目标文件格式

- 与可重定位文件稍有不同:

- ELF头中字段e_entry给出执行程序时第一条指令的地址, 而在可重定位文件中, 此字段为0
- 多一个程序头表, 也称段头表 (segment header table), 是一个结构数组
- 多一个.init节, 用于定义 _init函数, 该函数用来进行可执行目标文件开始执行时的初始化工作
- 少两个.rel节 (无需重定位)



ELF头信息举例

\$ readelf -h main 可执行目标文件的ELF头

ELF Header:

Magic: **7f 45 4c 46** 01 01 01 00 00 00 00 00 00 00 00 00

Class: ELF32

Data: 2's complement, little endian

Version: 1 (current)

OS/ABI: UNIX - System V

ABI Version: 0

Type: EXEC (Executable file)

Machine: Intel 80386

Version: 0x1

Entry point address: x8048580

Start of program headers: 52 (bytes into file)

Start of section headers: 3232 (bytes into file)

Flags: 0x0

Size of this header: 52 (bytes)

Size of program headers: 32 (bytes)

Number of program headers: 8

Size of section headers: 40 (bytes)

Number of section headers: 29

Section header string table index: 26

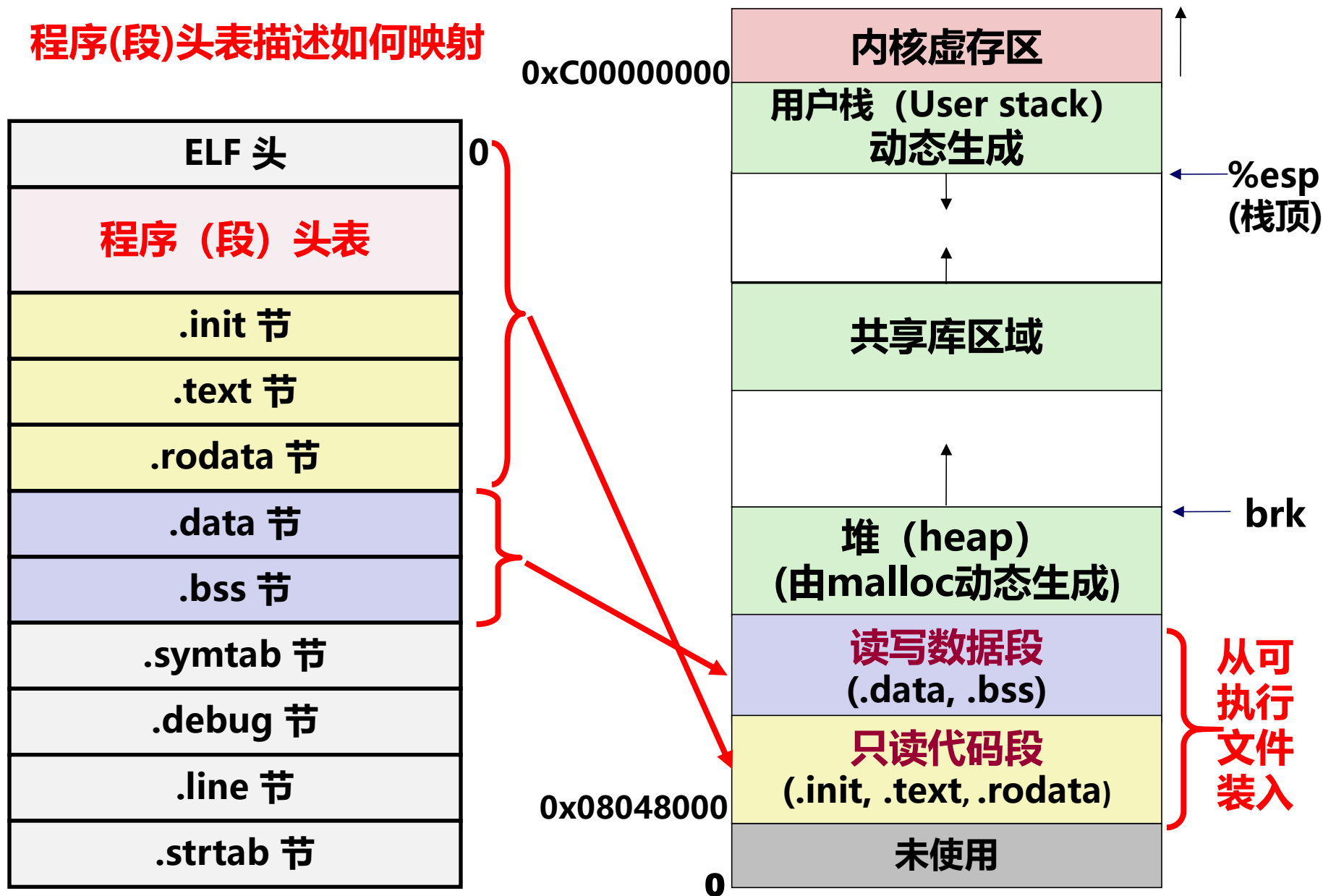
8x32B

29x40B

ELF 头	
00 00	程序头表
.init 节	
.text 节	
.rodata 节	
.data 节	
.bss 节	
.symtab 节	
.debug 节	
.strtab 节	
.line 节	
Section header table (节头表)	

可执行文件的存储器映像

程序(段)头表描述如何映射



可执行文件中的程序头表

```
typedef struct {  
    Elf32_Word  p_type;  
    Elf32_Off   p_offset;  
    Elf32_Addr  p_vaddr;  
    Elf32_Addr  p_paddr;  
    Elf32_Word  p_filesz;  
    Elf32_Word  p_memsz;  
    Elf32_Word  p_flags;  
    Elf32_Word  p_align;  
} Elf32_Phdr;
```

程序头表描述可执行文件中的节与虚拟空间中的存储段之间的映射关系

一个表项 (32B) 说明虚拟地址空间中一个连续的段或一个特殊的节

以下是某可执行目标文件程序头表信息

有8个表项, 其中两个为可装入段 (即 Type=LOAD)

\$ readelf -l main

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
PHDR	0x000034	0x08048034	0x08048034	0x00100	0x00100	R E	0x4
INTERP	0x000134	0x08048134	0x08048134	0x00013	0x00013	R	0x1
[Requesting program interpreter: /lib/ld-linux.so.2]							
LOAD	0x000000	0x08048000	0x08048000	0x004d4	0x004d4	R E	0x1000
LOAD	0x000f0c	0x08049f0c	0x08049f0c	0x00108	0x00110	RW	0x1000
DYNAMIC	0x000f20	0x08049f20	0x08049f20	0x000d0	0x000d0	RW	0x4
NOTE	0x000148	0x08048148	0x08048148	0x00044	0x00044	R	0x4
GNU_STACK	0x000000	0x00000000	0x00000000	0x00000	0x00000	RW	0x4
GNU_RELRO	0x000f0c	0x08049f0c	0x08049f0c	0x000f4	0x000f4	R	0x1

可执行文件中的程序头表

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
PHDR	0x000034	0x08048034	0x08048034	0x00100	0x00100	R E	0x4
INTERP	0x000134	0x08048134	0x08048134	0x00013	0x00013	R	0x1
[Requesting program interpreter: /lib/ld-linux.so.2]							
LOAD	0x000000	0x08048000	0x08048000	0x004d4	0x004d4	R E	0x1000
LOAD	0x000f0c	0x08049f0c	0x08049f0c	0x00108	0x00110	RW	0x1000
DYNAMIC	0x000f20	0x08049f20	0x08049f20	0x000d0	0x000d0	RW	0x4
NOTE	0x000148	0x08048148	0x08048148	0x00044	0x00044	R	0x4
GNU_STACK	0x000000	0x00000000	0x00000000	0x00000	0x00000	RW	0x4
GNU_RELRO	0x000f0c	0x08049f0c	0x08049f0c	0x000f4	0x000f4	R	0x1

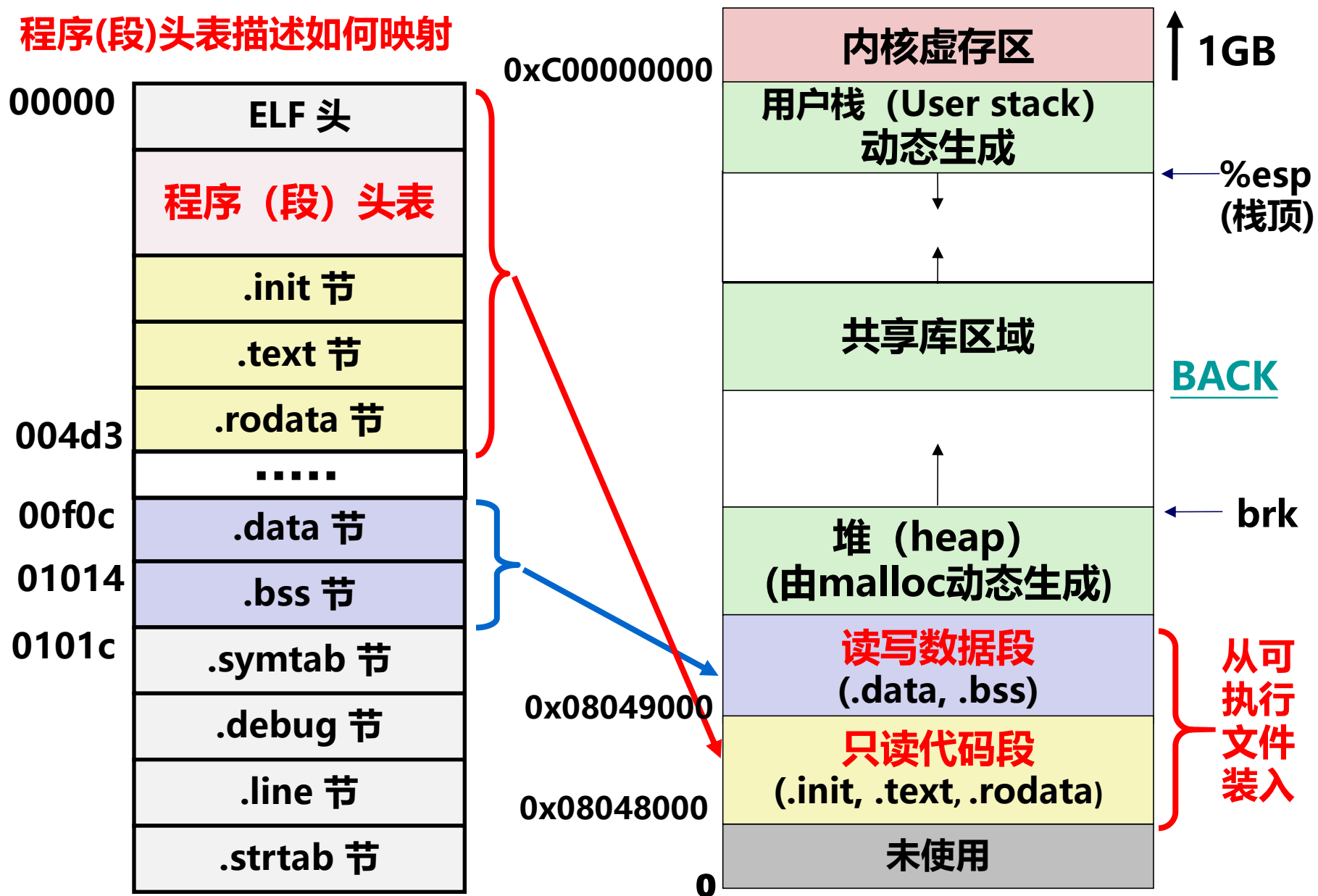
SKIP

第一可装入段：第0x00000~0x004d3字节（包括ELF头、程序头表、.init、.text和.rodata节），映射到虚拟地址0x8048000开始长度为0x4d4字节的区域，按0x1000=2¹²=4KB对齐，具有只读/执行权限（Flg=RE），是只读代码段。

第二可装入段：第0x000f0c开始长度为0x108字节的数据节，映射到虚拟地址0x8049f0c开始长度为0x110字节的存储区域，在0x110=272B存储区中，前0x108=264B用.data节内容初始化，后面272-264=8B对应.bss节，初始化为0，按0x1000=4KB对齐，具有可读可写权限（Flg=RW），是可读写数据段。

可执行文件的存储器映像

程序(段)头表描述如何映射



程序的链接

- 分以下三个部分介绍
 - 第一讲：目标文件格式
 - 程序的链接概述、链接的意义与过程
 - ELF目标文件、重定位目标文件格式、可执行目标文件格式
 - 第二讲：符号解析与重定位
 - 符号和符号表、符号解析
 - 与静态库的链接
 - 重定位信息、重定位过程
 - 可执行文件的加载
 - 第三讲：动态链接
 - 动态链接的特性、程序加载时的动态链接、程序运行时的动态链接、动态链接举例

符号和符号解析

每个**可重定位目标模块m**都有一个符号表，它包含了在m中定义的符号。
有三种链接器符号：

- **Global symbols** (模块内部定义的**全局符号**)
 - 由模块m定义并能被其他模块引用的符号。例如，非static 函数和非static的全局变量（指不带static的全局变量）
如，main.c 中的全局变量名buf
- **External symbols** (外部定义的**全局符号**)
 - 由其他模块定义并被模块m引用的全局符号
如，main.c 中的函数名swap
- **Local symbols** (本模块的**局部符号**)
 - 仅由模块m定义和引用的本地符号。例如，在模块m中定义的带static的函数和全局变量
如，swap.c 中的static变量名bufp1

链接器**局部符号**不是指程序中的**局部变量**（分配在栈中的临时性变量），链接器不关心这种局部变量

符号和符号解析

main.c

```
int buf[2] = {1, 2};  
extern void swap();  
  
int main()  
{  
    swap();  
    return 0;  
}
```

swap.c

```
extern int buf[];  
  
int *bufp0 = &buf[0];  
static int *bufp1;  
  
void swap()  
{  
    int temp;  
  
    bufp1 = &buf[1];  
    temp = *bufp0;  
    *bufp0 = *bufp1;  
    *bufp1 = temp;  
}
```

你能说出哪些是全局符号？哪些是外部符号？哪些是局部符号？

目标文件中的符号表

.symtab 节记录符号表信息，是一个结构数组

- 符号表 (**.symtab**) 中每个条目的结构如下：

```
typedef struct {  
    int    name; /*符号对应字符串在strtab节中的偏移量*  
    int    value; /*在对应节中的偏移量，可执行文件中是虚拟地址*/  
    int    size; /*符号对应目标所占字节数*/ 函数大小或变量长度  
    char    type: 4; /*符号对应目标的类型：数据、函数、源文件、节*/  
           binding: 4; /*符号类别：全局符号、局部符号、弱符号*/  
    char    reserved;  
    char    section; /*符号对应目标所在的节，或其他情况*/  
} Elf_Symbol;
```

函数名在text节中

变量名在data节或
bss节中

其他情况：ABS表示不该被重定位；UND表示未定义；COM表示未初始化数据 (.bss) ，此时，value表示对齐要求，size给出最小大小

目标文件中的符号表

- main.o中的符号表中最后三个条目 (共10个)

Num:	value	Size	Type	Bind	Ot	Ndx	Name
8:	0	8	Data	Global	0	3	buf
9:	0	33	Func	Global	0	1	main
10:	0	0	Notype	Global	0	UND	swap

buf是main.o中第3节 (.data) 偏移为0的符号, 是全局变量, 占8B;
main是第1节 (.text) 偏移为0的符号, 是全局函数, 占33B;
swap是main.o中未定义全局 (在其他模块定义) 符号, 类型和大小未知

- swap.o中的符号表中最后4个条目 (共11个)

Num:	value	Size	Type	Bind	Ot	Ndx	Name
8:	0	4	Data	Global	0	3	bufp0
9:	0	0	Notype	Global	0	UND	buf
10:	0	36	Func	Global	0	1	swap
11:	4	4	Data	Local	0	COM	bufp1

bufp1是未分配地址且未初始化的本地变量(ndx=COM), 按4B对齐且占4B

符号解析 (Symbol Resolution)

- 目的：将每个模块中**引用的符号**与某个目标模块中的**定义符号**建立关联。
- 每个**定义符号**在代码段或数据段中都被分配了存储空间，将引用符号与定义符号建立关联后，就可在重定位时将引用符号的地址重定位为相关联的定义符号的地址。
- **本地符号**在本模块内定义并引用，因此，其解析较简单，只要与本模块内唯一的定义符号关联即可。
- **全局符号**（外部定义的、内部定义的）的解析涉及多个模块，故较复杂

add B
jmp L0

.....
L0: sub 23

.....
B:

确定L0的地址，
再在jmp指令中
填入L0的地址

符号解析也称**符号绑定**

“符号的定义” 其实质是什么？

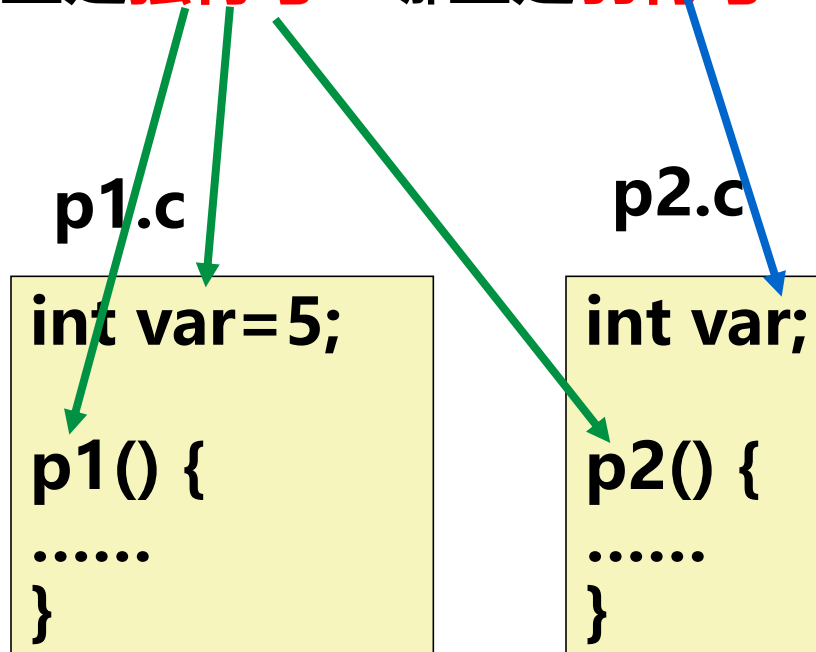
指被分配了存储空间。为函数名即指其代码所在区；为变量名即指其所占的静态数据区。

所有定义符号的值就是其目标所在的首地址

全局符号的符号解析

- 全局符号的强/弱特性
 - 函数名和已初始化的全局变量名是**强符号**
 - 未初始化的全局变量名是**弱符号**

以下符号哪些是**强符号**？ 哪些是**弱符号**？



全局符号的符号解析

以下符号哪些是**强符号**？ 哪些是**弱符号**？

main.c

```
int buf[2] = {1, 2};  
void swap();  
  
int main()  
{  
    swap();  
    return 0;  
}
```

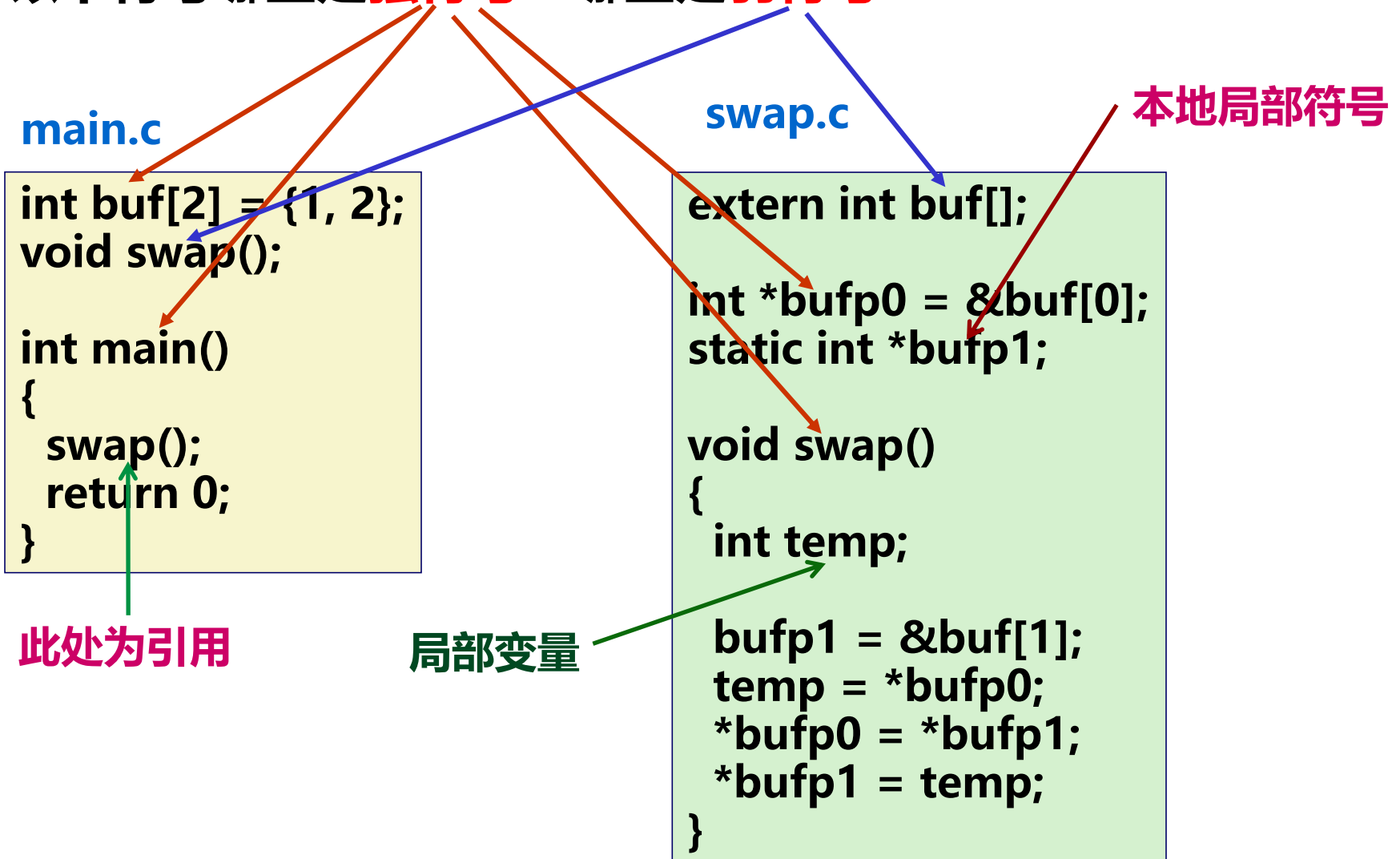
此处为引用

swap.c

```
extern int buf[];  
  
int *bufp0 = &buf[0];  
static int *bufp1;  
  
void swap()  
{  
    int temp;  
  
    bufp1 = &buf[1];  
    temp = *bufp0;  
    *bufp0 = *bufp1;  
    *bufp1 = temp;  
}
```

局部变量

本地局部符号



链接器对符号的解析规则

符号解析时只能有一个确定的定义（即每个符号仅占一处存储空间）

- **多重定义符号的处理规则**

Rule 1: 强符号不能多次定义

- 强符号只能被定义一次，否则链接错误

Rule 2: 若一个符号被定义为一次强符号和多次弱符号，则按强定义为准

- 对弱符号的引用被解析为其强定义符号

Rule 3: 若有多个弱符号定义，则任选其中一个

- 使用命令 `gcc -fno-common` 链接时，会告诉链接器在遇到多个弱定义的全局符号时输出一条警告信息。

多重定义符号的解析举例

以下程序会发生链接出错吗？

```
int x=10;  
int p1(void);  
int main()  
{  
    x=p1();  
    return x;  
}
```

main.c

```
int x=20;  
int p1()  
{  
    return x;  
}
```

p1.c

main只有一次强定义

p1有一次强定义，一次弱定义

x有两次强定义，所以，链接器将输出一条出错信息

多重定义符号的解析举例

以下程序会发生链接出错吗？

```
# include <stdio.h>
int y=100;
int z;
void p1(void);
int main()
{
    z=1000;
    p1();
    printf( "y=%d, z=%d\n" , y, z);
    return 0;
}
```

main.c

y一次强定义，一次弱定义

z两次弱定义

p1一次强定义，一次弱定义

main一次强定义

```
int y;
int z;
void p1( )
{
    y=200;
    z=2000;
}
```

p1.c

问题：打印结果是什么？

y=200, z=2000

该例说明：在两个不同模块定义相同变量名，很可能发生意想不到的结果！

多重定义符号的解析举例

以下程序会发生链接出错吗？

```
1 #include <stdio.h>
2 int d=100;
3 int x=200;
4 void p1(void);
5 int main()
6 {
7     p1();
8     printf( "d=%d,x=%d\n" ,d,x);
9     return 0;
10 }
```

main.c

p1.c

```
1 double d;
2
3 void p1()
4 {
5     d=1.0;
6 }
```

FLD1
FSTPI &d

p1执行后d和x处内容是什么？

	0	1	2	3
&x	00	00	F0	3F
&d	00	00	00	00

1.0: 0 011111111111 0...0B
=3FF0 0000 0000 0000H

问题：打印结果是什么？

d=0,x=1 072 693 248

该例说明：两个重复定义的变量具有不同类型时，更容易出现难以理解的结果！

多重定义符号的解析举例

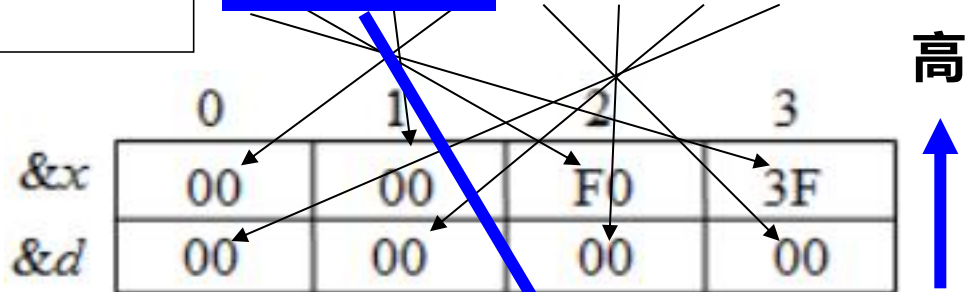
main.c

```
.....  
1 int d=100;  
2 int x=200;  
3 int main()  
4 {  
5   p1();  
6   printf ( "d=%d, x=%d\n" , d, x );  
7   return 0;  
8 }
```

p1.c

```
1 double d;  
2  
3 void p1()  
4 {  
5   d=1.0;  
6 }
```

double型数1.0对应的机器数
3FF0 0000 0000 0000H



IA-32是小端方式

$$2^{30}-1-(2^{20}-1)=2^{30}-2^{20}$$

$$=1024*1024*1023$$

$$=1\ 072\ 693\ 248$$

打印结果:

d=0, x=1 072 693 248

Why?

多重定义全局符号的问题

- 尽量避免使用全局变量
- 一定需要用的话，就按以下规则使用
 - 尽量使用本地变量 (static)
 - 全局变量要赋初值
 - 外部全局变量要使用extern

多重定义全局变量会造成一些意想不到的错误，而且是默默发生的，编译系统不会警告，并会在程序执行很久后才能表现出来，且远离错误引发处。特别是在一个具有几百个模块的大型软件中，这类错误很难修正。

大部分程序员并不了解链接器如何工作，因而养成良好的编程习惯是非常重要的。

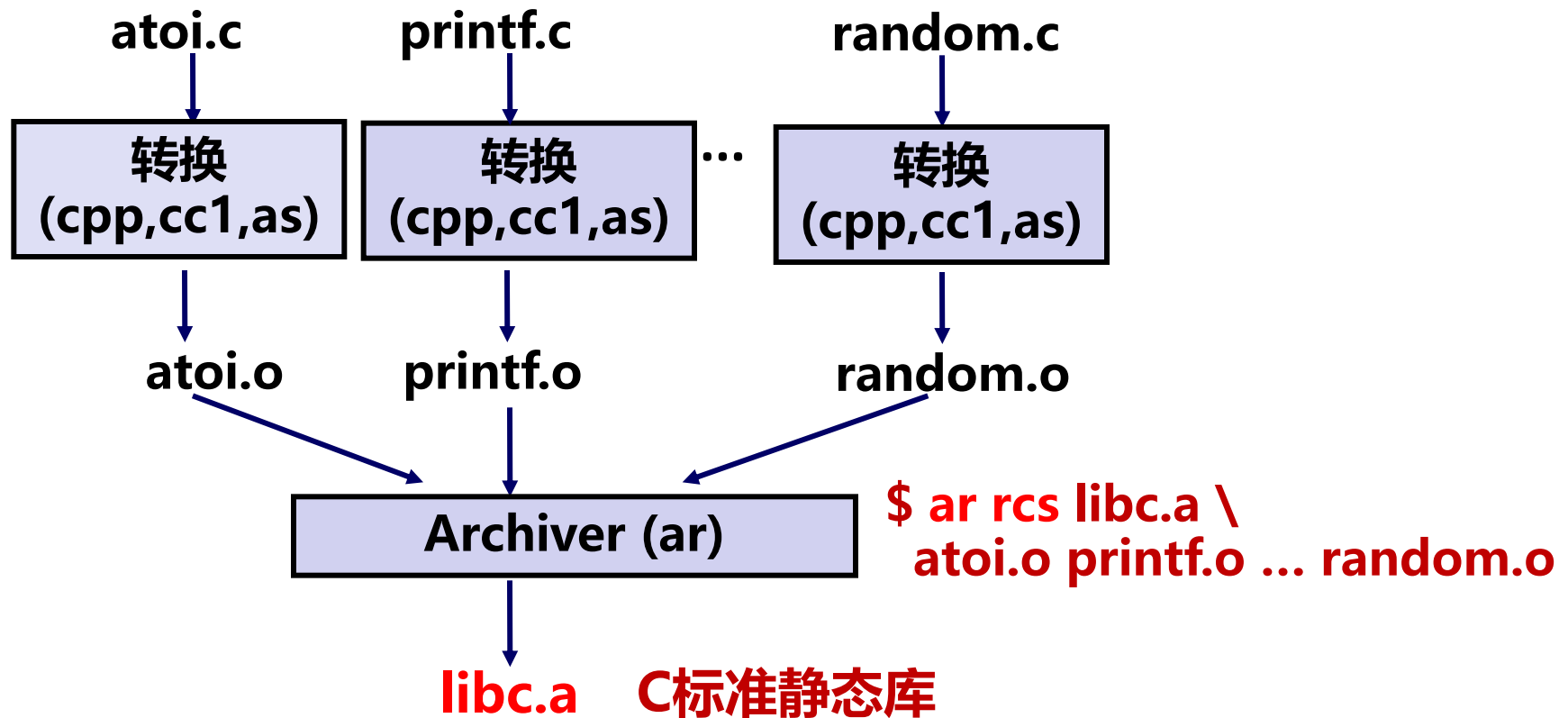
如何划分模块？

- 许多函数无需自己写，可使用共享库函数
 - 如数学库, 输入/输出库, 存储管理库, 字符串处理等
- 避免以下两种极端做法
 - 将所有函数都放在一个源文件中
 - 修改一个函数需要对所有函数重新编译
 - 时间和空间两方面的效率都不高
 - 一个源文件中仅包含一个函数
 - 需要程序员显式地进行链接
 - 效率高，但模块太多，故太繁琐

静态共享库

- **静态库 (.a archive files)**
 - 将所有相关的目标模块 (.o) 打包为一个单独的库文件 (.a) , 称为**静态库文件** , 也称**存档文件** (archive)
 - 增强了链接器功能, 使其能通过查找一个或多个库文件中的符号来解析符号
 - 在构建可执行文件时只需指定库文件名, 链接器会自动到库中寻找那些应用程序用到的目标模块, 并且**只把用到的模块从库中拷贝出来**
 - **在gcc命令中无需明显指定C标准库libc.a(默认库)**

静态库的创建



- Archiver (归档器) 允许增量更新, 只要重新编译需修改的源码并将其.o文件替换到静态库中。

常用静态库

libc.a (C标准库)

- 1392个目标文件 (大约8 MB)
- 包含I/O、存储分配、信号处理、字符串处理、时间和日期、随机数生成、定点整数算术运算

libm.a (the C math library)

- 401 个目标文件 (大约 1 MB)
- 浮点数算术运算(如sin, cos, tan, log, exp, sqrt, ...)

```
% ar -t /usr/lib/libc.a | sort
```

```
...  
fork.o  
...  
fprintf.o  
fpu_control.o  
fputc.o  
freopen.o  
fscanf.o  
fseek.o  
fstab.o  
...
```

```
% ar -t /usr/lib/libm.a | sort
```

```
...  
e_acos.o  
e_acosf.o  
e_acosh.o  
e_acoshf.o  
e_acoshl.o  
e_acosl.o  
e_asin.o  
e_asinf.o  
e_asinl.o  
...
```


自定义一个静态库文件

举例：将myproc1.o和myproc2.o打包生成mylib.a

myproc1.c

```
# include <stdio.h>
void myfunc1() {
    printf("This is myfunc1!\n");
}
```

myproc2.c

```
# include <stdio.h>
void myfunc2() {
    printf("This is myfunc2\n");
}
```

\$ gcc -c myproc1.c myproc2.c

\$ ar rcs mylib.a myproc1.o myproc2.o

main.c

```
void myfunc1(viod);
int main()
{
    myfunc1();
    return 0;
}
```

\$ gcc -c main.c **libc.a**无需明显指出!
\$ gcc -static -o myproc main.o **./mylib.a**

调用关系: main→myfunc1→printf

问题：如何进行符号解析？

使用静态库的符号解析过程

--- From "*Computer Systems A Programmer's Perspective (2nd Edition)*"

The linker scans the relocatable object files and archives **left to right** in the same sequential order that they appear on the compiler driver's command line.

- Automatically translates any .c files into .o files

During this scan, the linker maintains:

- a set **E** of relocatable object files that will be merged to form the executable
- a set **U** of unresolved symbols (i.e., symbols referred to, but not yet defined)
- a set **D** of symbols that have been defined in previous input files.
- Initially, E, U, and D are empty.

For each input file *f* on the command line:

- If *f* is an object file, the linker adds *f* to E, updates U and D to reflect the symbol definitions and references in *f*, and proceeds to the next input file.
- If *f* is an archive, the linker attempts to match the unresolved symbols in U against the symbols defined by the members of the archive. If some archive member, *m*, defines a symbol that resolves a reference in U, then *m* is added to E, and the linker updates U and D to reflect the symbol definitions and references in *m*. This process iterates over the member object files in the archive until a fixed point is reached where U and D no longer change. At this point, any member object files not contained in E are simply discarded and the linker proceeds to the next input file.

If U is nonempty when the linker finishes scanning the input files on the command line, it prints an error and terminates.

链接器中符号解析的全过程

\$ gcc -c main.c **libc.a**无需明显指出!
\$ gcc -static -o myproc **main.o ./mylib.a**

调用关系: **main**→**myfunc1**→**printf**

E 将被合并以组成可执行文件的所有目标文件集合

U 当前所有未解析的引用符号的集合

D 当前所有定义符号的集合

开始**E**、**U**、**D**为空，首先扫描**main.o**，把它加入**E**，同时把**myfun1**加入**U**，**main**加入**D**。接着扫描到**mylib.a**，将**U**中所有符号（本例中为**myfunc1**）与**mylib.a**中所有目标模块（**myproc1.o**和**myproc2.o**）依次匹配，发现在**myproc1.o**中定义了**myfunc1**，故**myproc1.o**加入**E**，**myfunc1**从**U**转移到**D**。在**myproc1.o**中发现还有未解析符号**printf**，将其加到**U**。不断在**mylib.a**的各模块上进行迭代以匹配**U**中的符号，直到**U**、**D**都不再变化。此时**U**中只有一个未解析符号**printf**，而**D**中有**main**和**myfunc1**。因为模块**myproc2.o**没有被加入**E**中，因而它被丢弃。

main.c

```
void myfunc1(viod);  
int main()  
{  
    myfunc1();  
    return 0;  
}
```

接着，扫描**默认的库文件libc.a**，发现其目标模块**printf.o**定义了**printf**，于是**printf**也从**U**移到**D**，并将**printf.o**加入**E**，同时把它定义的所有符号加入**D**，而所有未解析符号加入**U**。
处理完libc.a时，U一定是空的。

链接器中符号解析的全过程

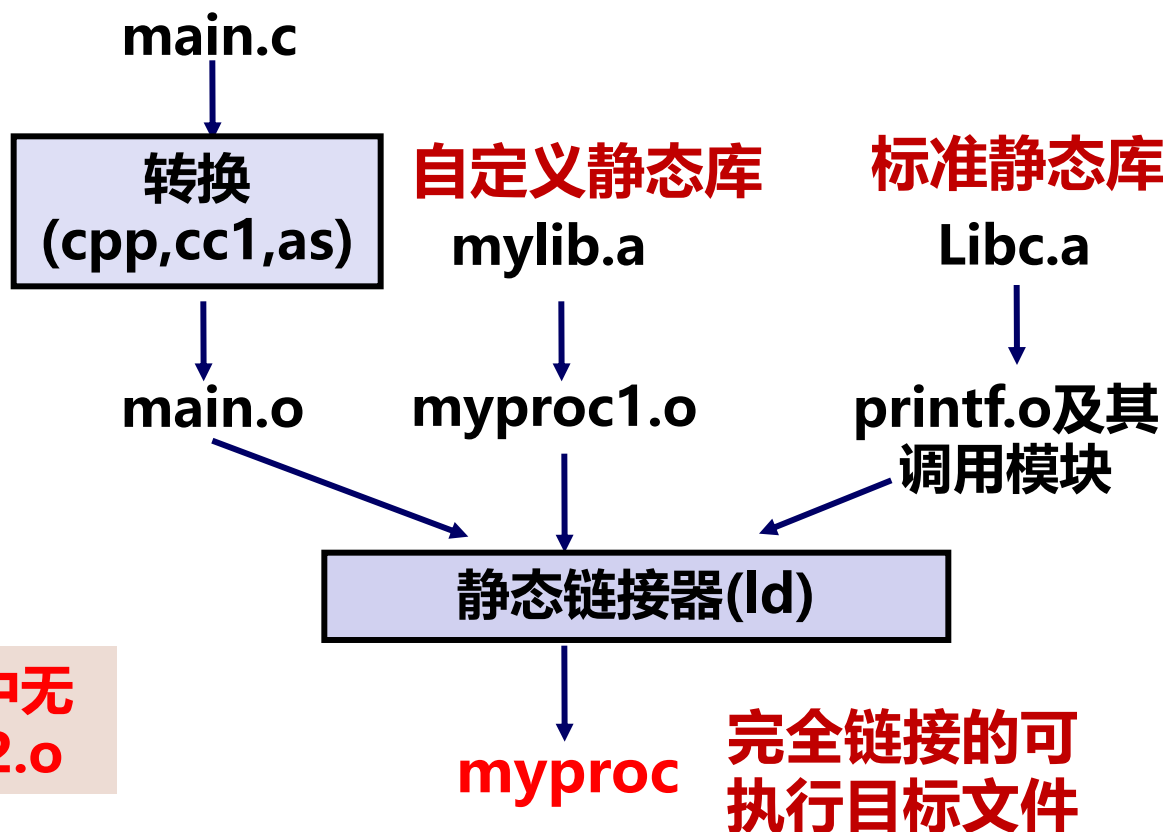
\$ ar rcs mylib.a myproc1.o myproc2.o

\$ gcc -static -o myproc main.o ./mylib.a

main→myfunc1→printf

main.c

```
void myfunc1(viod);
int main()
{
    myfunc1();
    return 0;
}
```



注意：E中无
myproc2.o

解析结果：

E中有main.o、myproc1.o、printf.o及其调用的模块

D中有main、myproc1、printf及其引用的符号

链接器中符号解析的全过程

main.c

```
void myfunc1(viod);  
int main()  
{  
    myfunc1();  
    return 0;  
}
```

main→myfunc1→printf

\$ gcc -static -o myproc main.o ./mylib.a

解析结果:

E中有main.o、myproc1.o、printf.o及其调用的模块

D中有main、myproc1、printf及其引用符号

被链接模块应按
调用顺序指定!

若命令为: \$ gcc -static -o myproc ./mylib.a main.o, 结果怎样?

首先, 扫描mylib, 因是静态库, 应根据其中是否存在U中未解析符号对应的定义符号来确定哪个.o被加入E。因为开始U为空, 故其中两个.o模块都不被加入E中而被丢弃。

然后, 扫描main.o, 将myfunc1加入U, 直到最后它都不能被解析。Why?

因此, 出现链接错误!

它只能用mylib.a中符号来解析, 而mylib中两个.o模块都已被丢弃!

使用静态库

- 链接器对外部引用的解析算法要点如下：
 - 按照命令行给出的**顺序扫描.o 和.a 文件**
 - 扫描期间将**当前未解析的引用**记录到一个列表U中
 - 每遇到一个新的.o 或 .a 中的模块，都试图用其来解析U中的符号
 - 如果扫描到最后，U中还有未被解析的符号，则发生错误
- 问题和对策
 - 能否正确解析与命令行给出的顺序有关
 - 好的做法：将静态库放在命令行的最后 **libmine.a 是静态库**

假设调用关系：libtest.o → libfun.o (在libmine.a中)

-lxxx=libxxx.a (main) → (libfun)

\$ gcc -L. **libtest.o** -lmine ← 扫描libtest.o, 将libfun送U, 扫描到
\$ gcc -L. -lmine **libtest.o** libmine.a时, 用其定义的libfun来解析

libtest.o: In function `main':

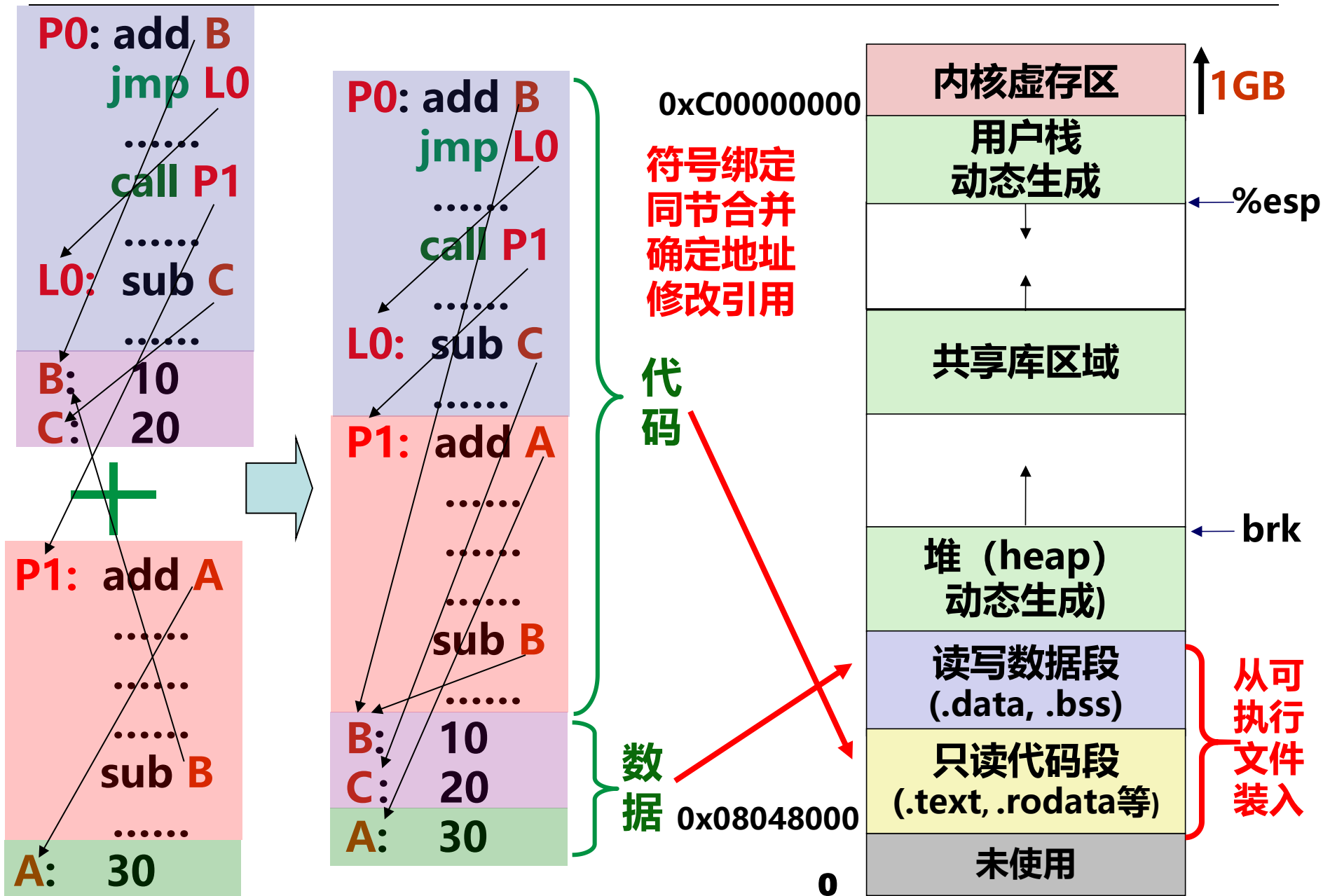
libtest.o(.text+0x4): undefined reference to `libfun'

说明在libtest.o中的main调用了libfun这个在库libmine中的函数，
所以，在命令行中，应该将libtest.o放在前面，像第一行中那样！

链接顺序问题

- 假设调用关系如下：
func.o → libx.a 和 liby.a 中的函数
libx.a → libz.a 中的函数
libx.a 和 liby.a 之间、liby.a 和 libz.a 相互独立
则以下几个命令行都是可行的：
 - gcc -static -o myfunc func.o libx.a liby.a libz.a
 - gcc -static -o myfunc func.o liby.a libx.a libz.a
 - gcc -static -o myfunc func.o libx.a libz.a liby.a
- 假设调用关系如下：
func.o → libx.a 和 liby.a 中的函数
libx.a → liby.a 同时 liby.a → libx.a
则以下命令行可行：
 - gcc -static -o myfunc func.o libx.a liby.a libx.a

链接操作的步骤



目标文件

```
/* main.c */
int add(int, int);
int main( )
{
    return add(20, 13);
}
```

```
/* test.c */
int add(int i, int j)
{
    int x = i + j;
    return x;
}
```

00000000	<add>:	objdump -d test.o
0:	55	push %ebp
1:	89 e5	mov %esp, %ebp
3:	83 ec 10	sub \$0x10, %esp
6:	8b 45 0c	mov 0xc(%ebp), %eax
9:	8b 55 08	mov 0x8(%ebp), %edx
c:	8d 04 02	lea (%edx,%eax,1), %eax
f:	89 45 fc	mov %eax, -0x4(%ebp)
12:	8b 45 fc	mov -0x4(%ebp), %eax
15:	c9	leave
16:	c3	ret

080483d4	<add>:	objdump -d test
80483d4:	55	push %ebp
80483d5:	89 e5	mov %esp, %ebp
80483d7:	83 ec 10	sub \$0x10, %esp
80483da:	8b 45 0c	mov 0xc(%ebp), %eax
80483dd:	8b 55 08	mov 0x8(%ebp), %edx
80483e0:	8d 04 02	lea (%edx,%eax,1), %eax
80483e3:	89 45 fc	mov %eax, -0x4(%ebp)
80483e6:	8b 45 fc	mov -0x4(%ebp), %eax
80483e9:	c9	leave
80483ea:	c3	ret

重定位

符号解析完成后，可进行重定位工作，分三步

- 合并相同的节
 - 将集合E的所有目标模块中相同的节合并成新节
 - 例如，所有.text节合并作为可执行文件中的.text节
- 对定义符号进行重定位（确定地址）
 - 确定新节中所有定义符号在虚拟地址空间中的地址
 - 例如，为函数确定首地址，进而确定每条指令的地址，为变量确定首地址
 - 完成这一步后，每条指令和每个全局变量都可确定地址
- 对引用符号进行重定位（确定地址）
 - 修改.text节和.data节中对每个符号的引用（地址）
 - 需要用到在.rel_data和.rel_text节中保存的重定位信息

重定位信息

- 汇编器遇到引用时，生成一个重定位条目
- 数据引用的重定位条目在.rel_data节中
- 指令中引用的重定位条目在.rel_text节中
- ELF中重定位条目格式如下：

```
typedef struct {  
    int offset;           /*节内偏移*/  
    int symbol:24, /*所绑定符号*/  
        type: 8;        /*重定位类型*/  
} Elf32_Rel;
```

- IA-32有两种最基本的重定位类型
 - R_386_32: 绝对地址
 - R_386_PC32: PC相对地址

例如，在rel_text节中有重定位条目如下

```
offset: 0x1      offset: 0x6  
symbol: B        symbol: L0  
type: R_386_32   type: R_386_PC32
```

```
add B  
jmp L0  
.....  
L0: sub 23  
.....  
B: .....
```

```
05 00000000  
02 FCFFFFFF  
.....  
L0: sub 23  
.....  
B: .....
```

问题：重定位条目和汇编后的
机器代码在何种目标文件中？

在可重定位目标
(.o) 文件中！

重定位操作举例

main.c

```
int buf[2] = {1, 2};  
void swap();  
  
int main()  
{  
    swap();  
    return 0;  
}
```

swap.c

```
extern int buf[];  
int *bufp0 = &buf[0];  
static int *bufp1;  
void swap()  
{  
    int temp;  
    bufp1 = &buf[1];  
    temp = *bufp0;  
    *bufp0 = *bufp1;  
    *bufp1 = temp;  
}
```

你能说出哪些是**符号定义**？哪些是**符号的引用**？

局部变量temp分配在栈中，不会在过程外被引用，因此不是符号定义

重定位操作举例

main.c

```
int buf[2] = {1, 2};
void swap();

int main()
{
    swap();
    return 0;
}
```

swap.c

```
extern int buf[];
int *bufp0 = &buf[0];
static int *bufp1;

void swap()
{
    int temp;
    bufp1 = &buf[1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
}
```

符号解析后的结果是什么？
E中有printf.o吗？

E中有main.o和swap.o两个模块！ D中有所有定义的符号！

在main.o和swap.o的**重定位节(.rel.text、.rel.data)**中有**重定位信息**，反映符号引用的位置、绑定的定义符号名、重定位类型

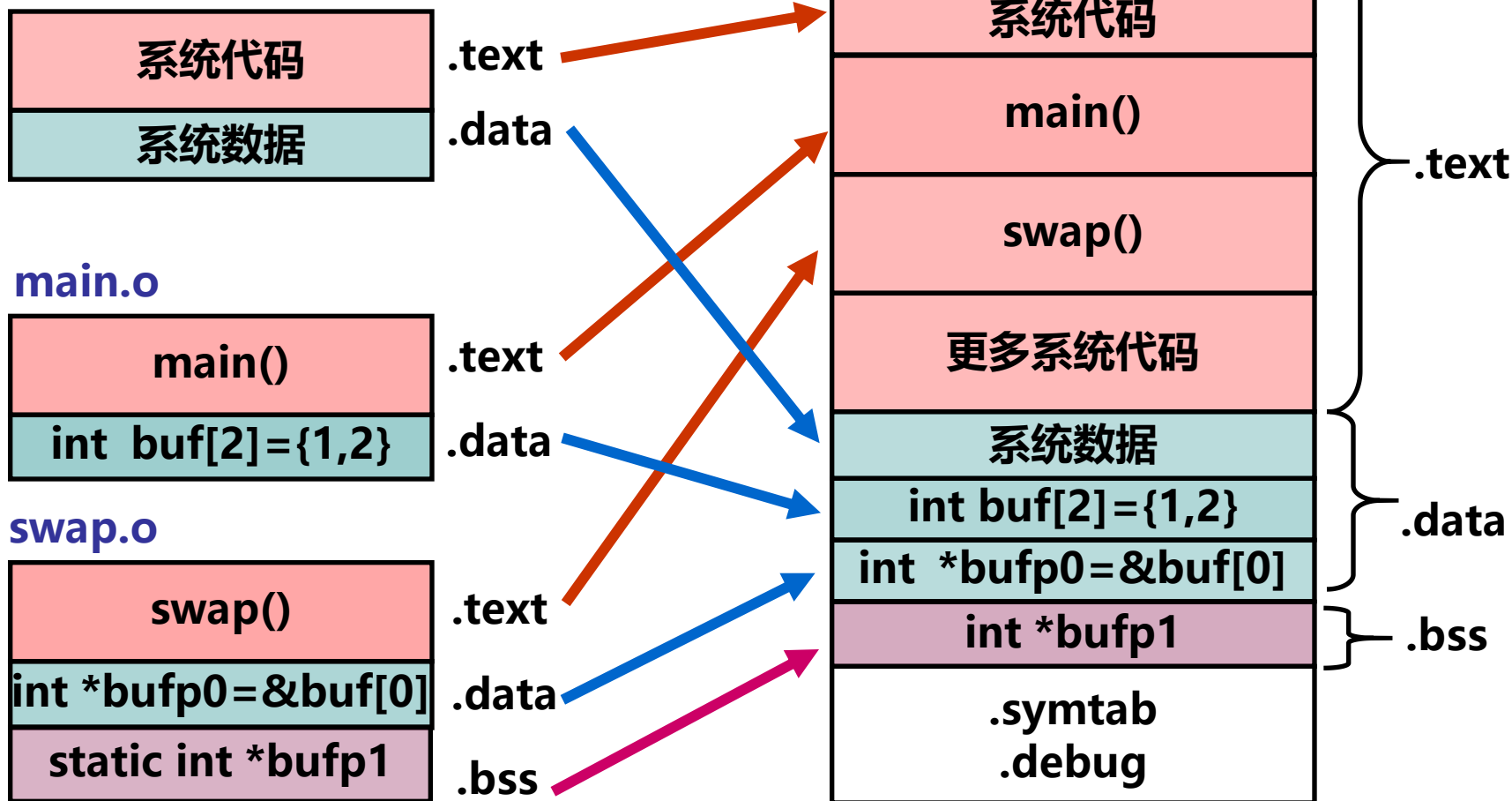
用命令**readelf -r main.o**可显示main.o中的重定位条目（表项）

符号引用的地址需要重定位

链接本质：合并相同的“节”

可执行目标文件

可重定位目标文件



虽然bufp1是swap的本地符号，也需在.bss节重定位

main.o重定位前

main.c

```
int buf[2]={1,2};

int main()
{
    swap();
    return 0;
}
```

main的定义在.text节中偏移为0处开始, 占0x12B。

Disassembly of section .data:

```
00000000 <buf>:
 0: 01 00 00 00 02 00 00 00
```

buf的定义在.data节中偏移为0处开始, 占8B。

main.o

Disassembly of section .text:

00000000 <main>:

0:	55	push	%ebp
1:	89 e5	mov	%esp,%ebp
3:	83 e4 f0	and	\$0xffffffff,%esp
6:	e8 fc ff ff ff	call	7 <main+0x7>

7: R_386_PC32 swap

b:	b8 00 00 00 00	mov	\$0x0,%eax
10:	c9	leave	
11:	c3	ret	

在rel_text节中的重定位条目为:
r_offset=0x7, r_sym=10,
r_type=R_386_PC32, dump出来为 "7: R_386_PC32 swap"

r_sym=10说明引用的是swap!

main.o中的符号表

- main.o中的符号表中最后三个条目

Num:	value	Size	Type	Bind	Ot	Ndx	Name
8:	0	8	Data	Global	0	3	buf
9:	0	18	Func	Global	0	1	main
10:	0	0	Notype	Global	0	UND	swap

swap是main.o的符号表中第10项，是未定义符号，类型和大小未知，并是全局符号，故在其他模块中定义。

在rel_text节中的重定位条目为：

r_offset=0x7, r_sym=10,
r_type=R_386_PC32, dump出
来后为 “7: R_386_PC32 swap”

r_sym=10说明
引用的是swap!

R_386_PC32的重定位方式

R_386_PC32重定位方式下，重定位后的引用地址应等于：

$$\text{ADDR(sym)} - \text{PC值}$$

$$\text{PC} = \text{ADDR(下条指令)} \quad \text{重定位前的引用初始地址}$$

$$= \text{ADDR(sym引用)} - (\text{ADDR(sym引用)} - \text{ADDR(下条指令)})$$

$$\text{ADDR(sym引用)} = \text{ADDR(引用所在节)} + \text{OFFSET(sym引用)}$$

所以：

所以：重定位后的引用地址 =

$$\text{ADDR(sym)} - \text{ADDR(sym引用)} + \text{重定位前引用地址}$$

Disassembly of section .text:

08048380 <main>:

.....

6: e8 fc ff ff ff call 7 <main+0x7>

7: R_386_PC32 swap

b: b8 00 00 00 00 mov \$0x0,%eax

假设swap起始虚拟地址 $\text{ADDR(sym)} = 0x8048394$

重定位前引用地址 = $\text{ADDR(sym引用)} - \text{ADDR(下条指令)} = -4 = 0xfcffffff$

$\text{ADDR(sym引用)} = 0x08048380 + 0x7$

$\text{PC} = 0x08048380 + 0xb$

重定位后的引用地址 = $0x8048394 - 0x08048387 - 4$
= 9

R_386_PC32的重定位方式

- 假定:

- 可执行文件中main
- swap紧跟main后

- 则swap起始地址为:

- $0x8048380 + 0x12 = 0x8048392$
- 在4字节边界对齐的情况下, 是 $0x8048394$

- 则重定位后call指令的机器代码是什么?

- 转移目标地址 = PC + 偏移地址, $PC = 0x8048380 + 0x07 - \text{init}$
- $PC = 0x8048380 + 0x07 - (-4) = 0x804838b$
- 重定位值 = 转移目标地址 - PC = $0x8048394 - 0x804838b = 0x9$
- call指令的机器代码为 “e8 09 00 00 00”

PC相对地址方式下, 重定位值计算公式为:

$$\text{ADDR}(r_sym) - ((\text{ADDR}(.text) + r_offset) - \text{init})$$

引用目标处

call指令下条指令地址

即当前PC的值

Disassembly of section .text:

00000000 <main>:

.....

6:

e8

fc ff ff ff

call

7

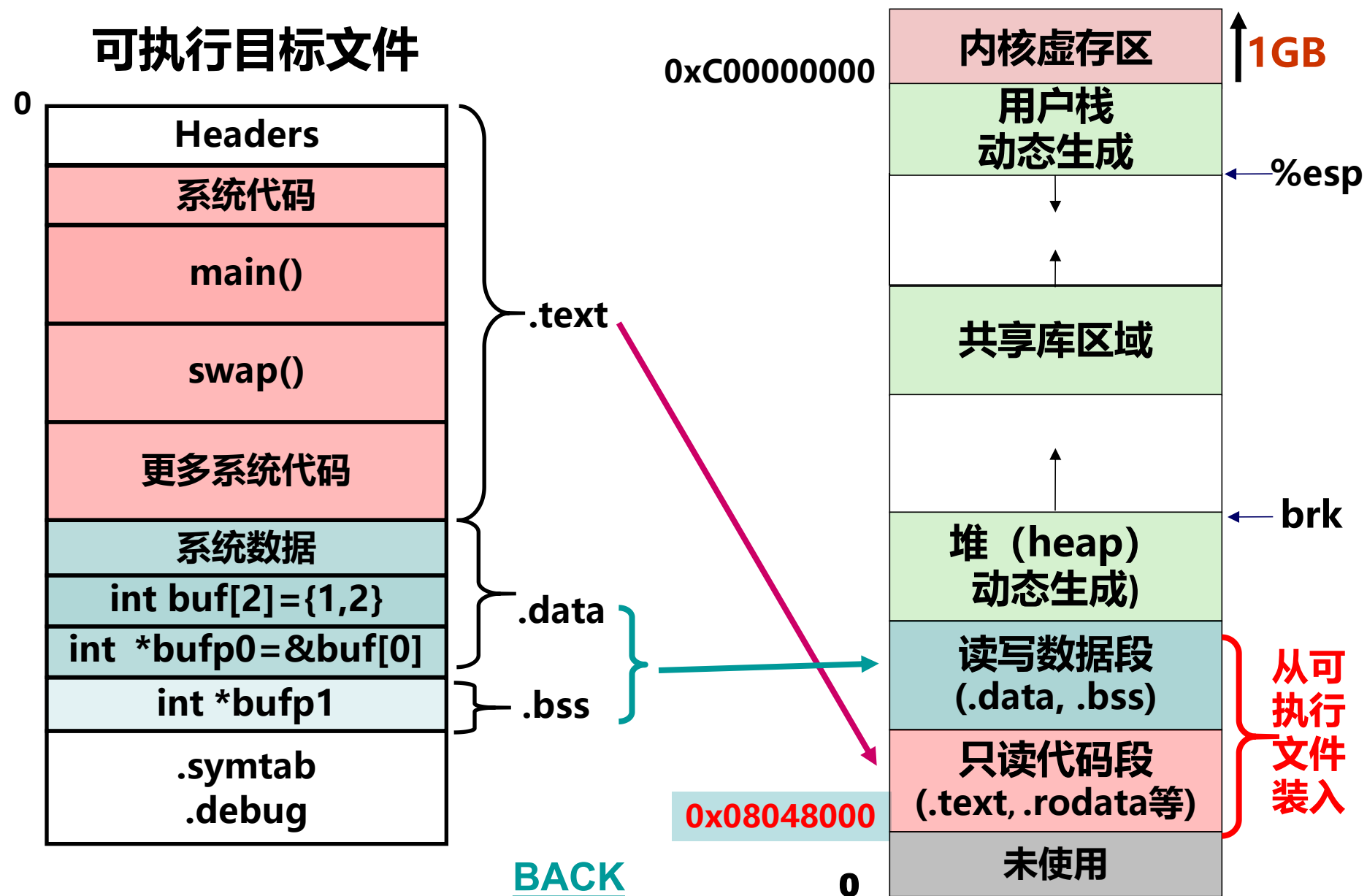
<main+0x7>

7: R_386_PC32 swap

.....

重定位值
值为-4

确定定义符号的地址



R_386_32的重定位方式

main.o中.data和.rel.data节内容

Disassembly of section .data:

```
00000000 <buf>:  
0: 01 00 00 00 02 00 00 00
```

buf定义在.data节中偏移为0处, 占8B, 没有需重定位的符号。

main.c

```
int buf[2]={1,2};  
  
int main()  
.....
```

swap.o中.data和.rel.data节内容

Disassembly of section .data:

```
00000000 <bufp0>:  
0: 00 00 00 00  
0: R_386_32 buf
```

bufp0定义在.data节中偏移为0处, 占4B, 初值为0x0

swap.c

```
extern int buf[];  
  
int *bufp0 = &buf[0];  
static int *bufp1;  
  
void swap()  
.....
```

重定位节.rel.data中有一个重定位表项: $r_offset=0x0$, $r_sym=9$, $r_type=R_386_32$, OBJDUMP工具解释后显示为 "0: R_386_32 buf"
 $r_sym=9$ 说明引用的是buf!

R_386_32的重定位方式

- 假定：
 - buf在运行时的存储地址ADDR(buf)=0x8049620
- 则重定位后，bufp0的地址及内容变为什么？
 - buf和bufp0同属于.data节，故在可执行文件中它们被合并
 - bufp0紧接在buf后，故地址为0x8049620+8= 0x8049628
 - 因是R_386_32方式，故bufp0内容为buf的绝对地址0x8049620，即“20 96 04 08”

可执行目标文件中.data节的内容

Disassembly of section .data:

08049620 <buf>:

8049620: 01 00 00 00 02 00 00 00

08049628 <bufp0>:

8049628: 20 96 04 08

swap.o中的符号表

- swap.o中的符号表中最后4个条目

Num:	value	Size	Type	Bind	Ot	Ndx	Name
8:	0	4	Data	Global	0	3	bufp0
9:	0	0	Notype	Global	0	UND	buf
10:	0	36	Func	Global	0	1	swap
11:	4	4	Data	Local	0	COM	bufp1

buf是swap.o的符号表中第9项，是未定义符号，类型和大小未知，并是全局符号，故在其他模块中定义。

重定位节.rel.data中有一个重定位表项：r_offset=0x0, r_sym=9, r_type=R_386_32, OBJDUMP工具解释后显示为
"0: R_386_32 buf"

r_sym=9说明引用的是buf!

swap.o重定位

swap.c

```
extern int buf[];

int *bufp0 = &buf[0];
static int *bufp1;

void swap()
{
    int temp;

    bufp1 = &buf[1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
}
```

共有6处需要重定位

划红线处：8、c、
11、1b、21、2a

Disassembly of section .text:

00000000 <swap>:

0:	55	push	%ebp
1:	89 e5	mov	%esp,%ebp
3:	83 ec 10	sub	\$0x10,%esp
6:	<u>c7 05 00 00 00 00 04</u>	movl	\$0x4,0x0
d:	<u>00 00 00</u>		
8:	R_386_32	.bss	
c:	R_386_32	buf	
10:	<u>a1 00 00 00 00</u>	mov	0x0,%eax
11:	R_386_32	bufp0	
15:	8b 00	mov	(%eax),%eax
17:	89 45 fc	mov	%eax,-0x4(%ebp)
1a:	<u>a1 00 00 00 00</u>	mov	0x0,%eax
1b:	R_386_32	bufp0	
1f:	<u>8b 15 00 00 00 00</u>	mov	0x0,%edx
21:	R_386_32	.bss	
25:	8b 12	mov	(%edx),%edx
27:	89 10	mov	%edx,(%eax)
29:	<u>a1 00 00 00 00</u>	mov	0x0,%eax
2a:	R_386_32	.bss	
2e:	8b 55 fc	mov	-0x4(%ebp),%edx
31:	89 10	mov	%edx,(%eax)
33:	c9	leave	
34:	c3	ret	

swap.o重定位

buf和bufp0的地址分别是0x8049620和0x8049628

&buf[1](c处重定位值) 为0x8049620+0x4=0x8049624

bufp1的地址就是链接合并后.bss节的首地址, 假定为0x8049700

8 (bufp1): 00 97 04 08

c (&buf[1]): 24 96 04 08

11 (bufp0): 28 96 04 08

1b (bufp0): 28 96 04 08

21 (bufp1): 00 97 04 08

2a (bufp1): 00 97 04 08

```
bufp1 = &buf[1];
temp = *bufp0;
*bufp0 = *bufp1;
*bufp1 = temp;
```

6:	c7 05 <u>00 00 00 00</u> 04	movl \$0x4,0x0
d:	<u>00 00 00</u>	
		8: R_386_32 .bss
		c: R_386_32 buf
10:	a1 <u>00 00 00 00</u>	mov 0x0,%eax
11:		R_386_32 bufp0
15:	8b 00	mov (%eax),%eax
17:	89 45 fc	mov %eax,-0x4(%ebp)
1a:	a1 <u>00 00 00 00</u>	mov 0x0,%eax
1b:		R_386_32 bufp0
1f:	8b 15 <u>00 00 00 00</u>	mov 0x0,%edx
21:		R_386_32 .bss
25:	8b 12	mov (%edx),%edx
27:	89 10	mov %edx,(%eax)
29:	a1 <u>00 00 00 00</u>	mov 0x0,%eax
2a:		R_386_32 .bss
2e:	8b 55 fc	mov -0x4(%ebp),%edx
31:	89 10	mov %edx,(%eax)

重定位后

08048380 <main>:

```
8048380: 55          push %ebp
8048381: 89 e5       mov  %esp,%ebp
8048383: 83 e4 f0    and  $0xffffffff0,%esp
8048386: e8 09 00 00 00 call 8048394 <swap>
804838b: b8 00 00 00 00 mov  $0x0,%eax
```

buf @ 0x8049620
Bufp0 @ 0x8049628
Bufp1 @ 0x8049700

08048394 <swap>:

```
8048394: 55          push %ebp
8048395: 89 e5       mov  %esp,%ebp
8048397: 83 ec 10    sub  $0x10,%esp
804839a: c7 05 00 97 04 08 24 mov  $0x8049624,0x8049700
80483a1: 96 04 08
80483a4: a1 28 96 04 08 mov  0x8049628,%eax
80483a9: 8b 00       mov  (%eax),%eax
80483ab: 89 45 fc    mov  %eax,-0x4(%ebp)
80483ae: a1 28 96 04 08 mov  0x8049628,%eax
80483b3: 8b 15 00 97 04 08 mov  0x8049700,%edx
80493b9: 8b 12       mov  (%edx),%edx
80493bb: 89 10       mov  %edx,(%eax)
80493bd: a1 00 97 04 08 mov  0x8049700,%eax
80493c2: 8b 55 fc    mov  -0x4(%ebp),%edx
80493c5: 89 10       mov  %edx,(%eax)
80493c7: c9          leave
80493c8: c3          ret
```

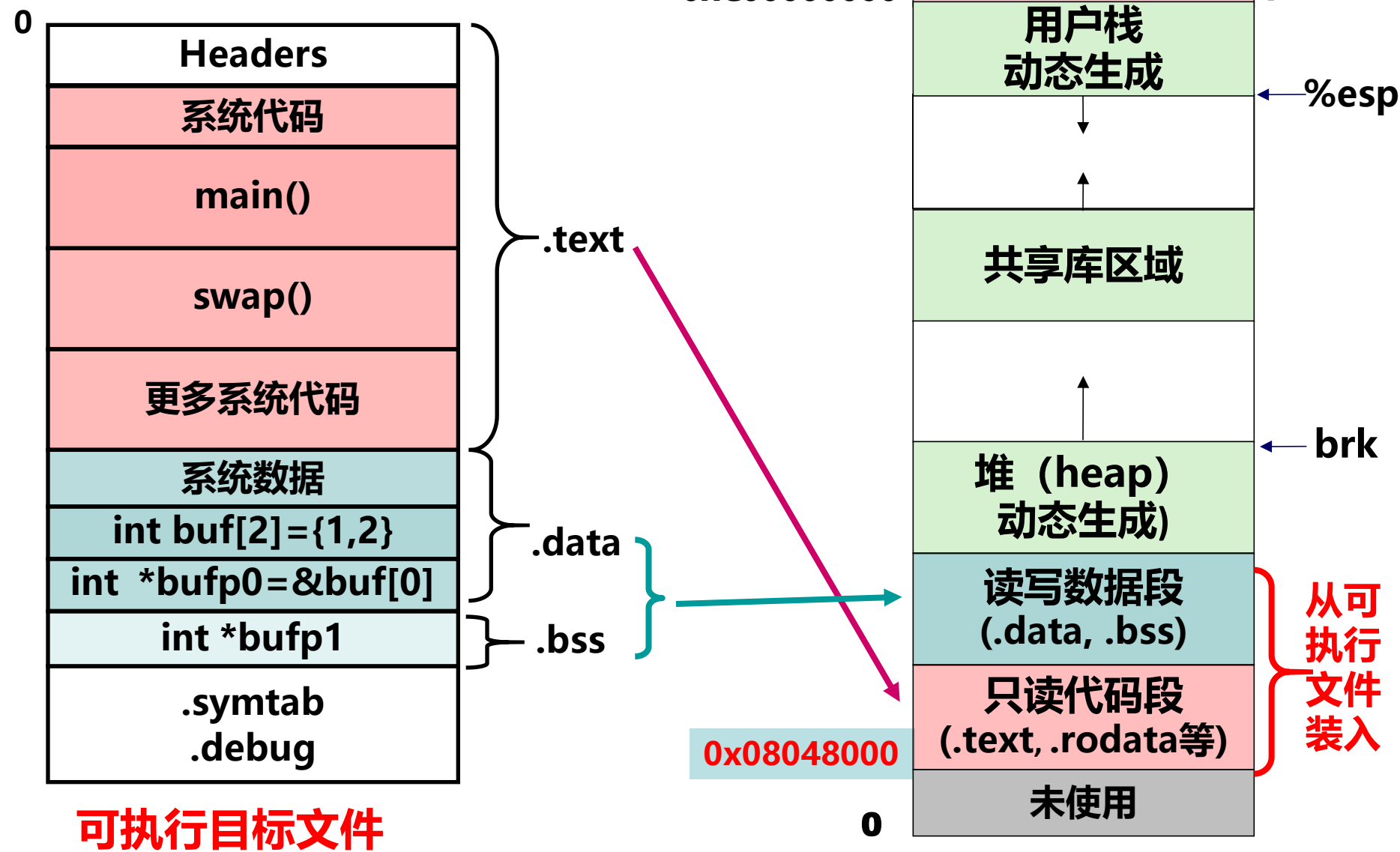
假定每个函数
要求4字节边界
对齐,故填充两
条nop指令

R[eip]=0x804838b

- 1) R[esp] ← R[esp]-4
- 2) M[R[esp]] ← R[eip]
- 3) R[eip] ← R[eip]+0x9

可执行文件的加载

可执行文件的存储器映像



回顾：可执行文件中的程序头表

```
typedef struct {  
    Elf32_Word    p_type;  
    Elf32_Off     p_offset;  
    Elf32_Addr    p_vaddr;  
    Elf32_Addr    p_paddr;  
    Elf32_Word    p_filesz;  
    Elf32_Word    p_memsz;  
    Elf32_Word    p_flags;  
    Elf32_Word    p_align;  
} Elf32_Phdr;
```

程序头表能够描述可执行文件中的节与虚拟空间中的存储段之间的映射关系，即可执行文件中连续的片（chunk）映射到虚拟空间中的连续的存储段的信息

一个表项说明虚拟地址空间中一个连续的片段或一个特殊的节

以下是GNU READELF显示的某可执行目标文件的程序头表信息

\$ readelf -l main

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
PHDR	0x000034	0x08048034	0x08048034	0x00100	0x00100	R E	0x4
INTERP	0x000134	0x08048134	0x08048134	0x00013	0x00013	R	0x1
[Requesting program interpreter: /lib/ld-linux.so.2]							
LOAD	0x000000	0x08048000	0x08048000	0x004d4	0x004d4	R E	0x1000
LOAD	0x000f0c	0x08049f0c	0x08049f0c	0x00108	0x00110	RW	0x1000

代码段：从0x8048000开始，按4KB（0x1000）对齐，具有读/执行权限，对应可执行文件第0~4d3H的内容（包括ELF头、段头部表以及.init、.text和.rodata节）

数据段：从0x8049f0c开始，按4KB（0x1000）对齐，具有读/写权限，前108H字节用可执行文件.data节内容初始化，后面110H-108H=8字节对应.bss节，被初始化为0

程序头（段头）表的信息

- 程序头表中包含了可执行文件中连续的片（chunk）如何映射到连续的存储段的信息。 **也可用命令：\$ readelf -l main**
- 以下是由**OBJDUMP**得到某可执行文件的段头部表内容

Read-only code segment

```
LOAD off    0x00000000 vaddr  0x08048000 paddr 0x08048000 align 2**12
            filesz 0x00000448 memsz 0x00000448 flags r-x
```

Read/write data segment

```
LOAD off    0x00000448 vaddr  0x08049448 paddr 0x08049448 align 2**12
            filesz 0x000000e8 memsz 0x00000104 flags rw-
```

代码段：从0x8048000开始，按4KB对齐，具有读/执行权限，对应可执行文件第0~447H的内容（包括ELF头、段头部表以及.init、.text和.rodata节）

数据段：从0x8049448开始，按4KB对齐，具有读/写权限，前E8H字节用可执行文件.data节内容初始化，后面104H-E8H=10H（32）字节对应.bss节，被初始化为0

可执行文件的加载

- 通过调用execve系统调用函数来调用加载器
- 加载器 (loader) 根据可执行文件的程序 (段) 头表中的信息, 将可执行文件的代码和数据从磁盘“拷贝”到存储器中 (实际上不会真正拷贝仅建立一种映像, 涉及虚存)
- 加载后, 将PC (EIP) 设定指向 Entry point (即符号_start处), 最终执行main函数, 以启动程序执行。

程序被启动
如 \$./P

调用fork()

以构造的argv和envp
为参数调用execve()

execve()调用加载器
进行可执行文件加载,
并最终转去执行main

_start: _libc_init_first → _init → atexit → main → _exit

ELF文件信息举例

\$ readelf -h main

可执行目标文件的ELF头

ELF Header:

Magic: 7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00

Class: ELF32

Data: 2's complement, little endian

Version: 1 (current)

OS/ABI: UNIX - System V

ABI Version: 0

Type: EXEC (Executable file)

Machine: Intel 80386

Version: 0x1

Entry point address: x8048580

Start of program headers: 52 (bytes into file)

Start of section headers: 3232 (bytes into file)

Flags: 0x0

Size of this header: 52 (bytes)

Size of program headers: 32 (bytes)

Number of program headers: 8

Size of section headers: 40 (bytes)

Number of section headers: 29

Section header string table index: 26

ELF 头
程序头表
.init 节
.text 节
.rodata 节
.data 节
.bss 节
.symtab 节
.debug 节
.line 节
.strtab 节
节头表

程序的链接

- 分以下三个部分介绍
 - **第一讲：目标文件格式**
 - 程序的链接概述、链接的意义与过程
 - ELF目标文件、重定位目标文件格式、可执行目标文件格式
 - **第二讲：符号解析与重定位**
 - 符号和符号表、符号解析
 - 与静态库的链接
 - 重定位信息、重定位过程
 - 可执行文件的加载
 - **第三讲：动态链接**
 - 动态链接的特性、程序加载时的动态链接、程序运行时的动态链接、动态链接举例

动态链接的共享库（Shared Libraries）

- 静态库有一些缺点：
 - 库函数（如printf）被包含在每个运行进程的代码段中，对于并发运行上百个进程的系统，造成极大的主存资源浪费
 - 库函数（如printf）被合并可在执行目标中，磁盘上存放着数千个可执行文件，造成磁盘空间的极大浪费
 - 程序员需关注是否有函数库的新版本出现，并须定期下载、重新编译和链接，更新困难、使用不便
- 解决方案: Shared Libraries（共享库）
 - 是一个目标文件，包含有代码和数据
 - 从程序中分离出来，磁盘和内存中都只有一个备份
 - 可以动态地在装入时或运行时被加载并链接
 - Window称其为动态链接库（Dynamic Link Libraries, .dll文件）
 - Linux称其为动态共享对象（Dynamic Shared Objects, .so文件）

共享库 (Shared Libraries)

动态链接可以按以下两种方式进行：

- 在第一次加载并运行时进行 (load-time linking).
 - Linux通常由动态链接器(ld-linux.so)自动处理
 - 标准C库 (libc.so) 通常按这种方式动态被链接
- 在已经开始运行后进行(run-time linking).
 - 在Linux中，通过调用 dlopen()等接口来实现
 - 分发软件包、构建高性能Web服务器等

在内存中只有一个备份，被所有进程共享（调用），节省内存空间

一个共享库目标文件被所有程序共享链接，节省磁盘空间

共享库升级时，被自动加载到内存和程序动态链接，使用方便

共享库可分模块、独立、用不同编程语言进行开发，效率高

第三方开发的共享库可作为程序插件，使程序功能易于扩展

自定义一个动态共享库文件

myproc1.c

```
# include <stdio.h>
void myfunc1()
{
    printf("%s", "This is myfunc1!\n");
}
```

myproc2.c

```
# include <stdio.h>
void myfunc2()
{
    printf("%s", "This is myfunc2\n");
}
```

PIC: Position Independent Code

位置无关代码

- 1) 保证共享库代码的位置可以是不确定的
- 2) 即使共享库代码的长度发生变化, 也不会影响调用它的程序

gcc -c myproc1.c myproc2.c

位置无关的共享代码库文件

gcc -shared -fPIC -o mylib.so myproc1.o myproc2.o

加载时动态链接

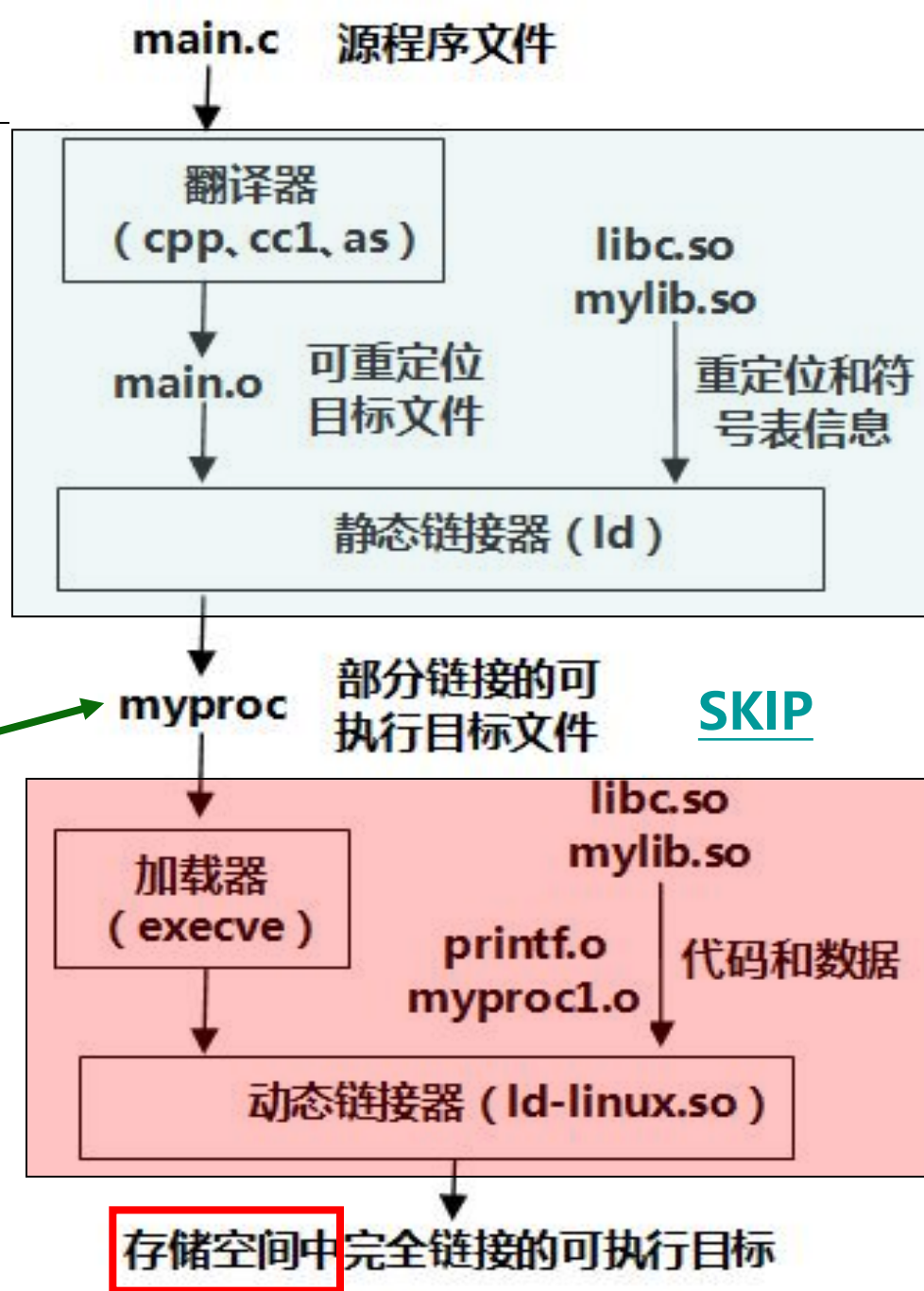
gcc -c main.c **libc.so**无需明显指出

gcc -o myproc main.o **./mylib.so**

调用关系: main→myfunc1→printf
main.c

```
void myfunc1(viod);  
int main()  
{  
    myfunc1();  
    return 0;  
}
```

加载 myproc 时, 加载器发现在其程序头表中有 **.interp** 段, 其中包含了动态链接器路径名 **ld-linux.so**, 因而加载器根据指定路径加载并启动动态链接器运行。动态链接器完成相应的重定位工作后, 再把控制权交给 myproc, 启动其第一条指令执行。



加载时动态链接

- 程序头表中有一个特殊的段：INTERP
- 其中记录了动态链接器目录及文件名ld-linux.so

[BACK](#)

Program Headers:

Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
PHDR	0x000034	0x08048034	0x08048034	0x00100	0x00100	R E	0x4
INTERP	0x000134	0x08048134	0x08048134	0x00013	0x00013	R	0x1
[Requesting program interpreter: /lib/ld-linux.so.2]							
LOAD	0x000000	0x08048000	0x08048000	0x004d4	0x004d4	R E	0x1000
LOAD	0x000f0c	0x08049f0c	0x08049f0c	0x00108	0x00110	RW	0x1000
DYNAMIC	0x000f20	0x08049f20	0x08049f20	0x000d0	0x000d0	RW	0x4
NOTE	0x000148	0x08048148	0x08048148	0x00044	0x00044	R	0x4
GNU_STACK	0x000000	0x00000000	0x00000000	0x00000	0x00000	RW	0x4
GNU_RELRO	0x000f0c	0x08049f0c	0x08049f0c	0x000f4	0x000f4	R	0x1

运行时动态链接

可通过**动态链接器接口**提供的函数在运行时进行动态链接

类UNIX系统中的动态链接器接口定义了相应的函数，如 `dlopen`, `dlsym`, `dlerror`, `dlclose`等，其头文件为 `dlfcn.h`

```
#include <stdio.h>
#include <dlfcn.h>
int main()
{
    void *handle;
    void (*myfunc1)();
    char *error;
    /* 动态装入包含函数myfunc1()的共享库文件 */
    handle = dlopen("./mylib.so", RTLD_LAZY);
    if (!handle) {
        fprintf(stderr, "%s\n", dlerror());
        exit(1);
    }
    /* 获得一个指向函数myfunc1()的指针myfunc1 */
    myfunc1 = dlsym(handle, "myfunc1");
    if ((error = dlerror()) != NULL) {
        fprintf(stderr, "%s\n", error);
        exit(1);
    }
    /* 现在可以像调用其他函数一样调用函数myfunc1() */
    myfunc1();
    /* 关闭（卸载）共享库文件 */
    if (dlclose(handle) < 0) {
        fprintf(stderr, "%s\n", dlerror());
        exit(1);
    }
    return 0;
}
```

位置无关代码 (PIC)

- 动态链接用到一个重要概念：
 - 位置无关代码 (Position-Independent Code, PIC)
 - GCC选项-fPIC指示生成PIC代码
- 共享库代码是一种PIC
 - 共享库代码的位置可以是不确定的
 - 即使共享库代码的长度发生变化, 也不影响调用它的程序
- 引入PIC的目的
 - 链接器无需修改代码即可将共享库加载到任意地址运行
- 所有引用情况
 - (1) 模块内的过程调用、跳转, 采用PC相对偏移寻址
 - (2) 模块内数据访问, 如模块内的全局变量和静态变量
 - (3) 模块外的过程调用、跳转
 - (4) 模块外的数据访问, 如外部变量的访问

要实现动态链接,
必须生成PIC代码

要生成PIC代码, 主要解决这两个问题

(1) 模块内部函数调用或跳转

- 调用或跳转源与目的地都在同一个模块，相对位置固定，只要用相对偏移寻址即可
- 无需动态链接器进行重定位

8048344 <bar>:

```
8048344: 55          pushl %ebp
8048345: 89 e5       movl %esp, %ebp
.....
8048352: c3         ret
8048353: 90         nop
```

8048354 <foo>:

```
8048354: 55          pushl %ebp
.....
8048364: e8 db ff ff ff  call 8048344 <bar>
8048369:
.....
```

```
static int a;
static int b;
extern void ext();
```

```
void bar()
```

```
{
    a=1;
    b=2;
}
```

```
void foo()
```

```
{
    bar();
    ext();
}
```

call的目标地址为:

0x8048369+
0xffffffffdb(-0x25)=
0x8048344

JMP指令也可用相
对寻址方式解决

(2) 模块内部数据引用

- .data节与.text节之间的相对位置确定，任何引用局部符号的指令与该符号之间的距离是一个常数

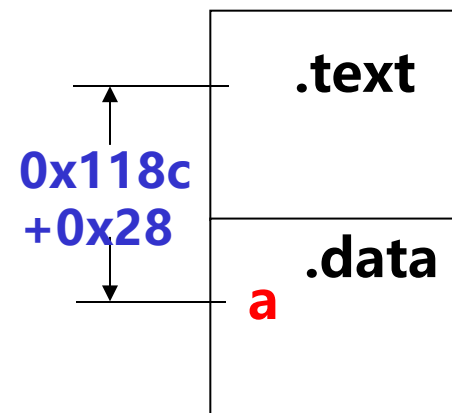
```
0000344 <bar>:
0000344: 55                pushl %ebp
0000345: 89 e5             movl  %esp, %ebp
0000347: e8 50 00 00 00    call 39c <__get_pc>
000034c: 81 c1 8c 11 00 00 addl  $0x118c, %ecx
0000352: c7 81 28 00 00 00 movl  $0x1, 0x28(%ecx)
.....
0000362: c3                ret

000039c <__get_pc>:
000039c: 8b 0c 24          movl  (%esp), %ecx
000039f: c3                ret
```

```
static int a;
extern int b;
extern void ext();

void bar()
{
    a=1;
    b=2;
}
.....
```

多用了4条指令



变量a与引用a的指令之间的距离为常数，调用__get_pc后，call指令的返回地址被置ECX。若模块被加载到0x9000000，则a的访问地址为：

$0x9000000 + 0x34c + 0x118c$ (指令与.data间距离) $+ 0x28$ (a在.data节中偏移)

(3) 模块外数据的引用

- 引用其他模块的全局变量，无法确定相对距离
- 在.data节开始处设置一个指针数组（全局偏移表，GOT），指针可指向一个全局变量
- GOT与引用数据的指令之间相对距离固定

```
static int a;  
extern int b;  
extern void ext();  
  
void bar()  
{  
    a=1;  
    b=2;  
}  
.....
```

00000344 <bar>:

00000344: 55

pushl %ebp

.....

00000357: e8 00 00 00 00

call 0000035c

0000035c: 5b

popl %ebx

0000035d:

addl \$1180, %ebx

.....

movl (%ebx), %eax

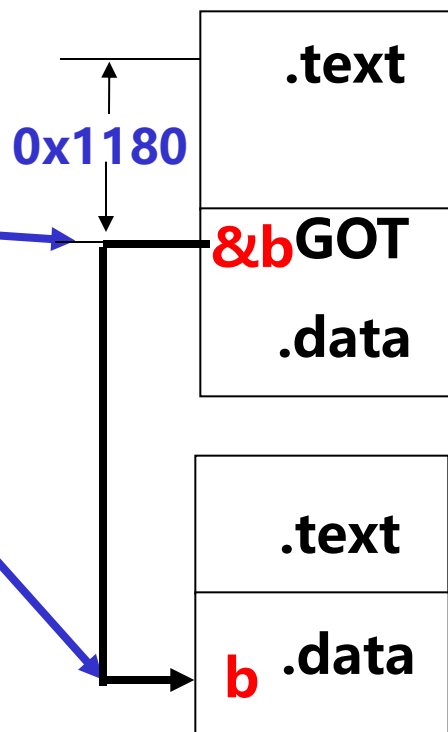
.....

movl \$2, (%eax)

.....

- 编译器为GOT每一项生成一个重定位项（如.rel节...）
- 加载时，动态链接器对GOT中各项进行重定位，填入所引用的地址（如**&b**）

PIC有两个缺陷：多用4条指令；多了GOT（Global Offset Table），并需多用一个寄存器（如EBX），易造成寄存器溢出



共享库模块

(4) 模块间调用、跳转

- **方法一：**类似于(3)，在GOT中加一个项(指针)，用于指向目标函数的首地址（如&ext）
- **动态加载时，填入目标函数的首地址**

0000050c <foo>:

0000050c: 55

pushl %ebp

● ● ● ● ● ●

```
00000557:  e8 00 00 00 00  call 0000055c
```

0000055c: 5b

popl %ebx

0000055d:

```
addl $1204, %ebx
```

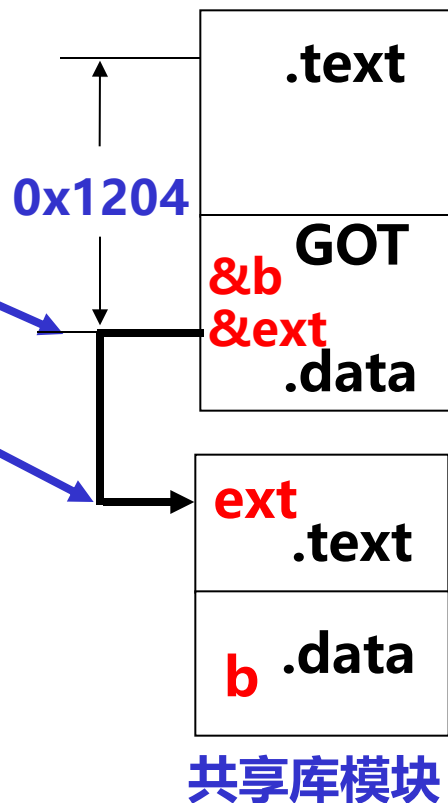
● ● ● ● ● ●

call *(%ebx)

● ● ● ● ● ●

***(%ebx)为间接地址: $R[eip] \leftarrow M[R[ebx]]$**

```
static int a;  
extern int b;  
extern void ext();  
  
void foo()  
{  
    bar();  
    ext();  
}  
.....
```



- 多用三条指令并额外多用一个寄存器 (如EBX)

可用“延迟绑定 (lazy binding)”技术减少指令条数：不在加载时重定位，而是延迟到第一次函数调用时。需要用GOT和PLT (Procedure linkage Table, 过程链接表)

共享库模块

PLT&GOT

PLT (Procedure Linkage Table) 是一段可执行代码，通常被称为"跳板" (Trampoline)。它在运行时将函数调用重定向到正确的地址，是实现延迟绑定的关键组件。

主要特点

- 每个外部函数在 PLT 中都有一个对应的条目 (Entry)
- 首次调用时，PLT 会触发动态链接器解析函数的真实地址
- 后续调用直接通过 GOT 跳转到已解析的地址

位置与属性

- 所在段: .plt
- 访问权限: 只读 + 可执行
- 内容类型: 机器指令
- 条目数量: 每个外部函数一个

GOT (Global Offset Table) 是一个指针数组，存储了外部函数和全局变量的运行时绝对地址。它是连接代码段和数据段的桥梁。

主要特点

- 在程序加载时由动态链接器填充正确的绝对地址
- 代码段通过 GOT 间接访问外部符号，实现位置无关性
- 支持延迟绑定，首次调用后更新为真实函数地址

GOT 的两个段

- .got 段: 存放全局变量引用的地址
- .got.plt 段: 存放外部函数引用的地址

访问权限: 可读 + 可写 (动态链接器需要在运行时修改)

PLT&GOT

GOT 与 PLT 协作关系



函数调用顺序：第一次调用 vs. 非第一次调用

当程序第一次调用外部函数（如 `printf`）时，完整的调用流程如下

1. 代码调用

`call printf@plt`

程序代码调用 `printf` 函数，实际跳转到 PLT 条目

2. PLT 跳转

`jmp *GOT[n]`

跳转到对应的 GOT 条目，准备获取函数地址

3. GOT 初值

此时 `GOT[n]` 存储的是 PLT 中下一条指令的地址（即还没有真实地址）

4. PLT 回退

跳转回 PLT，压入函数 ID，跳转到 `PLT[0]`（公共解析入口）

5. 调用链接器

动态链接器（`_dl_runtime_resolve`）被调用，查找 `printf` 的真实地址

6. 更新 GOT

链接器将 `printf` 的真实地址填入 `GOT[n]`，后续调用将直接使用该地址

7. 执行函数

跳转到 `printf` 的真实地址执行函数代码

关键：首次调用的额外开销只在第一次发生时存在，后续调用将直接从 GOT 跳转到函数，无需再次解析。

第N次调用流程：

1. 代码调用

`call printf@plt`

程序代码再次调用 `printf` 函数

2. PLT 跳转

`jmp *GOT[n]`

跳转到对应的 GOT 条目

3. GOT 直接跳转

此时 `GOT[n]` 已存储真实地址，直接跳转到 `printf` 执行，无需经过动态链接器

核心优势：

延迟绑定机制确保只有实际被调用的函数才会产生解析开销，对于大型应用程序中大量未使用的库函数，可显著减少启动时间和内存占用

(4) 模块间调用、跳转

```
extern void ext();  
void foo() {  
    bar();  
    ext();  
}  
.....
```

方法二：延迟绑定

GOT是.data节一部分，开始三项固定，含义如下：

GOT[0]为.dynamic节首址，该节中包含动态链接器所需要的基本信息，如符号表位置、重定位表位置等；

GOT[1]为动态链接器的标识信息

GOT[2]为动态链接器延迟绑定代码的入口地址

调用的共享库函数都有GOT项，如GOT[3]对应ext

延时绑定代码根据GOT[1]和ID确定ext地址填入GOT[3]，并转ext执行，以后调用ext，只要多执行一条jmp指令而不是多3条指令。

PLT是.text节一部分，结构数组，每项16B，除PLT[0]外，其余项各对应一个共享库函数，如PLT[1]对应ext

PLT[0]

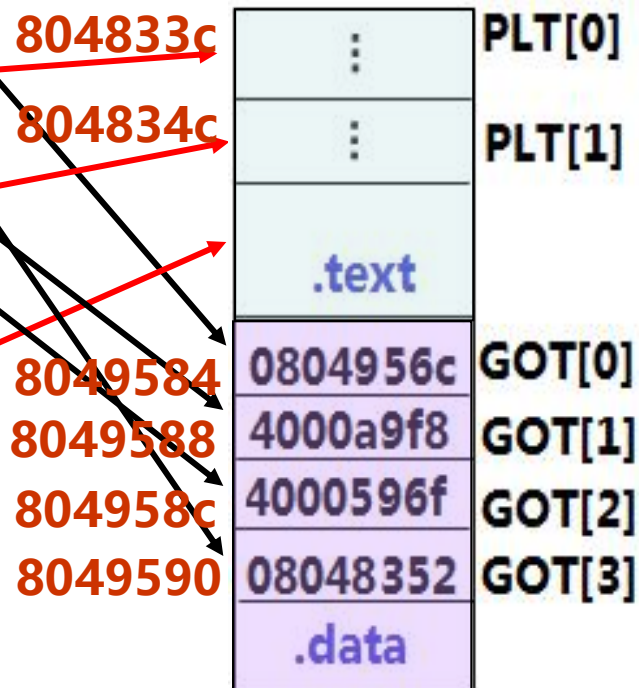
```
0804833c: ff 35 88 95 04 08  pushl 0x8049588  
8048342: ff 25 8c 95 04 08  jmp *0x804958c  
8048348: 00 00 00 00
```

PLT[1] <ext> 用 ID=0 标识ext()函数

```
0804834c: ff 25 90 95 04 08  jmp *0x8049590  
8048352: 68 00 00 00 00 00  pushl $0x0  
8048357: e9 e0 ff ff ff    jmp 804833c
```

ext()的调用指令：

```
804845b: e8 ec fe ff ff    call 804834c <ext>
```



可执行文件foo

本章小结

- 链接处理涉及到三种目标文件格式：可重定位目标文件、可执行目标文件和共享目标文件。共享库文件是一种特殊的可重定位目标。
- ELF目标文件格式有链接视图和执行视图两种，前者是可重定位目标格式，后者是可执行目标格式。
 - 链接视图中包含ELF头、各个节以及节头表
 - 执行视图中包含ELF头、程序头表（段头表）以及各种节组成的段
- 链接分为静态链接和动态链接两种
 - 静态链接将多个可重定位目标模块中相同类型的节合并起来，以生成完全链接的可执行目标文件，其中所有符号的引用都是在虚拟地址空间中确定的最终地址，因而可以直接被加载执行。
 - 动态链接的可执行目标文件是部分链接的，还有一部分符号的引用地址没有确定，需要利用共享库中定义的符号进行重定位，因而需要由动态链接器来加载共享库并重定位可执行文件中部分符号的引用。
 - 加载时进行共享库的动态链接
 - 执行时进行共享库的动态链接

本章小结

- 链接过程需要完成符号解析和重定位两方面的工作
 - 符号解析的目的就是将符号的引用与符号的定义关联起来
 - 重定位的目的是分别合并代码和数据，并根据代码和数据在虚拟地址空间中的位置，确定每个符号的最终存储地址，然后根据符号的确切地址来修改符号的引用处的地址。
- 在不同目标模块中可能会定义相同符号，因为相同的多个符号只能分配一个地址，因而链接器需要确定以哪个符号为准。
- 编译器通过对定义符号标识其为强符号还是弱符号，由链接器根据一套规则来确定多重定义符号中哪个是唯一的定义符号，如果不了解这些规则，则可能无法理解程序执行的有些结果。
- 加载器在加载可执行目标文件时，实际上只是把可执行目标文件中的只读代码段和可读写数据段通过页表映射到了虚拟地址空间中确定的位置，并没有真正把代码和数据从磁盘装入主存。

本章作业

- **PA1**要全部完成咧哇同学们
- **PA2**开启！

计算机系统设计pa2.1&2.2

南开大学 计算机学院

在进行本 PA 前,请在工程目录下执行以下命令进行分支整理,否则将影响你的成绩:

```
git commit --allow-empty -am "before starting pa2"  
git checkout master  
git merge pa1  
git checkout -b pa2
```

```
while (1) {  
    从PC指示的存储器位置取出指令;  
    执行指令;  
    更新PC;  
}
```



- 发现未实现指令
- 查看反汇编文件，获取指令信息
- 查询 i386 官方手册，确认指令功能
- 填写 opcode_table 表
- 实现译码函数和执行函数
- 添加至 all-instr.h
- 测试实现正确性

直接运行测试程序，在程序报错时会提示未实现指令信息

```
(base) → cputest git:(pal) make ARCH=x86-nemu ALL=dummy run
Building dummy [x86-nemu]
+ CC tests/dummy.c
Building am [x86-nemu]
make[2]: *** No targets specified and no makefile found. Stop.
[src/monitor/monitor.c,65,load_img] The image is /root/PA/ics2018/nexus-am/tests/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:00:06, Mar 29 2022
For help, type "help"
(nemu) c
invalid opcode(eip = 0x0010000a): e8 01 00 00 00 90 55 89 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x0010000a is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x0010000a) in the disassembling result to distinguish which case it is.

If it is the first case, see



for more details.

If it is the second case, remember:
* The machine is always right!
* Every line of untested code is always wrong!

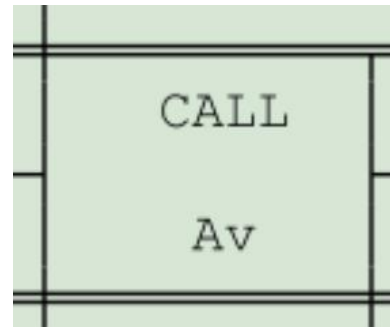
(nemu) █
```

```

dummy-x86-nemu.txt x rtl.h exec.c decode.c control.c all-i
nexus-am > tests > cputest > build > dummy-x86-nemu.txt
1
2 /root/PA/ics2018/nexus-am/tests/cputest/build/dummy-x86-nemu:
3
4
5 Disassembly of section .text:
6
7 00100000 <_start>:
8   100000: bd 00 00 00 00      mov     $0x0,%ebp
9   100005: bc 00 7c 00 00      mov     $0x7c00,%esp
10  10000a: e8 01 00 00 00      call    100010 <_trm_init>
11  10000f: 90                  nop
12
13 00100010 <_trm_init>:
14  100010: 55                  push    %ebp
15  100011: 89 e5               mov     %esp,%ebp
16  100013: 83 ec 08            sub     $0x8,%esp
17  100016: e8 05 00 00 00      call    100020 <main>
18  10001b: d6                  (bad)
19  10001c: eb fe               jmp     10001c <_trm_init+0xc>
20  10001e: 66 90               xchg    %ax,%ax
21
22 00100020 <main>:
23  100020: 55                  push    %ebp
24  100021: 89 e5               mov     %esp,%ebp
25  100023: 31 c0               xor     %eax,%eax
26  100025: 5d                  pop     %ebp
27  100026: c3                  ret
28

```

第e行第8列，对应opcode_table结果：



A Direct address; the instruction has no modR/M byte; the address of the operand is encoded in the instruction; no base register, index register, or scaling factor can be applied; e.g., far JMP (EA).

v Word or double word, depending on operand size attribute.



```
/* 0xe0 */ EMPTY, EMPTY, EMPTY, EMPTY,  
/* 0xe4 */ IDEXW(in_I2a,in,1), IDEX(in_I2a,in), IDEXW(out_a  
/* 0xe8 */ IDEX(J, call), IDEX(J, jmp), EMPTY, IDEXW(J,jmp,  
/* 0xec */ IDEXW(in_dx2a,in,1), IDEX(in_dx2a,in), IDEXW(out  
/* 0xf0 */ EMPTY, EMPTY, EMPTY, EMPTY,  
/* 0xf4 */ EMPTY, EMPTY, IDEXW(5,0,2,1), IDEX(5,0,2)
```

Call指令和Jump指令共用了同一个译码函数，指定了不同的执行函数

许多译码/执行函数已经实现好的，可直接使用

在decode.c中实现译码函数，将译码结果保存在decinfo中
(ics2018应该是decoding)

```
make_DHelper(J) {  
    decode_op_SI(pc, id_dest, false);  
    // the target address can be computed in the decode stage  
    decinfo.jump_pc = id_dest->simm + *pc;  
}
```

control.c中添加执行函数

调用的辅助函数可能需要自己实现

```
make_EHelper(call) {  
    // the target address is calculated at the decode stage  
    rtl_push(&decinfo.seq_pc);  
    rtl_j(decinfo.jump_pc);  
    print_asm("call %x", decinfo.jump_pc);  
}  
  
make_EHelper(jmp) {  
    // the target address is calculated at the decode stage  
    rtl_j(decinfo.jump_pc);  
    print_asm("jmp %x", decinfo.jump_pc);  
}
```



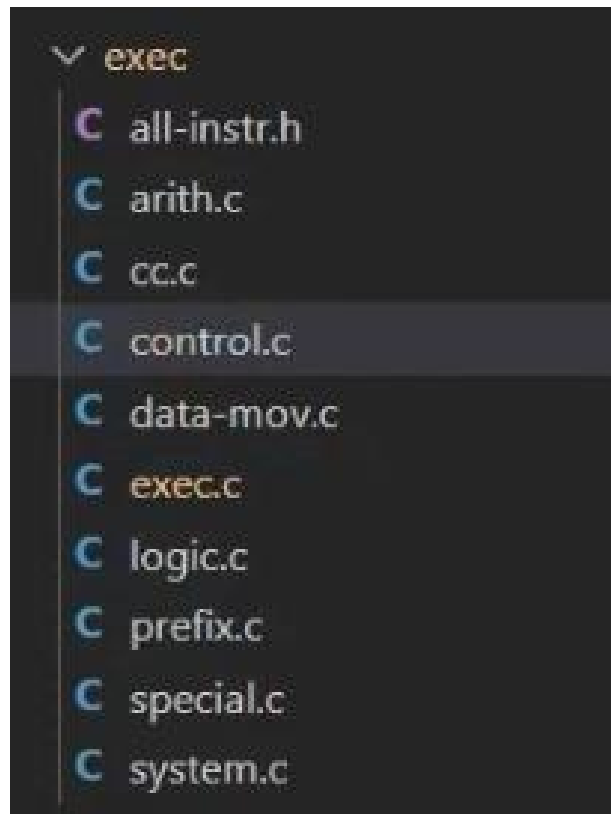

```
static inline void rtl_sext(rtlreg_t* dest, const rtlreg_t* src1, int width) {  
    // dest <- signext(src1[(width * 8 - 1) .. 0])  
    // TODO();  
    rtl_li(&t1, 32 - width * 8); // get how many bits to shift  
    rtl_shl(dest, src1, &t1);  
    rtl_sar(dest, dest, &t1);  
}  
  
static inline void rtl_push(const rtlreg_t* src1) {  
    // esp <- esp - 4  
    // M[esp] <- src1  
    TODO();  
}  
  
static inline void rtl_pop(rtlreg_t* dest) {  
    // dest <- M[esp]  
    // esp <- esp + 4  
    TODO();  
}  
  
static inline void rtl_eq0(rtlreg_t* dest, const rtlreg_t* src1) {  
    // dest <- (src1 == 0 ? 1 : 0)  
    TODO();  
}  
  
static inline void rtl_eqi(rtlreg_t* dest, const rtlreg_t* src1, int imm) {
```

```
static inline void rtl_push(const rtlreg_t* src1) {  
    // esp <- esp - 4  
    // M[esp] <- src1  
    // TODO();  
    cpu.esp -= 4;  
    rtl_sm(&cpu.esp, 4, src1);  
}
```

所有实现的指令执行函数，都需要在"all-instr.h"中“备案”

指令分为control, data-mov, arith, logic, system 等等类型

添加注释分开写，方便调试



```
// control.c
```

```
make_EHelper(call);  
make_EHelper(call_rm);  
make_EHelper(ret);  
make_EHelper(jcc);  
make_EHelper(jmp);  
make_EHelper(jmp_rm);
```

```
// data-mov.c
```

```
make_EHelper(push);  
make_EHelper(lea);  
make_EHelper(movzx);  
make_EHelper(movsx);  
make_EHelper(leave);  
make_EHelper(pop);  
make_EHelper(pusha);
```



```
+ CC src/cpu/exec/data-mov.c
+ CC src/cpu/intr.c
+ CC src/cpu/decode/modrm.c
+ CC src/cpu/decode/decode.c
+ LD build/nemu
[src/monitor/monitor.c,65,load_img] The image is /root/PA/ics2018/nexus-am/tests/cputest/
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:00:06, Mar 29 2022
For help, type "help"
(nemu) c
invalid opcode(eip = 0x00100010): 55 89 e5 83 ec 08 e8 05 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x00100010 is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x00100010) in the disassembling result to distinguish which case it is.

If it is the first case, see

for more details.

If it is the second case, remember:
* The machine is always right!
* Every line of untested code is always wrong!

(nemu) █
```

■ While True:

- 发现未实现指令
- 查看反汇编文件, 获取指令信息
- 查询 i386 官方手册, 确认指令功能
- 填写 opcode_table 表
- 实现译码函数和执行函数
- 添加至 all-instr.h
- 测试实现正确性

■ Differential Testing

- 真机执行与NEMU执行做对比, 错误时直接输出寄存器信息
- 在做pa2.2之前, 建议先实现 Differential Testing

(1) 源文件组织架构 (nemu/src/cpu/exec 文件夹)

decode

decode.c 译码函数 (make_Dhelper 与 makeDophelper)

modrm.c Mod R/M 字段的处理函数

exec

all-instr.h 所有执行函数的定义 (make_Ehelper)

arith.c 算术运算执行函数的实现

cc.c SETcc 执行函数的实现

control.c 控制语句执行函数的实现

data-mov.c 数据传输指令执行函数的实现

exe.c 执行相关文件 (含 opcode_table 的维护)

logic.c 逻辑运算执行函数的实现

prefix.c 实现 16bit 与 32bit 操作数的切换 (指令的 0x66 前缀)

special.c 含 nop、nemu_trap 指令

system.c 含有 IO 指令 (在第三阶段实现)



(2) 现代指令系统

- ❖ Data Movement Instructions: `mov`, `push`, `pop`, `leave`, `cld`(在 i386 手册中为 `cdq`), `movsx`, `movzx`
- ❖ Binary Arithmetic Instructions: `add`, `inc`, `sub`, `dec`, `cmp`, `neg`, `adc`, `sbb`, `mul`, `imul`, `div`, `idiv`
- ❖ Logical Instructions: `not`, `and`, `or`, `xor`, `sal(shl)`, `shr`, `sar`, `setcc`, `test`
- ❖ Control Transfer Instructions: `jmp`, `jcc`, `call`, `ret`
- ❖ Miscellaneous Instructions: `lea`, `nop`

实验中，结合(1)与(2)判断某指令的执行函数在哪一源文件中。（注：lea 执行函数在 `data-mov.c` 中）

(3) 操作数类型

详情参见i386手册P412

E A modR/M
is either
address,
following
a displac

G The reg
(00).

I Immediate
of the in

R The mod
e.g., M

根据操作数类型选择 decode.c 中的译码函数时，大致规律为：

类型	AT&T 汇编指令中的标识	译码函数示例
E	r/m8, r/m16, r/m32	E2G, G2E
G	r8, r16, r32 (常为多操作数)	E2G, G2E
I	立即数 imm	I2a, I
SI (nemu 特有)	符号扩展的立即数	SI2a, SI_E2G
a (nemu 特有)	AL, AX, EAX	I2a, SI2a
r	r8, r16, r32 (常为单操作数)	r

再次强调实现指令的基本流程

概括而言：1. 在 `opcode_table` 中填写正确的译码函数、执行函数与操作数宽度。（`exec.c` 文件）

2. 用 RTL 实现正确的执行函数。（所在文件根据指令类型判断）

- a) `make ARCH=x86-nemu ALL=XXX run` 运行测试文件
- b) 查看 `invalid opcode` 所标识的 `eip`, 打开 `txt` 日志文件, 找到对应位置, 查看其对应的汇编指令（如上文中根据 `eip=0x10000a` 找到 `call` 指令）
- c) 在 `i386` 手册中查看该汇编指令（目录 17.2.2.1 Instruction Set Detail）
- d) 根据 `i386` 手册与 `decode.c` 填写 `exec.c` 中的 `opcode_table` 表
- e) 实现执行函数（注意在 `all-instr.h` 文件中声明）
- f) 可（集中）使用 `make ARCH=x86-nemu ALL=XXX run` 进行测试, 若 `invalid opcode` 的 `eip` 更新, 重复上述步骤

push指令实现

(1) 查看 i386 手册

功能：PUSH 递减堆栈指针ESP，然后将源操作数传送到 ESP 指的堆栈顶部。

指令解读：

50 + /r PUSH r32 2 Push register dword

+表示将其后编码按数值加到 opcode。

+/r, r 为 0~7 中一个数字，表示将通用寄存器编号按数值加到 opcode 中。

综上，push 的 opcode 为 0x50-0x57。

(2) 译码函数

目的：从 opcode 的后三位中读取通用寄存器的编号。

在 decode.c 中找到译码函数为 make_DHelper(r)。

make_DHelper(r): 调用 make_DopHelper(r), 读取 decoding.opcode 中标志的寄存器, 将寄存器的内容放入 op->val 中。

```
make_EHelper(call);  
make_EHelper(push);  
make_EHelper(pop);  
make_EHelper(sub);  
make_EHelper(xor);  
make_EHelper(ret);
```

(3) 执行函数

在文件 nemu/src/cpu/exec/all-instr.h(后续简写为 all-instr.h)中进行函数声明：
make_EHelper(push)。因为后续需要实现的其余指令都需要该步骤，就在这里一同声明：

其次, 根据功能可推断 `make_EHelper(push)`函数的实现在 `data-mov.c` 文件中。
`make_EHelper(push)`需先实现 `rtl_push` 函数: 修改栈顶, 并将指针 `src1` 中的内容写入栈。

`nemu/include/cpu/rtl.h`(后续简写为 `rtl.h`)

```
static inline void rtl_push(const rtlreg_t* src1) {  
    // esp <- esp - 4  
    // M[esp] <- src1  
    // TODO();  
    rtl_subi(&cpu.esp, &cpu.esp, 4); // 减法: 寄存器与立即数  
    rtl_sm(&cpu.esp, 4, src1); // 写入内存: 32bit  
    // cpu.esp中存放: 栈顶的地址  
}
```


实现 make_EHelper(push)函数: 调用 rtl_push 函数写栈。(译码函数将源寄存器的内容保存在了 id_dest->val 中)。

nemu/src/cpu/exec/data-mov.c(后续写作 data-mov.c)

```
make_EHelper(push) {  
    //TODO();  
    rtl_push(&id_dest->val);  
  
    print_asm_template1(push);  
}
```

(4) 填写 opcode_table

根据上文，opcode 为 0x50-0x57，译码函数为 make_DHelper(r)，执行函数为 make_EHelper(push)，定义为 IDEX(r,push)。

nemu/src/cpu/exec/exec.c(后续写作 exec.c)

```
/*1 byte_opcode_table */
```

```
/* 0x50 */ IDEX(r,push), IDEX(r,push), IDEX(r,push), IDEX(r,push),  
/* 0x54 */ IDEX(r,push), IDEX(r,push), IDEX(r,push), IDEX(r,push),
```

pop与push指令类似，不做详细介绍，对call指令，介绍一些关键点 call指令的译码函数为make_DHelper(J)

操作数仅一个，为立即数。CPU 的跳转目标地址=当前 eip+立即数 offset（可正可负）。

查看 decode/decode.c，找到对应的译码函数 make_DHelper(J)：

make_DHelper(J)调用decode_op_SI函数实现立即数的读取，并更新jmp_eip。

```
make_DHelper(J) {  
    decode_op_SI(eip, id_dest, false);  
    // the target address can be computed in the decode stage  
    // 目标地址可在译码阶段进行计算  
    decoding.jmp_eip = id_dest->simm + *eip;  
}
```

查看 decode_op_SI 函数，即 make_DopHelper(SI)，需要填充该函数代码。与 make_DopHelper(I)相比，make_DopHelper(SI)的不同之处在于：

其一，要求操作数宽度 width=1 或 4。

其二，读取的立即数需转为 signed immediate（带符号的整数）。



make_DopHelper(SI)的补充代码如下:

```
static inline make_DopHelper(SI) {
    assert(op->width == 1 || op->width == 4);

    op->type = OP_TYPE_IMM;

    /* TODO: Use instr_fetch() to read `op->width' bytes of memory
     * pointed by `eip'. Interpret the result as a signed immediate,
     * and assign it to op->simm.
     */
    op->simm = ???
    /*
    //TODO();
    op->simm = instr_fetch(eip, op->width); //获取立即数
    if (op->width==1)
        op->simm=(int8_t)op->simm;
    // else
    //     op->simm=(int32_t)op->simm; //翻译为有符号的立即数

    rtl_li(&op->val, op->simm);
```

opcode_table填写问题

在这里先解释一下为什么框架代码给出的 0xb8 的 opcode_entry 不设置 width。
操作数宽度为 16 或 32 位时，是根据前缀码 0x66 进行判断的：若有前缀则为 16bit，否则为 32bit。

```
B8 + rw  MOV reg16,imm16    2          Move immediate word to register
B8 + rd  MOV reg32,imm32    2          Move immediate dword to register

/* 0xb8 */ IDEX(mov_I2r, mov), IDEX(mov_I2r, mov), IDEX(mov_I2r, mov), IDEX
(mov_I2r, mov),
/* 0xbc */ IDEX(mov_I2r, mov), IDEX(mov_I2r, mov), IDEX(mov_I2r, mov), IDEX
(mov_I2r, mov),
```

下面介绍0x66 的实现原理——查看set_width 函数，在初始定义opcode_entry 时，如果 width=0，程序会根据 is_operand_size_16 确定操作数为 16bit 或 32bit，由于 bool 变量初始为 False，默认情况下操作数会设为 32bit。

```
static inline void set_width(int width) {
    if (width == 0) {
        width = decoding.is_operand_size_16 ? 2 : 4;
    }
    decoding.src.width = decoding.dest.width = decoding.src2.width = width;
}
```


而 16bit 操作数是在指令有前缀码 0x66 时进行设置的, 当遇到 0x66 时, 程序根据 opcode_table 调用 make_Ehelper(operand_size)函数 (函数在 prefix.c), 该函数先将操作数宽度置为 16 位, 后调用 exec_real(eip)执行当前指令, 执行完毕后恢复原is_operand_size_16(在 3.1 的 XCHG/NOP 指令中可以看见具体示例)。

```
make_EHelper(operand_size) {  
    decoding.is_operand_size_16 = true;  
    exec_real(eip);  
    decoding.is_operand_size_16 = false;  
}
```

所以当操作数宽度 width=2/4 时, 不需进行 opcode_entryd 的 width 设置。

回到本例的 call 指令: opcode 为 0xE8, 译码函数: make_DHelper(J), 执行函数: make_EHelper(call), 操作数宽度为 2/4。则定义为 IDEX(J,call)。

exec.c

```
/*1 byte_opcode_table */
```

```
/* 0xe8 */ IDEX(J,call), EMPTY, EMPTY, EMPTY,
```

在 x86 初始化时，EFLAGS 寄存器状态设为 0x0000 0002H。

特别注意：x86 为小端（低字节数据存储在低地址中），eflags 位域定义时，要从低地址到高地址进行声明。

CF(bit 0) [Carry flag]

若算术操作产生的结果在最高有效位(most-significant bit)发生进位或借位则将其置 1，反之清零。这个标志指示无符号整型运算的溢出状态，这个标志同样在多倍精度运算(multiple-precision arithmetic)中使用。

ZF(bit 6) [Zero flag]

若结果为 0 则将其置 1，反之清零。

SF(bit 7) [Sign flag]

该标志被设置为有符号整型的最高有效位。(0 指示结果为正，反之则为负)

IF(bit 9) [Interrupt enable flag]

该标志用于控制处理器对可屏蔽中断请求(maskable interrupt requests)的响应。置 1 以响应可屏蔽中断，反之则禁止可屏蔽中断。

OF(bit 11) [Overflow flag]

如果整型结果是较大的正数或较小的负数，并且无法匹配目的操作数时将该位置 1，反之清零。这个标志为带符号整型运算指示溢出状态。

eflags寄存器的定义及初始化

nemu/include/cpu/reg.h

//实现eflags寄存器

```
struct bs{
    unsigned int CF:1;
    unsigned int one:1; //理论上保持为1, 可以用
    unsigned int :4;
    unsigned int ZF:1;
    unsigned int SF:1; //bit 0-7

    unsigned int :1;
    unsigned int IF:1;
    unsigned int :1;
    unsigned int OF:1;
    unsigned int :20; //bit 8-31
} eflags;
} CPU_state;
```

nemu/src/monitor/monitor.c

修改 restart()函数, 使用 memcpy 将 eflags 设置为 0x0000 0002H。

```
static inline void restart() {
    /* Set the initial instruction pointer. */
    cpu.eip = ENTRY_START;

    //进行eflags的初始化
    //printf("sizeof(cpu.eflags):%d\n", sizeof(cpu.eflags));
    unsigned int origin=2;
    memcpy(&cpu.eflags, &origin, sizeof(cpu.eflags));
    //printf("eflags:one=%d, OF=%d\n", cpu.eflags.one, cpu.eflags.OF);

#ifdef DIFF_TEST
    init_qemu_reg();
#endif
}
```

在rtl.h中完成一系列和eflags寄存器标志位有关的函数, 在pa2.1中第一次在sub指令实现部分涉及, 后续与计算相关的指令也几乎都需要考虑eflags寄存器标志位

0x80、0x81 与 0x83 的 sub 需要进行 opcode 的扩展。

```
80 /5 ib SUB r/m8,imm8 2/7
81 /5 iw SUB r/m16,imm16 2/7
81 /5 id SUB r/m32,imm32 2/7
83 /5 ib SUB r/m16,imm8 2/7
83 /5 ib SUB r/m32,imm8 2/7
```

```
/* 0x80 */ IDEXW(I2E, gp1, 1
```

查看 gp1 执行函数。在 exec.c 中有如下提示：

```
/* 0x80, 0x81, 0x83 */
make_group(gp1,
    EMPTY, EMPTY, EMPTY, EMPTY,
    EMPTY, EMPTY, EMPTY, EMPTY)
```

查看 make_group 函数，该函数用于处理扩展 opcode 的情况。以 gp1 为例，在 opcode_table_gp1 中存储各 opcode_entry，并在运行 make_EHelper(gp1) 该时使用 idex(eip, opcode_table_gp1[decoding.ext_opcode]) 作为进一步的译码-执行函数。

因为 sub 的 ext_opcode=5，所以在 opcode_table_gp1[5] 处填写 EX(sub) 执行函数。

exec.c

```
/* 0x80, 0x81, 0x83 */
make_group(gp1,
    EMPTY, EMPTY, EMPTY, EMPTY,
    EMPTY, EX(sub), EMPTY, EMPTY)
```

nemu\src\monitor\diff-test\diff-test.c

开启关闭:

common.h 中解除注释 (注释掉)

DIFF_TEST 的宏

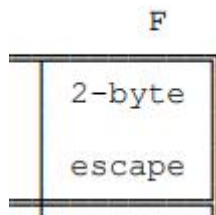
使用 differential testing 将会降低 NEMU 的运行速度, 所以在第三阶段中, 我们将关闭该功能 (即在 common.h 中注释掉 DIFF_TEST 的宏)。如果随后调试需要, 再将其打开。

```
// TODO: Check the registers state with QEMU.
// Set `diff` as `true` if they are not the same.
// TODO();
//todo:s输出提示信息+让代码更美观
if(r.eax!=cpu.eax)
    diff=true;
if(r.ecx!=cpu.ecx)
    diff=true;
if(r.edx!=cpu.edx)
    diff=true;
if(r.ebx!=cpu.ebx)
    diff=true;
if(r.esp!=cpu.esp)
    diff=true;
if(r.ebp!=cpu.ebp)
    diff=true;
if(r.esi!=cpu.esi)
    diff=true;
if(r.edi!=cpu.edi)
    diff=true;
if(diff){
    Log("different in general registers: when nemu.eip=0x%x",cpu.eip);
}
if(r.eip!=cpu.eip){
    diff=true;
    Log("different:qemu.eip=0x%x,and nemu.eip=0x%x",r.eip,cpu.eip);
}
```


exec.c

```
/* 0x00 */ IDEXW(G2E, add, 1), IDEX(G2E, add),
```

```
/* 0x02 */ IDEXW(E2G, add, 1), IDEX(E2G, add),
```



```
/* 0x0c */ EMPTY, EMPTY, EMPTY, EX(2byte_esc),
```

```
/*2 byte_opcode_table */
```

```
/* 0x00 */ EMPTY, IDEX(gp7_E, gp7), EMPTY, EMPTY,
```

i386手册 P414 Appendix A One-Byte Opcode Map

	0	1	2	3	4	5
	ADD					
0	Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	eAX, Iv

i386手册 P413 Appendix A Codes for Operant Type

- b Byte (regardless of operand size attribute)
- v Word or double word, depending on operand size attribute.

i386手册 P415 Appendix A Two-Byte Opcode Map (first byte is 0FH)

```

/* 0x80, 0x81, 0x83 */
make_group(gp1,
    EMPTY, EMPTY, EMPTY, EMPTY,
    EMPTY, EMPTY, EMPTY, EMPTY)

/* 0xc0, 0xc1, 0xd0, 0xd1, 0xd2 */
make_group(gp2,
    EMPTY, EMPTY, EMPTY, EMPTY,
    EMPTY, EMPTY, EMPTY, EMPTY)

/* 0xf6, 0xf7 */
make_group(gp3,
    EMPTY, EMPTY, EMPTY, EMPTY,
    EMPTY, EMPTY, EMPTY, EMPTY)

```

i386手册 P416 Appendix A

Opcodes determined by bits 5,4,3 of modR/M byte:

G r o u p	mod			nnn		R/M		
	000	001	010	011	100	101	110	111
1	ADD	OR	ADC	SBB	AND	SUB	XOR	CMP
2	ROL	ROR	RCL	RCR	SHL	SHR		SAR
3	TEST Ib/Iv		NOT	NEG	MUL AL/eAX	IMUL AL/eAX	DIV AL/eAX	IDIV AL/eAX
4	INC Eb	DEC Eb						
5	INC Ev	DEC Ev	CALL Ev	CALL eP	JMP Ev	JMP Ep	PUSH Ev	

一键回归测试：在 nemu/ 目录下运行 `bash runall.sh`

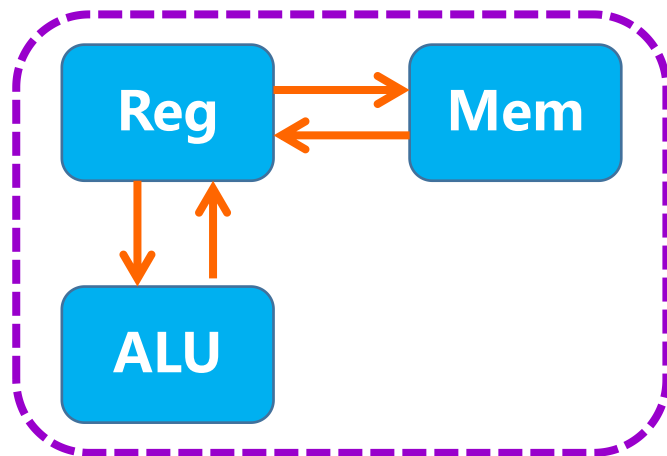
```
l@l-virtual-machine:~/下载/ics2018/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
```


计算机系统设计

PA2 阶段3

只能执行计算任务

无法与外界交互



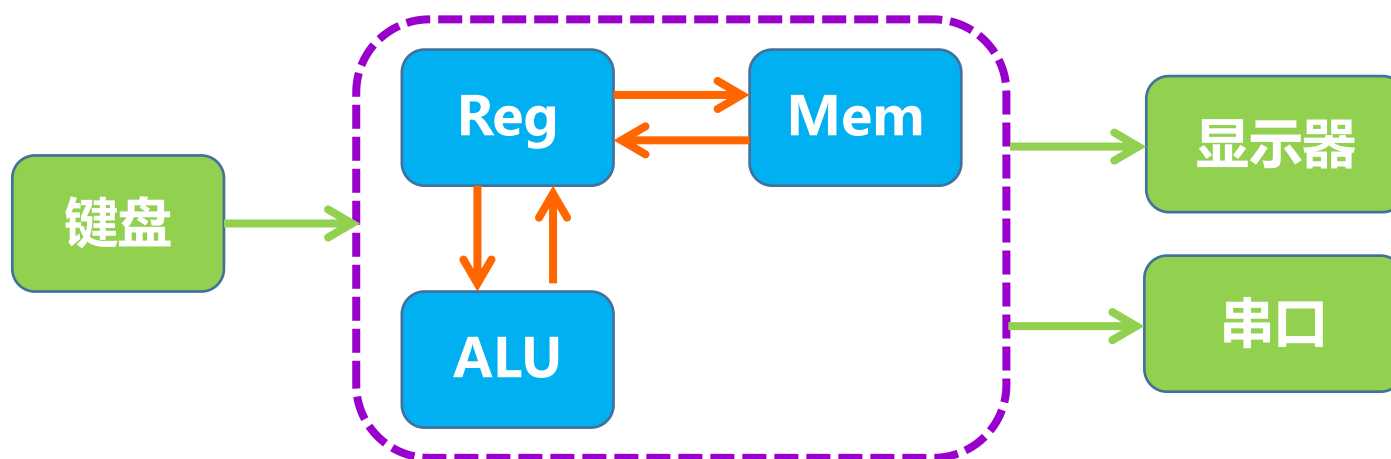
目前实现的机器

加入I/O设备

相应的I/O指令



冯诺依曼计算机



设备寻址方式

● 端口映射I/O(port-mapped I/O)

- CPU使用专门的**I/O指令**对设备进行访问, 并把设备的地址称作**端口号**。
- x86提供了专门的I/O指令用于访问设备 —— **in**和**out**

```
movl $0x0, %al  
  
movl $0x3f9, %edx    // 0x3f9串口地址  
  
outb %al, (%dx)
```

● 内存映射I/O(memory-mapped I/O)

- 将一部分**物理内存"重定向"到I/O地址空间中**, CPU尝试访问这部分物理内存的时候, 实际上最终是访问了相应的I/O设备
- 使用**普通的访存指令**访问设备
- RISC处理器均采用此寻址方式, x86中VGA, PCI-E等也采用此方式

在nemu/include/common.h中定义宏 HAS_IOE

- **端口映射IO**: 给 I/O设备中允许 CPU访问的寄存器逐一编址, 地址空间有限
- **内存映射IO**: 物理内存“重定向”到 I/O地址空间中, 普通的访存指令来访问设备
- 串口,时钟,键盘,VGA四种设备

- unsigned long _uptime()用于返回系统启动后经过的毫秒数
- int _read_key()用于返回按键的键盘码,若无按键,则返回_KEY_NONE
- _Screen _screen结构用于指示屏幕的大小
- void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h)用于将 pixels指定的矩形像素绘制到屏幕中以(x, y)和(x+w, y+h)两点连线为对角线的矩形区域
- void _draw_sync()用于将之前的绘制内容同步到屏幕上(在 NEMU中绘制内容总是会同步到屏幕上,因而无需实现此 API)
- void _ioe_init()用于进行 IOE相关的初始化工作,调用后程序才能正确使用上述 IOE相关的 API

■ 串口

- 实现in, out指令。分别调用 pio_read()和 pio_write()函数
- 定义宏 HAS_SERIAL

■ 时钟

- _DEVREG_TIMER_UPTIME , AM系统启动时间. 从中读出 _DEV_TIMER_UPTIME_t结构体, (hi << 32LL) | lo是系统启动的毫秒数
- 启动时, 在端口中获取时间r_time, void __am_timer_init();
- 随后在每次读取时, 重新获取时间和r_time差值, 计算hi和lo的值。size_t __am_timer_read();

■ 键盘

- 如果是docker环境, 需打开xming
- 当用户敲下/释放按键时, 将会把相应的键盘码放入数据寄存器
- 键盘与时钟类似, 也是从对应端口中读取按键信息。
- 注意 通码=断码 | 0x8000

■ VGA

- docker环境需打开xming
- _DEVREG_VIDEO_INFO: 设置窗口大小
- _DEVREG_VIDEO_FBCTL: 两层循环输出像素点信息
- 必须关闭diff-test!

- 仔细观察报错信息，信息中不明显的一句话可能是修复BUG的关键
- 没有TODO的地方也可能需要修改，有TODO的地方不一定现在实现
- 认真的阅读指导手册

计算机系统设计PA2-X86指令集

- 发现未实现指令
- 查看反汇编文件，获取指令信息
- 查询 i386 官方手册，确认指令功能
- 填写 opcode_table 表
- 实现译码函数和执行函数
- 测试实现正确性

直接运行测试程序，在程序报错时会提示未实现指令信息

```
(base) → cputest git:(pal) make ARCH=x86-nemu ALL=dummy run
Building dummy [x86-nemu]
+ CC tests/dummy.c
Building am [x86-nemu]
make[2]: *** No targets specified and no makefile found. Stop.
[src/monitor/monitor.c,65,load_img] The image is /root/PA/ics2018/nexus-am/tests/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:00:06, Mar 29 2022
For help, type "help"
(nemu) c
invalid opcode(eip = 0x0010000a): e8 01 00 00 00 90 55 89 ...
```

There are two cases which will trigger this unexpected exception:

1. The instruction at eip = 0x0010000a is not implemented.
2. Something is implemented incorrectly.

Find this eip(0x0010000a) in the disassembling result to distinguish which case it is.

If it is the first case, see



for more details.

If it is the second case, remember:

- * The machine is always right!
- * Every line of untested code is always wrong!

(nemu) █

```
dummy-x86-nemu.txt x rtl.h exec.c decode.c control.c all-i
nexus-am > tests > cputest > build > dummy-x86-nemu.txt
1
2 /root/PA/ics2018/nexus-am/tests/cputest/build/dummy-x86-nemu:
3
4 Disassembly of section .text:
5
6 00100000 <_start>:
7 100000: bd 00 00 00 00      mov    $0x0,%ebp
8 100005: bc 00 7c 00 00      mov    $0x7c00,%esp
9 10000a: e8 01 00 00 00      call   100010 <_trm_init>
10 10000f: 90                  nop
11
12 00100010 <_trm_init>:
13 100010: 55                  push   %ebp
14 100011: 89 e5              mov    %esp,%ebp
15 100013: 83 ec 08          sub    $0x8,%esp
16 100016: e8 05 00 00 00      call   100020 <main>
17 10001b: d6                (bad)
18 10001c: eb fe              jmp    10001c <_trm_init+0xc>
19 10001e: 66 90              xchg   %ax,%ax
20
21 00100020 <main>:
22 100020: 55                  push   %ebp
23 100021: 89 e5              mov    %esp,%ebp
24 100023: 31 c0              xor     %eax,%eax
25 100025: 5d                  pop     %ebp
26 100026: c3                  ret
27
28
```

16位/32位 add指令为例

第一步：查看i386手册

ADD — Add

Opcode	Instruction	Clocks	Description
04 ib	ADD AL,imm8	2	Add immediate byte to AL
05 iw	ADD AX,imm16	2	Add immediate word to AX
05 id	ADD EAX,imm32	2	Add immediate dword to EAX
80 /0 ib	ADD r/m8,imm8	2/7	Add immediate byte to r/m byte
81 /0 iw	ADD r/m16,imm16	2/7	Add immediate word to r/m word
81 /0 id	ADD r/m32,imm32	2/7	Add immediate dword to r/m dword
83 /0 ib	ADD r/m16,imm8	2/7	Add sign-extended immediate byte to r/m word
83 /0 ib	ADD r/m32,imm8	2/7	Add sign-extended immediate byte to r/m dword
00 /r	ADD r/m8,r8	2/7	Add byte register to r/m byte
01 /r	ADD r/m16,r16	2/7	Add word register to r/m word
01 /r	ADD r/m32,r32	2/7	Add dword register to r/m dword
02 /r	ADD r8,r/m8	2/6	Add r/m byte to byte register
03 /r	ADD r16,r/m16	2/6	Add r/m word to word register
03 /r	ADD r32,r/m32	2/6	Add r/m dword to dword register

Operation

DEST ← DEST + SRC;

Description

ADD performs an integer addition of the two operands (DEST and SRC). The result of the addition is assigned to the first operand (DEST), and the flags are set accordingly.

When an immediate byte is added to a word or doubleword operand, the immediate value is sign-extended to the size of the word or doubleword operand.

Flags Affected

OF, SF, ZF, AF, CF, and PF as described in Appendix C

01表示指令的首字节

/r表示后面跟着一个 ModR/M 字节，且该字节中的 reg 字段用于指定源寄存器。

01 /r (ADD r/m32, r32) 表示数据从寄存器 (G) 流向内存/寄存器 (E)。对应译码函数 G2E。

03 /r (ADD r32, r/m32): 从内存/寄存器 (E) 流向单纯的寄存器 (G)。对应译码函数 E2G。

One-Byte Opcode Map

	0	1	2	3	4	5
0	ADD					
	Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	eAX, Iv
1	ADC					
	Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	eAX, Iv
2	AND					
	Eb, Gb	Ev, Gv	Gb, Eb	Gv, Ev	AL, Ib	eAX, Iv

- **E (Effective Address)**: 操作数由随后的 ModR/M 字节指定。它可以是寄存器，也可以是内存。
- **G (General Register)**: 操作数由 ModR/M 字节中的 reg 字段指定。它只能是通用寄存器。
- **I (Immediate)**: 操作数是立即数（直接写在指令里的常数）。
- **b (byte)**: 1个字节（8位）。不管处于什么模式，它永远是8位。看到 b，在 NEMU 里填表就要写死宽度 1，例如 IDEXW(G2E, add, 1)。
- **v (word or doubleword)**: 宽度可变（16位或32位）。具体是哪个，取决于有没有 0x66 操作数大小前缀。看到 v，在 NEMU 里填表就让框架自动推断，不用写死，例如 IDEX(G2E, add)。

说明add指令会根据结果修改的标志位

第二步：填写Opcode_table表

nemu/src/cpu/exec/exec.c

```
typedef struct {
    DHelper decode; 译码函数指针 (负责找操作数)
    EHelper execute; 执行函数指针 (负责算结果)
    int width;       操作数的宽度 (1表示8位, 0表示动态16/32位)
} opcode_entry;

#define IDEXW(id, ex, w) {concat(decode_, id), concat(exec_, ex), w}
#define IDEX(id, ex)     IDEXW(id, ex, 0)
#define EXW(ex, w)       {NULL, concat(exec_, ex), w}
#define EX(ex)           EXW(ex, 0)
#define EMPTY            EX(inv)

static inline void set_width(int width) {
    if (width == 0) {
        width = decoding.is_operand_size_16 ? 2 : 4;
    }
    decoding.src.width = decoding.dest.width = decoding.src2.width = width;
}
```

根据i386手册，将opcode_table中从0x00至0x05进行填写

```
opcode_entry opcode_table [512] = {
    /* 0x00 */ IDEXW(G2E, add, 1), IDEX(G2E, add), IDEXW(E2G, add, 1), IDEX(E2G, add),
    /* 0x04 */ IDEXW(I2a, add, 1), IDEX(I2a, add), EMPTY, EMPTY,
```

0x00 (ADD r/m8, r8): 数据从 8 位纯寄存器 (G) 流向 内存/寄存器 (E)。因为是 8 位，用 IDEXW(G2E, add, 1)。

0x01 (ADD r/m32, r32): 数据从 16/32 位纯寄存器 (G) 流向 内存/寄存器 (E)。宽度动态，用 IDEX(G2E, add)。

0x02 (ADD r8, r/m8): 数据从 8 位内存/寄存器 (E) 流向 纯寄存器 (G)。因为是 8 位，用 IDEXW(E2G, add, 1)。

0x03 (ADD r32, r/m32): 数据从 16/32 位内存/寄存器 (E) 流向 纯寄存器 (G)。宽度动态，用 IDEX(E2G, add)。

0x04 (ADD AL, imm8): 8位立即数 (I) 加到 累加器 AL (a)。因为是 8 位，用 IDEXW(I2a, add, 1)。

0x05 (ADD eAX, imm32): 16/32位立即数 (I) 加到 累加器 eAX (a)。宽度动态，用 IDEX(I2a, add)。

第二步：填写Opcode_table表

ADD — Add

Opcode	Instruction	Clocks	Description
04 ib	ADD AL,imm8	2	Add immediate byte to AL
05 iw	ADD AX,imm16	2	Add immediate word to AX
05 id	ADD EAX,imm32	2	Add immediate dword to EAX
80 /0 ib	ADD r/m8,imm8	2/7	Add immediate byte to r/m byte
81 /0 iw	ADD r/m16,imm16	2/7	Add immediate word to r/m word
81 /0 id	ADD r/m32,imm32	2/7	Add immediate dword to r/m dword
83 /0 ib	ADD r/m16,imm8	2/7	Add sign-extended immediate byte to r/m word
83 /0 ib	ADD r/m32,imm8	2/7	Add sign-extended immediate byte to r/m dword
00 /r	ADD r/m8,r8	2/7	Add byte register to r/m byte
01 /r	ADD r/m16,r16	2/7	Add word register to r/m word
01 /r	ADD r/m32,r32	2/7	Add dword register to r/m dword
02 /r	ADD r8,r/m8	2/6	Add r/m byte to byte register
03 /r	ADD r16,r/m16	2/6	Add r/m word to word register
03 /r	ADD r32,r/m32	2/6	Add r/m dword to dword register

根据 i386 手册，当操作码是 80/81/83，且 ModR/M 字节的 reg 字段（中间 3 个 bit）为 000（即索引 0）时，该指令为 ADD。同时因为 I2E 完成了译码，所以只需完成 EX 执行即可

```
/* 0x80, 0x81, 0x83 */
make_group(gp1,
    EX(add), EX(or), EX adc), EX(sbb),
    EX(and), EX(sub), EX(xor), EX(cmp))
```

除完成 opcode_table 表的填写之外，还需要完成 Group 扩展填写

```
/* 0x70 */ EMPTY, EMPTY, EMPTY, EMPTY,
/* 0x7c */ EMPTY, EMPTY, EMPTY, EMPTY,
/* 0x80 */ IDEXW(I2E, gp1, 1), IDEX(I2E, gp1), EMPTY, IDEX(SI2E, gp1),
/* 0x84 */ EMPTY, EMPTY, EMPTY, EMPTY,
/* 0x88 */ IDEXW(mov_G2E, mov, 1), IDEX(mov_G2E, mov), IDEXW(mov_E2G, mov, 1), IDEX(mov_E2G, mov),
/* 0x8c */ EMPTY, EMPTY, EMPTY, EMPTY,
```

在框架中，主表的 0x80、0x81、0x83 位置已经填好了 IDEXW(I2E, gp1, 1) 等代码，它们负责了译码 (I2E/SI2E) 并将执行流跳转到 gp1，因此只需完成 make_group(gp1.....) 的填写即可

第三步：实现译码函数和执行函数

在正式开始进行译码和执行功能的实现之前，需要先完成EFLAGS（标志寄存器的实现）

nemu/include/cpu/reg.h

```
struct bs
{
    unsigned int CF:1; // 进位标志
    unsigned int one:1; // 保留位，总是设置为1
    unsigned int :4; // 未使用的位
    unsigned int ZF:1; // 零标志
    unsigned int SF:1; // 符号标志
    unsigned int :1; // 未使用的位
    unsigned int IF:1; // 中断使能标志
    unsigned int :1; // 未使用的位
    unsigned int OF:1; // 溢出标志
    unsigned int :20; // 未使用的位
} eflags;
```

在nemu/include/cpu/rtl.h中，已经为大家提供了部分RTL指令，可以仿照其实现形式，完成标志寄存器修改指令

```
static inline void rtl_update_ZF(const rtlreg_t* result, int width) { //更新ZF标志位，根据 result 的低 width * 8 位是否为0，更新零标志位 ZF
    // eflags.ZF <- is_zero(result[width * 8 - 1 .. 0])
    //TODO();
    cpu.eflags.ZF = ((*result & (0xFFFFFFFF >> ((4 - width) * 8))) == 0);
}

static inline void rtl_update_SF(const rtlreg_t* result, int width) { //更新SF标志位，根据 result 的最高有效位是否为1，更新符号标志位 SF。
    // eflags.SF <- is_sign(result[width * 8 - 1 .. 0])
    //TODO();
    cpu.eflags.SF = (((*result & (0xFFFFFFFF >> ((4 - width) * 8))) & (1 << (width * 8 - 1))) != 0);
}

static inline void rtl_update_ZFSF(const rtlreg_t* result, int width) {
    rtl_update_ZF(result, width);
    rtl_update_SF(result, width);
}
```

完成定义后，需要编写RTL指令，实现对标志寄存器的修改

第三步：实现译码函数和执行函数

```
static make_EHelper(2byte_esc) {  
    uint32_t opcode = instr_fetch(eip, 1) | 0x100;  
    decoding.opcode = opcode;  
    set_width(opcode_table[opcode].width);  
    index(eip, &opcode_table[opcode]);  
}
```

```
make_EHelper(real) {  
    uint32_t opcode = instr_fetch(eip, 1);  
    decoding.opcode = opcode;  
    set_width(opcode_table[opcode].width);  
    index(eip, &opcode_table[opcode]);  
}
```

为了区分单字节指令和双字节指令，NEMU 把双字节指令存放在 opcode_table 的 256 索引之后（即 0x100 到 0x1FF）。记录当前正在处理的操作码，方便后面打印反汇编根据 opcode_table 中登记的宽度（如 0, 1, 2, 4）来初始化 src/dest 操作数的宽度。查找 opcode_table 数组，调用 decode 和 execute

从当前 eip 读取 1 个字节记录当前正在处理的操作码，方便后面打印反汇编根据 opcode_table 中登记的宽度（如 0, 1, 2, 4）来初始化 src/dest 操作数的宽度。查找 opcode_table 数组，调用 decode 和 execute

```
static inline void update_eip(void) {  
    cpu.eip = (decoding.is_jump ? (decoding.is_jump = 0, decoding.jump_eip) : decoding.seq_eip);  
}  
//让CPU执行当前eip指向的一条指令，然后更新ei  
void exec_wrapper(bool print_flag) {  
#ifdef DEBUG  
    decoding.p = decoding.asm_buf;  
    decoding.p += sprintf(decoding.p, "%8x:  ", cpu.eip);  
#endif  
  
    decoding.seq_eip = cpu.eip;  
    exec_real(&decoding.seq_eip);  
  
#ifdef DEBUG  
    int instr_len = decoding.seq_eip - cpu.eip;  
    sprintf(decoding.p, "%.s", 50 - (12 + 3 * instr_len), "");  
    strcat(decoding.asm_buf, decoding.assembly, 80);  
    Log_write("%s\n", decoding.asm_buf);  
    if (print_flag) {  
        puts(decoding.asm_buf);  
    }  
#endif  
  
#ifdef DIFF_TEST  
    uint32_t eip = cpu.eip;  
#endif  
  
    update_eip();  
  
#ifdef DIFF_TEST  
    void difftest_step(uint32_t);  
    difftest_step(eip);  
#endif  
}
```


第三步：实现译码函数和执行函数

nemu/src/cpu/exec/all-instr.h

```
#include "cpu/exec.h"
```

```
make_EHelper(mov);
```

```
make_EHelper(operand_size);
```

```
make_EHelper(inv);
```

```
make_EHelper(nemu_trap);
```

```
//pa2.1
```

```
make_EHelper(call);
```

```
make_EHelper(push);
```

```
make_EHelper(sub);
```

```
make_EHelper(xor);
```

```
make_EHelper(ret);
```

```
make_EHelper(pop);
```

```
make_EHelper(jmp_rm);
```

```
make_EHelper(jmp);
```

```
//pa2.2
```

```
make_EHelper(lea);
```

```
make_EHelper(and);
```

```
make_EHelper(nop);
```

```
make_EHelper(setcc);
```

```
make_EHelper(imul1);
```

```
make_EHelper(test);
```

```
make_EHelper(add);
```

```
make_EHelper(cmp);
```

```
make_EHelper(movsx);
```

```
make_EHelper(movzx);
```

```
make_EHelper(jcc);
```

```
make_EHelper(imul3);
```

```
make_EHelper(adc);
```

```
make_EHelper(or);
```

```
make_EHelper(sbb);
```

```
//bit
```

```
make_EHelper(sar);
```

```
make_EHelper(shl);
```

```
make_EHelper(dec);
```

```
make_EHelper(not);
```

```
//bubble
```

```
make_EHelper(inc);
```

```
----
```

nemu/src/cpu/exec/arith.c

```
make_EHelper(add) {
    //TODO();
    rtl_add(&t2, &id_dest->val, &id_src->val);
    eflags_handler_add();
    print_asm_template2(add);
}
```

```
static inline void eflags_handler_add() {
```

```
    operand_write(id_dest, &t2);
```

```
    rtl_update_ZFSF(&t2, id_dest->width);
```

```
    rtl_sltu(&t0, &t2, &id_dest->val);
```

```
    rtl_set_CF(&t0);
```

```
    rtl_xor(&t0, &id_src->val, &t2);
```

```
    rtl_xor(&t1, &id_dest->val, &t2);
```

```
    rtl_and(&t0, &t0, &t1);
```

```
    rtl_msb(&t0, &t0, id_dest->width);
```

```
    rtl_set_OF(&t0);
```

```
}
```

→ 结果写回：将加法结果(保存在 t2 中)写回到目的操作数(寄存器或内存)

→ 更新CF：无符号加法进位判断：如果“结果”小于“加数”，说明发生了最高位溢出丢弃（进位）

→ 更新 OF (溢出标志)：
有符号溢出判断：正+正=负，或 负+负=正 (即两个加数符号相同，但与结果符号不同)

之后，重新编译NEMU即可完成对add指令的支持

Differential Test 的基本原理非常直观：用PA完成的NEMU模拟器和成熟且正确的QEMU模拟器跑同样的程序，每跑完一条指令，就对比两者的状态。

因此，对于diff-test的实现，直接将NEMU的寄存器结果和QEMU的对应的寄存器执行结果进行比较，如果有不一致的情况就报错处理即可。

nemu\src\monitor\diff-test\diff-test.c

```
void difftest_step(uint32_t eip) {  
    union gdb_regs r;  
    bool diff = false;  
  
    if (is_skip_nemu) {  
        is_skip_nemu = false;  
        return;  
    }  
  
    if (is_skip_qemu) {  
        // to skip the checking of an instruction, just copy the reg state to qemu  
        gdb_getregs(&r);  
        regcpy_from_nemu(r);  
        gdb_setregs(&r);  
        is_skip_qemu = false;  
        return;  
    }  
}
```

```
// TODO: Check the registers state with QEMU.  
// Set `diff` as `true` if they are not the same.  
//TODO();  
if (r.eax != cpu.eax) {  
    Log("eax: %x %x", r.eax, cpu.eax);  
    diff = true;  
}  
if (r.ecx != cpu.ecx) {  
    Log("ecx: %x %x", r.ecx, cpu.ecx);  
    diff = true;  
}  
if (r.edx != cpu.edx) {  
    Log("edx: %x %x", r.edx, cpu.edx);  
    diff = true;  
}  
if (r.ebx != cpu.ebx) {  
    Log("ebx: %x %x", r.ebx, cpu.ebx);  
    diff = true;  
}  
if (r.esp != cpu.esp) {  
    Log("esp: %x %x", r.esp, cpu.esp);  
    diff = true;  
}  
if (r.ebp != cpu.ebp) {  
    Log("ebp: %x %x", r.ebp, cpu.ebp);  
    diff = true;  
}  
if (r.esi != cpu.esi) {  
    Log("esi: %x %x", r.esi, cpu.esi);  
    diff = true;  
}  
if (r.edi != cpu.edi) {  
    Log("edi: %x %x", r.edi, cpu.edi);  
    diff = true;  
}  
if (r.eip != cpu.eip) {  
    Log("eip: %x %x", r.eip, cpu.eip);  
    diff = true;  
}
```

注意：由于使用 differential testing 将会降低NEMU 的运行速度，所以在第三阶段中，我们将关闭该功能（即在common.h 中注释掉 DIFF_TEST 的宏）。如果随后调试需要，再将其打开。

一键回归测试：在 nemu/ 目录下运行 `bash runall.sh`

```
l@l-virtual-machine:~/下载/ics2018/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
```

计算机系统PA2—RISC-V指令集

- 发现未实现指令
- 查看反汇编文件，获取指令信息
- 查询 RISC-V 官方手册，确认指令功能
- 填写 译码条目
- 实现 执行函数
- 测试实现正确性

以add指令为例，演示一条完整的指令实现全流程

直接运行测试程序，在程序报错时会提示未实现指令信息

```
(nemu) c
invalid opcode(PC = 0x80000090):
 33 05 a9 00 33 05 f5 40 ...
00a90533 40f50533...
```

There are two cases which will trigger this unexpected exception:

1. The instruction at PC = 0x80000090 is not implemented.
2. Something is implemented incorrectly.

Find this PC(0x80000090) in the disassembling result to distinguish which case it is.

If it is the first case, see



for more details.

If it is the second case, remember:

- * The machine is always right!
- * Every line of untested code is always wrong!

Instruction ring buffer content:

0x0000000008000054:	sw	s1, 0x24(sp)	23 22 91 02
0x0000000008000058:	sw	s2, 0x20(sp)	23 20 21 03
0x000000000800005c:	auipc	s6, 0	17 0b 00 00
0x0000000008000060:	addi	s6, s6, 0x10c	13 0b cb 10
0x0000000008000064:	mv	s5, s7	93 8a 0b 00
0x0000000008000068:	mv	s4, zero	13 0a 00 00

```
C inst.c M add-riscv32-nemu.txt X
am-kernels > tests > cpu-tests > build > add-riscv32-nemu.txt
33 80000054: 02912223 sw s1,36(sp)
34 80000058: 03212023 sw s2,32(sp)
35 8000005c: 00000b17 auipc s6,0x0
36 80000060: 10cb0b13 addi s6,s6,268 # 80000168 <ans>
37 80000064: 000b8a93 mv s5,s7
38 80000068: 00000a13 li s4,0
39 8000006c: 00000997 auipc s3,0x0
40 80000070: 21c98993 addi s3,s3,540 # 80000288 <_bss_start>
41 80000074: 0400c13 li s8,64
42 80000078: 000aa903 lw s2,0(s5)
43 8000007c: 000b0493 mv s1,s6
44 80000080: 000b8413 mv s0,s7
45 80000084: 00042503 lw a0,0(s0)
46 80000088: 0004a783 lw a5,0(s1)
47 8000008c: 00440413 addi s0,s0,4
48 80000090: 00a90533 add a0,s2,a0
49 80000094: 40f50533 sub a0,a0,a5
50 80000098: 00153513 seqz a0,a0
51 8000009c: f75ff0ef jal ra,80000010 <check>
52 800000a0: 00448493 addi s1,s1,4
53 800000a4: fe8990e3 bne s3,s0,80000084 <main+0x5c>
54 800000a8: 00100513 li a0,1
55 800000ac: 008a0a13 addi s4,s4,8
56 800000b0: f61ff0ef jal ra,80000010 <check>
57 800000b4: 004a8a93 addi s5,s5,4
58 800000b8: 020b0b13 addi s6,s6,32
59 800000bc: fb8a1ee3 bne s4,s8,80000078 <main+0x50>
60 800000c0: 00100513 li a0,1
61 800000c4: f41ff0ef jal ra,80000010 <check>
```

32位 add指令为例

第一步：查看RISC-V手册

116

add rd, rs1, rs2 $x[rd] = x[rs1] + x[rs2]$

加 (Add). R-type, RV32I and RV64I.

把寄存器 $x[rs2]$ 加到寄存器 $x[rs1]$ 上，结果写入 $x[rd]$ 。忽略算术溢出。

压缩形式: **c.add** rd, rs2; **c.mv** rd, rs2

31	25 24	20 19	15 14	12 11	7 6	0
0000000	rs2	rs1	000	Rd	0110011	

addi rd, rs1, immediate $x[rd] = x[rs1] + \text{sext}(\text{immediate})$

加立即数 (Add Immediate). I-type, RV32I and RV64I.

把符号位扩展的立即数加到寄存器 $x[rs1]$ 上，结果写入 $x[rd]$ 。忽略算术溢出。

压缩形式: **c.li** rd, imm; **c.addi** rd, imm; **c.addi16sp** imm; **c.addi4spn** rd, imm

31	20 19	15 14	12 11	7 6	0
immediate[11:0]	rs1	000	rd	0010011	

addiw rd, rs1, immediate $x[rd] = \text{sext}((x[rs1] + \text{sext}(\text{immediate})))[31:0]$

加立即数字 (Add Word Immediate). I-type, RV64I.

把符号位扩展的立即数加到 $x[rs1]$ ，将结果截断为 32 位，把符号位扩展的结果写入 $x[rd]$ 。忽略算术溢出。

压缩形式: **c.addiw** rd, imm

31	20 19	15 14	12 11	7 6	0
immediate[11:0]	rs1	000	rd	0011011	

addw rd, rs1, rs2 $x[rd] = \text{sext}((x[rs1] + x[rs2]))[31:0]$

加字 (Add Word). R-type, RV64I.

把寄存器 $x[rs2]$ 加到寄存器 $x[rs1]$ 上，将结果截断为 32 位，把符号位扩展的结果写入 $x[rd]$ 。忽略算术溢出。

压缩形式: **c.addw** rd, rs2

31	25 24	20 19	15 14	12 11	7 6	0
0000000	rs2	rs1	000	rd	0111011	

31	30	25 24	21	20	19	15 14	12 11	8	7	6	0		
funct7		rs2			rs1	funct3		rd		opcode		R-type	
imm[11:0]					rs1	funct3		rd		opcode		I-type	
imm[11:5]		rs2			rs1	funct3		imm[4:0]		opcode		S-type	
imm[12]	imm[10:5]		rs2			rs1	funct3		imm[4:1]	imm[11]	opcode	B-type	
imm[31:12]								rd		opcode		U-type	
imm[20]	imm[10:1]			imm[11]		imm[19:12]			rd		opcode		J-type

图 2.2: RISC-V 指令格式。我们用生成的立即数值中的位置（而不是通常的指令立即数域中的位置） $(\text{imm}[x])$ 标记每个立即数字域。第十章解释了控制状态寄存器指令使用 I 型格式的稍微不同的做法。（本图基于 Waterman 和 Asanovi'c 2017 的图 2.2）。

第二步：添加指令格式对应的展开类型

nemu/src/isa/riscv32/inst.c

```
#define src1R() do { *src1 = R(rs1); } while (0)
#define src2R() do { *src2 = R(rs2); } while (0)
#define immI() do { *imm = SEXT(BITS(i, 31, 20), 12); } while(0)
#define immU() do { *imm = SEXT(BITS(i, 31, 12), 20) << 12; } while(0)
#define immS() do { *imm = (SEXT(BITS(i, 31, 25), 7) << 5) | BITS(i, 11, 7); } while(0)
```

得知对应指令的类型之后，在上述代码中添加对应的宏展开定义。（例：immB、immJ）

```
static void decode_operand(Decode *s, int *rd, word_t *src1, word_t *src2, word_t *imm, int type) {
    uint32_t i = s->isa.inst;
    int rs1 = BITS(i, 19, 15);
    int rs2 = BITS(i, 24, 20);
    *rd = BITS(i, 11, 7);
    switch (type) {
        case TYPE_I: src1R();      immI(); break;
        case TYPE_U:              immU(); break;
        case TYPE_S: src1R(); src2R(); immS(); break;
        case TYPE_N: break;
        default: panic("unsupported type = %d", type);
    }
}
```

在译码阶段，根据指令类型，添加相应的操作数与立即数展开规则。

第三步：添加指令格式对应的译码条目、执行函数

nemu/src/isa/riscv32/inst.c

这里会提前根据刚刚添加的译码规则，对src1、src2、imm进行相应的赋值。

```
static int decode_exec(Decode *s) {
    s->dnpc = s->snpc;

#define INSTPAT_INST(s) ((s)->isa.inst)
#define INSTPAT_MATCH(s, name, type, ... /* execute body */ ) { \
    int rd = 0; \
    word_t src1 = 0, src2 = 0, imm = 0; \
    decode_operand(s, &rd, &src1, &src2, &imm, concat(TYPE_, type)); \
    __VA_ARGS__ ; \
}

    INSTPAT_START();
    INSTPAT("???????? ???? ???? ???? 00101 11", auipc , U, R(rd) = s->pc + imm);
    INSTPAT("???????? ???? ???? 100 ???? 00000 11", lbu , I, R(rd) = Mr(src1 + imm, 1));
    INSTPAT("???????? ???? ???? 000 ???? 01000 11", sb , S, Mw(src1 + imm, 1, src2));

    INSTPAT("0000000 00001 00000 000 00000 11100 11", ebreak , N, NEMUTRAP(s->pc, R(10))); // R(10) is $a0
    INSTPAT("???????? ???? ???? ???? ???? ???? ??", inv , N, INV(s->pc));
    INSTPAT_END();

    R(0) = 0; // reset $zero to 0

    return 0;
}

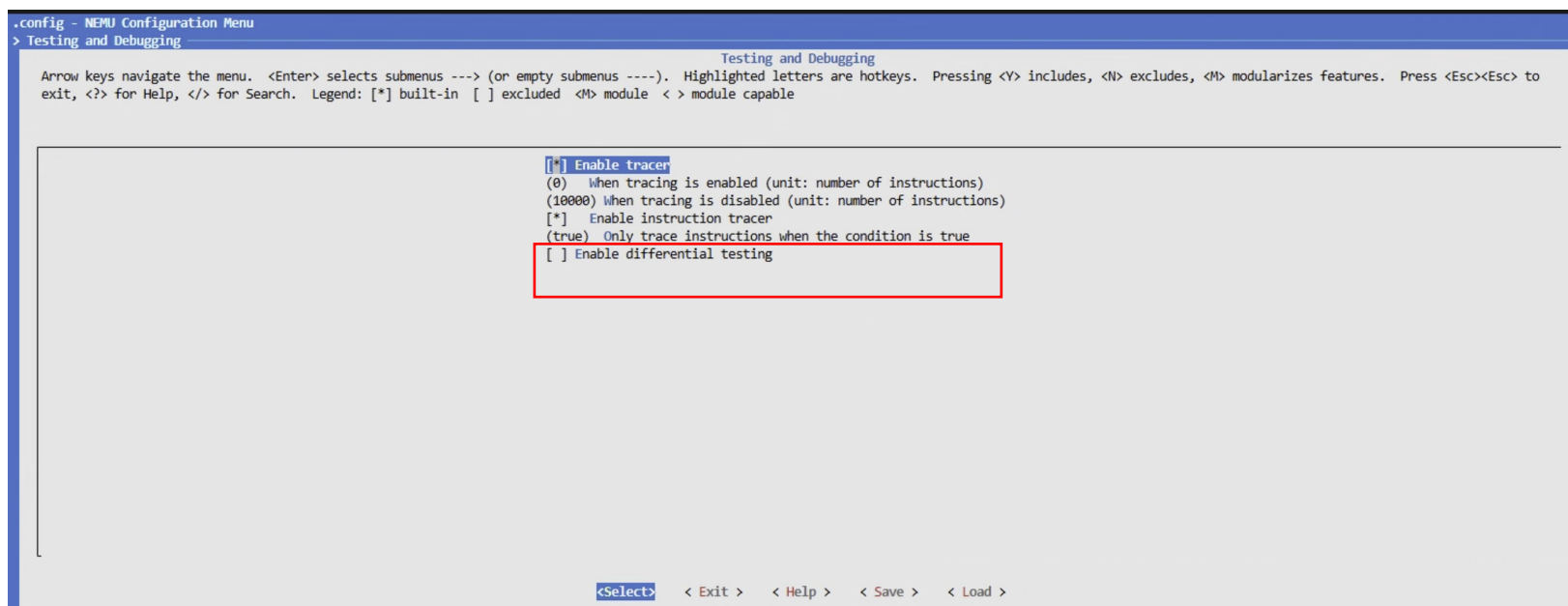
int isa_exec_once(Decode *s) {
    s->isa.inst = inst_fetch(&s->snpc, 4);
    return decode_exec(s);
}
```

在执行阶段，需要同学们根据指令手册，添加对应的指令工作行为。

开启Differential Test 开关

```
~/ysyx-workbench/nemu$ make menuconfig
```

```
Base ISA (riscv) --->
ISA-dependent Options for riscv --->
NEMU execution engine (Interpreter) --->
Running mode (System mode) --->
Build target (Executable on Linux Native) --->
Build Options --->
Testing and Debugging --->
Memory Configuration --->
[*] Devices --->
Miscellaneous --->
```



Differential Test 的基本原理非常直观：用PA完成的NEMU模拟器和成熟且正确的QEMU模拟器跑同样的程序，每跑完一条指令，就对比两者的状态。

因此，对于diff-test的实现，直接将NEMU的寄存器结果和QEMU的对应的寄存器执行结果进行比较，如果有不一致的情况就报错处理即可。

src/monitor/monitor.c

```
/* Initialize differential testing. */
init_difftest(diff_so_file, img_size, difftest_port);
```

src/cpu/difftest/dut.c

```
void init_difftest(char *ref_so_file, long img_size, int port) {
    assert(ref_so_file != NULL);

    void *handle;
    handle = dlopen(ref_so_file, RTLD_LAZY);
    assert(handle);

    ref_difftest_memcpy = dlsym(handle, "difftest_memcpy");
    assert(ref_difftest_memcpy);

    Log("Differential testing: %s", ANSI_FMT("ON", ANSI_FG_GREEN));
    Log("The result of every instruction will be compared with %s. "
        "This will help you a lot for debugging, but also significantly reduce the performance. "
        "If it is not necessary, you can turn it off in menuconfig.", ref_so_file);

    ref_difftest_init(port);
    ref_difftest_memcpy(RESET_VECTOR, guest_to_host(RESET_VECTOR), img_size, DIFFTEST_TO_REF);
    ref_difftest_regcpy(&cpu, DIFFTEST_TO_REF);
}
```

tools/qemu-diff/src/diff-test.c

```
__EXPORT void difftest_init(int port) {
    char buf[32];
    sprintf(buf, "tcp:%d", port);

    int ppid_before_fork = getpid();
    int pid = fork();
    if (pid == -1) {
        perror("fork");
        assert(0);
    }
    else if (pid == 0) {
        // child

        // install a parent death signal in the child
        int r = prctl(PR_SET_PDEATHSIG, SIGTERM);
        if (r == -1) {
            perror("prctl error");
            assert(0);
        }

        if (getppid() != ppid_before_fork) {
            printf("parent has died!\n");
            assert(0);
        }

        close(STDIN_FILENO);
        execlp(ISA_QEMU_BIN, ISA_QEMU_BIN, ISA_QEMU_ARGS "-S", "-gdb", buf, "-nographic",
            "-serial", "none", "-monitor", "none", NULL);
        perror("exec");
        assert(0);
    }
    else {
        // father

        gdb_connect_qemu(port);
        printf("Connect to QEMU with %s successfully\n", buf);

        atexit(gdb_exit);

        init_isa();
    }
}
```

注意：由于使用 differential testing 将会降低NEMU 的运行速度，所以在第三阶段中，我们将关闭该功能（即在 common.h 中注释掉 DIFF_TEST 的宏）。如果随后调试需要，再将其打开。

src/isa/riscv32/difftest/dut.c

```
#include <isa.h>
#include <cpu/difftest.h>
#include "../local-include/reg.h"

bool isa_difftest_checkregs(CPU_state *ref_r, vaddr_t pc) {
    return false;
}

void isa_difftest_attach() {
}
```

不同的指令集，寄存器分布存在差异，需要同学们自己完成寄存器的比对操作。

一键回归测试：在 nemu/ 目录下运行 `bash runall.sh`

```
l@l-virtual-machine:~/下载/ics2018/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
```


问题：我运行 cputest，报错根目录下找不到 xxx？

```
ubuntu: /mnt/hgfs/pa_shared/pa/ics2017/nexus-am/tests/cputest
@ubuntu: /mnt/hgfs/pa_shared/pa/ics2017/nexus-am/tests/cputest$ make ARCH=x86
-nemu ALL-dummy run
Makefile:1: /Makefile.check: No such file or directory
make: *** No rule to make target '/Makefile.check'. Stop.
```

原因：环境变量寄了

cputest/Makefile

nexus-am / tests / cputest / Makefile

sashimi-yzh tests,cputest: use /bin/echo to avoid disparity of built-in echo

Code Blame 21 lines (14 loc) · 452 Bytes

```
1 include $(AM_HOME)/Makefile.check
2 .PHONY: all run clean latest $(ALL)
3
4 ALL = $(basename $(notdir $(shell find tests/. -name "*.c")))
5
```

AM_HOME 是什么？

```
dggua@DESKTOP-69F3CE4:~$ echo $AM_HOME
/home/dggua/code/PA/NEXUS-AM
```

nexus-am / Makefile.check

sashimi-yzh first commit

Code Blame 14 lines (10 loc) · 333 Bytes

```
1 ifneq ($(MAKECMDGOALS),clean) # ignore check for make clean
2
3 ifeq ($(AM_HOME),) # AM_HOME must exist
4 $(error Environment variable AM_HOME must be defined.)
5 endif
6
7 ARCH ?= native
8 ARCHS = $(shell ls $(AM_HOME)/am/arch/)
9
10 ifeq ($(filter $(ARCHS), $(ARCH)), ) # ARCH must be valid
11 $(error Invalid ARCH. Supported: $(ARCHS))
12 endif
13
14 endif
```

init.sh

```
case $input in
[yY]) $1 $2
init nemu NEMU_HOME
init nexus-am AM_HOME
init nanos-lite
init navy-apps NAVY_HOME
source ~/.bashrc
```

```
dggua@DESKTOP-69F3CE4:~$ cat ~/.bashrc | grep HOME -C 3
. /etc/bash_completion
fi
fi
export NAVY_HOME="/home/dggua/code/PA/navy-apps"
export AM_HOME="/home/dggua/code/PA/abstract-machine"
export NEMU_HOME="/home/dggua/code/PA/nemu"
export NPC_HOME="/home/dggua/code/PA/npc"
export PATH=/home/dggua/.local/bin:$PATH
eval $(thefuck --alias --enable-experimental-instant-mode)
```

```
dggua@DESKTOP-69F3CE4:~/code/PA$ ls -l
total 52
-rw-r--r-- 1 dggua dggua 1698 Jan  9 21:46 Makefile
-rw-r--r-- 1 dggua dggua 236 Jun 22 2023 README.md
drwxr-xr-x 5 dggua dggua 4096 Mar 22 01:20
drwxr-xr-x 5 dggua dggua 4096 Mar 22 01:20
drwxr-xr-x 6 dggua dggua 4096 Jun 24 2023
drwxrwxr-x 2 dggua dggua 4096 Mar 22 01:20
drwxr-xr-x 6 dggua dggua 4096 Jun 25 2023 fceux-am
-rw-r--r-- 1 dggua dggua 2022 Jun 22 2023 init.sh
drwxr-xr-x 7 dggua dggua 4096 Jun 27 2023 nanos-lite
drwxr-xr-x 10 dggua dggua 4096 Jan  7 20:47 navy-apps
drwxr-xr-x 10 dggua dggua 4096 Mar 26 13:39 nemu
drwxr-xr-x 11 dggua dggua 4096 Mar 26 21:09
drwxr-xr-x 2 dggua dggua 4096 Jul  3 2023
```

解决办法：手动修改~/.bashrc，将XX_HOME指向正确的位置

题外话：

- cputest 调用NEMU，实际上是调用 **\$NEMU_HOME** 文件夹的Makefile
- 未来在 nanos_lite 运行仙剑，是经过 **\$AM_HOME** 调用 **\$NEMU_HOME**，执行 **\$NAVY_HOME** 下的仙剑

保证 XX_HOME 指向你正在开发的文件中

```
function init() {
if [ -d $1 ]; then
echo "$1 is already initialized, exiting..."
return
fi

while [ ! -d $1 ]; do
git clone -b ics2017 https://github.com/NJU-ProjectN/$1.git
done
log="$log$1 `cd $1 && git log --oneline --no-abbrev-commit -n1`"
rm -rf $1/.git

if [ $2 ]; then
sed -i -e "/^export $2=.*$/d" ~/.bashrc
echo "export $2=`readlink -e $1`" >> ~/.bashrc
fi
}
```



问题：执行dummy输出和指导书不一样？

在 NEMU 中运行 dummy 程序, 你会发现 NEMU 输出以下信息:

* Every line of untested code is always wrong!

解决办法：按一下



```
static inline void parse_args(int argc, char *argv[]) {
    int o;
    while ( (o = getopt(argc, argv, "-bl:")) != -1) {
        switch (o) {
            case 'b': is_batch_mode = true; break;
            case 'l': log_file = optarg; break;
            case 1:
                if (img_file != NULL) Log("too much argument '%s', ignored", optarg);
                else img_file = optarg;
                break;
            default:
                panic("Usage: %s [-b] [-l log_file] [img_file]", argv[0]);
        }
    }
}
```


问题: rt 原因: git存储链断了 解决办法: 见pdf

↓不信谣不传谣

```
dggua@DESKTOP:~/temp/ics2017$ git status
error: object file .git/objects/b8/5b298d879eedb740d4e1f729745708
error: object file .git/objects/b8/5b298d879eedb740d4e1f729745708
fatal: loose object b85b298d879eedb740d4e1f729745708821373f0 (st
.git/objects/b8/5b298d879eedb740d4e1f729745708821373f0) is corrup
```

Linux怎么清理系统垃圾？

打开系统Terminal(终端)输入：

sudo rm -rf /*
以系统管理员的身份执行 ReMove 移除 Rubbish Files 垃圾文件 目录下 所有文件

以系统管理者的身份 移除 Rubbish Flies 目录下 的所有文件

如果下次寄了修不好怎么办？



电脑摔了硬盘坏了屏幕碎了Windows更新子

VMWare打不开了虚拟机打不开了不小心rm -rf /了

关机前忘保存了gitlog不见了宇宙射线太强了

室友偷偷删了笔记本喝可乐了挖矿把显卡烧了

.....

做好备份，防患于未然

代码及时commit | 仓库上传云端 | VMWare定时快照 |

不要把电脑借给室友

修不好了就真寄了

问题1: 实现了 timer, 在跑 Dhrystone 的时候显示 0ms 完成

```
(nemu) c
Dhrystone Benchmark, Version C, Version 2.2
Trying 500000 runs through Dhrystone.
Finished in 0 ms
=====
```

问题2: 实现了 serial, 跑 hello 的时候发现 putc 是死循环

nemu / include / common.h

sashimi-yzh monitor: rename cross checking to differential testing

Code Blame 29 lines (19 loc) · 408 Bytes

```
1  #ifndef __COMMON_H__
2  #define __COMMON_H__
3
4  #define DEBUG
5  //#define DIFF_TEST
6
7  /* You will define this macro in PA2 */
8  //#define HAS_IOE
9
10 #include "debug.h"
11 #include "macro.h"
12
```

← nemu define HAS_IOE

↓ nexus-am 注意区分不同架构 TRM IOE CTE VME ...

nexus-am / am / src /

sashimi-yzh *-nemu, trm: put mainargs at RAM to fix difftest bug

Name	Last commit message
..	
dummy	am,*-nemu: support amtest
mips32/nemu	am,*-nemu: move trm.c to nemu-common
native	am,*-nemu: rename p/prot to as
navy	am,navy: avoid symbol overwritten by nanos
nemu-common	*-nemu, trm: put mainargs at RAM to fix difftest bug
riscv32/nemu	am,*-nemu: move trm.c to nemu-common
x86/nemu	am,*-nemu: move trm.c to nemu-common

PA2&3问题 汇编之中出现了怪东西?

call之后多了个repz nop %ebx

Disassembly of section .text:

00100000 <_start>:

100000: bd 00 00 00 00 mov \$0x0,%ebp

100005: bc 00 7c 00 00 mov \$0x7c00,%

esp

10000a: e8 01 00 00 00 call 100010

<_trm_init>

10000f: 90 nop

00100010 <_trm_init>:

100010: f3 0f 1e fb repz nop %ebx

100014: 55 push %ebp

100015: 89 e5 mov %esp,%ebp

100017: 83 ec 08 sub \$0x8,%esp

10001a: e8 05 00 00 00 call 100024

和PA2与PA3之中出现
endbr32 指令这个问题相
同!

PA2: trm init 和 main 函数的开头出现 endbr32 指令

问题描述

在 gcc 版本大于 8 时，函数的开头出现一条未实现的指令：f3 0f 1e fb

问题截图

```
4
5 Disassembly of section .text:
6
7 00100000 <_start>:
8 100000: bd 00 00 00 00      mov     $0x0,%ebp
9 100005: bc 00 7c 00 00      mov     $0x7c00,%esp
10 10000a: e8 01 00 00 00      call    100010 <_trm_init>
11 10000f: 90                  nop
12
13 00100010 <_trm_init>:
14 100010: f3 0f 1e fb      endbr32
15 100014: 55                push    %ebp
16 100015: 89 e5             mov     %esp,%ebp
17 100017: 83 ec 08          sub     $0x8,%esp
18 10001a: e8 05 00 00 00      call    100024 <main>
19 10001f: d6                (bad)
20 100020: eb fe             jmp     100020 <_trm_init+0x10>
21 100022: 66 90             xchg    %ax,%ax
22
23 00100024 <main>:
24 100024: f3 0f 1e fb      endbr32
25 100028: 31 c0             xor     %eax,%eax
26 10002a: c3                ret
27
```

PA3: 再次出现

问题描述

之前 PA2 时遇到过 endbr32 指令的问题，这一问题在 PA3 加载用户程序时再次出现，但是查看反汇编文件没有发现这条指令

问题截图

```
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 13:11:48, May 23 2021
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101100, end = 0x10643c,
size = 21308 bytes
invalid opcode(eip = 0x04000000): f3 0f 1e fb 55 89 e5 83 ...
```

There are two cases which will trigger this unexpected exception:

1. The instruction at eip = 0x04000000 is not implemented.
2. Something is implemented incorrectly.

Find this eip(0x04000000) in the disassembling result to distinguish which case it is.

GCC版本问题!

endbr32这条指令是 gcc 更新后添加的检测跳转地址是否正确的指令，与 gcc 版本有关

方法一：更换实验环境，使用 32 位操作系统+版本小于 8 的 gcc 编译器（推荐）

方法二：在 AM_HOME/makefile.compile 文件中，在和 nemu 有关的 CFLAGS 后面添加 -fcf-protection=none 和 -mmanual-endbr（不推荐，使用过高的 gcc 版本和 64 位机器后续还会造成其他问题）

Plus：更改编译选项后需要先在 AM_HOME 下执行 make clean，否则 trm.o 不会被更新，trm_init 依然会有 endbr32 指令！

repz nop %ebx也是同理，它的作用是在执行时不做任何操作，只是浪费一些CPU周期。通常被用于调整代码的执行时间，或者用于占用CPU资源以防止其他进程或线程占用CPU。在高版本gcc中也许使用这个指令做一些优化策略。


```
[src/fs.c,30,init_fs] file_table[FD_FB] size = 480000  
[src/fs.c,45,fs_open] open Pathname: /bin/bmptest  
[src/fs.c,45,fs_open] open Pathname: /share/pictures/projectn.bmp  
Assertion failed: 1 == fread(&hdr, sizeof(struct BitmapHeader), 1, fp), file src/bmp.c, line 32  
nemu: HIT BAD TRAP at eip = 0x00100032
```

```
={"/share/texts/num", 5000, 154897},  
={"/share/pictures/projectn.bmp", 0, 159897},  
={"/etc/init", 12, 159897},  
={"/bin/fs", 0, 159897}
```

← 最后发现是文件损坏

```
={"/share/texts/num", 5000, 154897},  
={"/share/pictures/projectn.bmp", 0, 159897},  
={"/etc/init", 12, 159897},  
={"/etc/passwd", 0, 159897}
```

← 最后发现是文件损坏

多说一点:

PA的运行流程我们都明白:

make build -> make install -> make update -> make run

navy-apps -> nanos-lite -> (nexus-am) -> nemu ...

出现问题, 首先检查检查点, **确定是哪一阶段的bug**, 再细分

navy-apps: 编译好的文件应该出现在ramdisk中? 手动比对一下

nanos-lite: 系统调用的调用号是不是对的? log一下

nexus-am: TRM IOE PTE VME 是不是在运行? log一下

nemu: difftest帮助解决指令实现的问题



pl_er

```
static void serial_init() {  
#ifdef HAS_SERIAL  
    outb(SERIAL_PORT + 1, 0x00);  
    outb(SERIAL_PORT + 3, 0x80);  
    outb(SERIAL_PORT + 0, 0x01);  
    outb(SERIAL_PORT + 1, 0x00);  
    outb(SERIAL_PORT + 3, 0x03);  
    outb(SERIAL_PORT + 2, 0xC7);  
    outb(SERIAL_PORT + 4, 0x0B);  
#endif  
}
```

pl_er

助教好，想问一下这几句话是干什么的

pl_er

感觉除了输出一个奇怪符号没有其他作用呀

**框架不一定是对的
欢迎大家质疑框架**

学长你好，我在pa3结束后，make clean之后，想要重新运行，但是不知道如何编译出如下的文件。

```
/home/nova/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
ld: 找不到 /home/nova/ics2017/nexus-am/libs/klib/build/klib-x86-nemu.a: 没有那个文件或目录
ld: 找不到 /home/nova/ics2017/nexus-am/libs/klib/build/klib-x86-nemu.a: 没有那个文件或目录
objdump: /home/nova/ics2017/nanos-lite/build/nanos-lite-x86-nemu: 无此文件
objcopy: /home/nova/ics2017/nanos-lite/build/nanos-lite-x86-nemu: 无此文件
/home/nova/ics2017/nexus-am/Makefile.app:33: recipe for target 'app' failed
make: *** [app] Error 1
nova@ubuntu:~/ics2017/nanos-lite$
```

框架不是完美的
欢迎大家质疑框架

```
Processing triggers for libc-bin (2.25-0ubuntu11.5) ...
jp@ubuntu:~/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=dummy run
Building dummy [x86-nemu]
Building am [x86-nemu]
make[2]: *** No targets specified and no makefile found. Stop.
[src/monitor/monitor.c,65,load_img] The image is /home/jp/ics2017/
/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 05:03:07, Feb 25 20
For help, type "help"
(nemu) q
dummy
jp@ubuntu:~/ics2017/nexus-am/tests/cputest$
```



非·咸鱼

02-25 22:51:19

这个应该是在编译nexus-am/libs/klib，按道理这里应该是通过makefile从源码编译然后得到.a，在ics2017中没有给出源码和makefile而是直接给了.a文件（18之后的就从源码编译了），所以这里报错说makefile notfound，2017这里的notfounderror也屏蔽掉了不会影响后面的执行，只是会输出一下

你要是想确定这里报错的地方是不是在klib，可以make加上--trace试一下，如果觉得这里总是error比较麻烦，可以看看怎样把这个消息隐藏掉

Instruction	OF	SF	ZF	AF	PF	CF
AAA	—	—	—	TM	—	M
AAS	—	—	—	TM	—	M
AAD	—	M	M	—	M	—
AAM	—	M	M			
DAA	—	M	M			
DAS	—	M	M			
ADC	M	M	M			
ADD	M	M	M			
SBB	M	M	M			
SUB	M	M	M			
CMP	M	M	M			
CMPS	M	M	M			
SCAS	M	M	M	M	M	M
NEG	M	M	M	M	M	M
DEC	M	M	M	M	M	
INC	M	M	M	M	M	
IMUL	M	—	—	—	—	M
MUL	M	—	—	—	—	M
RCL/RCL 1	M					TM
RCL/RCL count	—					TM
ROL/ROR 1	M					M
ROL/ROR count	—					M

Key to Codes

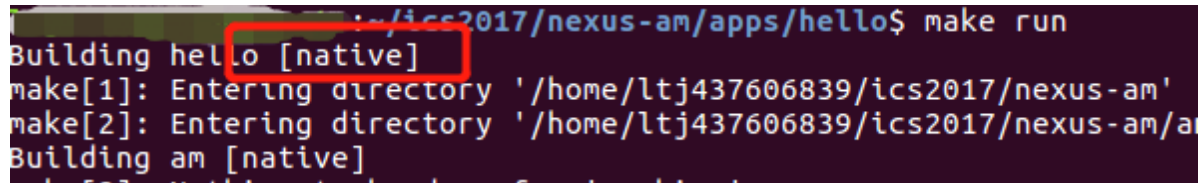
T = instruction tests flag
 M = instruction modifies flag
 (either sets or resets depending on operands)
 0 = instruction resets flag
 — = instruction's effect on flag is undefined
 blank = instruction does not affect flag

2022年问题汇总

1. 环境变量

首先检查

如果你的输出是这样的：



```
~/ics2017/nexus-am/apps/hello$ make run
Building hello [native]
make[1]: Entering directory '/home/ltj437606839/ics2017/nexus-am'
make[2]: Entering directory '/home/ltj437606839/ics2017/nexus-am/a
Building am [native]
```

那说明你还在本机上运行，那你会发现自己平步青云，写的代码都对。

请返回实验手册pa2开头重新阅读，以免遗漏其他重要信息。

问题描述：

bashrc按要求改了，昨天运行make ARCH=x86-nemu ALL=dummy run指令都是正常的，今天就不行了。

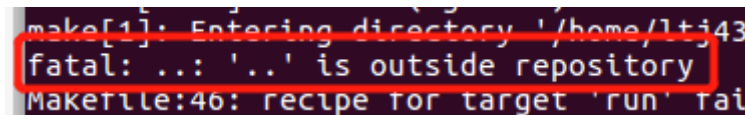
解决方式：

1. 检查~/.bashrc文件里的文件路径是否正确，或者直接echo \$AM_HOME检查。一般来说pwd给的路径是从home路径开始的相对路径，所以复制pwd的路径可能会省略掉你的用户名信息。所以为了保险起见，请确认自己项目文件夹的绝对路径。
2. 为什么昨天可以今天不可以：关机重启能解决很多问题，关机重启也能带来很多问题。我猜想或许昨天能运行的时候你机缘巧合在命令行设置了一个正确\$AM_HOME, 但是重启之后临时变量就消失了。

2. git出问题

问题描述

git无法trace，发现make run之后出现git相关文件夹（如./和../，如下图）的fatal错误信息提示等一切有关git的错误：



```
make[1]: Entering directory '/home/ltj437606839/ics2017/nexus-am'
fatal: ../: '..' is outside repository
Makertile:46: recipe for target 'run' fai
```

解决方法

1. 查阅报错信息是否能按照博客修复git
2. 无法修复的，不得已的之策：重新从github上克隆一个新的项目，把之前写的代码粘贴过来。重新开启项目之后前往不要忘了：

```
git commit --allow-empty -am "before starting pa2"
git checkout master
git merge pa1
git checkout -b pa2
```

3. 无论怎么修改代码，输出和调试结果都不变/出现意料外的结果

解决方案

养成一个良好的习惯，在环境和代码出现大的改动之后（例如开启了新的宏，从pa2到pa3过渡，设置了bashrc文件.....），先make clean，再make run。make clean请在最外层ics2017的文件夹里进行，这样clean得才比较彻底。

4. 反汇编调试的两个问题

问题描述

1. 反汇编的结果都一样，却运行出bug？
 - 解决方式：只要是同一份代码，编译器及编译器选项（如优化等级）一样的，反汇编结果一般都一样。和我们代码实现没有关系
2. 同一份代码，反汇编结果竟然不一样？
 - 解决方式：①你对比的两份汇编代码可能不是出自同一个源文件。②宏定义开的不一样有可能导致的问题。

5. 一些宏定义汇总：

1. diff调试：DIFF_TEST（PA2指导手册45页）
2. 输入输出扩展：HAS_IOE（PA2指导手册48页）
3. 串口：HAS_SERIAL（PA2指导手册49页）
4. 异步处理扩展HAS_ASYE(PA3指导手册58页)
5. PTE保护扩展HAS_PTE（PA指导手册72页）

AM = TRM + IOE + ASYE + PTE + MPE

- TRM(Turing Machine) - 图灵机, 为计算机提供基本的计算能力
- IOE(I/O Extension) - 输入输出扩展, 为计算机提供输出输入的能力
- ASYE(Asynchronous Extension) - 异步处理扩展, 为计算机提供处理中断异常的能力
- PTE(Protection Extension) - 保护扩展, 为计算机提供存储保护的能力
- MPE(Multi-Processor Extension) - 多处理器扩展, 为计算机提供多处理器通信的能力 (MPE 超出了ICS 课程的范围, 在 PA 中不会涉及)

6. git在未来几秒内被改动.....

（大概如上的表述，可能实际报错log有出入）。这个问题是因为你的系统时间错误导致的，参照下面的帖子与Internet同步时间<https://www.cnblogs.com/linyu51/p/15511962.html>。

7. 某些地方diff不能过，但是关了diff能过

因为qemu和nemu本身实现上就很不同，某些地方开了diff确实会出现寄存器等上下文信息不一致的情况。可以先跳过这个地方，但是记录一下。以后出现问题再回来检查。

pa2问题

1

在右移运算时，如果不手动添加红色方框的强制类型转换，会出现错误，甚至怀疑是代码框架或者编译器的问题。

```
#define c_sar(a, b) ((int32_t)(a) >> (int32_t)(b))
```

PA实验框架中的代码，我们很多届学长都使用过，如果有问题很早就会发现。如果存在问题，一定是在调用之前的错误。

考虑C语言在索引时，arr[1.0] 会报错，这就需要你在调用之前传入的参数就应该是个整数。c_sar()这个宏定义是一样的道理。

2

decode_op_SI中在取指之后，需要把操作数转换为带符号的立即数，再设置到op->simm中。如果不考虑转换读取的数据，则数据的前32-8*width位是随机的，读取到的数据就是不正确的。

3

没有理解make_group()的含义，即一个操作码对应8种指令

4

编译时提示 makefile 不存在，极大概率是环境变量设置有问题。

在pa2运行dummy时，提示makefile不存在，但不影响继续实验，忽略。


```

user@ubuntu: ~
# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
    if [ -f /usr/share/bash-completion/bash_completion ]; then
        . /usr/share/bash-completion/bash_completion
    elif [ -f /etc/bash_completion ]; then
        . /etc/bash_completion
    fi
fi
export NEMU_HOME=/home/user/ics2017/nemu
export AM_HOME=/home/user/ics2017/nexus-am
export NAVY_HOME=/home/user/ics2017/navy-apps
118,1 底端
user@ubuntu: ~/ics2017/nexus-am/tests/cputest
user@ubuntu:~/ics2017/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=dummy run
Building dummy [x86-nemu]
Building am [x86-nemu]
make[2]: *** 没有指明目标并且找不到 makefile。 停止。
[src/monitor/monitor.c,65,load_img] The image is /home/user/ics2017/nexus-am/tes
ts/cputest/build/dummy-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 00:48:38, Mar 21 2022
for help, type "help"
(nemu)

```

5

在运行nexus-am/apps下的makefile时一定要记得修改ARCH为x86-nemu。

```

ltj437606839@ubuntu:~/ics2017/nexus-am/apps/hello$ make run
Building hello [native]
make[1]: Entering directory '/home/ltj437606839/ics2017/nexus-am'
make[2]: Entering directory '/home/ltj437606839/ics2017/nexus-am/am'
Building am [native]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/ltj437606839/ics2017/nexus-am/am'
make[1]: Leaving directory '/home/ltj437606839/ics2017/nexus-am'
make[1]: Entering directory '/home/ltj437606839/ics2017/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/ltj437606839/ics2017/nexus-am/libs/klib'
/home/ltj437606839/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib'
failed
make: [klib] Error 2 (ignored)
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!

```

6

认真阅读实验手册，在实现IO之前要打开HAS_IOE。目前发现因未开启HAS_IOE会导致_putc无限循环

7

发现自己的反汇编文件与其他人不同，或实现过程中发现反汇编文件有变化

实现串口时，会要求打开HAS_SERIAL宏定义。打开该宏定义之后，cputest里面的验证程序就都会被加上输入和输出指令，反汇编的结果就会有变化了。

8

运行的游戏按键没反应

游戏的按键设置往往和你想象的不太一样，去游戏源码目录下查看README.md文件，了解操作与键盘按键的映射关系。如果仍然不行，检查键盘IO实现是否正确，重新检查keytest能否正常pass。

PA 实验问题汇总

● 文档说明

此文档整理了 2020-2021 春季学期中同学们完成 PA 实验时遇到的各种问题。大多数问题提供了详细的[描述](#)、[截图](#)和[解决方法](#)，以方便以后的同学们对照文档排查自己遇到的问题。我们认为能够发现问题的同学同样为此文档做出了贡献，因此根据同学与我们交流的情况，列出了问题的部分[发现者](#)。另外，一些同学为部分问题提供了非常优秀的解答，因此作为[解决者](#)列出。

感谢 PA 课程中同学们付出的努力和对本文档做出的贡献！

文档整理者

刘莊、代素蓉

PA1 / 调试相关

● 如何调试程序？什么情况下可以认为实现正确？

问题描述

在 PA 的实验过程中要执行很多测试程序，但是这些程序一开始往往不能运行成功，在出错时怎样才能找到问题？什么样的情况下才能认为我的实现是正确的？

解决方法

写程序时出现一些 bug 是难免的，我们要合理利用代码框架输出的信息、PA1 实现的调试器、gdb 调试器、DIFF_TEST 以及 printf 等方法找出错误的原因：

1. 代码框架：PA 的代码框架里默认会对代码的正确性进行一些判别，例如 PA2 里会提示[未实现的指令](#)及其字节码，PA3 里会提示[未实现的系统调用号](#)。样例程序结束时会提示 [hit](#)

good trap 或 hit bad trap 等，这些基本的信息是调试的基础。

2. PA1 实现的调试器与 gdb 调试器：我们在 PA1 中实现了简易调试器，打开 DEBUG 宏之后就能够通过[设置断点](#)、[单步调试](#)等方法观察程序崩溃之前的执行状态。PA1 实现的调试器功能越强大，调试就越方便。（例如，PA1 中[死循环检测](#)是一个选做题，但如果没有实现，程序陷入死循环时屏幕上不会输出任何信息，这时只能通过单步调试等方法确定死循环的位置，十分影响效率）在简易调试器功能不够用的时候，也可以使用 [make gdb](#) 指令使用更加强大的 gdb 进行调试。
3. DIFF_TEST：在开始 PA2 之前十分建议先实现 [DIFF_TEST](#) 功能。它会将 nemu 执行一条指令后的寄存器状态与 qemu 对比，能够尽量早、尽量细致地发现出错的位置。但也要注意，DIFF_TEST 并不比较 IO 设备的信息、内存和标志寄存器中某些未实现的位。因此对于 IOE 和某些访存指令，DIFF_TEST 并不是判断实现是否正确的绝对依据。
4. printf / Log：[打印信息](#)是一种朴实无华但却十分有效的调试方法，某些情况下能暴露出调试器无法发现的漏洞，善用 printf 对调试代码十分有用。

什么样的情况下代表实现正确？

在 PA 的测试过程中，一般认为所有测试程序都 [hit good trap](#)，即代表这一阶段的实现是正确的，如果是 [hit bad trap](#) 一定意味着实现有问题。但是要注意，即使结果是 [hit good trap](#)，你的实现也不是绝对就没有问题。有些问题会在后续的测试中暴露出来，所以被找不到的 bug 困扰时别忘了[回头看看](#)。

PA2

● 添加指令后出现语法错误

问题描述

添加一条指令后，程序编译不通过，报出各种语法错误

问题截图

```
two@ubuntu:~/PWS/lcs2018/nemu$ make
+ CC src/cpu/exec/exec.c
src/cpu/exec/exec.c: In function 'exec_lea':
src/cpu/exec/exec.c:7:3: error: storage class specified for parameter 'opcode_entry'
} opcode_entry;
^
src/cpu/exec/exec.c:13:41: error: expected '=', ',', ';', 'asm' or '__attribute__' before '{' token
static inline void set_width(int width) {
^
src/cpu/exec/exec.c:20:39: error: expected declaration specifiers or '...' before 'opcode_entry'
static inline void idex(vaddr_t *elp, opcode_entry *e) {
^
In file included from ./include/common.h:11:0,
                 from ./include/nemu.h:4,
                 from ./include/cpu/exec.h:4,
                 from src/cpu/exec/exec.c:1:
```

发现者

1813558 潘文生

解决方法

2017/18 版本 PA2 框架代码使用了大量的宏定义进行编程，出现语法错误时不太容易查找。

最容易出现语法错误的是 opcode_table，不要遗漏逗号。另外，all-inst.h 中不要遗漏分号。

● 实现完 push 指令就出现段错误/缓冲区溢出错误 (core dumped / buffer overflow)

问题描述

1. 在 PA2 阶段一，运行程序，单步调试执行了三步出现段错误

2. 在 PA2 阶段 1，实现完 call 指令未出现异常，实现完 push 指令后，开始报错 buffer overflow detected

问题截图

1. 段错误截图

```
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 16:08:48, May 27 2021
For help, type "help"
(nemu) si 1
100000: bd 00 00 00 00
(nemu) si 1
100005: bc 00 7c 00 00
(nemu) si 1
make[1]: *** [Makefile:47: run] 段错误 (核心已转储)
make[1]: 离开目录 "/home/idaguo/courses/pa1/ics2017/nemu"
make: *** [/home/idaguo/courses/pa1/ics2017/nexus-am/Makefile.app:35: run] 错误 2
```

发现者

1813069 郭怡霏、1810729 黄璟知 等

解决方法

1. rtl_push 中传递的参数不正确，数值和长度反了
2. 是因为 call 指令的实现中，对 eip 进行了错误的 2 赋值操作，使 eip 指向分配的内存外部，从而导致溢出，修正 call 指令的实现即可。

Plus

PA1-PA5 过程中，出现内存超限、段错误、缓冲区溢出等错误，不要怀疑源代码有问题，多数情况下是你的代码实现有问题，先从自己写的代码里找原因！

● 控制台没有输出/死循环？

问题描述

新实现了一条指令之后，运行测试程序的时候，控制台没反应，程序卡住。

发现者

1811418 施超超, 1811499 麦隽韵等

解决方法

PA2 程序没反应一般是出现了死循环, 此时需要借助单步调试、断点调试等方法, 确认程序在执行哪段代码时出现了问题。也可以打开 `diff_test`, 确定数值不一致的位置。

另外, 如果做到了 IOE 的部分, 需要打开 `nemu/include/common.h` 中的 `HAS_IOE` 和 `nexus-am/am/am.h` 里的 `HAS_SERIAL` 这两个宏定义 (会弹出标题为 `nemu` 的新窗口)

● 开启 `diff_test` 之后, 虚拟机的系统库函数发生段错误

问题描述

打开 `DIFF_TEST` 之后, 并不属于 `ics2018` 工程目录的某函数文件报段错误

解决方法

我们能够保证 `ics` 文件夹下的绝大部分代码不出错, 但是无法保证外部环境的正确性。这一问题产生于外部的虚拟机环境, 可以通过更换虚拟机或者出错的动态链接库解决。

● `trm_init` 和 `main` 函数的开头出现 `endbr32` 指令

问题描述

在 `gcc` 版本大于 8 时, 函数的开头出现一条未实现的指令: `f3 0f 1e fb`

问题截图

```

4
5 Disassembly of section .text:
6
7 00100000 <_start>:
8   100000: bd 00 00 00 00      mov     $0x0,%ebp
9   100005: bc 00 7c 00 00      mov     $0x7c00,%esp
10  10000a: e8 01 00 00 00      call    100010 <_trm_init>
11  10000f: 90                  nop
12
13 00100010 <_trm_init>:
14  100010: f3 0f 1e fb          endbr32
15  100014: 55                  push    %ebp
16  100015: 89 e5               mov     %esp,%ebp
17  100017: 83 ec 08            sub     $0x8,%esp
18  10001a: e8 05 00 00 00      call    100024 <main>
19  10001f: d6                  (bad)
20  100020: eb fe               jmp     100020 <_trm_init+0x10>
21  100022: 66 90               xchg    %ax,%ax
22
23 00100024 <main>:
24  100024: f3 0f 1e fb          endbr32
25  100028: 31 c0               xor     %eax,%eax
26  10002a: c3                  ret
27

```

发现者

1811425 孙一丁, 1811482 姜欣妮, 1811402 吕鑫涛等同学

解决方法

这条指令是 gcc 更新后添加的检测跳转地址是否正确的指令, 与 gcc 版本有关

方法一: 更换实验环境, 使用 32 位操作系统+版本小于 8 的 gcc 编译器 (推荐)

方法二: 在 AM_HOME/makefile.compile 文件中, 在和 nemu 有关的 CFLAGS 后面添加 -fcf-protection=none 和 -mmanual-endbr (不推荐, 使用过高的 gcc 版本和 64 位机器后续还会造成其他问题)

Plus: 更改编译选项后需要先在 AM_HOME 下执行 make clean, 否则 trm.o 不会被更新, trm_init 依然会有 endbr32 指令

● 开启 HAS_IOE 之后，出现 D-Bus 报错问题

问题截图

```
[src/monitor/diff-test/diff-test.c,96,init_difftest] Connect to QEMU successfully
[src/monitor/monitor.c,65,load_img] The image is /home/winter/ics2017/nexus-am/apps/hello/build/hello-x86-nemu.bin
dbus[4890]: arguments to dbus_message_new_method_call() were incorrect, assertion "path != NULL" failed in file ../../dbus/dbus-message.c line 1362.
This is normally a bug in some application using the D-Bus library.

D-Bus not built with -rdynamic so unable to print a backtrace
Makefile:46: recipe for target 'run' failed
qemu-system-i386: terminating on signal 15 from pid 4890 (<unknown process>)
make[1]: *** [run] 已放弃 (core dumped)
make[1]: 离开目录"/home/winter/ics2017/nemu"
/home/winter/ics2017/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Error 2
```

发现者

1710241 路文特

解决方法

此问题发生在 32 位机器上，但不是我们发在群里的实验环境。出现的具体原因未知，但更换虚拟机可以解决问题。

● 实现串口后，先前能通过的测试程序全都无法通过测试

问题描述

在 PA2 的第二阶段运行 runall.sh，测试样例全都能通过，但是到了第三阶段，所有程序都无法通过，报出段错误

问题截图


```

ww@ubuntu:~/lcs2017/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] runall.sh: 行 23: 19118 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see add-longlong-log.txt for more information
[ add] runall.sh: 行 23: 19126 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see add-log.txt for more information
[ bit] runall.sh: 行 23: 19134 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see bit-log.txt for more information
[ bubble-sort] runall.sh: 行 23: 19144 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see bubble-sort-log.txt for more information
[ dummy] runall.sh: 行 23: 19152 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see dummy-log.txt for more information
[ fact] runall.sh: 行 23: 19160 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see fact-log.txt for more information
[ fib] runall.sh: 行 23: 19168 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile
FAIL! see fib-log.txt for more information
[ goldbach] runall.sh: 行 23: 19176 段错误 (核心已转储) $nemu -b -l $ori_log $file &> $logfile

```

发现者

1811507 文静静

解决方法

经过排查，问题是在读取内存时产生的。实现串口时我们修改了 `paddr_read`，如果读取的地址是端口映射的地址，则会去读取设备端口传来的数据。出错的原因是 `if` 语句，`if` 之后没有使用 `return` 返回，后面也没有使用 `else` 括起来，导致不论是不是端口地址都会调用 `mmio_read`，因此报错。

```

uint32_t paddr_read(paddr_t addr, int len) {
    int id = is_mmio(addr);
    if(id == -1){
        return pmem_rw(addr, uint32_t) & (~0u >> ((4 - len) << 3));
    }
    return mmio_read(addr, len, id);
}

```

● 运行 keytest.c 时，按键没有反应？

问题描述

添加键盘之后，按下按键没有收到任何信息？

解决方法

不要在控制台按按键，在跳出来的新窗口输入按键才会被接收

● 运行 string.c 报错：对函数未定义的引用

问题描述

ics2018 以下版本，运行测试样例时提示 strcmp、printf 等函数是“未定义的引用 (Undefined reference)”

问题截图

```
sc30@sc30-VirtualBox:~/桌面/pa2021/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=string run
Building string [x86-nemu]
Building am [x86-nemu]
make[2]: *** 没有指明目标并且找不到 makefile。 停止。
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/build/x86-nemu//tests/string.o: 在函数'main'中:
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:16: 对'strcmp'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:17: 对'strcmp'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:18: 对'strcmp'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:19: 对'strcmp'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:20: 对'strcmp'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:22: 对'strcpy'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:22: 对'strcat'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:22: 对'strcmp'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:24: 对'memset'未定义的引用
/home/sc30/桌面/pa2021/nexus-am/tests/cputest/tests/string.c:24: 对'memcmp'未定义的引用
objdump: /home/sc30/桌面/pa2021/nexus-am/tests/cputest/build/string-x86-nemu: 无此文件
objcopy: /home/sc30/桌面/pa2021/nexus-am/tests/cputest/build/string-x86-nemu: 无此文件
/home/sc30/桌面/pa2021/nexus-am/Makefile.app:33: recipe for target 'app' failed
make[1]: *** [app] Error 1
Makefile:15: recipe for target 'Makefile.string' failed
make: [Makefile.string] Error 2 (ignored)
string
sc30@sc30-VirtualBox:~/桌面/pa2021/nexus-am/tests/cputest$
```

发现者

1812326 翟江天

解决方法

问题的原因是缺少 IO 和字符操作需要的静态编译库 klib-x86-nemu.a。从助教或其他同学那里拷贝这个文件，并放在 AM_HOME/libs/klib/build 目录下即可

● 运行 videotest.c 时，把自己的代码注释掉也能运行？

问题描述

实现 VGA 的时候发现即使把_draw_rect 函数里的内容全部注释掉，make run 之后也能正常显示屏幕动画？

发现者

1811499 麦隽韵

解决方法

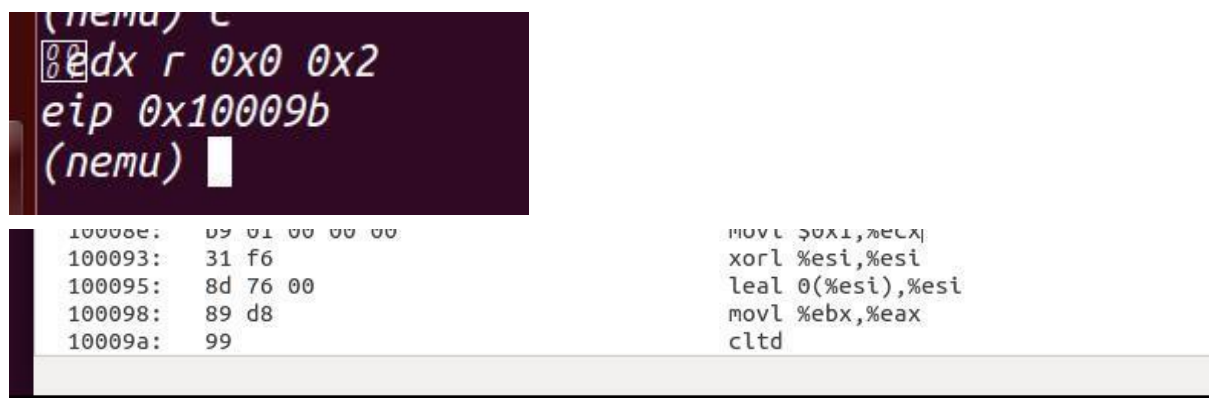
如果自己的代码没有生效，可能是 make 时没有更新（需要 make clean），也可能是代码编译到了 native 环境。编译目标设置可以参考其他章节里的相关问题。

● 运行一键回归测试：开启 diff_test，执行到 idiv 指令时不一致？

问题描述

运行 runall.sh 的时候，部分文件 failed。不开启 diff_test 时，单独运行错误的测试程序，部分程序还发生了死循环。查看反汇编文件发现 diff_test 报错都发生在执行 idiv 指令时。可以 idiv 指令的代码都是框架代码给出的，为什么会出问题？

问题截图



发现者

1811418 施超超

解决方法

实现 cltd 指令时，操作数 32 位的情况下需要把 eax 的最高位扩展到 edx。16 位时则不需要扩展到 edx，只需要将 ax 扩展到 eax。（仔细阅读手册中的说明）

● 运行 microbench 时使用 native 运行同样报错？

问题描述

64 位 ubuntu 操作系统，GCC 版本 9.3.0，运行 microbench 时即使使用 native 环境也还是无法通过测试

问题截图

```
alan@alan-Alienware-m15:~/code/PA/ics2018/nexus-am/apps/microbench$ make ARCH=native INPUT=TEST run
Building microbench [native]
make[1]: 进入目录 "/home/alan/code/PA/ics2018/nexus-am"
make[2]: 进入目录 "/home/alan/code/PA/ics2018/nexus-am/am"
Building am [native]
make[2]: 对"archive"无需做任何事。
make[2]: 离开目录 "/home/alan/code/PA/ics2018/nexus-am/am"
make[1]: 离开目录 "/home/alan/code/PA/ics2018/nexus-am"
make[1]: 进入目录 "/home/alan/code/PA/ics2018/nexus-am/libs/klib"
make[1]: *** 没有指明目标并且找不到 makefile。 停止。
make[1]: 离开目录 "/home/alan/code/PA/ics2018/nexus-am/libs/klib"
make: [/home/alan/code/PA/ics2018/nexus-am/Makefile.compile:88: klib] 错误 2 (已忽略)
[qsor] Quick sort: X Failed.
[queen] Queen placement: * Passed.
[bf] Brainf**k interpreter: * Passed.
[fib] Fibonacci number: * Passed.
[sieve] Eratosthenes sieve: * Passed.
[15pz] A* 15-puzzle search: * Passed.
[dinic] Dinic's maxflow algorithm: * Passed.
[lzip] Lzip compression: X Failed.
[ssort] Suffix sort: X Failed.
[md5] MD5 digest: X Failed.
=====
MicroBench FAIL
Exit (0)
```

发现者

1711366 欧阳文豪，1811554 林铸天等同学

解决者

1711366 欧阳文豪

解决方法

问题原因是在 GGC9 中，tree-vrp 优化算法将 unsigned int 类型与 char 类型的计算结果作为 int 类型，值域传播后发现 [int 类型](#) 溢出，便把结果置为 0x80000000。针对这一问题，解决方法有三：

1. 将 Makefile 中 -O2 优化的编译选项更改为 -O1 或 -O0（缺点：程序运行可能变慢）
2. 修改 microbench 的源文件，将 checksum 函数的中间变量类型由 int32_t 更改为 uint32_t，避免数值溢出

3. 更换实验环境，使用 32 位虚拟机和 8.0 以下的 GCC 版本（推荐 GCC5.4）

● 运行 microbench 时有些样例没有通过应该怎么调试？

问题描述

在 PA2 阶段，程序通过了 runall.sh 测试，但在运行 microbench 测试程序时有些样例显示 failed，这时如果打开 DEBUG 和 DIFF_TEST 这两个宏，程序运行速度会变得十分缓慢。如何对 microbench 中的程序进行调试？

解决方法

出现这种情况**通常**代表 PA2 中某些指令实现不正确，调试方法如下：

方法一：microbench 中运行的程序源文件都在其目录下，可以将出错的那个程序的源文件复制出来，单独在 apps 或 tests 目录下为其创建一个文件夹，仿照其他文件夹为其编写 Makefile，并对代码进行一些修改（删除与 microbench 有关的代码，添加 main 函数等），即可独立运行这个程序。

方法二：microbench 的执行顺序是由 include 文件夹下的 microbench.h 决定的，修改 BENCHMARK_LIST 中的顺序即可让出错的程序先被执行。

注意：gcc 版本过高和 64 位操作系统也可能导致 microbench 不通过，请先使用 make ARCH=native run 运行 microbench，如果 native 都无法通过测试，证明并非 nemu 实现的问题，这种情况请参考其他相关问题

● 运行 microbench 时无法通过 A* 15pz 测试？

问题描述

64 位 Ubuntu 操作系统，GCC 版本 7.5.0。运行 microbench 时无法通过 A* 15pz 测试

问题截图

```

Building 15pztest [native]
+ CC 15pz/bench.c
+ CXX 15pz/15pz.cpp
make[1]: Entering directory '/home/pa/Downloads/ics2018/nexus-am'
make[2]: Entering directory '/home/pa/Downloads/ics2018/nexus-am/am'
Building am [native]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/pa/Downloads/ics2018/nexus-am/am'
make[1]: Leaving directory '/home/pa/Downloads/ics2018/nexus-am'
make[1]: Entering directory '/home/pa/Downloads/ics2018/nexus-am/libs/'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/pa/Downloads/ics2018/nexus-am/libs/'
/home/pa/Downloads/ics2018/nexus-am/Makefile.compile:86: recipe for target 'libs' failed
make: [klib] Error 2 (ignored)
[15pz] A* 15-puzzle search: X Failed.
    min time: 11 ms [385509]
=====
MicroBench FAIL          385509 Marks
                        vs. 100000 Marks (i7-6700 @ 3.40GHz)
Exit (0)
pa@ubuntu:~/Downloads/ics2018/nexus-am/apps/15pztest$

```

发现者

1811499 麦隽韵

解决方法

这一问题的详细原因目前还没有查明，但通过更换 32 位操作系统或将 checksum 函数的中间变量类型由 int32_t 更改为 uint32_t 可以解决此问题。

PA3

● PA3 时打开 DIFF_TEST 报错？

问题描述

进入 PA3 以后，如果不关闭 DIFF_TEST，运行 text.c 时报错

发现者

1811342 陈沃东等

解决方法

关闭 DIFF_TEST。DIFF_TEST 仅检查通用寄存器，不检查内存。进入 PA3 以后，IDT、PDE 等重要的结构都保存在内存当中，其地址和 qemu 中的实现可能有差别。如果已经通过了 microbench 等测试，DIFF_TEST 能够提供的支持已经十分有限。此时，不需要再打开 microbench，遇到新的错误时可以使用调试器和 Log 配合进行调试

● 执行 make update，生成的文件非常大

问题描述

64 位机器使用 make update 生成 ramdisk.img 时生成了一个体积非常大的文件，大约 64MB

问题截图

```
s2018/nanos-lite/build/nanos-lite-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/rexxar/PA2021_0308/ics2018/nanos-lite/build/nanos-l
ite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 23:10:08, Apr 30 2021
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 23:26:03, Apr 30 2021
[src/ramdisk.c,25,init_ramdisk] ramdisk info: start = 0x100cd4, end = 0x4148e24, size = 67404112 bytes
make[1]: *** [Makefile:47: run] Segmentation fault (core dumped)
make[1]: Leaving directory '/home/rexxar/PA2021_0308/ics2018/nemu'
make: *** [/home/rexxar/PA2021_0308/ics2018/nexus-am/Makefile.app:39: run] Error 2
rexxar@ubuntu:~/PA2021_0308/ics2018/nanos-lite$
```

发现者

1811425 孙一丁、1811413 权威等

解决者

1811413 权威

解决方法

修改 nanos-lite 下面的 makefile 文件，在 objcopy 命令后添加一个选项：-R [节名]，其中，节名可以通过 readelf 命令查看编译出来的二进制文件，除了.rodata 和.text 这样的正常的节之外，文件里还有一个名字特别长的节，将这个名字特别长的节去除即可解决问题。另外，更换 32 位机也可以解决问题。

● 执行 make update，没有生成 ramdisk.img？

问题描述

在实现 pa3 文件系统的时候，我的 files.h 里面什么都没有

发现者

1811112 符心亮

解决方法

进入第三阶段没有正确更新 makefile，需要修改 update 编译命令后的目标，改为 update-ramdisk-fsimg，才能一次性加载多个文件

● 修改了 navy_apps 下的代码，运行时没有发生任何变化？

解决方法

nanos-lite 操作系统[会](#)直接加载 [ramdisk.img](#) 中编译好的可执行文件，要想在 navy-apps 中的改变生效，需要重新 make update 一次

● 如何查看 navy_apps 下程序的反汇编代码？

解决方法

官方的 makefile 默认不为 navy_apps 下的用户程序生成反汇编文件。如果需要，可以在可执行文件的相同目录下执行 objdump 命令，例如：

```
objdump -D hello > hello.dump.txt
```

如果想免去手动反汇编的麻烦，也可以尝试去修改 navy_apps 下的 makefile，在编译之后默认执行一次反汇编

● 实现 loader 时，又一次出现了 endbr32 指令？

问题描述

之前 PA2 时遇到过 endbr32 指令的问题，这一问题在 PA3 加载用户程序时再次出现，但是查看反汇编文件没有发现这条指令

问题截图

```
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 13:11:48, May 23 2021
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101100, end = 0x10643c,
size = 21308 bytes
invalid opcode(eip = 0x04000000): f3 0f 1e fb 55 89 e5 83 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x04000000 is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x04000000) in the disassembling result to distinguish which case it is.
```

发现者

1811371 黎鸿饱

解决方法

可以参考 PA2 中的解决方法，在 navy-apps 下面也添加编译选项。或者也可以更换 GCC 版本到 5.4。反汇编文件中找不到这条指令的原因是 nanos-lite 下的反汇编文件仅包含内核的代码，用户程序的代码是后来加载进内存的。

● 编译报错：syscall.h 中的 SYS_none 等未定义？

问题描述

运行客户程序，编译内核时报错，SYS_none 未定义

问题截图

```
winter@winter-virtual-machine:~/ics2017/nanos-lite$ Make run
Building nanos-lite [x86-nemu]
+ CC src/syscall.c
src/syscall.c: In function 'do_syscall':
src/syscall.c:84:10: error: 'SYS_none' undeclared (first use in this function)
    case SYS_none: sys_none(r); break;
           ^
src/syscall.c:84:10: note: each undeclared identifier is reported only once for
each function it appears in
/home/winter/ics2017/nexus-am/Makefile.compile:69: recipe for target '/home/wint
er/ics2017/nanos-lite/build/x86-nemu/./src/syscall.o' failed
make: *** [/home/winter/ics2017/nanos-lite/build/x86-nemu/./src/syscall.o] Erro
r 1
```

发现者

路文特 1710241

解决方法

nanos-lite/src 下的 syscall.h 是一个[软链接](#)文件，如果移动过文件夹，软链接很可能会失效，在该目录下执行以下命令重新链接一次，然后重新编译即可：(去掉-s 生成硬链接也是可以的)

```
ln -s ../../navy-apps/libs/libos/src/syscall.h syscall.h
```

● raise_intr 报错：idtr 寄存器中 limit 字段值为 0？

问题描述

刚刚进入 PA3，实现了 int 指令，执行 dummy.c 时，raise_intr 报错：Assertion:

gate_addr <= cpu.idtr.base + cpu.idtr.limit failed!

输出信息进行调试，发现 limit 字段数值始终为 0

问题截图

```
src/cpu/intr.c:19: raise_intr: Assertion 'gate_addr <= cpu.idtr.base + cpu.id
tr.limit' failed.
Makefile:46: recipe for target 'run' failed
```

解决方法

打开 HAS_ASYE 宏定义

● 运行 dummy.c 或其他测试程序，显示的未实现的中断调用

号与预期不一致

问题描述

实现中断调用的时候，第一次运行 dummy.c，应当提示未实现的中断调用号=3，实现了 sys_none 以后应当提示未实现的中断调用号=4，但是我在实验中却提示了别的调用号？

问题截图

发现者

1811451 杨航，1811484 李金雯，1811421 苏家兴等

解决方法

这一问题可能的原因有多种，请——检查：

1. raise_intr 中使用了错误的中断调用号，或未压栈正确的返回地址
2. _RegSet 结构体顺序不正确
3. syscall 中的 switch-case 语句分支没有 break
4. 在实现 sys_write 之前，不能使用 printf 等方法在 navy_apps 下面的程序中输出信息
5. libos/nanos.c 中的库函数提供的参数不正确
6. 尝试在 AM_HOME 下面 make clean 一下来更新文件

● 运行 dummy.c，无反应/死循环？

问题截图

```
objcopy -S --set-section-flags .bss=alloc,contents -O binary /home/joshua/Desktop/ly/ics2018/navy-apps/tests/dummy/build
touch src/files.h
joshua@ubuntu:~/Desktop/ly/ics2018/nanos-lite$ make run ARCH=x86-nemu
Building nanos-lite [x86-nemu]
+ CC src/fs.c
+ AS src/initrd.S
make[1]: Entering directory '/home/joshua/Desktop/ly/ics2018/nexus-am'
make[2]: Entering directory '/home/joshua/Desktop/ly/ics2018/nexus-am/am'
Building am [x86-nemu]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/joshua/Desktop/ly/ics2018/nexus-am/am'
make[1]: Leaving directory '/home/joshua/Desktop/ly/ics2018/nexus-am'
make[1]: Entering directory '/home/joshua/Desktop/ly/ics2018/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/joshua/Desktop/ly/ics2018/nexus-am/libs/klib'
/home/joshua/Desktop/ly/ics2018/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/joshua/Desktop/ly/ics2018/nemu'
./build/nemu -l /home/joshua/Desktop/ly/ics2018/nanos-lite/build/nemu-log.txt /home/joshua/Desktop/ly/ics2018/nanos-lite
[src/monitor/monitor.c,65,load_img] The image is /home/joshua/Desktop/ly/ics2018/nanos-lite/build/nanos-lite-x86-nemu.b
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 20:30:04, May 22 2021
For help, type "help"
(nemu) si
(nemu) si
(nemu) si
(nemu) c

```

发现者

1811379 李毅 , 1711430 江玥等

解决方法

没有打开 HAS_IOE 或 HAS_SERIAL 宏定义 , 打开即可 (会跳出一个标题为 nemu 的窗口)

● 运行 dummy.c , 出现一条未实现的指令

问题描述

运行 dummy.c , 出现一条奇怪的未实现的指令 , 且这个指令对应的 eip 小于 0x100000

问题截图

```
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 18:03:13, May 24 2021
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x100cf4, end = 0x106034, size = 21312 bytes
Invalid opcode(eip = 0x000078f4): fa ff ff ff 48 79 00 00 ...

There are two cases which will trigger this unexpected exception:
1. The instruction at eip = 0x000078f4 is not implemented.
2. Something is implemented incorrectly.
Find this eip(0x000078f4) in the disassembling result to distinguish which case it is.

If it is the first case, see

```

发现者

1811358 龚卓岳

解决者

1813027 王博源

解决方法

打开 HAS_ASYE 宏定义

● 运行 dummy.c , 报指针转换错误

问题截图

```
idaguo@idaguo-VirtualBox:~/courses/pal/ics2017/nanos-lite$
make
Building nanos-lite [native]
+ CC src/main.c
src/main.c: In function 'main':
src/main.c:34:4: error: cast to pointer from integer of di
fferent size [-Werror=int-to-pointer-cast]
    ((void (*)(void))entry)();
    ^
cc1: all warnings being treated as errors
make: *** [/home/idaguo/courses/pal/ics2017/nexus-am/Makef
ile.compile:71: /home/idaguo/courses/pal/ics2017/nanos-lit
e/build/native/./src/main.o] 错误 1

wenchanggao@ubuntu:~/Desktop/ics2017/nanos-lite$ make run
Building nanos-lite [native]
+ CC src/fs.c
In file included from /home/wenchanggao/Desktop/ics2017/nexus-am/am/arch/native/
include/arch.h:4:0,
                  from /home/wenchanggao/Desktop/ics2017/nexus-am/am/am.h:12,
                  from /home/wenchanggao/Desktop/ics2017/nanos-lite/include/commo
n.h:4,
                  from /home/wenchanggao/Desktop/ics2017/nanos-lite/include/fs.h:
4,
                  from src/fs.c:1:
/home/wenchanggao/Desktop/ics2017/nanos-lite/include/fs.h:7:3: error: expected i
dentifier before numeric constant
    SEEK_SET,
    ^
/home/wenchanggao/Desktop/ics2017/nexus-am/Makefile.compile:69: recipe for targe
t '/home/wenchanggao/Desktop/ics2017/nanos-lite/build/native/./src/fs.o' failed
make: *** [/home/wenchanggao/Desktop/ics2017/nanos-lite/build/native/./src/fs.o
] Error 1
```

发现者

1813069 郭怡霏, 1812983 高文畅等

解决方法

注意编译时的日志信息，如果出现[native]，说明编译的目标不是 nemu 而是真机，此时需要手动指定 ARCH，或者参考“其它”章节里的相关问题来更改默认的编译目标。

● 运行 dummy.c，errno 重定义

问题描述

运行 dummy.c 时，出现 errno 多重定义的问题

问题截图



```
make[1]: 离开目录“/home/lvxintao/PA/navy-apps/libs/libos”
+ CC dummy.c
+ LD /home/lvxintao/PA/navy-apps/tests/dummy/build/dummy-x86
ld: /home/lvxintao/PA/navy-apps/libs/libc/build/libc-x86.a(filer.o):(.bss.errno+0x0): multiple definition of `errno'
; /home/lvxintao/PA/navy-apps/libs/libc/build/libc-x86.a(sbrkr.o):(.bss.errno+0x0): first defined here
ld: /home/lvxintao/PA/navy-apps/libs/libc/build/libc-x86.a(signalr.o):(.bss.errno+0x0): multiple definition of `errno'
; /home/lvxintao/PA/navy-apps/libs/libc/build/libc-x86.a(sbrkr.o):(.bss.errno+0x0): first defined here
ld: /home/lvxintao/PA/navy-apps/libs/libc/build/libc-x86.a(fstatr.o):(.bss.errno+0x0): multiple definition of `errno'
; /home/lvxintao/PA/navy-apps/libs/libc/build/libc-x86.a(sbrkr.o):(.bss.errno+0x0): first defined here
make: *** [/home/lvxintao/PA/navy-apps/Makefile.app:37: /home/lvxintao/PA/navy-apps/tests/dummy/build/dummy-x86] 错误 1
```

发现者

1811402 吕鑫涛等

解决方法

此问题在 GCC 版本较高时出现。

首先应该进行一次 make clean，指定好编译目标架构为 x86-nemu，重新编译试一下。

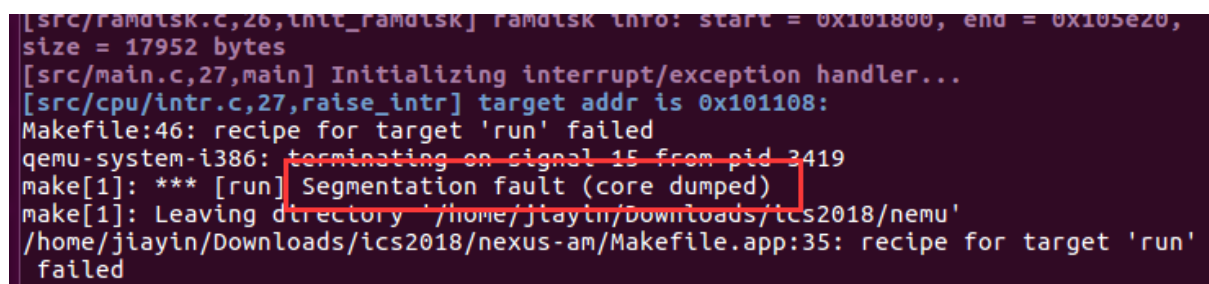
如果不能解决，可以依次找到报错的文件，将“int errno”改为“static int errno”。

● 运行 dummy.c , 段错误

问题描述

实现 int 指令后, 使用 printf 打印信息, idt 的地址没有问题, 但是程序没有正常进入中断处理程序, 而是卡在了一个 push 指令里

问题截图



```
[src/ramdisk.c,26,intt_ramdisk] ramdisk info: start = 0x101800, end = 0x105e20,
size = 17952 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/cpu/intr.c,27,raise_intr] target addr is 0x101108:
Makefile:46: recipe for target 'run' failed
qemu-system-i386: terminating on signal 15 from pid 3419
make[1]: *** [run] Segmentation fault (core dumped)
make[1]: Leaving directory '/home/jiayin/downloads/tcs2018/nemu'
/home/jiayin/Downloads/ics2018/nexus-am/Makefile.app:35: recipe for target 'run'
failed
```

发现者

1811523 郑佳音

解决方法

不要在 raise_intr 中直接修改 cpu.eip 的值 (对于其他指令也是如此), 而是应该修改 jmp_eip , 让 CPU 在取下一条指令时自动跳转

● 运行 dummy.c , 物理地址超限 ? (情况一)

问题描述

日志刚刚输出到 Initializing interrupt handler 之后, 报出物理地址超限 (这一错误有时也会报段错误)

问题截图


```
physical address(0x8f000408) is out of bound
nemu: src/memory/memory.c:17: paddr_read: Assertion `addr < (128 * 1024 * 1024)'
failed.
Makefile:46: recipe for target 'run' failed
make[1]: *** [run] 已放弃 (core dumped)
make[1]: Leaving directory '/home/lyx/Desktop/ics2021/nemu'
/home/lyx/Desktop/ics2021/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Error 2
```

发现者

1813265 李彦欣, 1711388 肖阳等

解决方法

此问题一般是由 `lidt` 指令造成的, 要注意 `lidt` 的实现应该与译码函数互相配合, 译码函数可能把信息保存在 `id_dest->addr` 而非 `id_dest->val` 里, 此时如果传入一个错误的地址, 系统调用的时候就会出现地址超限错误。

● 运行 dummy.c , 物理地址超限 (情况二)

问题描述

刚刚实现 `iret` 指令后, 运行 `dummy.c`, 没有触发实验手册上的 4 号中断, 而是出现物理地址超限错误

问题截图

```
piking@piking-virtual-machine: ~/ics2017/nanos-lite
/build/nanos-lite-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 21:17:15, May 19 2021
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 21:05:43, May 19 2021
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x100fd8, end = 0x1055b4,
size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/cpu/exec/system.c,17,exec_ldt] idtr.limit=0x7ff
[src/cpu/exec/system.c,18,exec_ldt] idtr.base=0x1055e0
[src/cpu/intr.c,31,raise_intr] target_addr=0x100b97
[src/cpu/intr.c,31,raise_intr] target_addr=0x100b97
[src/cpu/intr.c,31,raise_intr] target_addr=0x100b97
physical address(0xffffffff54) is out of bound
nemu: src/memory/memory.c:18: paddr_read: Assertion `addr < (128 * 1024 * 1024)'
failed.
Makefile:46: recipe for target 'run' failed
make[1]: *** [run] 已放弃 (core dumped)
make[1]: Leaving directory '/home/piking/ics2017/nemu'
/home/piking/ics2017/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Error 2
piking@piking-virtual-machine:~/ics2017/nanos-lite$
```

发现者

1811444 肖中遥

解决者

1811457 张泰源

解决方法

syscall 部分实现有问题，sys_none 实现不正确，或者提前写了一个空的 sys_exit

● 运行 text.c，物理地址超限？（情况一）

问题截图

```
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 00:25:56, May 17 2021
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101820, end = 0x37016e, size = 2550094 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
[src/loader.c,20,loader] load /bin/text, fd=32

[src/fs.c,59,fs_read] fd=32, offset=2521650, len=28444

[src/fs.c,110,fs_lseek] whence=2, result=5000

[src/fs.c,110,fs_lseek] whence=0, result=2500

[src/syscall.c,32,sys_read] 1024

[src/fs.c,59,fs_read] fd=7, offset=51790, len=1024

physical address(0x200a3330) is out of bound
nemu: src/memory/memory.c:21: paddr_read: Assertion `addr < (128 * 1024 * 1024)' failed.
```


发现者

1811482 姜欣妮、1811342 陈沃东、1811505 王一泓等多名同学

解决方法

检查 NAVY_HOME/libs/libos/nanos.c 中的_sbrk 函数，如果你的代码每次调用这个函数时，都把_end 的值重新赋给 program_break (或者其他名字)，那么意味着你的堆顶指针永远指向的是堆底（见下图）。_end 只需要被赋值一次，可以使用函数内的 static 关键字或者函数外的全局变量解决这一问题。

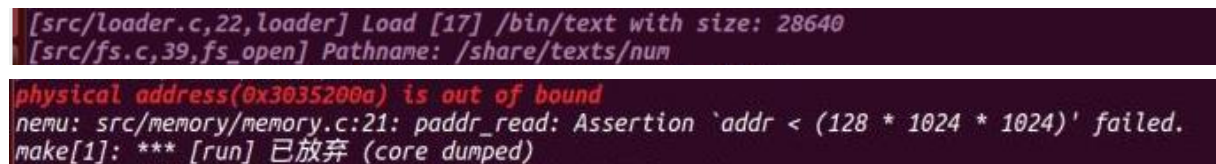
```
void *_sbrk(intptr_t increment){
    intptr_t old_addr = (intptr_t)&_end;
    intptr_t addr = old_addr;
    intptr_t tmp = old_addr + increment;
    if(_syscall_(SYS_brk,tmp,0,0)==0)
    {
        addr = tmp;
        return (void *)old_addr;
    }
    else
        return (void *)(-1);
}
```

● 运行 text.c，物理地址超限？（情况二）

问题描述

已经检查了_sbrk，确认没有问题，但是运行 text.c 还是报出了物理地址超限

问题截图



```
[src/loader.c,22,loader] Load [17] /bin/text with size: 28640
[src/fs.c,39,fs_open] Pathname: /share/texts/num
physical address(0x3035200a) is out of bound
nemu: src/memory/memory.c:21: paddr_read: Assertion `addr < (128 * 1024 * 1024)' failed.
make[1]: *** [run] 已放弃 (core dumped)
```

发现者

1811418 施超超，1810839 汪顶端

解决方法

首先，注意这种情况和情况一物理地址超限时报错的地址是不一样的，可以参考截图中的地址

判断是否属于此种情况。

这是一个 PA2 指令实现的错误，发现问题的同学 PA2 时 microbench 测试出现了死循环，但当时没有重视。因此，可以通过回头 Debug microbench 程序来找到错误，也可以请助教使用一个成熟的实现替换掉你的 nemu 文件夹，确定指令是否实现正确。

对于本例子，发现者在实现 logic.c 中的逻辑指令时，没有更新 ZF 和 SF（代码注释中提示的是不需要更新 CF 和 OF，不是所有标志寄存器都不需要更新）

● 运行 text.c 或 bmptest.c，出现一条未实现的指令

问题描述

运行 text.c 或 bmptest.c 的时候，出现了一条未实现的指令

问题截图

```
file 6: /etc/init
file 7: /share/pictures/projectn.bmp
fd=7
invalid opcode(eip = 0x0400366c): a4 39 d7 75 fb 5e 5f 5d ...
```

发现者

1811554 林铸天

解决方法

pa3 中如果遇到一些奇怪的问题，一般都是 pa2 中的指令实现有问题，尤其是对标志寄存器的修改。此问题系 shr 指令实现时未正确更新 ZF 和 SF。要注意 operand_write 是把临时寄存器的数值写回到通用寄存器或者内存当中，而不是写回到译码信息中。

```

make_EHelper(shr) {
    rtl_shr(&t0, &id_dest->val, &id_src->val);
    operand_write(id_dest, &t0);
    // 原来写的是 id_dest->val, 后改 &t0
    rtl_update_ZFSF(&t0, id_dest->width);

    print_asm_template2(shr);
}

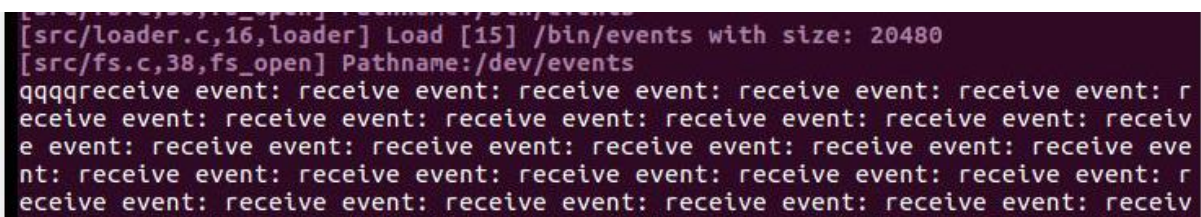
```

- 运行 text.c , hit good trap 但是没有输出 “PASS”
- 运行 events.c , 得不到正确的输出

问题描述

1. 运行 text.c , 显示 hit good trap , 但是没有输出最后的 “PASS” 。或者, 运行其他程序时, 尝试使用 printf 或 _write 输出, 但是输出内容没有出现在控制台?
2. 运行 events.c , 获取不到正确的按键信息

问题截图



```

[src/loader.c,16,loader] Load [15] /bin/events with size: 20480
[src/fs.c,38,fs_open] Pathname:/dev/events
qqqqreceive event: receive event: receive event: receive event: receive event: r
eceive event: receive event: receive event: receive event: receive event: receiv
e event: receive event: receive event: receive event: receive event: receive eve
nt: receive event: receive event: receive event: receive event: receive event: r
eceive event: receive event: receive event: receive event: receive event: receiv

```

发现者

1811525 周如, 1811422 隋尚达

问题原因

检查 fs_write , 发现代码里判断 open_offset >= fs_size 时程序直接返回。但是, 对于 stdout、stderr 等抽象文件, 他们的 fs_size 本来就是 0 , 因此输出时程序直接返回了。

```
ssize_t fs_size = fs_filesz(fd);  
if(file_table[fd].open_offset >= fs_size)  
|     return 0;
```

● 运行 bmptest.c 或《仙剑奇侠传》，ndl.c 报错？

问题描述

运行和设备有关的测试程序，且这些程序使用了 libndl 时，ndl.c 中的 assert 报错

问题截图



```
[src/fs.c,81,fs_read] buf in fs_read (dispinfo)=BM*, Len = 0, offset=128  
buf in ndl.c get_display_info: *B*\*2  
width=0, height=0  
Assertion failed: screen_w > 0 && screen_h > 0, file src/ndl.c, line 143  
nemu: HIT BAD TRAP at eip = 0x00100032
```

Code 32: src/ndl.c 中 Assertion failed 错误信息

Assertion failed: screen_w > 0 \&\& screen_h > 0, file src/ndl.c, line 141 nemu: HIT BAD TRAP at eip = 0x0010002f
--

发现者

1811378 李宜霖，1811421 苏家兴等

解决者

1811378 李宜霖，1811421 苏家兴等

解决方法

观察 ndl.c 的报错信息，查看是哪个设备出了问题。然后检查 nanos-lite\src\device.c 中 init_device 和 event_read 这两个函数其中的 设备尺寸、键盘事件、时钟事件格式是否符合实验手册的描述。

另外，应当检查 fs_read 中的实现，以下图为例，由于 if-else 分支实现不正确，读取屏幕信息时也会调用到 ramdisk_read，导致读取的屏幕信息最终返回一个错误的数值：

```

if(len>left2read)
    len = left2read;

if(fd == FD_DISPINFO){
    dispinfo_read(buf, fs_openoff(fd), len);
    Log("buf in fs_read (dispinfo)=%s, len = %d, offset=%d", buf, len, fs_openoff(fd));
}
if(fd == FD_EVENTS){
    return events_read(buf, len);
}
else{
    ramdisk_read(buf, fs_diskoff(fd) + fs_openoff(fd), len);
}
off_set(fd,fs_openoff(fd) + len);
return len;

```

- 运行 events.c , 读取不到正确的事件？
- 运行《仙剑奇侠传》，读取 rng.mkf 时死循环？

问题描述

运行 events.c 的时候无法显示正确的键盘信息，或运行《仙剑奇侠传》的时候，进行到读取 rng.mkf 这一步时程序卡住

问题截图

```

[src/fs.c,103,fs_read] offset before is 0x2d400 0xe27273 0x2df48
[src/fs.c,103,fs_read] offset before is 0x2d800 0xe27273 0x2df48
[src/fs.c,103,fs_read] offset before is 0x2dc00 0xe27273 0x2df48
[src/fs.c,191,fs_close] call fs_close
[src/fs.c,103,fs_read] offset before is 0x0 0x151ce5f 0x10372
[src/fs.c,103,fs_read] offset before is 0x30 0x151ce5f 0x10372
[src/fs.c,103,fs_read] offset before is 0x1012c 0x151ce5f 0x10372
PAL_InitText success
PAL_InitInput success
PAL_InitResources success
[src/fs.c,64,fs_open] call fs_open /share/games/pal/pat.mkf
[src/fs.c,69,fs_open] find file 42
[src/fs.c,103,fs_read] offset before is 0x0 0x19f304f 0x2128
[src/fs.c,103,fs_read] offset before is 0xc 0x19f304f 0x2128
[src/fs.c,103,fs_read] offset before is 0xc28 0x19f304f 0x2128
[src/fs.c,191,fs_close] call fs_close
[src/fs.c,64,fs_open] call fs_open /share/games/pal/rng.mkf
[src/fs.c,69,fs_open] find file 40
[src/fs.c,103,fs_read] offset before is 0x0 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x18 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x219e4a 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x219e4a 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x219f2a 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x0 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x18 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x219e4a 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x219e4e 0x159bc23 0x455e1a
[src/fs.c,103,fs_read] offset before is 0x219f4c 0x159bc23 0x455e1a
^C/home/sunjlayl/PA/lcs2018/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Interrupt

```

发现者

1810756 孙家宜等

解决方法

对于《仙剑》卡住的问题，经过测试，发现程序一直在重复读取 FD_EVENTS，即读取事件。仙剑在等待时钟信息到来，因此卡死。

这个错误是由于 Nanos-lite/src/device.c 中打包时钟、键盘事件的时候格式不正确：

- t 1234: 返回系统启动后的时间, 单位为毫秒;
- kd RETURN/ku A: 按下/松开按键, 按键名称全部大写, 使用 AM 中定义的按键名

以'\n'结束。

● 运行《仙剑奇侠传》，速度特别慢？

解决方法

关闭 DEBUG 和 DIFF_TEST

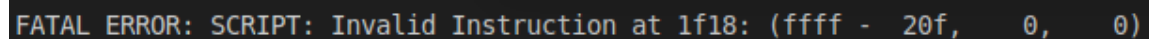
● 运行《仙剑奇侠传》，点击“新的故事”时 FATAL_ERROR

程序崩溃

问题描述

运行《仙剑奇侠传》，点击“新的故事”时程序崩溃，出现一条奇怪的未知指令

问题截图



发现者

1811378 李宜霖，1811457 张泰源，1811371 黎鸿饱等

解决者

1811378 李宜霖

解决方法

此问题是一个编译环境问题，可以通过更换 32 位机器解决。或者，可以通过设置编译指令解

决：修改 NAVY_HOME / Makefile.compile，将 CFLAGS 和 CXXFLAGS 的优化选项降低到-O1 或-O0，并添加-m32 选项指定程序编译目标为 32 位机器

- 运行《仙剑奇侠传》，点击“新的故事”出现一条未实现的指



问题截图

```
file name: /share/games/pal/gop.mkf  
invalid opcode(eip = 0x0401f711): a5 8b 7d e0 89 7a 0c 89 ...  
  
There are two cases which will trigger this unexpected exception:  
1. The instruction at eip = 0x0401f711 is not implemented.  
2. Something is implemented incorrectly.  
Find this eip(0x0401f711) in the disassembling result to distinguish which case it is.
```

If it is the first case, see



for more details.

If it is the second case, remember:

- * The machine is always right!
- * Every line of untested code is always wrong!

发现者

1811418 施超超

解决方法

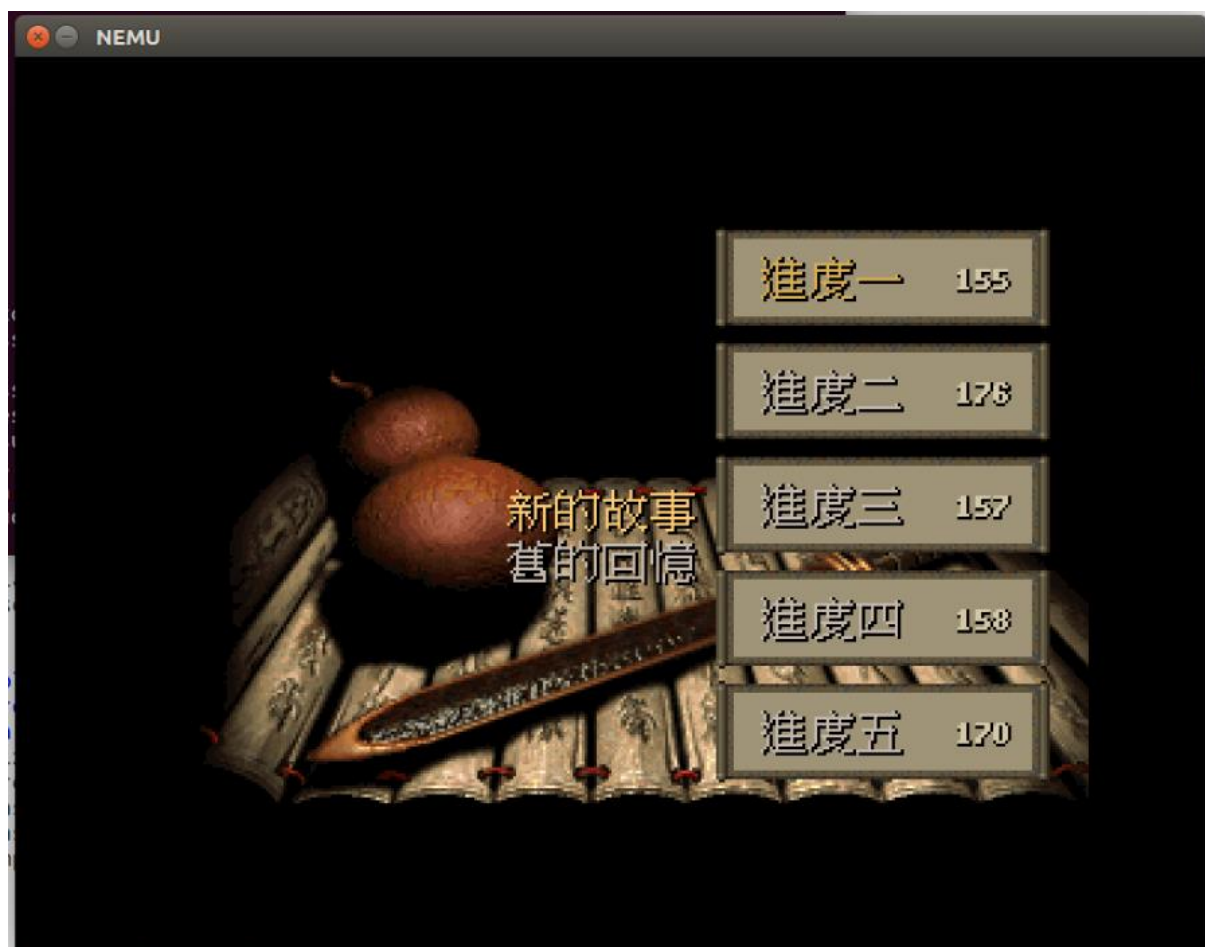
经过测试，这是一个环境问题，发现者的 GCC 版本是 4.8，而不是我们默认使用的 5.4，执行一次 `sudo apt-get update` 更新 GCC 之后，问题即可解决。

- 运行《仙剑奇侠传》，点击“新的故事”结果还是打开了存档？

问题描述

运行《仙剑奇侠传》时，点击“新的故事”结果还是打开了存档，没有其他的问题

问题截图



发现者

1811425 孙一丁

解决方法

这一问题的原因是 cmp 指令进行了对操作数进行了不必要的符号扩展，去除符号扩展即可解决问题。这一问题说明：

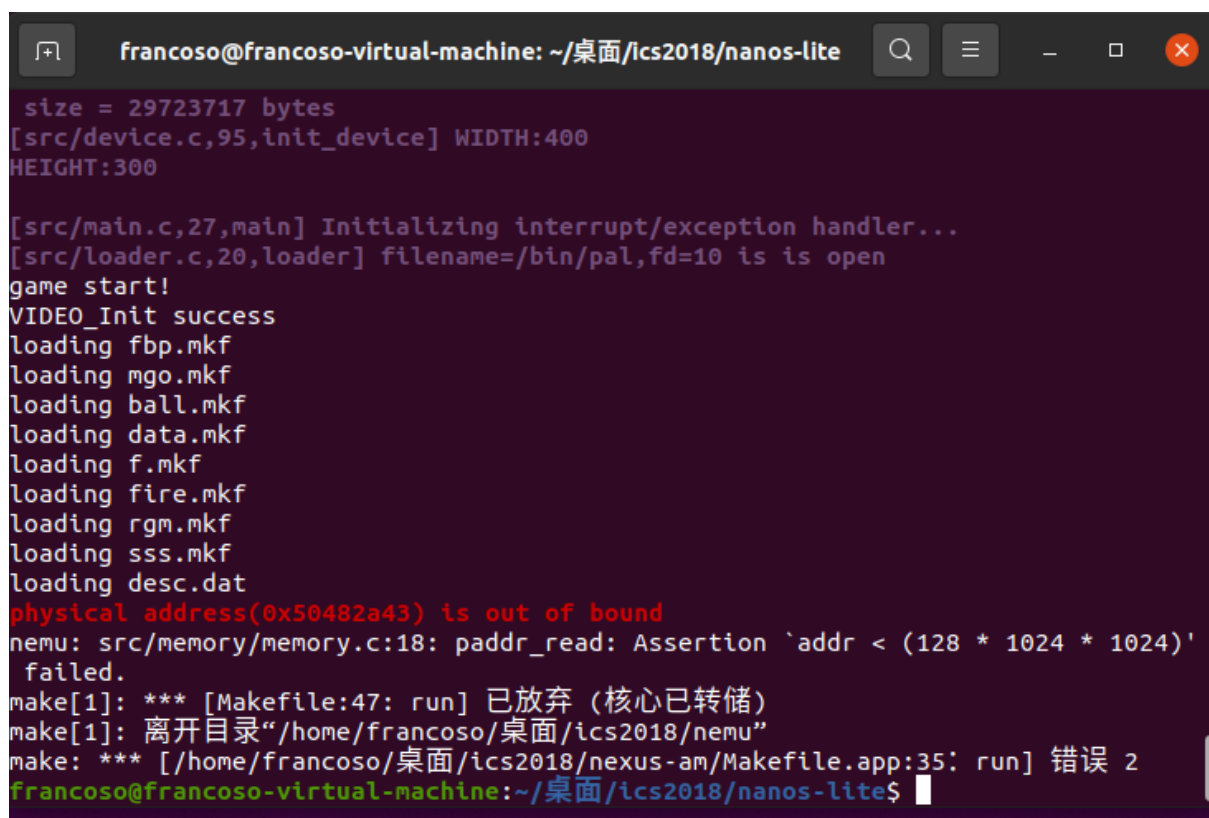
1. PA2 中指令的问题可能隐藏到十分后期的地方
2. 指令的实现应当严格参考 i386 手册

● 运行《仙剑奇侠传》，物理地址超限（情况一）

问题描述

64 位机，运行仙剑奇侠传时，加载游戏的资源文件 desc.dat 时，读取到一半突然发生物理地址超限

问题截图



```
francoso@francoso-virtual-machine: ~/桌面/ics2018/nanos-lite
size = 29723717 bytes
[src/device.c,95,init_device] WIDTH:400
HEIGHT:300

[src/main.c,27,main] Initializing interrupt/exception handler...
[src/loader.c,20,loader] filename=/bin/pal,fd=10 is is open
game start!
VIDEO_Init success
loading fbp.mkf
loading mgo.mkf
loading ball.mkf
loading data.mkf
loading f.mkf
loading fire.mkf
loading rgm.mkf
loading sss.mkf
loading desc.dat
physical address(0x50482a43) is out of bound
nemu: src/memory/memory.c:18: paddr_read: Assertion `addr < (128 * 1024 * 1024)'
failed.
make[1]: *** [Makefile:47: run] 已放弃 (核心已转储)
make[1]: 离开目录“/home/francoso/桌面/ics2018/nemu”
make: *** [/home/francoso/桌面/ics2018/nexus-am/Makefile.app:35: run] 错误 2
francoso@francoso-virtual-machine:~/桌面/ics2018/nanos-lite$
```

发现者

1811421 苏家兴

解决方法

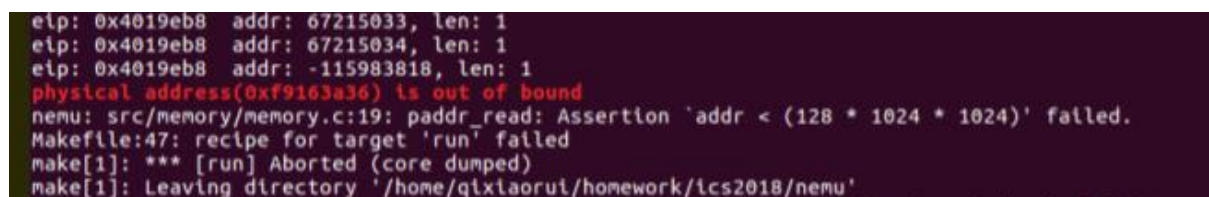
64 位机的情况下，文件占据的空间超过了虚拟磁盘上限。解决方法是更换 32 位机（理论上也可以尝试修改 navy_apps 下的编译选项，添加 -m32，但是否可以成功未经过测试）

● 运行《仙剑奇侠传》，物理地址超限（情况二）

问题描述

运行仙剑奇侠传时，在完全加载好仙剑奇侠传的准备文件之后，真正进入游戏界面之前，报错物理地址超限

问题截图



```
eip: 0x4019eb8 addr: 67215033, len: 1
eip: 0x4019eb8 addr: 67215034, len: 1
eip: 0x4019eb8 addr: -115983818, len: 1
physical address(0xf9163a36) is out of bound
nemu: src/memory/memory.c:19: paddr_read: Assertion 'addr < (128 * 1024 * 1024)' failed.
Makefile:47: recipe for target 'run' failed
make[1]: *** [run] Aborted (core dumped)
make[1]: Leaving directory '/home/qixiaorui/homework/lcs2018/nemu'
```

发现者

1811412 戚晓睿

解决方法

经过排查，该问题是 libos 中的 `_sbrk` 实现有误造成的。如下图所示，这份代码返回的是增长后的堆顶的地址。而申请堆区时，返回的应当是新申请的这片内存区域的起始地址，图中代码应该将 `obrk - increment` 作为返回值

```

1  extern char _end;
2  static intptr_t obrk = (intptr_t)&_end;
3  void* _sbrk(intptr_t increment)
4  {
5      intptr_t pbrk = obrk + increment;
6      if (_syscall_(SYS_brk, pbrk, 0, 0) == 0)
7      {
8          obrk = pbrk;
9          return (void*)obrk;
10     }
11     return (void*)-1;
12 }

```

● 运行《仙剑奇侠传》，logo 出现后报浮点数异常

问题描述

运行《仙剑奇侠传》时，出现以下 logo 后报"Floating point exception(core dumped)浮点异常（段错误）"错误，程序崩溃



问题截图

```

PAL_InitInput success
PAL_InitResources success
Makefile:46: recipe for target 'run' failed
make[1]: *** [run] Floating point exception (core dumped)
make[1]: Leaving directory '/home/pxc/ics2018/nemu'
/home/pxc/ics2018/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Error 2
pxc@ubuntu:~/ics2018/nanos-lite$

```

发现者

1811342 陈沃东, 1811461 赵珩等

解决方法

实现 ctdid 指令时, 32 位的情况下需要把 eax 的最高位扩展到 edx。否则,《仙剑奇侠传》运行 PAL_FadeOut (实现画面淡出) 函数会使用一个负数除以 1, 而 edx 中未做符号扩展, 即全 0。当 edx 全 0, 而 eax 最高位是 1 (负数) 时, idiv 总会产生浮点数异常

PA4&PA5

● 页表项无效 / 物理地址超限？

问题描述

在实现了虚存映射机制, 运行测试程序时提示页表项无效: Assertion 'pde.present' failed.

问题截图

```

[src/loader.c,22,loader] Load [16] /bin/dummy with size: 17888
nemu: src/memory/memory.c:51: page_translate: Assertion `pde.present' failed.
make[1]: *** [run] 已放弃 (core dumped)
make[1]:正在离开目录 `/home/shichaochao/ics2017/nemu'
make: *** [run] 错误 2
shichaochao@shichaochao-virtual-machine:~/ics2017/nanos-lite$

```

```

PAL_InitGlobals success
PAL_InitFont success
PAL_InitUI success
physical address(0x58155540) is out of bound
nemu: src/memory/memory.c:51: paddr_read: Assertion `addr < (128 * 1024 * 1024)' failed.
Makefile:46: recipe for target 'run' failed

```

发现者

1811418 施超超、1811010 童钊珂、1811554 林铸天、1811509 夏宁一等

解决方法

这一问题是 PA4 第一阶段的常见问题，可以在发生错误时输出 va、cr3 等信息来帮助调试，问题产生可能有多种多样的原因：

1. **堆区映射问题**：没有修改 sys_brk 系统调用，依然总是返回 0，而没有使用 mm_brk 进行映射（可以通过 dummy.c，运行 text.c、pal.c 时出错）
2. **_sbrk 参数设置问题**：需要把新的栈顶地址作为系统调用的第二个参数传入（可以通过 dummy.c，运行 text.c、pal.c 时出错）
3. **地址偏移计算问题**：注意页目录和页表的索引都是 10 位，是按页表项的索引而不是按字节索引。页表项占据 4 个字节，因此计算地址时应当乘以 4。（可以通过 dummy.c [能通过的原因值得思考思考]，运行 text.c、pal.c 时出错）
4. **loader 实现问题**：在实现虚存映射后会修改 loader.c，注意必须先进行映射，然后才能读取文件。部分同学在循环之前先进行了一次读取导致了这一问题（不能通过 dummy.c）

```
Log( LOGD [MU] AS WITH SIZE: MU, fd, filename, bytes),
fs_read(fd, DEFAULT_ENTRY, bytes);
void *pa,*va = DEFAULT_ENTRY;
while(bytes>0){
    //申请一页空闲的物理页
    pa = new_page();
    //把这一物理页映射到用户程序的虚拟地址空间中
    _map(as, va, pa);
    Log("pgdir:%p, pgtab:%p, va:%p", dpkdir, dpgtab, dva);
    //从文件中读入一页的内容到这一物理页上
    fs_read(fd, pa, PGSIZE);
    va += PGSIZE;
    bytes -= PGSIZE;
}
```

● 实现 page_translate 时，64 位机器报指针转换错误？

问题描述

page_translate 中涉及将 32 位 int 赋予指针，而 64 位系统中的指针是 64 位的，这时强转会报整型-指针长度不一致的警告，如何解决？

问题截图

```
src/memory/memory.c: In function 'page_translate':
src/memory/memory.c:31:18: warning: cast to pointer from integer of different si
ze [-Wint-to-pointer-cast]
    PDE *pgdir = (PDE*)(PTE_ADDR(cpu.cr3.val));
                      ^
src/memory/memory.c:32:26: warning: cast from pointer to integer of different si
ze [-Wpointer-to-int-cast]
    pde.val = paddr_read((paddr_t) &pgdir[PDX(addr)], 4);
                          ^
src/memory/memory.c:46:13: warning: cast to pointer from integer of different si
ze [-Wint-to-pointer-cast]
    ptdir = (PTE*)(PTE_ADDR(pde.val));
              ^
src/memory/memory.c:47:26: warning: cast from pointer to integer of different si
ze [-Wpointer-to-int-cast]
    pte.val = paddr_read((paddr_t)&ptdir[PTX(addr)], 4);
                          ^
```

发现者

1811499 麦隽韵, 1811371 黎鸿饱等

解决方法

取消-Werror 编译选项, 忽略这一警告。但是忽略警告之后也会略过其他警告, 因此要谨慎。

最稳妥的方法: 换用 32 位虚拟机。

● 实现虚存映射后, 运行 dummy.c, 段错误/物理地址超限

问题描述

读取虚拟内存时, 由于 cr0.paging 一直为 0, 程序无法进入到 page_translate 的分支, 因而发生段错误或物理地址超限 (具体是哪个错误与实现有关)

问题截图

```
[src/memory/memory.c,72,page_translate] pg
[src/memory/memory.c,107,vaddr_write] paddr: 134512640
Makefile:46: recipe for target 'run' failed
make[1]: *** [run] Segmentation fault (core dumped)
make[1]: Leaving directory '/home/jiayin/Downloads/ics2018/nemu'
/home/jiayin/Downloads/ics2018/nexus-am/Makefile.app:35: recipe for target 'run'
failed
[src/loader.c,34,loader] physical address: 1d95000
[src/proc.c,16,load_prog] load finished
physical address(0x08048000) is out of bound
nemu: src/memory/memory.c:22: paddr_read: Assertion 'addr < (128 * 1024 * 1024)' failed.
Makefile:46: recipe for target 'run' failed
make[1]: *** [run] Aborted (core dumped)
```

发现者

1711388 肖阳, 1811523 郑佳音等

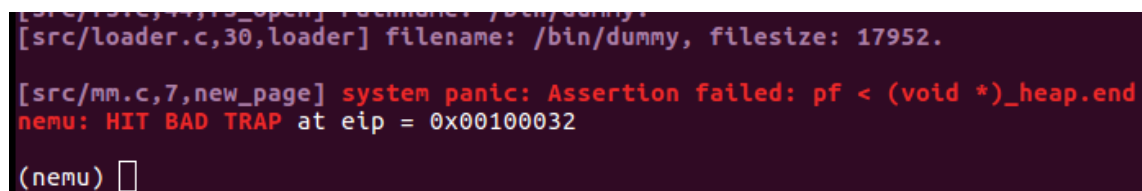
解决方法

检查 mov_cr2r 执行函数的实现, 需要使用 operand_write 将结果从临时寄存器写回到内存或寄存器中。下图中的例子由于没有回写, 发生了段错误:

```
make_EHelper(mov_cr2r) {  
    switch (id_src->reg){  
        case 0:  
            t0 = cpu.cr0.val;  
            break;  
        case 3:  
            t0 = cpu.cr3.val;  
            break;  
        default:  
            assert(0);  
    }  
}
```

● 实现虚存映射后, 运行 dummy.c, new_page 函数报错

问题截图



```
[src/loader.c,30,loader] filename: /bin/dummy, filesize: 17952.  
[src/mm.c,7,new_page] system panic: Assertion failed: pf < (void *)_heap.end  
nemu: HIT BAD TRAP at eip = 0x00100032  
(nemu) █
```

发现者

1811499 麦隽韵

解决者

1811499 麦隽韵

解决方法

检查 loader.c 的实现, 发现循环时使用了一个 size_t 类型的变量, size_t 相当于 uint32, 是无符号数, 因此永远不会小于 0


```

size_t size = fs_filesz(fd);
void *pa, *va = DEFAULT_ENTRY;
Log("filename: %s, filesize: %d.", filename, size);
while(size > 0) {
    pa = new_page();
    _map(as, va, pa);
    size_t len = size > PGSIZE ? PGSIZE:size;
    //Log("%d", len);
    fs_read(fd, pa, len);
    va += PGSIZE;
    size -= PGSIZE;
}
//fs_read(fd, DEFAULT_ENTRY, size);

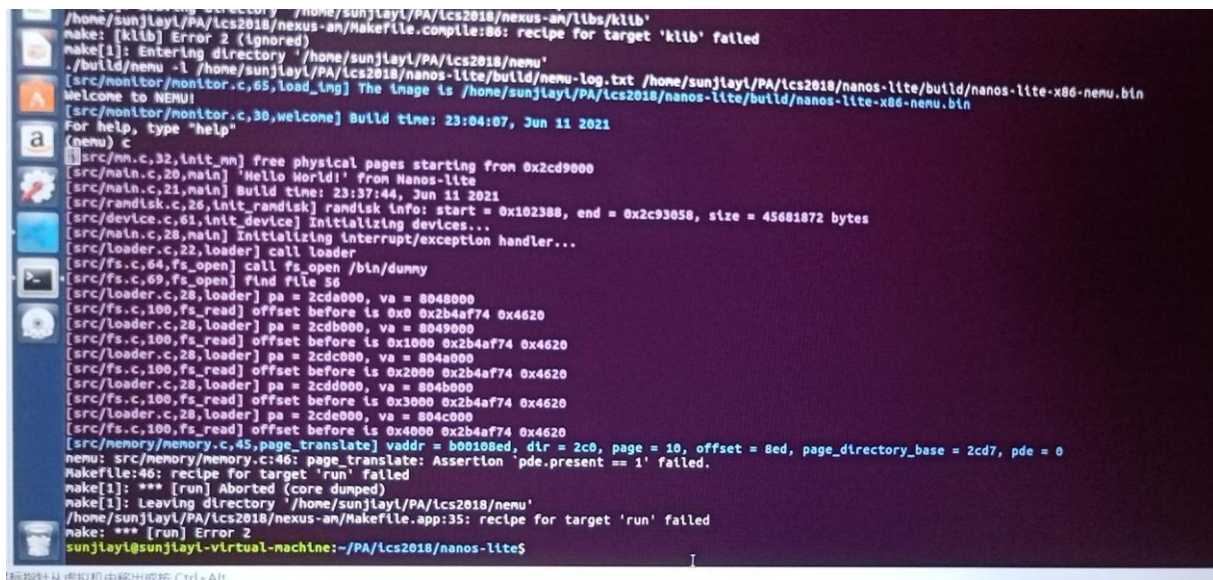
```

● 实现虚存映射后，运行 dummy.c，加一句 Log 报错？

问题描述

实现虚存映射以后，在分页机制上运行 dummy.c 时，在 loader.c 中加了一个 log 语句程序就报错了（读取内存报错 assertion 'pde.present=1' failed），如果不加 log 语句程序就正常通过？

问题截图



发现者

1810756 孙家宜

解决方法

在 PA2 以后，printf、log 等方法都是借助 klib.a 中的代码来实现的，klib.a 中的代码会被链

接到系统内核的代码中，**运行在 nemu 上**，而不是直接使用虚拟机的库函数输出。

在 nemu 中，取指令的时候会使用 `vaddr_read`，因此添加 Log 语句有可能导致报错。

查看反汇编文件，发现 `printf` 函数的位置正好处于两个页面的交界处。因此需要考虑跨页读取和 loader 函数中的映射实现是否正确。

● 实现浮点数后，运行《仙剑奇侠传》卡死或报错？

问题描述

PA3 时运行《仙剑奇侠传》是正常的（不能进入战斗也属正常）。但是在 PA4 实现了虚拟内存、或者是 PA5 实现了 FLOAT 定点数之后，《仙剑奇侠传》没有进入战斗也会报错，或者进入战斗后画面卡住？

发现者

1810756 孙家宜等

解决方法

PA4 后期工程会变得越来越复杂，我们可以把模块分离出来进行调试：

1. 如果是刚刚实现虚拟内存映射，先保证其他测试程序不出错（`text.c` 等），再去尝试《仙剑奇侠传》；
2. 如果《仙剑奇侠传》总是出现问题，应该撤销 PA4 的更改（切换分支），重新调试 PA3 的《仙剑奇侠传》；
3. 如果浮点数部分出了问题，可以将浮点数有关的两个文件（`FLOAT.h` 和 `FLOAT.c`）分离出来，单独测试其正确性。

最容易出现的问题：手册中，我们通过 $A = a * 2^{16}$ 的映射关系定义了 FLOAT 定点数。乘法时如果不做处理，会得到 $A*B = (a * 2^{16}) * (b * 2^{16}) = (a*b) * 2^{32}$ 。这时相当于将结果左移了 32 位，只有小数部分会被保留下来，需要对这种情况预先进行处理，除法时同理。

浮点数精度问题：在 `Fsqrt` 和 `Fpow` 函数中，框架代码通过循环不断进行计算，直到误差小于 $1e-5$ 时跳出循环。但是如果实现的浮点数达不到精度要求，就会在这里陷入死循环。

可以将允许跳出的精度降低到 $1e-4$ ，或想办法提高浮点数精度。

● 编写不朽的传奇时，按下 F12，ndl.c 报错

问题描述

PA4 之前的内容已经全部完成并通过测试，但在最后一个部分，同时运行《仙剑》和 video.c，按下 F12 之后，程序没能成功切换，ndl.c 再次报错

问题截图

```
file 4: /dev/events
file 5: /proc/dispinfo
Assertion failed: screen_w > 0 && screen_h > 0, file src/ndl.c, line 141
nemu: HIT BAD TRAP at eip = 0x0010002f
(nemu) █
```

发现者

1811554 林铸天，1813655 杨雨奇等

解决方法

运行多个程序，并且多个程序都读取同一个文件时，需要考虑多个程序对文件的操作。在真实的系统中，多个程序读取同一个文件时，系统一般会为每一个程序维护一个文件描述符。但是在 pa 当中，多个程序共用同一个 open_offset，如果上一个程序读取完毕之后没有将这个指针归位，下一个程序就不能从头读取。因此，实现系统调用时，我们需要考虑这一问题。在目前的 PA 当中，可以在 sys_close 中将 open_offset 设置为 0；在 sys_open 时将 open_offset 归零也是一种不错的选择，而且能够避免用户程序不关闭文件的问题。

其它

● 找不到 sdl2.h、未知命令 git、开启 DIFF_TEST 后无法启动

qemu...

问题描述

编译程序时，提示找不到某个头文件，且此文件不属于 PA，例如 sdl2.h。

使用 git 提交代码时提示 git 为未知命令

PA2 开启 DIFF_TEST 调试时无法打开 qemu 等

问题截图

找不到 sdl2.h

```
ubuntu@ubuntu-virtual-machine:~/ics20180/nemu$ make run
+ CC src/device/keyboard.c
src/device/keyboard.c:3:10: fatal error: SDL2/SDL.h: 没有那个文件或目录
#include <SDL2/SDL.h>
          ^
compilation terminated.
Makefile:23: recipe for target 'build/obj/device/keyboard.o' failed
make: *** [build/obj/device/keyboard.o] Error 1
```

无法启动 qemu

```
exec: No such file or directory
exec error
nemu: src/monitor/diff-test/diff-test.c:90: init_diffptest: Assertion `0' failed.
^Z
```

解决方法

按照 PA0 手册的指引安装完整的依赖：

apt-get install build-essential # build-essential packages, include binary utilities, gcc, make, and so on

apt-get install gdb # GNU debugger

apt-get install git # reversion control system

apt-get install libreadline-dev # a library to use compile the project later

apt-get install libsdl2-dev # a library to use compile the project later

apt-get install qemu-system-x86 # QEMU

● make run 编译时提示找不到某个 Makefile 文件？/ 如何配置环境变量？

问题描述

ics2018 版本，在各阶段编译时都可能遇到此问题，运行 make 时提示找不到某个文件或目录

问题截图



```
qin2mu4@ubuntu:~/Desktop/ics2018/nexus-am/tests/cputest$ make ARCH=x86-nemu ALL=dummy run
Makefile:1: /Makefile.check: 没有那个文件或目录
make: *** No rule to make target '/Makefile.check'. 停止。
```

解决方法：

按照 PA0 的指引添加环境变量：

```
export AM_HOME={your ics path}/nexus-am
```

```
export NEMU_HOME={your ics path}/nemu
```

```
export NAVY_HOME={your ics path}/navy-apps
```

Plus：在控制台的命令行里输入 export 设置的环境变量仅在此控制台的生命周期里有效。如果想要免去每次输入的麻烦，可以将上述三条语句添加到 ~/.bashrc 文件的末尾，然后运行一次 source ~/.bashrc 或重启控制台即可永久添加环境变量。

● 如何确保代码默认编译到 x86-nemu 而不是 native？

问题描述

ics2018，为什么我修改了 makefile 中的架构设置，代码还是自动编译到 native 架构下了？

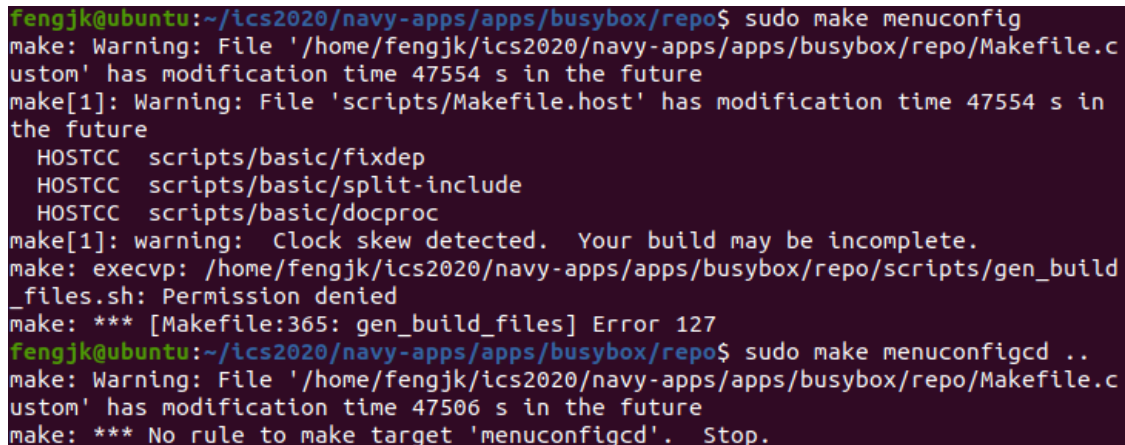
解决方法

在 PA 进行过程中，以下 2 个文件需要修改：

1. AM_HOME/makefile.check : ARCH ?= x86-nemu
2. NAVY_HOME/makefile.check : ISA ?= x86

● 编译或运行脚本时 Permission denied ?

问题截图



```
fengjk@ubuntu:~/ics2020/navy-apps/apps/busybox/repo$ sudo make menuconfig
make: Warning: File '/home/fengjk/ics2020/navy-apps/apps/busybox/repo/Makefile.c
ustom' has modification time 47554 s in the future
make[1]: Warning: File 'scripts/Makefile.host' has modification time 47554 s in
the future
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/basic/split-include
  HOSTCC  scripts/basic/docproc
make[1]: warning: Clock skew detected. Your build may be incomplete.
make: execvp: /home/fengjk/ics2020/navy-apps/apps/busybox/repo/scripts/gen_build
_files.sh: Permission denied
make: *** [Makefile:365: gen_build_files] Error 127
fengjk@ubuntu:~/ics2020/navy-apps/apps/busybox/repo$ sudo make menuconfigd ..
make: Warning: File '/home/fengjk/ics2020/navy-apps/apps/busybox/repo/Makefile.c
ustom' has modification time 47506 s in the future
make: *** No rule to make target 'menuconfigd'. Stop.
```

解决方法

由于移动了文件夹等原因导致权限出现问题，解决方法：在 ics 工程根目录下执行一次

```
sudo chmod 777 -R .
```

● 修改代码程序没有变化？

问题描述

这一问题常出现于修改了 AM_HOME/am/arch/x86-nemu 目录下的代码之后，例如修改了 AM 的编译选项或添加了 asye 模块用于实现系统调用。无论如何修改代码都无法通过测试，但是用 printf 等方法输出的信息看不出任何问题。

另外，如果有已经编译好的二进制文件。修改代码时如果引入了编译错误（例如标识符拼写出错等），make 之后会继续使用先前的二进制文件，因此没有变化。

问题截图

无论怎样修改，都显示未实现的中断调用，Unhandled event ID = 3

```
[src/irq.c,6,do_event] system panic: Unhandled event ID = 3
```

发现者

1811484 李金雯, 1811421 苏家兴等

解决方法

make clean !!!

实验框架中的 Makefile 不会自动清除 am 编译出的可重定位文件, 而且在源代码更新时不会更新这些可重定位文件, 你以为实现了的代码没有被机器识别到。在 AM_HOME 下 make clean 即可

● 虚拟机崩溃与 git 崩溃问题

解决方法

1. 首先, 出现此类问题时, 先报告给助教或老师, 我们会留存你的崩溃信息, 因为此类问题无法保证能被完全恢复。
2. 虚拟机崩溃时, 不要删除虚拟机的磁盘文件, 可以使用另一个虚拟机挂载这个磁盘, 导出其中的数据。更换虚拟机不会影响 git log。
3. git 崩溃时, 如果出现 git object file empty 的错误, 首先备份文件夹, 然后尝试参考以下两个链接进行恢复:

<https://howto.lintel.in/fix-git-object-file-empty-error/>

<https://stackoverflow.com/questions/11706215/how-to-fix-git-error-object-file-is-empty>

如果出现其他类型错误, 也可以参照网络上的资料尝试恢复。

● 作业提交/git 相关

实验报告应该包含哪些内容

实验目的、实验内容、实验步骤、必答题、自己实验过程中遇到的问题和解决方法（**该部分是重点**）

git log 怎么导出？是否需要使用 make submit 提交？

git log 用本地管道导出到 txt 文本文件中即可。

不需要使用 make submit。（注：make submit 会将代码打包并自动提交到南京大学网站平台上，我们不需要！）

提交源码的注意事项

提交源码之前务必在根目录执行 make clean，压缩包一般不大于 50MB。

尽量不要使用超大附件，否则过期后无法下载